Алгоритмы и игры на Си

С абсолютного нуля — учимся программировать, разбирая каждую строку

Учебник

Содержание

Введение. Зачем учить Си через алгоритмы и игры **1**
Почему именно Си . . . 1
Как устроена книга . . . 2
Как мы будем работать в каждой главе . . . 2
Что понадобится . . . 2
Маленькое обещание . . . 3

Глава 1. Компилятор и первая программа **4**
Что такое компилятор . . . 4
Ставим компилятор . . . 4
macOS . . . 5
Linux . . . 5
Windows . . . 5
Первая программа . . . 5
`#include <stdio.h>` . . . 6
Пустая строка . . . 6
`int main(void)` . . . 6
`{` и `}` . . . 6
`printf("Привет, мир!\n");` . . . 7
`return 0;` . . . 7
Компилируем и запускаем . . . 7
Включаем предупреждения (важная привычка) . . . 8
Частые ошибки . . . 8
Упражнения . . . 10
Что мы получили . . . 10

Глава 2. Минимум Си: переменные, ввод, ветвления, циклы, функции **11**
Шаг 1. Переменная — коробка для значения . . . 11
Основные типы . . . 12
Шаг 2. Спрашиваем у пользователя — `scanf` . . . 13
Арифметика . . . 13
Шаг 3. Принимаем решения — `if` и `else` . . . 14
Операторы сравнения и логики . . . 15
Шаг 4. Повторяем — цикл `while` . . . 15
Сокращения, которые ты будешь видеть всюду . . . 16
Шаг 5. Цикл `for` — когда знаем число повторений . . . 16
Шаг 6. Свои функции . . . 16
Складываем всё вместе . . . 17

Частые ошибки . . . 19
Упражнения . . . 19
Что мы получили . . . 20

Глава 3. Массивы, строки, указатели и память 21
Часть 1. Массив — много коробок в ряд . . . 21
Перебор массива циклом . . . 22
Заполнение массива вводом . . . 23
Часть 2. Строки — это массивы символов . . . 23
Чтение строки от пользователя . . . 25
Полезные функции для строк (`<string.h>`) . . . 25
Часть 3. Указатели — переменные, хранящие адрес . . . 26
Зачем это нужно: функция, меняющая аргумент . . . 27
Массив и указатель — близкие родственники . . . 28
Часть 4. Динамическая память — когда размер заранее неизвестен . . . 29
Замечание про русские буквы . . . 31
Частые ошибки . . . 31
Упражнения . . . 31
Что мы получили . . . 31

Глава 4. Числа: чётность, сумма цифр, переворот числа, факториал, FizzBuzz 33
Что мы сделаем . . . 33
Мостик: чётность через остаток . . . 33
Часть 1. Сумма цифр и переворот числа . . . 34
Разбираем задачу руками . . . 34
Шаг 1. Наивная версия — считаем сумму . . . 34
Шаг 2. Проблема — где взять перевёрнутое число? . . . 35
Шаг 3. Проблема — последняя цифра тратится дважды . . . 36
Шаг 4. Краевые случаи — ноль и минус . . . 36
Полная программа `04-digits.c` . . . 37
Проверяем . . . 38
Часть 2. Факториал и ловушка переполнения . . . 39
Разбираем задачу руками . . . 39
Шаг 1. Наивная версия на `int` . . . 39
Шаг 2. Проблема — это неправильный ответ! . . . 40
Шаг 3. Фикс — берём `long long` . . . 40
Шаг 4. Но и `long long` не бесконечен . . . 41
Полная программа `04-factorial.c` . . . 41
Проверяем . . . 42
Часть 3. FizzBuzz . . . 42
Разбираем задачу руками . . . 42
Шаг 1. Наивная версия — и её баг . . . 43
Шаг 2. Фикс — самую строгую проверку ставим первой . . . 43
Полная программа `04-fizzbuzz.c` . . . 44
Проверяем . . . 44
Частые ошибки . . . 45
Упражнения . . . 45
Что мы получили . . . 46

Глава 5. НОД Евклида, простые числа и решето Эратосфена 47
Что мы сделаем . . . 47
Часть 1. Наибольший общий делитель . . . 48
Разбираем задачу руками . . . 48
Шаг 1. Наивный перебор . . . 48
Шаг 2. Идея Евклида . . . 49
Шаг 3. Евклид через остаток . . . 50
Полная программа `05-gcd.c` . . . 51
Часть 2. Простое число . . . 52
Разбираем задачу руками . . . 52
Шаг 1. Наивная проверка . . . 53
Шаг 2. Почему достаточно до корня . . . 53
Полная программа `05-prime.c` . . . 54
Часть 3. Решето Эратосфена . . . 56
Разбираем задачу руками . . . 56
Шаг 1. Массив-флаги . . . 57
Шаг 2. Вычёркиваем . . . 57
Полная программа `05-sieve.c` . . . 58
Как это работает: трассировка решета . . . 59
Проверяем . . . 59
Частые ошибки . . . 60
Упражнения . . . 60
Что мы получили . . . 61

Глава 6. Алгоритмы над массивами: минимум/максимум, сумма, среднее, разворот, сдвиг 62
Что мы сделаем . . . 62
Часть 1. Минимум, максимум, сумма, среднее . . . 63
Разбираем задачу руками . . . 63
Шаг 1. Наивная версия — и её баг . . . 63
В чём проблема . . . 64
Шаг 2. Фикс — стартуем с первого элемента . . . 64
Среднее: не наступи на целочисленное деление . . . 65
Полная программа `06-minmax.c` . . . 65
Проверяем . . . 66
Часть 2. Разворот массива на месте . . . 67
Разбираем задачу руками . . . 67
Шаг 1. Меняем местами через временную переменную . . . 67
Шаг 2. Сводим индексы навстречу . . . 68
Почему условие именно `i < j` . . . 68
Полная программа `06-reverse.c` . . . 69
Проверяем . . . 70
Часть 3. Подсчёт значения и сдвиг влево . . . 70
Задача А. Сколько раз встречается число . . . 70
Задача Б. Циклический сдвиг влево на 1 . . . 71
Шаг 1. Наивно — и ловушка с границей . . . 71
Шаг 2. Фикс — останавливаем цикл на один шаг раньше . . . 71
Полная программа `06-count-rotate.c` . . . 72

Проверяем . . . 73
Частые ошибки . . . 74
Упражнения . . . 74
Что мы получили . . . 75

Глава 7. Поиск: линейный и бинарный **76**
Что мы сделаем . . . 76
Разбираем задачу руками . . . 76
Шаг 1. Наивный линейный поиск прямо в main . . . 77
Шаг 2. Что не так с наивной версией . . . 78
Шаг 3. Линейный поиск как функция . . . 78
Полная программа: линейный поиск . . . 79
Линейный поиск медленный — и вот почему . . . 80
Шаг 4. Идея бинарного поиска — руками . . . 81
Шаг 5. Почему `mid = low + (high - low) / 2`, а не `(low + high) / 2` . . . 82
Полная программа: бинарный поиск . . . 82
Как это работает: трассировка по шагам . . . 84
Во сколько раз бинарный быстрее . . . 85
Проверим на секундомере . . . 85
Проверяем . . . 86
Частые ошибки . . . 87
Упражнения . . . 88
Что мы получили . . . 88

Глава 8. Сортировки: пузырёк, выбором, вставками **89**
Что мы сделаем . . . 89
Общая заготовка: печать массива . . . 89
Часть 1. Сортировка пузырьком . . . 90
Идея руками . . . 90
Шаг 1. Наивная версия . . . 91
Шаг 2. Проблема: лишняя работа . . . 92
Шаг 3. Фикс: укорачиваем хвост и ставим флаг . . . 92
Полная программа . . . 93
Проверяем . . . 94
Часть 2. Сортировка выбором . . . 95
Идея руками . . . 95
Код . . . 95
Проверяем . . . 97
Часть 3. Сортировка вставками . . . 97
Идея руками . . . 97
Шаг 1. Наивная мысль и сразу аккуратная версия . . . 97
Проверяем . . . 99
Как это работает: чем три сортировки отличаются . . . 99
Сложность: почему это медленно и что бывает быстрее . . . 100
Быстрая сортировка (quicksort) . . . 100
Сортировка слиянием (merge sort) . . . 101
Готовое решение: `qsort` из `<stdlib.h>` . . . 101
Частые ошибки . . . 103

Упражнения . . . 103
Что мы получили . . . 104

Глава 9. Рекурсия: факториал, Фибоначчи, Ханойские башни 105
Что такое рекурсия . . . 105
Что будет без базы . . . 106
Стек вызовов простыми словами . . . 106
Часть 1. Факториал рекурсивно . . . 106
Пишем код . . . 107
Как это работает: раскрутка вызовов . . . 108
Проверяем . . . 109
Часть 2. Числа Фибоначчи и ловушка медленной рекурсии . . . 109
Наивная версия . . . 110
В чём проблема: дерево вызовов . . . 110
Фикс: один проход циклом . . . 111
Часть 3. Ханойские башни . . . 114
Разбираем руками . . . 114
Пишем код . . . 114
Проверяем . . . 116
Частые ошибки . . . 117
Упражнения . . . 117
Что мы получили . . . 118

Глава 10. Строковые алгоритмы: длина, переворот, палиндром, слова, анаграммы 119
Важное предупреждение про русские буквы . . . 119
Что мы сделаем . . . 119
Часть 1. Длина и переворот строки . . . 120
Разбираем задачу руками . . . 120
Шаг 1. Своя функция длины . . . 120
Шаг 2. Наивный переворот (и где он спотыкается) . . . 121
Шаг 3. Виноват `'\n'` от fgets — убираем его . . . 122
Полная программа: `10-reverse.c` . . . 123
Часть 2. Палиндром . . . 124
Идея . . . 124
Программа: `10-palindrome.c` . . . 125
Часть 3. Подсчёт слов . . . 127
Идея: ловим начало слова . . . 127
Программа: `10-words.c` . . . 127
Часть 4. Анаграммы . . . 129
Идея: считаем частоты букв . . . 129
Программа: `10-anagram.c` . . . 129
Частые ошибки . . . 132
Упражнения . . . 132
Что мы получили . . . 133

Глава 11. Динамические структуры данных: растущий массив, стек, очередь, связный список 134
Знакомимся: `struct` — свой составной тип . . . 134

Указатель на структуру и стрелка `->` . . . 135
`typedef` — короткое имя для типа . . . 136
Часть 1. Растущий массив . . . 136
Разбираем руками . . . 136
`realloc` — изменить размер уже выделенной памяти . . . 137
Наивно → запуск . . . 137
Часть 2. Стек (LIFO) . . . 139
Аналогия: стопка тарелок . . . 139
Строим на массиве . . . 139
Часть 3. Очередь (FIFO) . . . 141
Аналогия: очередь в магазине . . . 141
Проблема наивной очереди и кольцевой буфер . . . 142
Часть 4. Связный список . . . 144
Зачем он, если есть массив . . . 144
Описываем узел через `struct` . . . 144
Добавление в начало — по шагам . . . 145
Обход и освобождение . . . 146
Полная программа . . . 146
Частые ошибки . . . 148
Упражнения . . . 148
Что мы получили . . . 149

Глава 12. Игра «Угадай число» 150
Что мы сделаем . . . 150
Разбираем задачу руками . . . 151
Шаг 1. Откуда брать случайность . . . 151
Шаг 2. Почему `rand()` повторяется и что такое «засев» . . . 152
Шаг 3. Зерно из текущего времени — `time` . . . 153
Шаг 4. Сужаем диапазон до 1..100 . . . 154
Шаг 5. Одна попытка угадать . . . 155
Шаг 6. Игровой цикл — спрашиваем, пока не угадал . . . 156
Шаг 7. Считаем попытки . . . 157
Шаг 8. Защита от неправильного ввода . . . 158
Полная программа . . . 159
Как это работает . . . 161
Проверяем . . . 161
Частые ошибки . . . 162
Упражнения . . . 163
Что мы получили . . . 164

Глава 13. Игра «Камень, ножницы, бумага» 165
Что мы сделаем . . . 165
Разбираем задачу руками . . . 165
Шаг 1. Один раунд, выбор компьютера . . . 166
Шаг 2. Превращаем число в слово . . . 167
Шаг 3. Даём имена числам — `enum` . . . 168
Шаг 4. Кто победил — наивный способ . . . 168
Шаг 5. Красивый приём с остатком . . . 169

Шаг 6. Ввод игрока с проверкой . . . 170
Полная программа . . . 171
Как это работает . . . 174
Проверяем . . . 174
Частые ошибки . . . 175
Упражнения . . . 176
Что мы получили . . . 176

Глава 14. Игра «Виселица» 178
Что мы сделаем . . . 178
Разбираем задачу руками . . . 178
Шаг 1. Список слов и случайный выбор . . . 179
Шаг 2. Показываем слово прочерками . . . 181
Шаг 3. Читаем букву — и ловим коварный перевод строки . . . 182
Приводим к нижнему регистру . . . 183
Шаг 4. Запоминаем букву и открываем её . . . 184
Шаг 5. Жизни, победа и поражение . . . 185
Шаг 6. Рисуем виселицу . . . 186
Полная программа . . . 186
Как это работает . . . 189
Проверяем . . . 190
Частые ошибки . . . 192
Упражнения . . . 193
Что мы получили . . . 193

Глава 15. Игра «Крестики-нолики» 194
Что мы сделаем . . . 194
Разбираем задачу руками . . . 194
Двумерный массив — таблица в памяти . . . 195
Как это лежит в памяти . . . 196
Перебор двумерного массива — цикл в цикле . . . 196
Шаг 1. Рисуем пустое поле . . . 196
Шаг 2. Добавляем разделители и выносим печать в функцию . . . 197
Шаг 3. Один ход игрока . . . 199
Шаг 4. Проверяем ввод . . . 199
Шаг 5. Проверка победы — выносим в функцию . . . 201
Шаг 6. Проверка ничьей . . . 202
Полная программа . . . 203
Как это работает . . . 207
Проверяем . . . 207
Частые ошибки . . . 208
Упражнения . . . 209
Что мы получили . . . 209

Глава 16. Игра «Сапёр» 211
Что мы сделаем . . . 211
Разбираем задачу руками . . . 212
Две карты, наложенные друг на друга . . . 212

Сколько мин вокруг клетки . . . 212
Заливка пустой области . . . 213
Шаг 1. Поле и его печать . . . 214
Шаг 2. Расставляем мины случайно . . . 215
Первый ход не должен убивать . . . 216
Шаг 3. Считаем соседей . . . 218
Шаг 4. Открываем клетку и запускаем заливку . . . 219
Шаг 5. Главный цикл: ход, проигрыш, победа . . . 221
Полная программа . . . 222
Как это работает . . . 227
Проверяем . . . 228
Частые ошибки . . . 229
Упражнения . . . 230
Что мы получили . . . 230

Глава 17. Игра «Змейка» 232
Что мы сделаем . . . 232
Разбираем задачу руками . . . 232
Как хранить змейку . . . 233
Почему обычный ввод не подходит . . . 233
Шаг 1. Поле и неподвижная змейка . . . 234
Шаг 2. Заставляем змейку ползти . . . 236
Шаг 3. Ловим клавиши без Enter — сырой режим . . . 238
Включаем сырой режим . . . 239
Страховка: терминал надо вернуть в любом случае . . . 240
Читаем клавишу . . . 241
Где ломается защита от разворота . . . 243
Шаг 4. Еда, рост и счёт . . . 244
Шаг 5. Столкновения и конец игры . . . 246
Полная программа . . . 247
Как это работает . . . 253
Проверяем . . . 254
Частые ошибки . . . 255
Упражнения . . . 255
Что мы получили . . . 256

Глава 18. Что дальше 257
Что мы прошли . . . 257
Привычки, которые стоит закрепить . . . 257
Инструменты, которые сделают тебя сильнее . . . 258
Куда двигаться дальше . . . 258
Маленький совет напоследок . . . 259

Как готовиться к собеседованию по Си 260
Что проверяют на каждом уровне . . . 260
Как пользоваться задачами . . . 261
Частые ошибки на собеседовании (и как их избежать) . . . 261
Что повторить перед собеседованием . . . 261

Приложение А. Задачи с собеседований: Junior **263**
Как пользоваться . . . 263

Часть 1. Задачи на код **265**
Задача 1. Поменять местами две переменные . . . 265
Задача 2. Проверить чётность числа . . . 266
Задача 3. Максимум и минимум массива . . . 267
Задача 4. Развернуть строку на месте . . . 268
Задача 5. Проверка палиндрома . . . 270
Задача 6. Сколько цифр в числе . . . 271
Задача 7. Сумма цифр числа . . . 272
Задача 8. Подсчёт гласных в строке . . . 273
Задача 9. Факториал итеративно . . . 274
Задача 10. FizzBuzz . . . 276
Задача 11. Второй по величине элемент массива . . . 277
Задача 12. Проверка на простое число . . . 278

Часть 2. Каверзы: «что выведет / в чём ошибка» **280**
Каверза 1. `sizeof('a')` — сколько это? . . . 280
Каверза 2. Целочисленное деление `5 / 2` . . . 280
Каверза 3. `printf("%d", c)` для `char` . . . 281
Каверза 4. = вместо == в условии . . . 281
Каверза 5. Лишняя `;` после `for(...)` . . . 282
Каверза 6. `printf("%d %d", i++, ++i)` . . . 282
Каверза 7. Неинициализированная переменная . . . 283
Каверза 8. Остаток от отрицательного числа . . . 283

Мини-резюме **284**

Куда дальше **285**

Приложение Б. Задачи с собеседований: Middle **286**
Как пользоваться этим приложением . . . 286
Часть 1. Задачи на код . . . 287
Задача 1. Своя `strlen`, `strcpy`, `strcmp` . . . 287
Задача 2. Развернуть односвязный список . . . 289
Задача 3. Середина списка (быстрый и медленный указатель) . . . 290
Задача 4. Цикл в списке (черепаха и заяц, алгоритм Флойда) . . . 291
Задача 5. Подсчёт единичных битов (popcount) . . . 292
Задача 6. Степень ли двойки . . . 294
Задача 7. Удалить дубликаты из отсортированного массива на месте . . . 295
Задача 8. Число, встречающееся нечётное число раз (XOR) . . . 296
Задача 9. Корректный бинарный поиск . . . 297
Задача 10. Слияние двух отсортированных массивов . . . 298
Задача 11. Поменять местами два числа через указатели . . . 299
Часть 2. Каверзы . . . 300
Каверза 1. `char *s = "abc"` против `char s[] = "abc"` . . . 301
Каверза 2. `sizeof` массива внутри функции . . . 302
Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)` . . . 303

Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++` . . . 303
Каверза 5. `const char *p` против `char * const p` . . . 304
Каверза 6. Найди баг: утечка памяти . . . 304
Каверза 7. Найди баг: use-after-free и двойной free . . . 305
Каверза 8. Возврат адреса локальной переменной . . . 306
Мини-резюме . . . 307
Что дальше . . . 307

Приложение B. Задачи с собеседований: Senior **308**

Часть 1. Задачи на код **309**
Задача 1. Быстрая сортировка (quicksort) . . . 309
Задача 2. Сортировка слиянием (merge sort) . . . 311
Задача 3. Обобщённый стек через `void *` . . . 313
Задача 4. Свои `atoi` и `itoa` с обработкой краёв . . . 315
Задача 5. Таблица обработчиков на указателях на функции . . . 318
Задача 6. Кольцевой буфер (ring buffer) . . . 319
Задача 7. Хеш-таблица «строка → счётчик» с цепочками . . . 322
Задача 8. Точка пересечения двух односвязных списков . . . 324
Задача 9. Набросок LRU-кэша . . . 325
Задача 10. Разворот числа с защитой от переполнения . . . 327

Часть 2. Каверзы и теория **329**
Каверза 1. Что такое UB и где оно прячется . . . 329
Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать . . . 330
Каверза 3. `volatile` — зачем он нужен . . . 331
Каверза 4. Две роли `static` . . . 331
Каверза 5. Stack vs heap и причины segfault . . . 332
Каверза 6. Выравнивание и padding: что выведет `sizeof` . . . 332
Каверза 7. Опасный макрос `#define SQR(x) x*x` . . . 333
Каверза 8. Идиома `do { ... } while(0)` в макросах . . . 334
Каверза 9. Порядок вычисления аргументов не определён . . . 334
Каверза 10. Endianness — порядок байтов . . . 335

Итог практикума **336**
Напутствие . . . 336

Введение. Зачем учить Си через алгоритмы и игры

Эта книга для тех, кто **никогда не программировал**. Если ты умеешь открыть терминал и сохранить текстовый файл — этого достаточно. Всё остальное мы построим сами, с нуля, не пропуская ни одной мелочи.

Мы будем учить язык **Си** (по-английски — *C*). И учить не сухо, а на двух самых увлекательных вещах в программировании:

- **алгоритмы** — способы решать задачи: отсортировать числа, найти нужное среди тысяч, разложить число на простые множители;
- **игры** — настоящие, в которые можно играть прямо в терминале: «Угадай число», крестики-нолики, виселица, сапёр и даже змейка.

Каждая глава — это маленький проект, который мы соберём своими руками и запустим.

Почему именно Си

Си придумали в начале 1970-х, и с тех пор он никуда не делся. На нём написаны операционные системы (ядро Linux, части macOS и Windows), базы данных, игровые движки, прошивки микроконтроллеров. Когда тебе говорят «эта программа быстрая» — очень часто под капотом Си.

Для новичка у Си есть одно бесценное свойство: **он ничего не прячет**. В современных языках многое происходит «само собой» — память выделяется и освобождается, типы подбираются автоматически. Си заставляет тебя понимать, что происходит на самом деле: где лежат данные, сколько занимают байт, что такое указатель. Это тяжелее в начале, но даёт фундамент, на котором потом легко стоит любой другой язык.

Алгоритмы и игры — идеальный полигон для этого. Алгоритм заставляет думать чётко: «сделай вот это, потом вот это, повтори столько-то раз». Игра заставляет довести программу до конца: пока она не заработает, играть не получится.

Как устроена книга

Книга идёт **строго по порядку**. Каждая глава опирается на предыдущие. Не перепрыгивай: то, что разобрано в главе 2, в главе 8 уже не объясняется заново.

Три части:

1. **Основы.** Установим компилятор, напишем первую программу, разберём минимум языка: переменные, ввод-вывод, ветвления, циклы, функции, массивы, строки, указатели и память. Ровно столько, сколько нужно, чтобы двигаться дальше.
2. **Алгоритмы.** От простых числовых задач до сортировок, рекурсии и структур данных. Каждый алгоритм — сначала «руками» на бумаге, потом кодом.
3. **Игры.** Соберём шесть консольных игр, по нарастанию сложности. К этому моменту ты будешь уверенно читать и писать код.

Как мы будем работать в каждой главе

Один и тот же приём, который повторяется из главы в главу:

1. Ставим задачу простыми словами.
2. Решаем её **на бумаге**, чтобы понять суть.
3. Пишем **самую наивную** программу — пусть кривую.
4. **Запускаем** и смотрим, что вышло.
5. Находим проблему (ошибка, медленно, некрасиво).
6. **Чиним** — и снова запускаем.

Так ты увидишь программу в движении на каждом шаге, а не получишь готовый листинг неизвестно откуда. Именно так пишут код в реальности: не с первого раза набело, а постепенно, проверяя каждый кусок.

Что понадобится

- Компьютер с macOS или Linux. (Под Windows тоже можно — подскажу, где отличия, но примеры рассчитаны на macOS/Linux.)
- Терминал.
- Любой текстовый редактор: VS Code, Sublime, даже встроенный.
- Компилятор Си. Как его поставить — первая практическая глава.

Никаких платных программ и сложных сред разработки не нужно. Терминал и редактор — этого хватит на всю книгу.

Маленькое обещание

Будет момент, когда что-то не скомпилируется, выдаст непонятную ошибку или поведёт себя не так. Это **нормально и неизбежно** — так у всех, включая опытных. Я специально буду показывать такие моменты: вот сломалось, вот почему, вот как чинится. Умение читать ошибки — половина мастерства программиста.

Поехали. Следующая глава — ставим компилятор и запускаем первую программу.

Глава 1. Компилятор и первая программа

В этой главе мы поставим инструмент, который превращает наш текст в работающую программу, напишем классическое «Привет, мир!» и разберём в нём **каждый символ** — от первой решётки до последней точки с запятой. К концу главы ты будешь уметь скомпилировать и запустить программу на Си.

Что такое компилятор

Компьютер не понимает Си напрямую. Процессор понимает только **машинный код** — наборы чисел, команды вроде «сложи», «положи в память», «прыгни сюда». Писать в таких числах человеку невозможно.

Поэтому мы пишем на Си — понятном человеку языке, — а специальная программа переводит наш текст в машинный код. Эта программа-переводчик называется **компилятор** (compiler, от англ. *compile* — собирать).

Схема такая:

```
наш текст (file.c)  →  [компилятор]  →  исполняемый файл  →  запускаем
```

Файл с нашим кодом имеет расширение `.c`. Результат компиляции — **исполняемый файл**, готовая программа, которую можно запустить.

Ставим компилятор

Нам нужен компилятор `clang` или `gcc` — подойдёт любой, команды одинаковые. Проверим, есть ли он уже.

macOS

Открой терминал и набери:

```
cc --version
```

`cc` — это «C Compiler», стандартное имя компилятора в системе. Если увидишь что-то вроде `Apple clang version ...` — всё готово, переходи дальше. Если система предложит установить «Command Line Tools» — соглашайся, это и есть компилятор. Можно поставить и вручную:

```
xcode-select --install
```

Откроется окно установки, жми «Установить» и жди.

Linux

```
cc --version
```

Если не установлен, на Ubuntu/Debian:

```
sudo apt update
sudo apt install build-essential
```

`build-essential` — набор инструментов для сборки программ, включая компилятор `gcc`.

Windows

На Windows проще всего поставить **MSYS2** или **WSL** (подсистема Linux внутри Windows) и работать как в Linux. Либо MinGW-w64, который даёт `gcc`. Дальше в книге команды одинаковые, кроме главы про змейку — там есть пометка про отличия Windows.

Первая программа

Создай папку для книги, например `c-book`, зайди в неё и создай файл `hello.c`. Содержимое:

```c
#include <stdio.h>

int main(void)
{
    printf("Привет, мир!\n");
    return 0;
}
```

Сохрани. Теперь разберём **каждую строчку**.

`#include <stdio.h>`

Эта строка подключает к нашей программе чужой готовый код.

- `#include` — это **директива препроцессора**. Препроцессор — помощник компилятора, который срабатывает первым и подготавливает текст. Всё, что начинается с `#`, — команды для него. `include` по-английски «включить».
- `<stdio.h>` — имя файла, который мы включаем. Это **заголовочный файл** (header, расширение `.h`). `stdio` расшифровывается как *standard input/output* — стандартный ввод-вывод. В нём описана, среди прочего, функция `printf`, которой мы будем печатать на экран.
- Угловые скобки `<...>` означают «ищи этот файл среди системных», то есть среди тех, что идут вместе с компилятором.

Без этой строки компилятор не знал бы, что такое `printf`, и выдал бы ошибку.

Пустая строка

Просто для читаемости. Пустые строки Си игнорирует — они для людей, не для машины.

`int main(void)`

Это объявление **функции** по имени `main`. Функция — это кусок кода с именем, который можно выполнить. `main` — особенная: с неё начинается выполнение **любой** программы на Си. Когда ты запускаешь программу, система ищет `main` и выполняет то, что внутри.

Разберём по частям:

- `int` — это **тип** значения, которое функция вернёт в конце. `int` (от *integer* — целое) означает целое число. То есть `main` пообещала вернуть целое число.
- `main` — имя функции. Менять его нельзя: система ищет именно `main`.
- `(void)` — в скобках перечисляют, что функция принимает на вход. `void` по-английски «пустота, ничто». Значит, `main` ничего на вход не принимает.

`{` и `}`

Фигурные скобки обрамляют **тело** функции — всё, что она делает. Открывающая `{` — «здесь начинается код функции», закрывающая `}` — «здесь он заканчивается». Всё между ними выполняется по порядку, сверху вниз.

```
printf("Привет, мир!\n");
```

Это вызов функции `printf` — той самой, что описана в `stdio.h`. `printf` (от *print formatted* — печать с форматированием) выводит текст на экран.

- В круглых скобках — **аргумент**, то, что передаём функции. Здесь это текст `"Привет, мир!\n"`.
- Двойные кавычки `"..."` обрамляют **строку** — последовательность символов. Сам текст между кавычками появится на экране.
- `\n` — это **один** специальный символ, перевод строки (от *new line*). Обратный слеш `\` говорит: «следующий символ читай особым образом». `\n` означает «перейти на новую строку», как нажать Enter. Без него следующий вывод прилип бы к нашему тексту.
- `;` — **точка с запятой**. В Си ею заканчивается почти каждая команда (её называют *инструкция*, англ. *statement*). Это как точка в конце предложения. Забыть её — самая частая ошибка новичка, и компилятор сразу ругается.

```
return 0;
```

- `return` (англ. «вернуть») завершает функцию и возвращает значение туда, откуда её вызвали. Помнишь, `main` обещала вернуть `int` — целое число? Вот мы и возвращаем целое `0`.
- `0` по соглашению означает «программа отработала успешно, без ошибок». Любое другое число — сигнал, что что-то пошло не так. Этим кодом пользуется система и другие программы.
- И снова `;` в конце.

Компилируем и запускаем

В терминале, находясь в папке с `hello.c`:

```
cc hello.c -o hello
```

Разберём команду:

- `cc` — вызываем компилятор.
- `hello.c` — наш исходный файл.
- `-o hello` — **флаг** `-o` (от *output* — вывод) задаёт имя результата. Получится исполняемый файл `hello`. Без `-o` компилятор назвал бы результат `a.out` (историческое имя по умолчанию).

Если ошибок нет, команда отработает молча — это хорошо. Теперь запускаем:

```
./hello
```

`./` означает «файл в текущей папке». Без него система искала бы программу `hello` среди системных и не нашла. Результат:

```
Привет, мир!
```

Поздравляю — первая программа работает.

Включаем предупреждения (важная привычка)

Компилятор умеет предупреждать о подозрительных местах в коде, но по умолчанию молчит о многом. Заведём привычку компилировать с флагами:

```
cc -std=c11 -Wall -Wextra -O2 hello.c -o hello
```

- `-std=c11` — использовать стандарт языка **C11** (версия Си 2011 года). Так все будут на одной волне.
- `-Wall` — *warnings all*, включить основные предупреждения.
- `-Wextra` — добавить ещё дополнительные.
- `-O2` — *optimize level 2*, попросить компилятор сделать программу быстрее. Для учёбы не обязательно, но пусть будет.

Предупреждение (*warning*) — это не ошибка: программа соберётся. Но это компилятор говорит «тут что-то подозрительное, посмотри». **В этой книге мы пишем код так, чтобы предупреждений не было.** Считай их маленькими ошибками.

Частые ошибки

Здесь я не буду описывать ошибки словами — сломаю программу по-настоящему и покажу, что скажет компилятор. Читать эти сообщения — навык, который экономит часы, а выглядят они страшно только первые пару раз.

Забыл `;` в конце строки. Убираем точку с запятой после `printf`:

```
hello.c:5:38: error: expected ';' after expression
    5 |     printf("Привет, мир!\n")
      |                              ^
      |                              ;
1 error generated.
```

Разбираем сообщение по частям — устроены они все одинаково:

- `hello.c:5:38` — файл, строка 5, символ 38. Компилятор показывает **точное место**, курсор `^` под ним дублирует.
- `error` — это ошибка, файл не собран. Бывает ещё `warning` (предупреждение) — тогда программа соберётся, но что-то подозрительно.
- `expected ';' after expression` — «ожидал `;` после выражения».
- Ниже компилятор рисует саму строку и **подсказывает** прямо под курсором, что нужно вписать: `;`.

Важная тонкость: указано место, где **`;` не хватило**, — конец пятой строки. Компилятор дочитал до `return` на шестой строке и только там понял, что предыдущая инструкция не закончена. Поэтому ошибку часто надо искать на строку **выше** указанной.

Забыл `#include <stdio.h>`. Удаляем первую строку:

```
hello.c:3:5: error: call to undeclared library function 'printf'
with type 'int (const char *, ...)'; ISO C99 and later do not support
implicit function declarations [-Wimplicit-function-declaration]
    3 |     printf("Привет, мир!\n");
      |     ^
hello.c:3:5: note: include the header <stdio.h> or explicitly provide
a declaration for 'printf'
1 error generated.
```

«Вызов необъявленной библиотечной функции `printf`». Компилятор не знает, что это за имя, — мы же не подключили заголовок, где оно описано. И тут же строка `note:` подсказывает решение прямым текстом: «подключи заголовок `<stdio.h>`». Компилятор часто сам говорит, что делать, — не пролистывай эти строки.

Если увидишь в чужих текстах формулировку «implicit declaration of function» — это то же самое сообщение в старых версиях компиляторов. Раньше такой код собирался с предупреждением, теперь это ошибка.

Перепутал кавычки. Текст печатается в **двойных** кавычках `"..."`. Одинарные `'...'` в Си — это один символ, а не строка. Пишем `printf('Привет')` и получаем:

```
hq.c:5:12: error: character too large for enclosing character literal type
    5 |     printf('Привет');
      |            ^
```

«Символ не влезает в символьный литерал» — компилятор пытался понять `'Привет'` как **один** символ, а там их шесть. Одинарные кавычки — это ровно один знак: `'A'`, `'?'`, `'\n'`.

Запускаешь без `./`. Наберёшь `hello` — оболочка ответит:

```
zsh: command not found: hello
```

Она искала программу `hello` среди системных (в каталогах, которые перечислены в переменной `PATH`), а текущую папку туда не включают — из соображений безопасности. Нужно указать путь явно: `./hello`.

Файл не сохранён. Изменил код, но забыл сохранить — компилируется старая версия. Сохраняй перед каждой компиляцией.

Упражнения

1. Сделай так, чтобы программа печатала две строки: «Привет, мир!» и на следующей строке «Я учу Си». (Подсказка: можно два `printf`, а можно один с двумя `\n`.)
2. Убери `\n` из `printf` и посмотри, как изменится вывод. Где теперь оказывается приглашение терминала?
3. Намеренно убери `;` и прочитай, что скажет компилятор. Запомни этот текст ошибки — увидишь его ещё не раз.
4. Скомпилируй с `-Wall -Wextra` и убедись, что предупреждений нет.

Что мы получили

Мы поставили компилятор, написали, скомпилировали и запустили первую программу и разобрали в ней буквально каждый знак: `#include`, `main`, типы, фигурные скобки, `printf`, `\n`, `return`, точку с запятой. Это скелет любой программы на Си — дальше мы будем только наполнять его.

В следующей главе оживим программу: научимся хранить данные в переменных, спрашивать у пользователя числа, принимать решения и повторять действия в цикле. Это весь необходимый минимум, чтобы взяться за первые алгоритмы.

Итоговый файл этой главы — `code/01-hello.c`.

Глава 2. Минимум Си: переменные, ввод, ветвления, циклы, функции

Эта глава — фундамент. Мы возьмём наш «Привет, мир!» и будем наращивать его, пока не получим маленькую интерактивную программу. По дороге разберём всё, без чего не написать ни одного алгоритма: **переменные и типы**, **вывод чисел**, **ввод с клавиатуры**, **арифметику**, **ветвления** (`if`/`else`), **циклы** (`while`, `for`) и **свои функции**.

Глава длинная, но это единственная теоретическая глава такого объёма. Дальше будет в основном практика.

Шаг 1. Переменная — коробка для значения

Программе нужно где-то хранить данные: число, которое ввёл пользователь, промежуточный результат, счётчик. Для этого есть **переменные**.

Переменная — это **именованная коробочка в памяти**, в которой лежит значение. У коробочки есть имя (чтобы к ней обращаться) и тип (чтобы знать, что в ней лежит и сколько места занимает).

```c
#include <stdio.h>

int main(void)
{
    int age = 25;
    printf("Возраст: %d\n", age);
    return 0;
}
```

Разбор новых вещей:

- `int age = 25;` — мы **объявили** переменную. По частям:
  - `int` — тип: целое число (мы его уже видели у `main`).

  - `age` — имя переменной. Имена пишем латиницей, можно буквы, цифры (не первой) и _. Регистр важен: `age` и `Age` — разные.
  - `=` — оператор **присваивания**: «положи в коробку `age` значение справа». Это **не** «равно» из математики! Это команда «запиши». Запомни сразу: один `=` — присвоить, два `==` — сравнить (про сравнение чуть позже).
  - `25` — само значение.
  - `;` — конец инструкции.
- `printf("Возраст: %d\n", age);` — здесь появилось кое-что новое:
  - `%d` внутри строки — это **спецификатор формата**. Он означает «сюда подставь целое число в десятичной записи» (`d` — *decimal*). `printf` увидит `%d` и возьмёт следующий аргумент.
  - после строки, через запятую, мы передали `age`. Его значение и встанет на место `%d`.

Скомпилируй и запусти — увидишь `Возраст: 25`.

Основные типы

Пока хватит четырёх типов:

- `int` — целое число (например, −7, 0, 42). Обычно занимает 4 байта, вмещает примерно от −2 до +2 миллиардов.
- `double` — дробное число (3.14, −0.5). «Double» — двойная точность.
- `char` — один символ (`'A'`, `'?'`). Внутри это маленькое целое (код символа), занимает 1 байт.
- (`float` — тоже дробное, но менее точное; мы будем брать `double`.)

У каждого типа — свой спецификатор в `printf`:

```c
int    x = 42;     printf("%d\n", x);     // целое
double y = 3.14;   printf("%f\n", y);     // дробное
char   c = 'A';    printf("%c\n", c);     // символ
```

- `%d` — целое.
- `%f` — дробное (по умолчанию 6 знаков после запятой). Можно задать точность: `%.2f` напечатает два знака — `3.14`.
- `%c` — один символ.
- Заметь: `//` — это **комментарий**. Всё от `//` до конца строки компилятор игнорирует, это пометка для человека. Есть и многострочный вид: `/* ... */`.

Шаг 2. Спрашиваем у пользователя — `scanf`

Программа интереснее, когда реагирует на ввод. Считать число с клавиатуры умеет функция `scanf` (*scan formatted* — чтение с форматированием), тоже из `stdio.h`.

```c
#include <stdio.h>

int main(void)
{
    int age;
    printf("Сколько тебе лет? ");
    scanf("%d", &age);
    printf("Через год будет %d\n", age + 1);
    return 0;
}
```

Что нового:

- `int age;` — объявили переменную **без** начального значения. Пока в ней «мусор» — случайное содержимое памяти. Мы сейчас его заполним вводом.
- `scanf("%d", &age);` — читаем целое число и кладём в `age`:
  - `"%d"` — формат: «ожидаю целое число».
  - `&age` — а вот это важно. Знак `&` означает «**адрес** переменной `age`», то есть «где в памяти лежит коробка `age`». `scanf` нужно не значение, а место, **куда** записать прочитанное. Поэтому передаём адрес.
  - **Самая частая ошибка новичка** — забыть `&` и написать `scanf("%d", age)`. Компилятор предупредит, а программа, скорее всего, упадёт. Правило простое: в `scanf` перед обычной переменной почти всегда стоит `&`.
- `age + 1` — арифметика прямо в аргументе `printf`. Сначала посчитается `age + 1`, потом результат подставится в `%d`.

Запуск:

```
Сколько тебе лет? 25
Через год будет 26
```

Арифметика

Операторы как в математике:

- `+` сложение, `-` вычитание, `*` умножение, `/` деление, `%` остаток от деления.

Две ловушки, о которые спотыкаются все:

Целочисленное деление. Если оба числа целые, `/` делит нацело, дробную часть отбрасывает:

```
printf("%d\n", 7 / 2);   // напечатает 3, а не 3.5
printf("%d\n", 7 % 2);   // напечатает 1 — остаток
```

Чтобы получить дробный результат, хотя бы одно число должно быть дробным: `7.0 / 2` даст `3.5`.

Остаток `%` очень полезен. `n % 2` равно 0, если `n` чётное, и 1, если нечётное. Это пригодится постоянно.

Шаг 3. Принимаем решения — `if` и `else`

Программа должна уметь выбирать путь: «если число чётное — сделай так, иначе — эдак». Для этого есть `if` (англ. «если»).

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи число: ");
    scanf("%d", &n);

    if (n % 2 == 0) {
        printf("%d — чётное\n", n);
    } else {
        printf("%d — нечётное\n", n);
    }
    return 0;
}
```

Разбор:

- `if (условие) { ... }` — если условие истинно, выполняется блок в фигурных скобках. Условие пишут в круглых скобках.
- `n % 2 == 0` — условие. Здесь сразу два важных оператора:
  - `n % 2` — остаток от деления на 2.
  - `==` — оператор **сравнения** «равно». Возвращает «истину», если левая и правая части равны. Ещё раз: `==` сравнивает, `=` присваивает. Путаница между ними — классический баг. Если случайно написать `if (n = 0)`, компилятор с `-Wall` предупредит.

- `else { ... }` — «иначе». Выполняется, если условие `if` оказалось ложным. `else` не обязателен.

Операторы сравнения и логики

Сравнения дают «истину» или «ложь»:

- == равно, `!=` не равно;
- < меньше, > больше, <= меньше или равно, >= больше или равно.

В Си нет отдельного типа «логическое» в классическом смысле: **ложь — это 0, истина — любое ненулевое число** (обычно 1). Это пригодится.

Условия можно соединять:

- `&&` — «И» (оба условия истинны): `if (age >= 18 && age < 65)`.
- `||` — «ИЛИ» (хотя бы одно истинно): `if (c == 'y' || c == 'Y')`.
- `!` — «НЕ» (переворачивает): `if (!found)`.

Если веток несколько, их сцепляют через `else if`:

```
if (n > 0) {
    printf("положительное\n");
} else if (n < 0) {
    printf("отрицательное\n");
} else {
    printf("ноль\n");
}
```

Шаг 4. Повторяем — цикл `while`

Часто нужно повторить действие много раз. Самый понятный цикл — `while` (англ. «пока»): «пока условие истинно — повторяй тело».

Напечатаем числа от 1 до 5:

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    i = i + 1;
}
printf("\n");
```

Разбор:

- `int i = 1;` — заводим **счётчик** и ставим в 1.
- `while (i <= 5)` — пока `i` не больше 5, выполняй тело.
- внутри печатаем `i` и пробел.
- `i = i + 1;` — увеличиваем счётчик на 1. **Без этой строки условие никогда не станет ложным, и программа зациклится навсегда** — это называется бесконечный цикл. Если такое случилось, останови программу клавишами Ctrl+C.

Вывод: `1 2 3 4 5`.

Сокращения, которые ты будешь видеть всюду

`i = i + 1` пишут короче: `i++` (увеличить на 1) или `i += 1`. Аналогично `i--` (уменьшить), `i += 3` (прибавить 3), `i *= 2` (умножить на 2). Это просто более краткая запись того же самого.

Шаг 5. Цикл `for` — когда знаем число повторений

Когда у цикла есть счётчик с понятным началом, концом и шагом, удобнее `for`. Тот же вывод 1..5:

```c
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
printf("\n");
```

В скобках `for` — три части через `;`:

1. `int i = 1` — **инициализация**: выполняется один раз в начале.
2. `i <= 5` — **условие**: проверяется перед каждым повтором; пока истинно — цикл идёт.
3. `i++` — **шаг**: выполняется в конце каждого повтора.

Это ровно то же, что `while` выше, только всё про счётчик собрано в одном месте. `for` — самый частый цикл в книге.

Шаг 6. Свои функции

`printf` и `scanf` — чужие функции. Но мы можем писать **свои** — чтобы не повторять код и дать кускам логики понятные имена.

Напишем функцию, которая возводит число в квадрат:

```c
#include <stdio.h>
```

```c
int square(int x)
{
    return x * x;
}

int main(void)
{
    int n = 6;
    printf("%d в квадрате = %d\n", n, square(n));
    return 0;
}
```

Разбор `int square(int x)`:

- `int` (первый) — тип того, что функция **вернёт**. Здесь целое.
- `square` — имя функции, придумали сами.
- `(int x)` — **параметр**: функция принимает одно целое число и внутри называет его `x`. Параметры — это «вход» функции.
- `return x * x;` — посчитать `x * x` и **вернуть** наружу.

В `main` мы пишем `square(n)` — это **вызов**: передаём `n`, получаем обратно результат, который тут же подставляется в `printf`.

Важный момент: функцию нужно объявить **до** того места, где её вызывают. Поэтому `square` стоит выше `main`. (Есть способ объявить функцию заранее, а тело написать ниже, — про это в главах с алгоритмами.)

Функция, которая ничего не возвращает, имеет тип `void`:

```c
void hello(void)
{
    printf("Привет!\n");
}
```

`void` как тип результата = «ничего не возвращаю». Внутри такой функции `return;` можно писать без значения (или не писать вовсе).

Складываем всё вместе

Маленькая программа, использующая всё из этой главы: спрашивает число и печатает его таблицу умножения, отмечая чётные строки.

```c
#include <stdio.h>

int is_even(int x)
{
    return x % 2 == 0;
}

int main(void)
{
    int n;
    printf("Таблица умножения для какого числа? ");
    scanf("%d", &n);

    for (int i = 1; i <= 10; i++) {
        int result = n * i;
        if (is_even(result)) {
            printf("%2d x %2d = %3d  (чётное)\n", n, i, result);
        } else {
            printf("%2d x %2d = %3d\n", n, i, result);
        }
    }
    return 0;
}
```

Новое здесь только оформление: `%2d` и `%3d` — это `%d` с указанием **минимальной ширины** поля (2 и 3 знака). Числа выровняются в столбик, таблица станет аккуратной.

Функция `is_even` возвращает результат сравнения `x % 2 == 0` — помнишь, истина это 1, ложь это 0? Значит, `is_even` возвращает 1 для чётных и 0 для нечётных, и `if (is_even(result))` срабатывает именно на чётных.

Запуск:

```
Таблица умножения для какого числа? 4
 4 x  1 =   4  (чётное)
 4 x  2 =   8  (чётное)
 4 x  3 =  12  (чётное)
 4 x  4 =  16  (чётное)
 4 x  5 =  20  (чётное)
 4 x  6 =  24  (чётное)
```

```
 4 x  7 =  28  (чётное)
 4 x  8 =  32  (чётное)
 4 x  9 =  36  (чётное)
 4 x 10 =  40  (чётное)
```

Для четвёрки все строки вышли чётными — что логично: чётное число, умноженное на что угодно, остаётся чётным. Введи 3, и пометки начнут чередоваться:

```
Таблица умножения для какого числа? 3
 3 x  1 =   3
 3 x  2 =   6  (чётное)
 3 x  3 =   9
 3 x  4 =  12  (чётное)
 3 x  5 =  15
 3 x  6 =  18  (чётное)
 3 x  7 =  21
 3 x  8 =  24  (чётное)
 3 x  9 =  27
 3 x 10 =  30  (чётное)
```

Сохрани эту программу как `code/02-multiplication.c`.

Частые ошибки

`=` вместо `==` в условии. `if (n = 5)` всегда «истина» и испортит `n`. Нужно `if (n == 5)`. Компилятор с `-Wall` предупреждает.

Забытый `&` в `scanf`. `scanf("%d", n)` — почти всегда падение. Правильно `scanf("%d", &n)`.

Неверный спецификатор. Печатать `double` через `%d` или `int` через `%f` — мусор на экране. Тип и спецификатор должны совпадать.

Бесконечный цикл. Забыл менять счётчик в `while` — программа висит. Ctrl+C, и добавь `i++`.

Целочисленное деление врасплох. `double avg = sum / count;` где `sum` и `count` целые — деление произойдёт нацело **до** записи в `double`. Приведи к дробному: `(double)sum / count`.

Упражнения

1. Программа спрашивает два числа и печатает их сумму, разность, произведение и частное (частное — дробное, через `%.2f`).

2. Программа спрашивает число и говорит, положительное оно, отрицательное или ноль (через `if`/`else if`/`else`).
3. С помощью `for` напечатай все чётные числа от 2 до 20.
4. Напиши функцию `int max2(int a, int b)`, возвращающую большее из двух чисел, и проверь её в `main`.
5. Цикл `while`: спрашивай числа, пока пользователь не введёт 0, и в конце напечатай их сумму.

Что мы получили

Это был самый плотный по теории кусок книги. Теперь у тебя есть всё для алгоритмов: переменные и типы, ввод-вывод чисел, арифметика (и ловушки деления/остатка), ветвления `if`/`else`, логические операторы, циклы `while` и `for`, свои функции. Если что-то осталось туманным — не страшно: дальше мы будем применять это десятки раз, и оно уляжется на практике.

В следующей главе разберём три вещи, без которых не обойтись ни в сортировках, ни в играх: **массивы** (много значений под одним именем), **строки** (текст как набор символов) и **указатели** (адреса в памяти). После неё мы наконец возьмёмся за настоящие алгоритмы.

Глава 3. Массивы, строки, указатели и память

Один `int` хранит одно число. Но алгоритмы работают с **наборами**: отсортировать список оценок, найти слово в тексте, разложить колоду карт. Для этого нужны три кита, на которых стоит весь Си: **массивы** (много значений подряд), **строки** (текст) и **указатели** (адреса в памяти). Разберём их по очереди, как всегда — строя и запуская.

Часть 1. Массив — много коробок в ряд

Массив — это несколько переменных одного типа, лежащих в памяти **подряд**, под общим именем. К каждой обращаемся по **номеру** — *индексу*.

```c
#include <stdio.h>

int main(void)
{
    int marks[5] = {4, 5, 3, 5, 2};
    printf("Первая оценка: %d\n", marks[0]);
    printf("Третья оценка: %d\n", marks[2]);
    return 0;
}
```

Разбор:

- `int  marks[5]` — объявляем массив из **5** целых чисел. Число в квадратных скобках — **размер**, сколько элементов. Он фиксированный: массив на 5 не станет на 6.
- `= {4, 5, 3, 5, 2}` — сразу заполняем пятью значениями в фигурных скобках.
- `marks[0]` — обращение к элементу по индексу. **Внимание: нумерация с нуля!** Первый элемент — `marks[0]`, второй — `marks[1]`, …, пятый и последний — `marks[4]`. Это сбивает с толку поначалу, но так во всех серьёзных языках.

Скомпилируй и запусти:

```
Первая оценка: 4
Третья оценка: 3
```

Вот она, нумерация с нуля, во всей красе: «третья оценка» — это `marks[2]`, и печатается `3`, третье число из списка. Если бы мы написали `marks[3]`, получили бы `5` — четвёртый элемент. Привыкай сразу: индекс — это не «который по счёту», а «на сколько шагов отступить от начала».

Перебор массива циклом

Массив и цикл `for` созданы друг для друга:

```c
int marks[5] = {4, 5, 3, 5, 2};
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum = sum + marks[i];
}
printf("Сумма: %d\n", sum);
printf("Среднее: %.2f\n", (double)sum / 5);
```

- `i` пробегает от 0 до 4 (`i < 5`, не <=!). Если написать `i <= 5`, на последнем шаге обратимся к `marks[5]` — а его **нет**, индексы только 0..4. Это **выход за границы массива** — одна из самых опасных ошибок в Си. Язык **не проверяет** границы: он молча прочитает чужой кусок памяти или испортит данные. Следи за условием сам.
- `(double)sum / 5` — мы привели `sum` к дробному типу перед делением, иначе среднее посчиталось бы нацело. Запись `(тип)значение` называется **приведением типа** (cast).

Запуск:

```
Сумма: 19
Среднее: 3.80
```

Проверь руками: 4 + 5 + 3 + 5 + 2 = 19, делим на 5 — получается 3.8.

А теперь посмотри, что бывает без приведения. Напишем «естественно»:

```c
double avg = sum / 5;                /* тип переменной дробный — и что? */
printf("Среднее: %.2f\n", avg);
```

```
Среднее: 3.00
```

Три ровно. Обе части `sum / 5` целые, значит деление идёт нацело: 19 / 5 = 3, дробная часть выброшена **до** того, как результат попал в `double`. Тип переменной слева ни на что не влияет

— вычисляется сначала правая часть, по своим правилам. Ошибка тихая и оттого противная: `3.00` выглядит совершенно правдоподобно.

Хуже другой вариант — убрать `(double)` прямо в `printf`:

```
printf("Среднее: %.2f\n", sum / 5);   /* так делать нельзя */
```

Тут `%.2f` ждёт дробное число, а получает целое. Компилятор предупредит («format specifies type ‘double’ but the argument has type ‘int’»), а программа напечатает `0.00` — чистый мусор. Это к вопросу о том, зачем мы всегда собираем с `-Wall`.

Заполнение массива вводом

```
int n;
printf("Сколько чисел? ");
scanf("%d", &n);

int a[100];                    // место под 100 чисел
for (int i = 0; i < n; i++) {
    printf("a[%d] = ", i);
    scanf("%d", &a[i]);        // &a[i] — адрес i-го элемента
}
```

Здесь `&a[i]` — адрес конкретного элемента, куда `scanf` запишет число. Мы объявили массив с запасом (100), а используем только первые `n`. Так часто делают, пока не познакомимся с динамической памятью.

Часть 2. Строки — это массивы символов

В Си **нет** отдельного типа «строка». Строка — это **массив `char`**, в конце которого стоит специальный символ-метка `'\0'` (нулевой символ, код 0). Он говорит: «здесь текст закончился».

```
char name[] = "Анна";
```

Что произошло:

- `char name[]` — массив символов. Размер не указали — компилятор посчитает сам.
- `"Анна"` — компилятор разложит строку по буквам и **сам добавит** `'\0'` в конце. То есть в памяти лежит: `А`, `н`, `н`, `а`, `\0`. Хотя букв 4, размер массива — минимум 5 (плюс `\0`). (С русскими буквами в UTF-8 байт даже больше — об этом ниже.)

Печатают строку через `%s` (*string*):

```
printf("Привет, %s!\n", name);
```

`printf` берёт символы от начала и печатает, **пока не встретит** `'\0'`. Вот зачем нужен нулевой символ: без него `printf` не знал бы, где остановиться, и печатал бы мусор дальше по памяти.

Проверим это на живой программе — заодно посмотрим, сколько места строка занимает на самом деле:

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char name[] = "Анна";
    printf("Привет, %s!\n", name);
    printf("sizeof(name) = %zu\n", sizeof name);
    printf("strlen(name) = %zu\n", strlen(name));

    char latin[] = "Anna";
    printf("sizeof(latin) = %zu, strlen(latin) = %zu\n",
           sizeof latin, strlen(latin));
    return 0;
}
```

```
Привет, Анна!
sizeof(name) = 9
strlen(name) = 8
sizeof(latin) = 5, strlen(latin) = 4
```

Разбираем числа — они говорят о Си больше, чем абзац теории.

- `latin` — четыре латинские буквы. `strlen` даёт 4 (столько байт до `'\0'`), `sizeof` даёт 5 — на один больше, потому что сам `'\0'` тоже лежит в массиве и место занимает. Это правило работает всегда: `sizeof` массива-строки на единицу больше `strlen`.
- `name` — те же четыре буквы, но русские, и числа вдвое больше: 8 и
  9. Каждая кириллическая буква в UTF-8 занимает **два байта**. `strlen` считает байты, а не буквы, и про это ещё будет отдельный разговор в конце главы.
- `%zu` — спецификатор для типа `size_t`, который возвращают `sizeof` и `strlen`. Это беззнаковый целый тип «размер чего-то в байтах»; печатать его через `%d` формально неверно, компилятор поворчит.

- `sizeof name` пишется без скобок — это оператор, а не функция. Со скобками тоже можно (`sizeof(name)`), и для типов скобки обязательны: `sizeof(int)`.

Чтение строки от пользователя

Для слова без пробелов годится `%s` в `scanf`:

```
char word[64];
printf("Введи слово: ");
scanf("%63s", word);
printf("Ты ввёл: %s\n", word);
```

Тонкости:

- В `scanf("%63s", word)` перед `word` **нет** `&`! Имя массива само по себе уже означает адрес его начала (об этом — в части про указатели). Это исключение из правила «всегда `&`».
- `%63s` с числом — мы ограничили ввод 63 символами, чтобы не переполнить массив из 64 (последний байт — под `'\0'`). Без ограничения слишком длинный ввод **затрёт соседнюю память** — классическая уязвимость.

Чтобы прочитать **целую строку с пробелами**, `scanf("%s")` не годится (он читает до первого пробела). Берём `fgets`:

```
char line[100];
printf("Введи фразу: ");
fgets(line, sizeof line, stdin);
```

- `fgets(куда, сколько_максимум, откуда)` — читает строку.
- `sizeof line` — размер массива `line` в байтах (100). Оператор `sizeof` возвращает размер; так `fgets` не вылезет за границу.
- `stdin` — *standard input*, стандартный ввод (клавиатура).
- Особенность: `fgets` оставляет в строке символ перевода строки `'\n'` (от нажатия Enter). Часто его убирают. Как — увидим в главе про строковые алгоритмы.

Никогда не используй `gets`. Это старая функция чтения строки без ограничения длины; она настолько небезопасна, что её убрали из языка. Только `fgets`.

Полезные функции для строк (`<string.h>`)

Подключив `#include <string.h>`, получаем готовые инструменты:

- `strlen(s)` — длина строки (число символов до `'\0'`, сам `'\0'` не считается).
- `strcmp(a, b)` — сравнить две строки. Возвращает 0, если они **равны**. (Да, именно 0 при равенстве — это сбивает с толку.)

- `strcpy(dst, src)` — скопировать строку `src` в `dst`.

Сравнивать строки через `==` **нельзя** — это сравнит адреса, а не содержимое. Только `strcmp`. Запомни сразу, эта ошибка очень частая:

```
if (strcmp(word, "выход") == 0) {   // правильно
    printf("Пока!\n");
}
```

Часть 3. Указатели — переменные, хранящие адрес

Мы уже встречали `&` в `scanf` — «адрес переменной». Теперь разберём это до конца, потому что без указателей не написать функцию, меняющую свои аргументы, и не понять динамическую память.

У каждой переменной есть **адрес** — номер ячейки в памяти, где она лежит. **Указатель** — это переменная, которая хранит такой адрес.

```
#include <stdio.h>

int main(void)
{
    int x = 42;
    int *p = &x;     // p хранит адрес x

    printf("значение x:        %d\n", x);
    printf("адрес x:           %p\n", (void *)&x);
    printf("p хранит адрес:    %p\n", (void *)p);
    printf("значение по p:     %d\n", *p);

    *p = 100;        // меняем x через указатель
    printf("теперь x = %d\n", x);
    return 0;
}
```

Разбор — это ключевой момент всей главы:

- `int *p` — объявляем **указатель на int**. Звёздочка `*` в объявлении говорит: «`p` — не число, а адрес, по которому лежит число». Тип — «указатель на `int`».
- `= &x` — кладём в `p` **адрес** переменной `x`. Оператор `&` — «взять адрес».

- `*p` (в коде, не в объявлении) — оператор **разыменования**: «значение, лежащее **по** адресу `p`». Это как «открыть коробку, на которую указывает стрелка». `*p` — это и есть `x`, просто доступ к нему через адрес.
- `*p = 100;` — записать 100 туда, куда указывает `p`. Поскольку `p` указывает на `x`, изменится именно `x`. После этой строки `x` равен 100.
- `%p` — спецификатор для печати адреса; `(void *)` — приведение, которого требует `printf` для адресов. Конкретные числа адресов у тебя будут свои.

Запускаем (файл `code/03-pointers.c`, в нём же лежит `swap` из следующего раздела):

```
значение x:        42
адрес x:           0x7ff7bba0d9d4
p хранит адрес:    0x7ff7bba0d9d4
значение по p:     42
теперь x = 100
после swap: a=2 b=1
```

Смотри на две средние строки: адрес `x` и содержимое `p` — **одно и то же число**. Это и значит «указатель хранит адрес». А `*p` в четвёртой строке вернул 42 — то, что по этому адресу лежит.

Само число адреса (`0x7ff7bba0d9d4`) у тебя будет другим, и даже при повторных запусках оно меняется: система каждый раз кладёт программу в разное место памяти. Запоминать его бессмысленно — важно только то, что оба раза оно совпало.

Последняя строка — уже про `swap` из следующего раздела: `a` и `b` поменялись местами по-настоящему, потому что функция получила адреса.

Запутаться легко, поэтому короткая шпаргалка:

- `&x` — «**адрес** переменной x» (был у scanf).
- `int *p` — «p это **указатель**».
- `*p` — «**значение по** указателю p».

Зачем это нужно: функция, меняющая аргумент

В Си аргументы передаются **по значению** — функция получает копию. Поэтому такая функция не сработает:

```c
void bad_swap(int a, int b) {   // меняет местами КОПИИ — бесполезно
    int t = a; a = b; b = t;
}
```

Снаружи ничего не изменится. Чтобы функция поменяла **сами** переменные, передаём их **адреса**:

```
void swap(int *a, int *b)   // принимаем адреса
{
    int t = *a;             // запомнили значение по a
    *a = *b;                // в a положили значение по b
    *b = t;                 // в b положили запомненное
}

int main(void)
{
    int x = 1, y = 2;
    swap(&x, &y);           // передаём адреса x и y
    printf("x=%d y=%d\n", x, y);   // x=2 y=1
    return 0;
}
```

Разницу лучше один раз увидеть. Позовём обе функции подряд:

```
int x = 1, y = 2;
bad_swap(x, y);
printf("после bad_swap: x=%d y=%d\n", x, y);
swap(&x, &y);
printf("после swap:     x=%d y=%d\n", x, y);
```

```
после bad_swap: x=1 y=2
после swap:     x=2 y=1
```

`bad_swap` честно поменяла местами свои `a` и `b` — но это были копии, которые умерли на выходе из функции. Снаружи не изменилось ничего. `swap` получила адреса и полезла в оригиналы — и вот они поменялись.

Это и есть причина, по которой `scanf` требует `&`: ему нужен адрес, чтобы записать прочитанное в **твою** переменную, а не в копию.

Массив и указатель — близкие родственники

Имя массива в большинстве выражений превращается в **адрес его первого элемента**. Поэтому `scanf("%s", word)` работает без `&`: `word` — это уже адрес. И поэтому, когда передаёшь массив в функцию, функция получает адрес начала и может менять его содержимое. Этим мы будем пользоваться постоянно в сортировках.

Часть 4. Динамическая память — когда размер заранее неизвестен

`int a[100]` требует знать размер при написании кода. А если число элементов выяснится только во время работы? Тогда память просят у системы **во время выполнения** — это **динамическая память**.

Нужен `#include <stdlib.h>` и две функции: `malloc` (попросить память) и `free` (вернуть).

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    scanf("%d", &n);

    int *a = malloc(n * sizeof(int));   // память под n чисел
    if (a == NULL) {                    // вдруг память не дали
        printf("Не хватило памяти\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        a[i] = i * i;                   // к malloc-памяти — как к массиву
    }
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    free(a);                            // вернули память системе
    return 0;
}
```

Разбор:

- `malloc(n * sizeof(int))` — просим столько байт, сколько занимают `n` целых. `sizeof(int)` — размер одного `int` в байтах (обычно 4). `malloc` (*memory allocate*) возвращает **адрес** выделенного куска, поэтому результат кладём в указатель `int *a`.

- `if (a == NULL)` — `malloc` может вернуть `NULL` («нулевой адрес, никуда не указывает»), если памяти не хватило. Проверять это — хорошая привычка. `NULL` — специальное значение «указатель в никуда».
- `a[i]` — к динамической памяти обращаемся **точно как к массиву**. Указатель и массив здесь неотличимы в использовании.
- `free(a);` — **обязательно** возвращаем память, когда она больше не нужна. Если этого не делать, программа будет занимать всё больше памяти — это **утечка памяти** (memory leak). Правило: на каждый `malloc` однажды должен прийтись `free`.

Запуск (файл `code/03-dynamic.c`):

```
Сколько чисел? 4
0 1 4 9
```

Ввели 4 — программа попросила у системы место ровно под четыре числа и заполнила их квадратами индексов: 0^2, 1^2, 2^2, 3^2. Введи 10 — получишь десять чисел, и никакого `int a[100]` с запасом «на всякий случай» для этого не понадобилось. В этом и смысл динамической памяти: размер решается во время работы, а не при написании кода.

Кстати, о проверках. Ошибки с памятью коварны тем, что программа может выглядеть работающей и при этом быть сломанной. Ловить их помогает **санитайзер** — компилятор умеет встроить в программу проверки на каждое обращение к памяти:

```
cc -std=c11 -Wall -Wextra -g -fsanitize=address,undefined \
   code/03-dynamic.c -o dynamic
./dynamic
```

Попробуй специально ошибиться — написать `a[n] = 0` (на один элемент за границей выделенного). Обычная сборка проглотит это молча, а тут:

```
==96656==ERROR: AddressSanitizer: heap-buffer-overflow on address ...
WRITE of size 4 at 0x602000000120 thread T0
    #0 0x000102fccd74 in main dyn_oob.c:27
0x602000000120 is located 0 bytes after 16-byte region ...
```

«Запись за границей кучи, 0 байт после выделенного участка, файл `dyn_oob.c`, строка 27» — точный адрес ошибки вместо загадочного падения через полчаса работы. Держи этот флаг под рукой.

Оговорка про утечки: искать забытые `free` умеет отдельный кусок санитайзера, LeakSanitizer, и на macOS он **не работает** — попытка включить его через `ASAN_OPTIONS=detect_leaks=1` отвечает `detect_leaks is not supported on this platform`. На Linux та же команда покажет утечку со стеком вызовов. На маке для этого берут инструмент `leaks` из системы или Instruments.

Замечание про русские буквы

Современные системы хранят текст в кодировке **UTF-8**, где русская буква занимает **2 байта**, а не 1. Поэтому `strlen("Привет")` вернёт 12, а не 6, и посимвольная обработка кириллицы — отдельная морока. В этой книге, где важно считать **символы**, мы будем работать с латиницей и цифрами (по 1 байту). Для вывода сообщений русский текст полностью годится — `printf` печатает байты как есть, и всё выглядит правильно.

Частые ошибки

Выход за границы массива. `for (i = 0; i <= n; ...)` вместо `i < n`. Си не проверяет границы — получишь порчу памяти или падение.

Сравнение строк через `==`. Сравнивает адреса, не текст. Нужен `strcmp`.

Забыл `'\0'`. Если собираешь строку вручную по символам, не забудь поставить `'\0'` в конце, иначе `printf("%s")` улетит в мусор.

`malloc` без проверки на `NULL` и **без `free`.** Первое — возможное падение, второе — утечка.

`*` путается в объявлении и в использовании. В `int *p` звёздочка объявляет указатель; в `*p = 5` — разыменовывает. Это разные роли одного знака.

Упражнения

1. Заполни массив из 10 чисел вводом и напечатай его в обратном порядке.
2. Найди в массиве максимальный элемент (заведи переменную `max`, пройди циклом, обновляй её).
3. Напиши функцию `void fill(int *a, int n, int value)`, которая заполняет первые `n` элементов значением `value`.
4. Прочитай слово через `scanf("%63s", ...)` и напечатай его длину через `strlen`.
5. Через `malloc` выдели массив на введённое число элементов, заполни их кубами индексов (`i*i*i`), напечатай и не забудь `free`.

Что мы получили

Теперь у нас есть всё необходимое: массивы для наборов данных, строки как массивы символов с `'\0'`, указатели как адреса (и понимание, зачем `scanf` хочет `&`), динамическая память с парой `malloc`/`free`. Это последняя «теоретическая» глава.

Дальше — чистая практика. Следующая глава открывает часть про алгоритмы: начнём с простых, но показательных задач над числами — чётность, сумма цифр, переворот числа, факториал и знаменитый FizzBuzz.

Итоговые программы главы — `code/03-pointers.c` и `code/03-dynamic.c`.

Глава 4. Числа: чётность, сумма цифр, переворот числа, факториал, FizzBuzz

Хватит теории — займёмся первыми настоящими алгоритмами. Все они про числа, и все опираются на один скромный, но мощный инструмент: **остаток от деления** `%` и **целочисленное деление** `/`. С их помощью мы научимся разбирать число на цифры, считать их сумму, переворачивать число задом наперёд, вычислять факториал (и заодно наткнёмся на переполнение `int` — важнейшую ловушку Си) и напишем знаменитую задачку с собеседований — FizzBuzz.

Всё, что нужно из прошлых глав, у тебя уже есть: переменные и типы, `printf`/`scanf` с `&`, арифметика и `%`, `if`/`else`, циклы `while` и `for`, функции (см. главы 2 и 3). Новое тут — только пара приёмов и тип `long long`. Их разберём подробно.

Что мы сделаем

К концу главы у тебя будут три рабочие программы:

- `04-digits.c` — вводит число, печатает сумму его цифр и само число задом наперёд;
- `04-factorial.c` — считает факториал, корректно справляясь с переполнением и отрицательным вводом;
- `04-fizzbuzz.c` — печатает числа от 1 до 100, заменяя кратные 3 на `Fizz`, кратные 5 — на `Buzz`, кратные 15 — на `FizzBuzz`.

А главное — ты прочувствуешь два оператора, `%` и `/`, на которых держится половина числовых алгоритмов.

Мостик: чётность через остаток

Вспомним из главы 2: `%` — это **остаток от деления**. `7 % 2` равно 1, потому что $7 = 2 \cdot 3 + \mathbf{1}$. А `8 % 2` равно 0, потому что 8 делится на 2 без остатка.

Отсюда главный трюк:

`n % 2 == 0` — число чётное; `n % 2 != 0` — нечётное.

И шире: `n % k == 0` означает «`n` делится на `k` нацело». Это правило мы будем гонять всю главу. Кратно 3? `n % 3 == 0`. Кратно 10? `n % 10 == 0`. Запомни эту формулу — она ключ ко всему дальнейшему.

Часть 1. Сумма цифр и переворот числа

Разбираем задачу руками

Возьмём число 12345. Как достать из него отдельные цифры, не зная заранее, сколько их? Тут и пригодятся `%` и `/`.

Посмотри, что делает остаток от деления на 10:

```
12345 % 10 = 5   ← последняя цифра!
```

Деление на 10 нацело отбрасывает эту последнюю цифру:

```
12345 / 10 = 1234   ← число без последней цифры
```

Дальше повторяем то же самое с 1234, потом с 123, и так пока число не станет нулём. Распишем по шагам:

```
n = 12345   n % 10 = 5   n / 10 = 1234
n = 1234    n % 10 = 4   n / 10 = 123
n = 123     n % 10 = 3   n / 10 = 12
n = 12      n % 10 = 2   n / 10 = 1
n = 1       n % 10 = 1   n / 10 = 0   ← стоп, n стал 0
```

Мы «откусили» цифры по одной с конца: 5, 4, 3, 2, 1. Сложи их — вот и сумма цифр (15). А если приклеивать их в новое число в том порядке, в каком достаём, — получится перевёрнутое число (54321). Осталось оформить это кодом.

Шаг 1. Наивная версия — считаем сумму

Начнём с малого: только сумма цифр.

```c
#include <stdio.h>

int main(void)
{
    int n;
```

```
    printf("Введи целое число: ");
    scanf("%d", &n);

    int sum = 0;
    while (n > 0) {
        sum = sum + n % 10;   /* добавили последнюю цифру */
        n = n / 10;           /* отбросили её */
    }

    printf("Сумма цифр: %d\n", sum);
    return 0;
}
```

Разбор новых для главы мест (всё остальное — из глав 2–3):

- `int sum = 0;` — заводим «копилку» и обнуляем её. Очень важно стартовать с нуля: если оставить `sum` без значения, в ней будет мусор и сумма получится бессмысленной.
- `while (n > 0)` — крутим цикл, пока в числе ещё остались цифры. Как только `n` дошёл до 0 (все цифры откусили), условие станет ложным и цикл закончится.
- `sum = sum + n % 10;` — здесь важен **порядок операций**. `%` и `+`: сначала считается `n % 10` (последняя цифра), потом она прибавляется к `sum`. Скобки не нужны — `%` «сильнее» `+`, как умножение сильнее сложения в обычной математике.
- `n = n / 10;` — целочисленное деление (см. главу 2): дробная часть отбрасывается, число «укорачивается» на одну цифру справа.

Запустим с числом 12345:

```
Введи целое число: 12345
Сумма цифр: 15
```

Работает. 1+2+3+4+5 = 15 — верно.

Шаг 2. Проблема — где взять перевёрнутое число?

Нам нужна ещё и вторая величина — число задом наперёд. Заведём переменную `reversed` и будем «пристраивать» к ней цифры по мере того, как достаём.

Главный приём — вот эта строка:

```
reversed = reversed * 10 + digit;
```

Как она собирает число? Каждый шаг мы сдвигаем то, что уже накопили, на разряд влево (умножение на 10 дописывает справа ноль), а на освободившееся место ставим новую цифру. Распишем для 12345:

```
старт:           reversed = 0
цифра 5:  0*10 + 5  = 5
цифра 4:  5*10 + 4  = 54
цифра 3: 54*10 + 3  = 543
цифра 2: 543*10 + 2 = 5432
цифра 1: 5432*10 + 1 = 54321
```

Так из 0 постепенно вырастает 54321. Тот же приём (накопить число, дописывая цифры справа) встретится тебе ещё много раз.

Шаг 3. Проблема — последняя цифра тратится дважды

В наивной версии я писал `n % 10` дважды: один раз для суммы, другой раз понадобится для переворота. Но между ними нельзя менять `n`! Аккуратнее — посчитать `n % 10` **один раз**, положить в отдельную переменную `digit`, и пользоваться ею:

```c
int digit = n % 10;          /* последняя цифра — один раз */
sum = sum + digit;
reversed = reversed * 10 + digit;
n = n / 10;                  /* и только теперь укорачиваем n */
```

Это и понятнее, и надёжнее: видно, что обе операции работают с одной и той же цифрой, а укорачивание `n` происходит **после** них.

Шаг 4. Краевые случаи — ноль и минус

Наивный цикл `while (n > 0)` спотыкается на двух входах.

Ноль. Если ввести 0, условие `n > 0` ложно сразу, цикл не сделает ни шагу, и `sum`/`reversed` останутся нулями. Для нуля это как раз правильный ответ (сумма цифр нуля — 0, переворот — 0), так что нам повезло. Но держать этот случай в голове полезно.

Отрицательное число. Введи −90 — и `n > 0` тоже ложно сразу, программа выдаст нули. Неверно: у −90 сумма цифр 9. Чинится просто — в начале берём модуль (отбрасываем знак):

```c
if (n < 0) {
    n = -n;   /* унарный минус: меняет знак на противоположный */
}
```

`-n` — это **унарный минус**: один операнд, меняет знак. Для −90 он даст 90, и дальше всё работает как обычно. Знак мы при этом теряем — но для суммы цифр и переворота он и не нужен.

И ещё мелочь: после такой замены `n` испортится (мы же его делим в цикле), а нам хочется напечатать исходное число. Поэтому до цикла сохраним копию в `original`.

Полная программа `04-digits.c`

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        n = -n;   /* работаем с модулем, знак отбросим */
    }

    int sum = 0;        /* сюда копим сумму цифр */
    int reversed = 0;   /* сюда собираем перевёрнутое число */
    int original = n;   /* запомним исходное для печати */

    if (n == 0) {
        sum = 0;
        reversed = 0;
    }

    while (n > 0) {
        int digit = n % 10;          /* последняя цифра */
        sum = sum + digit;           /* добавили к сумме */
        reversed = reversed * 10 + digit;  /* приклеили к перевёрнутому */
        n = n / 10;                  /* отбросили последнюю цифру */
    }
```

```
    printf("Число:           %d\n", original);
    printf("Сумма цифр:      %d\n", sum);
    printf("Задом наперёд:   %d\n", reversed);
    return 0;
}
```

Новое здесь — проверка ввода:

- `if (scanf("%d", &n) != 1)` — `scanf` возвращает **число успешно прочитанных значений**. Мы просили одно (`%d`), значит при успехе он вернёт 1. Если пользователь ввёл буквы вместо цифр, вернётся не 1 — и мы вежливо выходим с `return 1`. По соглашению `return 0` из `main` значит «всё хорошо», а ненулевое — «была ошибка».

(Блок `if (n == 0)` тут для наглядности — он показывает, что для нуля ответы нулевые. Поскольку `sum` и `reversed` и так стартуют с 0, без этого блока программа отработала бы точно так же. Оставил его, чтобы краевой случай был виден явно.)

Проверяем

```
cc -std=c11 -Wall -Wextra -O2 code/04-digits.c -o digits
./digits
```

Прогоним на разных входах:

```
Введи целое число: 12345
Число:           12345
Сумма цифр:      15
Задом наперёд:   54321
```

```
Введи целое число: 100
Число:           100
Сумма цифр:      1
Задом наперёд:   1
```

Обрати внимание на 100: задом наперёд получилось `1`, а не `001`. Ведущие нули в числе не хранятся — `001` это просто `1`. Так и должно быть, когда мы переворачиваем именно **число**, а не строку.

```
Введи целое число: -90
Число:           90
Сумма цифр:      9
Задом наперёд:   9
```

Минус мы отбросили, дальше посчиталось правильно.

Часть 2. Факториал и ловушка переполнения

Разбираем задачу руками

Факториал числа `n` (пишут `n!`) — это произведение всех целых от 1 до `n`:

```
5! = 1 · 2 · 3 · 4 · 5 = 120
```

Отдельно: `0! = 1` (так договорились математики, и это удобно). А для отрицательных факториал не определён вовсе.

Считать будем циклом: завести накопитель `fact = 1` и домножать его на 2, 3, …, `n`. Стартуем именно с 1, а не с 0 — ведь умножение на 0 всё обнулит.

Шаг 1. Наивная версия на `int`

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    scanf("%d", &n);

    int fact = 1;
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %d\n", n, fact);
    return 0;
}
```

Разбор:

- `int fact = 1;` — накопитель произведения, стартует с 1.
- `for (int i = 2; i <= n; i++)` — идём от 2 до `n` включительно (`<=`). С 2, а не с 1: умножать на 1 бессмысленно, результат не меняется. Если `n` равно 0 или 1, цикл не выполнится ни разу, и `fact` останется 1 — как раз правильный ответ для `0!` и `1!`.
- `fact = fact * i;` — домножаем накопитель на очередное число.

Проверим на 5 и 12:

```
Факториал какого числа? 5
5! = 120
```

```
Факториал какого числа? 12
12! = 479001600
```

Отлично. А теперь введём 13:

```
Факториал какого числа? 13
13! = 1932053504
```

Шаг 2. Проблема — это неправильный ответ!

13! на самом деле равно **6 227 020 800**, а программа выдала 1 932 053 504. Откуда мусор?

Вспомни из главы 2: `int` вмещает примерно от −2 до +2 миллиардов (точнее, до 2 147 483 647). А 13! — больше шести миллиардов. Число **не помещается** в коробку `int`. При переполнении Си не падает и не предупреждает — он молча оставляет «обрезок», и дальше всё врёт. Это и есть **переполнение целого** (integer overflow) — одна из самых коварных ошибок: программа выглядит рабочей, а считает чушь.

Доведём до абсурда — попросим 20!:

```
Факториал какого числа? 20
20! = -2102132736
```

Факториал стал **отрицательным**! Произведение положительных чисел — отрицательное. Так бывает, когда переполнение задевает знаковый бит. Верный признак, что `int` лопнул.

Шаг 3. Фикс — берём `long long`

Нужна коробка побольше. В Си для этого есть тип `long long` — **целое увеличенной разрядности**, обычно 8 байт. Он вмещает примерно до 9.2·10^18 (девять квинтиллионов) — на порядки больше, чем `int`.

```
long long fact = 1;
```

Печатать `long long` нужно специальным спецификатором — `%lld` (буквы `ll` — *long long*, `d` — десятичное). Если оставить `%d`, компилятор с `-Wall` справедливо ругнётся, а вывод будет неверным:

```
printf("%d! = %lld\n", n, fact);
```

С `long long` 13! и 20! считаются правильно:

```
Факториал какого числа? 13
13! = 6227020800
```

```
Факториал какого числа? 20
20! = 2432902008176640000
```

Шаг 4. Но и `long long` не бесконечен

21! уже больше 9.2·10^18 — значит, переполнится даже `long long`. Бесконечно растить тип не выйдет (для совсем больших чисел нужна «длинная арифметика», это отдельная большая тема). Поэтому честная программа должна **сама** отказаться считать там, где соврёт:

```c
if (n > 20) {
    printf("Слишком большое: даже long long переполнится при n > 20.\n");
    return 1;
}
```

И заодно отсечём отрицательный ввод, для которого факториал не определён:

```c
if (n < 0) {
    printf("Факториал определён только для n >= 0.\n");
    return 1;
}
```

Полная программа `04-factorial.c`

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }
```

```
    if (n > 20) {
        printf("Слишком большое: даже long long переполнится при n > 20.\n");
        return 1;
    }

    long long fact = 1;   /* 1! = 1, и 0! = 1 — стартуем с единицы */
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %lld\n", n, fact);
    return 0;
}
```

Проверяем

```
cc -std=c11 -Wall -Wextra -O2 code/04-factorial.c -o factorial
```

```
Факториал какого числа? 5
5! = 120
```

```
Факториал какого числа? 21
Слишком большое: даже long long переполнится при n > 20.
```

```
Факториал какого числа? -3
Факториал определён только для n >= 0.
```

Запомни вывод из этой части: **прежде чем выбрать тип, прикинь, какие числа в нём окажутся.** Маленький `int` для счётчика — нормально. Для факториала, суммы больших величин, произведений — сразу думай про `long long`.

Часть 3. FizzBuzz

Разбираем задачу руками

Классическая задачка с собеседований. Печатаем числа от 1 до 100, но:

- если число кратно 3 — вместо него пишем `Fizz`;
- если кратно 5 — пишем `Buzz`;
- если кратно **и** 3, и 5 (то есть кратно 15) — пишем `FizzBuzz`;

- иначе — само число.

Кратность мы уже умеем проверять: `i % 3 == 0`, `i % 5 == 0`. Вся хитрость — в **порядке** проверок.

Шаг 1. Наивная версия — и её баг

Напишем «в лоб», как читается условие:

```
for (int i = 1; i <= 100; i++) {
    if (i % 3 == 0) {
        printf("Fizz\n");
    } else if (i % 5 == 0) {
        printf("Buzz\n");
    } else {
        printf("%d\n", i);
    }
}
```

Что выведет для 15? 15 кратно 3, значит сработает **первая** ветка — напечатается `Fizz`. А мы хотели `FizzBuzz`! Случай «кратно и 3, и 5» мы вообще не предусмотрели. Это и есть главная ловушка FizzBuzz, на которой спотыкаются на собеседованиях.

Шаг 2. Фикс — самую строгую проверку ставим первой

В цепочке `if / else if` ветки проверяются **сверху вниз**, и срабатывает **первая** подходящая, остальные пропускаются. Значит самое частное условие — «кратно 15» — должно идти **раньше** более общих «кратно 3» и «кратно 5». Тогда 15 поймается до того, как дойдёт до них:

```
if (i % 15 == 0) {
    printf("FizzBuzz\n");
} else if (i % 3 == 0) {
    printf("Fizz\n");
} else if (i % 5 == 0) {
    printf("Buzz\n");
} else {
    printf("%d\n", i);
}
```

Почему `i % 15`, а не `i % 3 == 0 && i % 5 == 0`? Это одно и то же: число делится и на 3, и на 5 ровно тогда, когда делится на их произведение 15. Можно писать любой из вариантов — `% 15` просто короче.

Полная программа `04-fizzbuzz.c`

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 100; i++) {
        if (i % 15 == 0) {
            printf("FizzBuzz\n");
        } else if (i % 3 == 0) {
            printf("Fizz\n");
        } else if (i % 5 == 0) {
            printf("Buzz\n");
        } else {
            printf("%d\n", i);
        }
    }
    return 0;
}
```

Эта программа ничего не вводит — поэтому `scanf` тут не нужен.

Проверяем

```
cc -std=c11 -Wall -Wextra -O2 code/04-fizzbuzz.c -o fizzbuzz
./fizzbuzz
```

Первые 16 строк:

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
```

```
Fizz
13
14
FizzBuzz
16
```

Видишь: 3, 6, 9, 12 → `Fizz`; 5, 10 → `Buzz`; а 15 → `FizzBuzz`, как и задумано. Порядок проверок решил всё.

Частые ошибки

Накопитель не обнулён/не выставлен в 1. Для суммы стартуй с `sum = 0`, для произведения — с `fact = 1`. Перепутаешь — получишь мусор или вечный ноль.

`while (n > 0)` и отрицательный ввод. Цикл откусывания цифр не работает с минусом — сначала возьми модуль через `n = -n`.

Переполнение `int`. Факториалы, произведения, суммы больших чисел легко вылетают за 2 миллиарда. Признак — внезапно отрицательный или явно неправильный результат. Лечится переходом на `long long` (и не забудь `%lld` в `printf`).

`%d` вместо `%lld` для `long long`. Спецификатор и тип должны совпадать, иначе мусор. Компилятор с `-Wall` предупредит — не игнорируй.

Неверный порядок в FizzBuzz. Если «кратно 3» проверяется раньше «кратно 15», числа вроде 15 и 30 получат `Fizz` вместо `FizzBuzz`. Самое строгое условие — первым.

Деление и остаток от нуля. `n % 0` и `n / 0` — это ошибка времени выполнения (программа упадёт). В наших программах делитель всегда 10, 3, 5, 15 — не ноль, так что мы в безопасности. Но если делитель приходит от пользователя — проверяй его на ноль.

Упражнения

1. Подсчёт цифр. Измени `04-digits.c` так, чтобы он ещё печатал, **сколько** цифр в числе (заведи счётчик и увеличивай его на каждом шаге цикла).
2. Палиндром. Число-палиндром читается одинаково в обе стороны (121, 1331). Используя готовый переворот, проверь: если `reversed` равен `original`, печатай «палиндром».
3. Сумма от 1 до n. Напиши функцию `long long sum_to(int n)`, возвращающую сумму всех чисел от 1 до `n` циклом. Проверь на больших `n` — почему тут тоже нужен `long long`?

4. Степень. Напиши `long long power(int base, int exp)`, считающую `base` в степени `exp` циклом (накопитель стартует с 1, домножаем `exp` раз). С какого момента результат переполнится?
5. FizzBuzz по-своему. Спроси у пользователя два делителя (вместо жёстких 3 и 5) и два слова, и сделай настраиваемый FizzBuzz от 1 до 100. Не забудь проверить, что делители не равны нулю.
6. Сумма чётных цифр. Переделай сумму цифр так, чтобы она складывала только **чётные** цифры числа (подсказка: внутри цикла добавь `if (digit % 2 == 0)`).

Что мы получили

Три рабочие программы — и, что важнее, чувство двух операторов, `%` и `/`, которыми разбирают числа на части. Ты увидел, как «откусывать» цифры с конца (`n % 10` и `n / 10`), как собирать число обратно (`r = r*10 + digit`), как копить сумму и произведение, и как порядок веток `if / else if` меняет результат (FizzBuzz). А ещё ты столкнулся с переполнением `int` вживую и научился выбирать тип под размер чисел — этот урок будет возвращаться всю книгу.

В следующей главе останемся в мире чисел, но копнём в теорию чисел: найдём **наибольший общий делитель** двух чисел алгоритмом Евклида (тем самым `%`, только хитрее) и научимся проверять, **простое** ли число. Это первые алгоритмы, у которых есть имя и история.

Итоговые программы главы — `code/04-digits.c`, `code/04-factorial.c` и `code/04-fizzbuzz.c`.

Глава 5. НОД Евклида, простые числа и решето Эратосфена

Числа умеют больше, чем складываться и делиться. У них есть скрытая структура: общие делители, простота, разложение на множители. В этой главе мы напишем три маленькие, но знаменитые программы — и каждую доведём от наивной, медленной версии до быстрой и красивой.

Сначала найдём **наибольший общий делитель** двух чисел и познакомимся с алгоритмом Евклида — ему больше двух тысяч лет, и он до сих пор в работе. Потом научимся проверять, **простое** ли число, и поймём, почему достаточно проверок «до корня». В конце соберём **решето Эратосфена** — оно одним махом находит все простые до заданной границы.

Что мы сделаем

К концу главы у тебя будут три программы:

- `05-gcd.c` — НОД двух чисел: сперва тупым перебором делителей, потом алгоритмом Евклида.
- `05-prime.c` — проверка числа на простоту, наивно и с оптимизацией «до корня».
- `05-sieve.c` — решето Эратосфена, печатающее все простые до 100.

Ничего нового из синтаксиса почти не понадобится: всё держится на циклах, остатке `%`, массивах и функциях из глав 2–4. Главное здесь — **идеи алгоритмов**. Их и будем разбирать руками.

Часть 1. Наибольший общий делитель

Разбираем задачу руками

Наибольший общий делитель (НОД) двух чисел — это самое большое число, на которое делятся **оба** без остатка.

Возьмём 48 и 36. Выпишем делители каждого:

- делители 48: 1, 2, 3, 4, 6, 8, 12, 16, 24, 48;
- делители 36: 1, 2, 3, 4, 6, 9, 12, 18, 36.

Общие у них: 1, 2, 3, 4, 6, 12. Самый большой — **12**. Значит, НОД(48, 36) = 12.

Так мы посчитали вручную: перебрали делители и выбрали наибольший общий. Эту же мысль можно записать кодом напрямую.

Шаг 1. Наивный перебор

Заведём результат и переберём все числа `d` от 1. Если `d` делит и `a`, и `b` — запоминаем его. Поскольку `d` растёт, последнее запомненное и будет наибольшим. Делитель не может быть больше меньшего из чисел, поэтому идём, пока `d` не больше обоих.

```
int gcd_slow(int a, int b)
{
    int result = 1;
    for (int d = 1; d <= a && d <= b; d++) {
        if (a % d == 0 && b % d == 0) {
            result = d;
        }
    }
    return result;
}
```

Разбор:

- `int result = 1;` — на 1 делится всё, так что 1 — безопасный старт.
- `for (int d = 1; d <= a && d <= b; d++)` — счётчик `d` идёт от 1, пока он не больше **и** `a`, **и** `b`. Связка `&&` — логическое «И» (см. главу 2): цикл живёт, пока оба условия истинны.
- `a % d == 0` — остаток от деления `a` на `d` равен нулю, то есть `d` делит `a` нацело. Помнишь из главы 2: `%` — это остаток.
- `a % d == 0 && b % d == 0` — `d` делит оба числа, значит он общий делитель.
- `result = d;` — запоминаем. Так как `d` только растёт, в `result` под конец останется самый большой общий делитель.

Запустим (НОД(48, 36)):

```
Перебором: НОД(48, 36) = 12
```

Верно. Но в чём проблема? Этот цикл крутится **столько раз, сколько меньшее число**. Для 48 и 36 — пустяк, 36 шагов. А для чисел вроде 1 000 000 007 это уже миллиард проверок. Медленно и некрасиво: мы перебираем всё подряд, хотя ответ можно получить за несколько шагов.

Шаг 2. Идея Евклида

Евклид заметил простую вещь. Пусть `a` больше `b`. Тогда **любой общий делитель `a` и `b` делит и их разность `a − b`**.

Почему? Если `d` делит `a` и делит `b`, то он делит и `a − b` (две величины, кратные `d`, дают в разности тоже кратное `d`). И наоборот. Значит, у пары `(a, b)` и у пары `(a − b, b)` **одни и те же** общие делители — а раз так, то и наибольший общий делитель один и тот же.

Отсюда рецепт: **большее число заменяем на разность большего и меньшего** и повторяем, пока числа не сравняются. Это сравнявшееся число и есть НОД.

Проследим на 48 и 36 руками:

```
(48, 36) -> 48 − 36 = 12 -> (36, 12)
(36, 12) -> 36 − 12 = 24 -> (24, 12)
(24, 12) -> 24 − 12 = 12 -> (12, 12)
числа сравнялись -> НОД = 12
```

Получили те же 12, но всего за три вычитания вместо перебора всех делителей. Запишем версию через вычитание:

```
int gcd_sub(int a, int b)
{
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}
```

- `while (a != b)` — крутимся, пока числа не равны (`!=` — «не равно»).
- из большего вычитаем меньшее, меньшее не трогаем.

- когда `a` и `b` сравнялись — выходим и возвращаем `a` (он же равен `b`).

Уже хорошо. Но и тут есть слабое место. Если числа сильно разные, например 1000000 и 1, вычитание будет отнимать по единице миллион раз. А ведь «отнять `b` из `a` столько раз, сколько влезет» — это ровно **взять остаток** `a % b`.

Шаг 3. Евклид через остаток

Заменим череду вычитаний одним остатком. Пока `b` не ноль, делаем `a` и `b` новой парой: `(b, a % b)`. Когда `b` станет нулём, ответ — в `a`.

```
int gcd(int a, int b)
{
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

Разбор по строкам — здесь важен порядок:

- `while (b != 0)` — работаем, пока второе число не обнулилось.
- `int t = b;` — заводим временную переменную и прячем в неё текущее `b`. Зачем? Сейчас мы испортим `b`, а его старое значение нужно будет положить в `a`.
- `b = a % b;` — новое `b` — это остаток от деления `a` на `b`.
- `a = t;` — а в `a` кладём то, что раньше было в `b` (мы сохранили это в `t`).

Эти три строки — классический **обмен с пересчётом**. Без `t` мы бы сначала затёрли `b` и потеряли нужное значение. (Похожий приём с временной переменной ты видел в `swap` из главы 3.)

Проследим на 48 и 36:

```
a=48 b=36 -> t=36, b=48%36=12, a=36 -> (36, 12)
a=36 b=12 -> t=12, b=36%12=0,  a=12 -> (12, 0)
b == 0 -> возвращаем a = 12
```

Два шага — и ответ. Для огромных чисел Евклид тоже укладывается в считаные десятки шагов. Вот зачем нужен был этот алгоритм.

Полная программа `05-gcd.c`

```c
#include <stdio.h>

/* Наибольший общий делитель двумя способами:
   1) наивный перебор делителей;
   2) алгоритм Евклида через остаток. */

int gcd_slow(int a, int b)
{
    int result = 1;
    for (int d = 1; d <= a && d <= b; d++) {
        if (a % d == 0 && b % d == 0) {
            result = d;
        }
    }
    return result;
}

int gcd(int a, int b)
{
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int main(void)
{
    int a, b;
    printf("Введи два числа через пробел: ");
    if (scanf("%d %d", &a, &b) != 2) {
        printf("Нужно два целых числа.\n");
        return 1;
    }
```

```
    printf("Перебором: НОД(%d, %d) = %d\n", a, b, gcd_slow(a, b));
    printf("Евклидом:  НОД(%d, %d) = %d\n", a, b, gcd(a, b));
    return 0;
}
```

Новое в `main`:

- `scanf("%d %d", &a, &b)` — читаем **два** числа за один вызов. Пробел в формате означает «тут может быть пробел или перевод строки» между числами. Перед каждой переменной — `&` (адрес, куда писать), как обычно для `scanf`.
- `if (scanf(...) != 2)` — `scanf` **возвращает**, сколько значений ему удалось прочитать. Мы ждём 2. Если пришло меньше (ввели буквы), ругаемся и выходим с кодом 1. `return 1;` из `main` — сигнал «программа завершилась с ошибкой» (0 — всё хорошо, не ноль — что-то не так). Проверять результат `scanf` — хорошая привычка из STYLE-требований.

Запуск:

```
Введи два числа через пробел: 48 36
Перебором: НОД(48, 36) = 12
Евклидом:  НОД(48, 36) = 12
```

И ещё пара примеров:

```
Введи два числа через пробел: 1071 462
Перебором: НОД(1071, 462) = 21
Евклидом:  НОД(1071, 462) = 21
```

```
Введи два числа через пробел: 17 5
Перебором: НОД(17, 5) = 1
Евклидом:  НОД(17, 5) = 1
```

17 и 5 не имеют общих делителей, кроме 1, — такие числа называют **взаимно простыми**.

Часть 2. Простое число

Разбираем задачу руками

Простое число — это целое больше 1, у которого ровно два делителя: 1 и оно само. 7 — простое (делится только на 1 и 7). 12 — нет: оно делится на 2, 3, 4, 6. Числа больше 1, которые не простые, называют **составными**. Числа 0 и 1 не считают простыми по определению.

Как проверить вручную? Берём число и пробуем поделить его на 2, 3, 4 и так далее. Если хоть на что-то поделилось без остатка — оно составное. Если ни на что не поделилось — простое.

Шаг 1. Наивная проверка

Переберём все возможные делители от 2 до `n - 1`. Нашёлся делитель — число составное, сразу возвращаем «нет».

```
int is_prime_slow(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}
```

Разбор:

- `if (n < 2) return 0;` — отсекаем 0, 1 и отрицательные: они не простые. Помни из главы 2: 0 — это «ложь/нет», 1 — «истина/да». Здесь функция возвращает именно такой ответ-флаг.
- `for (int i = 2; i < n; i++)` — пробуем делители от 2 до `n - 1`.
- `if (n % i == 0) return 0;` — нашёлся делитель → число составное, немедленно выходим из функции с ответом 0. `return` прекращает функцию сразу, остаток цикла не выполняется.
- `return 1;` — если цикл закончился и ни одного делителя не нашлось, число простое.

Работает. Но снова медленно: для большого `n` мы делаем почти `n` проверок. Можно ли меньше?

Шаг 2. Почему достаточно до корня

Ключевое наблюдение. Если `n` делится на какое-то `i`, то есть и парный делитель `n / i`. Например, у 36: 2 и 18, 3 и 12, 4 и 9, 6 и 6.

В каждой паре **один из множителей не больше квадратного корня из `n`**, а другой не меньше. Не может быть так, чтобы оба были больше корня — тогда их произведение было бы больше `n`. Значит, если у числа вообще есть делитель, найдётся делитель **не больше √n**.

Вывод: достаточно проверять `i` только до корня из `n`. Для 36 это проверки до 6 — а не до 35. Для миллиона — до тысячи вместо миллиона.

Как написать «до корня» без подключения `math.h` и функции `sqrt`? Очень просто: вместо `i <= sqrt(n)` пишем `i * i <= n`. Это то же самое условие, только без дробей и без библиотеки — сравниваем квадрат `i` с самим `n`.

```
int is_prime(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}
```

Изменилась одна деталь — условие цикла: `i * i <= n` вместо `i < n`. Цикл обрывается, как только `i` переваливает за корень. Ответы те же, а работы в разы меньше.

> Осторожно с очень большими `n`: `i * i` может **переполнить** `int` (см. ловушку переполнения из главы 4). Для чисел до сотен миллионов всё в порядке; для огромных взяли бы тип пошире (`long long`).

Полная программа `05-prime.c`

```
#include <stdio.h>

/* Проверка, простое ли число.
   is_prime_slow — делим на всё подряд до n-1.
   is_prime      — делим только до корня из n (i*i <= n). */

int is_prime_slow(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i < n; i++) {
```

```
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}

int is_prime(int n)
{
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }
    return 1;
}

int main(void)
{
    int n;
    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%d — простое число\n", n);
    } else {
        printf("%d — составное (или меньше 2)\n", n);
    }
    return 0;
}
```

(В файле лежат обе функции, чтобы их можно было сравнить, но в `main` мы зовём быструю `is_prime`. Функцию `is_prime_slow` компилятор тоже видит — поскольку она объявлена и используется как часть листинга, предупреждения «неиспользуемая функция» не возникает.)

Запуски:

```
Введи число: 7
7 — простое число
```

```
Введи число: 12
12 — составное (или меньше 2)
```

```
Введи число: 1
1 — составное (или меньше 2)
```

```
Введи число: 97
97 — простое число
```

Часть 3. Решето Эратосфена

Разбираем задачу руками

А если нужны не одно число, а **все простые до 100** сразу? Можно вызвать `is_prime` для каждого, но есть способ изящнее и быстрее, его придумал Эратосфен больше двух тысяч лет назад.

Идея — **вычёркивать**. Выпишем все числа от 2 до 100. Берём первое невычеркнутое — это 2, оно простое. Теперь вычеркнем всех его «родню»: 4, 6, 8, 10… — все числа, кратные 2 (они точно составные, раз делятся на 2).

Идём дальше. Следующее невычеркнутое — 3, тоже простое. Вычёркиваем кратные 3: 6, 9, 12, 15… (6 уже была вычеркнута — ничего, вычеркнем повторно, не страшно).

Дальше 4 — но она уже вычеркнута (как кратная 2), пропускаем. Берём 5, вычёркиваем 10, 15, 20… И так до конца. В конце все **невычеркнутые** числа — простые.

Маленькая хитрость про экономию. Когда мы дошли до числа `i` и хотим вычёркивать его кратные, начинать можно не с `2*i`, а сразу с `i*i`. Почему? Все кратные `i`, что меньше `i*i` (это `2*i`, `3*i`, …, `(i-1)*i`), уже вычеркнуты более маленькими множителями. Например, для `i = 5` числа 10, 15, 20 уже убрали двойка и тройка. Так что начинаем с 25.

И ещё: запускать вычёркивание имеет смысл, только пока `i*i` не больше границы. Если `i*i` уже больше 100, то у любого составного числа до 100 точно есть делитель поменьше — мы его уже вычеркнули. Это та же мысль «до корня», что и в проверке на простоту.

Шаг 1. Массив-флаги

Нам нужно где-то отмечать, вычеркнуто число или нет. Это идеальная работа для массива: заведём массив флагов, где индекс — само число.

```
int is_composite[N + 1] = {0};
```

- `is_composite` — массив флагов. `is_composite[k]` отвечает на вопрос «число `k` составное?»: 0 — пока считаем простым, 1 — вычеркнули как составное.
- `[N + 1]` — почему плюс один? Мы хотим обращаться по индексу вплоть до самого `N` (то есть `is_composite[100]`). Индексы массива идут с нуля, поэтому, чтобы существовал элемент с индексом 100, нужен массив длиной 101. Индексы 0 и 1 просто не используем.
- `= {0}` — короткая, но важная запись: **заполнить весь массив нулями**. Если в фигурных скобках указать хотя бы один элемент, остальные C обнулит сам. Без этой инициализации в массиве был бы мусор (см. главу 3), и «невычеркнутость» нельзя было бы отличить от случайных значений.
- `N` — у нас это не переменная, а **именованная константа**. Её задаёт строка `#define N 100` в начале файла. `#define ИМЯ значение` — указание препроцессору: «перед компиляцией замени везде в тексте `N` на `100`». Удобно: поменяешь одно число — изменится везде. Имена констант принято писать заглавными.

Шаг 2. Вычёркиваем

```
for (int i = 2; i * i <= N; i++) {
    if (is_composite[i] == 0) {
        for (int j = i * i; j <= N; j += i) {
            is_composite[j] = 1;
        }
    }
}
```

Разбор — здесь два вложенных цикла:

- внешний `for (int i = 2; i * i <= N; i++)` — перебираем кандидатов в «вычёркиватели». Условие `i * i <= N` — та самая граница «до корня».
- `if (is_composite[i] == 0)` — берёмся вычёркивать только от **ещё не вычеркнутого** `i`. Если `i` сам составной, его кратные уже убраны его делителями.
- внутренний `for (int j = i * i; j <= N; j += i)` — идём по кратным `i`, начиная с `i * i`. Шаг `j += i` (прибавить `i`) перескакивает сразу к следующему кратному: `i*i`, `i*i + i`, `i*i + 2i`, …
- `is_composite[j] = 1;` — отмечаем `j` как составное (вычёркиваем).

Вложенный цикл — новинка по сравнению с прошлыми главами, но ничего магического: это просто цикл внутри цикла. На каждом шаге внешнего цикла внутренний пробегает свои кратные целиком.

Полная программа `05-sieve.c`

```c
#include <stdio.h>

/* Решето Эратосфена: печатает все простые числа до N. */

#define N 100

int main(void)
{
    int is_composite[N + 1] = {0};   /* 0 = считаем простым, 1 = вычеркнуто */

    for (int i = 2; i * i <= N; i++) {
        if (is_composite[i] == 0) {
            for (int j = i * i; j <= N; j += i) {
                is_composite[j] = 1;
            }
        }
    }

    printf("Простые числа до %d:\n", N);
    int count = 0;
    for (int i = 2; i <= N; i++) {
        if (is_composite[i] == 0) {
            printf("%d ", i);
            count++;
        }
    }
    printf("\nВсего: %d\n", count);
    return 0;
}
```

Вторая половина — печать результата:

- проходим числа от 2 до `N` и печатаем те, что **не** вычеркнуты (`is_composite[i] == 0`).

- `count` считает, сколько простых нашли, — приятно увидеть итог.

Запуск:

```
Простые числа до 100:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Всего: 25
```

25 простых чисел до сотни — можешь сверить со справочником, всё на месте. Хочешь до 1000? Поменяй одну строку `#define N 100` на `#define N 1000`, перекомпилируй — и решето справится мгновенно.

Как это работает: трассировка решета

Чтобы решето уложилось окончательно, прокрутим его для маленького `N = 10` руками. Массив (индексы 2..10), все нули:

```
индекс:  2 3 4 5 6 7 8 9 10
старт:   0 0 0 0 0 0 0 0 0
```

- `i = 2`, `2*2=4 <= 10`. Двойка не вычеркнута → вычёркиваем 4, 6, 8, 10:

```
индекс:  2 3 4 5 6 7 8 9 10
i=2:     0 0 1 0 1 0 1 0 1
```

- `i = 3`, `3*3=9 <= 10`. Тройка не вычеркнута → вычёркиваем 9 (с `i*i=9`, дальше `12 > 10` — стоп):

```
индекс:  2 3 4 5 6 7 8 9 10
i=3:     0 0 1 0 1 0 1 1 1
```

- `i = 4`: `4*4=16 > 10` → внешний цикл останавливается.

Невычеркнутые (нули): 2, 3, 5, 7. Это и есть простые до 10. Сошлось.

Проверяем

Скомпилируй и запусти каждую программу:

```
cc -std=c11 -Wall -Wextra -O2 code/05-gcd.c   -o gcd
cc -std=c11 -Wall -Wextra -O2 code/05-prime.c -o prime
cc -std=c11 -Wall -Wextra -O2 code/05-sieve.c -o sieve
./gcd
```

```
./prime
./sieve
```

Ни одного предупреждения быть не должно. Ожидаемый вывод решета — 25 простых чисел, как выше. Для НОД и простоты проверь на своих числах.

Частые ошибки

Бесконечный цикл в Евклиде. Если перепутать порядок строк и написать сначала `b = a % b;`, а потом `int t = b;`, потеряешь старое `b`, и пара перестанет правильно обновляться. Сохраняй `b` в `t` **до** того, как его испортишь.

`<= n` вместо `< n` в наивной проверке простоты. С `i <= n` число поделится само на себя (`n % n == 0`), и любое число объявится составным. Делители проверяем строго до `n` (а лучше — до корня).

Забыл отсечь `n < 2`. Без проверки `if (n < 2)` единица или ноль проскочат пустой цикл и ошибочно объявятся «простыми».

`i <= sqrt` через дроби. Не тащи `math.h` ради `sqrt`: дробное сравнение может ошибиться на грани из-за неточности `double`. Целое `i * i <= n` надёжнее и проще.

Массив на `N`, а не на `N + 1`. Объявишь `is_composite[N]` и обратишься к `is_composite[N]` — это выход за границу (см. главу 3). Под индекс `N` нужен размер `N + 1`.

Забыл `= {0}`. Неинициализированный массив полон мусора, и решето выдаст случайную чепуху. Обнуляй флаги при объявлении.

Упражнения

1. В `05-gcd.c` добавь функцию `gcd_sub` (Евклид через вычитание из шага 2) и печатай её результат третьей строкой. Сверь, что все три способа дают одно и то же.
2. **Наименьшее общее кратное** (НОК) связано с НОД формулой `НОК(a, b) = a / gcd(a, b) * b`. Допиши функцию `lcm` и проверь: НОК(4, 6) = 12. (Подсказка: дели **до** умножения, чтобы не переполнить `int`.)
3. Сделай так, чтобы `05-prime.c` печатал не только «простое/нет», но и **наименьший делитель** составного числа (первое `i`, на которое разделилось).
4. На основе `is_prime` напечатай первые 20 простых чисел (заведи счётчик, перебирай числа подряд, пока не наберёшь 20).

5. Преврати решето в подсчёт: пусть `05-sieve.c` спрашивает `N` у пользователя и печатает только **количество** простых до него. (Подсказка: статический массив фиксированного размера здесь не годится — вспомни `malloc` из главы 3.)
6. Для каждого простого числа до 50 напечатай его «пару-близнеца», если она есть: простые-близнецы отличаются на 2 (например, 11 и 13, 17 и 19).

Что мы получили

Три классических алгоритма над числами — и в каждом мы прошли путь от наивной версии к умной. Алгоритм Евклида показал, как замена операции (вычитание → остаток) ускоряет всё в разы. Проверка простоты научила приёму «достаточно до корня» — он встречается постоянно. А решето Эратосфена впервые свело нас с массивом-флагами и вложенными циклами, работающими над набором данных сразу.

Эта последняя идея — обрабатывать **весь массив целиком** — и есть мостик в следующую главу. В главе 6 мы возьмёмся за алгоритмы над массивами вплотную: поиск максимума и минимума, разворот, линейный и двоичный поиск. Массивы из фона выйдут на первый план.

Итоговые программы главы — `code/05-gcd.c`, `code/05-prime.c` и `code/05-sieve.c`.

Глава 6. Алгоритмы над массивами: минимум/максимум, сумма, среднее, разворот, сдвиг

Массив у нас уже есть: мы умеем его объявить, заполнить вводом через цикл и пройтись по элементам. Пора заставить массив работать. В этой главе мы разберём пять задач, которые встречаются буквально везде — от обработки оценок до игровой логики:

- найти **минимум** и **максимум**;
- посчитать **сумму** и **среднее**;
- **развернуть** массив задом наперёд, не заводя второй массив;
- **посчитать**, сколько раз встречается заданное число;
- **сдвинуть** массив циклически влево на одну позицию.

Все они — про один проход циклом и пару переменных. Никакой новой тяжёлой теории: только приёмы, которые ты будешь повторять до конца книги.

Что мы сделаем

К концу главы у тебя будут три рабочие программы:

- `06-minmax.c` — вводит числа и печатает минимум, максимум, сумму и среднее.
- `06-reverse.c` — переворачивает массив **на месте**.
- `06-count-rotate.c` — считает, сколько раз встретилось число, и сдвигает массив влево.

И, что важнее, ты поймёшь две вещи, на которых спотыкается почти каждый новичок: **почему минимум/максимум нельзя начинать с нуля** и **почему разворот «двумя индексами» останавливается ровно посередине**.

Часть 1. Минимум, максимум, сумма, среднее

Разбираем задачу руками

Пусть ввели пять чисел: `7 −3 10 4 10`.

Сумму ты бы считал так: ведёшь «копилку», начинаешь с нуля, идёшь по числам и каждое добавляешь. `0 → 7 → 4 → 14 → 18 → 28`. Сумма 28. Среднее — сумма делить на количество: `28 / 5 = 5.6`.

А как ты ищешь минимум глазами? Смотришь на первое число и держишь его в голове как «пока самое маленькое». Потом смотришь на каждое следующее: если оно меньше того, что в голове, — запоминаешь новое. Дошёл до конца — в голове лежит минимум. Максимум — то же самое, только «больше».

Главное здесь — **с чего начать**. «В голове» сначала лежит **первое число массива**, а не ноль. Запомни этот момент, сейчас увидим, почему он критичен.

Шаг 1. Наивная версия — и её баг

Сделаем «в лоб»: заведём `min` и `max`, поставим их в 0 (кажется логичным — «пусто»), сумму тоже в 0, и пройдёмся.

```
int min = 0;
int max = 0;
int sum = 0;
for (int i = 0; i < n; i++) {
    if (a[i] < min) min = a[i];
    if (a[i] > max) max = a[i];
    sum += a[i];
}
```

- `sum += a[i];` — это короткая запись `sum = sum + a[i]` (видели в главе 2). К копилке прибавляем очередной элемент.
- `if (a[i] < min) min = a[i];` — если текущий элемент меньше нашего «рекордсмена», обновляем рекорд. Без фигурных скобок: когда в теле `if` одна инструкция, скобки можно опустить (но будь аккуратен).

Запускаем на числах `7 −3 10 4 10`:

```
Минимум: −3
Максимум: 10
```

Минимум и максимум совпали с ожиданием. Случайно. Теперь введём числа, которые **все положительные**: `7 3 10 4 9`.

```
Минимум: 0
Максимум: 10
```

Максимум верный, а минимум — **0**, хотя нуля в данных вообще нет! Вот он, баг.

В чём проблема

Мы стартовали `min` с нуля. Дальше условие `if (a[i] < min)` спрашивает «меньше ли элемент, чем 0?». Все наши числа положительные — значит, **ни одно** не меньше нуля, `min` так и остаётся 0. Программа уверенно врёт.

Если бы все числа были, наоборот, **отрицательными** (`-7 -3 -10`), то сломался бы `max`: он остался бы нулём, потому что ничего не больше нуля.

Ноль — это не «нейтральное пустое значение». Это конкретное число, которое влезает в сравнение и портит результат. Для суммы ноль правильный старт (прибавить ноль — ничего не меняет). А для минимума и максимума нужен старт, который **точно есть среди данных**.

Шаг 2. Фикс — стартуем с первого элемента

Правильное начало для `min` и `max` — **первый элемент массива**, `a[0]`. Он гарантированно из наших данных, поэтому ничего не исказит: в худшем случае минимум так и останется первым элементом, и это будет правдой.

```
int min = a[0];
int max = a[0];
int sum = 0;
for (int i = 0; i < n; i++) {
    if (a[i] < min) min = a[i];
    if (a[i] > max) max = a[i];
    sum += a[i];
}
```

Цикл по-прежнему идёт с `i = 0`. Да, на первом шаге мы сравниваем `a[0]` сам с собой — это не ошибка, просто `min` и `max` не изменятся. Лишнего тут нет, зато код проще.

Теперь на `7 3 10 4 9`:

```
Минимум: 3
Максимум: 10
```

Минимум правильный. Баг побеждён одной строкой.

Среднее: не наступи на целочисленное деление

Сумма у нас целая (`int`), количество тоже целое. Если написать `sum / n`, Си поделит **нацело** и отбросит дробную часть — `28 / 5` даст 5, а не 5.6 (мы разбирали эту ловушку в главе 2). Чтобы получить дробь, приводим сумму к `double`:

```
double avg = (double)sum / n;
```

`(double)sum` превращает целое в дробное **перед** делением. Тогда деление становится дробным, и `n` тоже автоматически превратится в дробное на время операции. Результат — настоящие 5.6.

Полная программа `06-minmax.c`

```
#include <stdio.h>

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Нужно положительное количество.\n");
        return 1;
    }

    int a[100];
    if (n > 100) {
        n = 100;
    }
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        scanf("%d", &a[i]);
    }

    int min = a[0];
    int max = a[0];
    int sum = 0;
    for (int i = 0; i < n; i++) {
        if (a[i] < min) {
```

```
            min = a[i];
        }
        if (a[i] > max) {
            max = a[i];
        }
        sum += a[i];
    }

    double avg = (double)sum / n;

    printf("Минимум: %d\n", min);
    printf("Максимум: %d\n", max);
    printf("Сумма:    %d\n", sum);
    printf("Среднее:  %.2f\n", avg);
    return 0;
}
```

Пара новых деталей по сравнению с прошлой главой:

- `if (scanf("%d", &n) != 1 || n <= 0)` — мы проверяем, что ввод удался. `scanf` возвращает, **сколько значений** он успешно прочитал. Если попросили одно (`%d`) и получили одно — вернёт `1`. Если пользователь ввёл буквы, вернёт не 1, и мы выходим с ошибкой. Заодно отсекаем `n <= 0`, чтобы дальше `a[0]` точно существовал.
- `return 1;` — выход из `main` с кодом, отличным от нуля. По традиции `0` значит «всё хорошо», а ненулевое — «была ошибка».
- `if (n > 100) n = 100;` — наш массив `a[100]` вмещает максимум 100 чисел. Если попросили больше, аккуратно ограничиваем `n`, чтобы не вылезти за границу массива. (Совсем правильно было бы выделить память под точное `n` через `malloc` из главы 3 — это будет в упражнениях.)

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 06-minmax.c -o 06-minmax
./06-minmax
```

Вводим пять чисел `7 -3 10 4 10` (каждое и количество — с новой строки):

```
Сколько чисел? 5
a[0] = 7
```

```
a[1] = -3
a[2] = 10
a[3] = 4
a[4] = 10
Минимум: -3
Максимум: 10
Сумма:    28
Среднее:  5.60
```

Минимум `-3`, максимум `10`, сумма `28`, среднее `5.60`. Всё сходится.

Часть 2. Разворот массива на месте

Разбираем задачу руками

Есть массив `1 2 3 4 5`, нужно получить `5 4 3 2 1`. Условие — **без второго массива**: переставляем элементы внутри того же самого.

Возьми пять карточек в ряд. Как перевернуть порядок руками? Меняешь местами **крайние**: первую с последней. Получилось `5 2 3 4 1`. Потом вторую с предпоследней: `5 4 3 2 1`. Средняя карточка (третья) остаётся на месте — ей и так быть в центре. Готово за два обмена.

Идея: два «пальца». Левый `i` стоит на первом элементе, правый `j` — на последнем. Меняем местами то, на что они показывают, и сдвигаем пальцы навстречу друг другу: `i` вправо, `j` влево. Останавливаемся, когда они встретились или прошли друг друга.

Шаг 1. Меняем местами через временную переменную

Обмен двух элементов массива — то же самое, что обмен двух переменных из главы 3, только теперь это элементы:

```
int t = a[i];
a[i] = a[j];
a[j] = t;
```

Почему нельзя короче, `a[i] = a[j]; a[j] = a[i];`? Потому что первая строка **затирает** `a[i]` значением `a[j]`, и старое `a[i]` теряется навсегда — вторая строка положит обратно то же `a[j]`. Поэтому нужна временная коробка `t`, куда мы прячем `a[i]` до перезаписи. (В главе 3 мы делали ровно это через указатели в функции `swap`; здесь обмен прямо на элементах массива.)

Шаг 2. Сводим индексы навстречу

```
int i = 0;
int j = n - 1;
while (i < j) {
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
    i++;
    j--;
}
```

- `int j = n - 1;` — правый индекс ставим на **последний** элемент. Помни: индексы идут `0 .. n-1`, поэтому последний — `n - 1`, а не `n`. Написать `a[n]` значило бы выйти за границу массива.
- `while (i < j)` — повторяем, **пока левый палец левее правого**.
- внутри — обмен через `t`.
- `i++; j--;` — двигаем пальцы навстречу: левый на шаг вправо, правый на шаг влево.

Почему условие именно `i < j`

Это сердце алгоритма, разберём по случаям.

Почему не `i <= j`? Когда `i == j`, оба пальца стоят на **одном и том же** элементе (это центр массива нечётной длины). Менять элемент сам с собой бессмысленно — лишняя работа. Хуже того, если продолжить после встречи, пальцы пойдут дальше и начнут **разворачивать массив обратно**: то, что уже поменяли местами, поменяется во второй раз и вернётся как было. Условие `i < j` останавливает нас ровно в нужный момент.

Посмотрим на `1 2 3 4 5` (нечётная длина, n = 5):

```
старт:  i=0 j=4   меняем a[0] и a[4] -> 5 2 3 4 1
шаг:    i=1 j=3   меняем a[1] и a[3] -> 5 4 3 2 1
шаг:    i=2 j=2   i < j ложно -> стоп (центр 3 не трогаем)
```

А на `1 2 3 4` (чётная длина, n = 4):

```
старт:  i=0 j=3   меняем a[0] и a[3] -> 4 2 3 1
шаг:    i=1 j=2   меняем a[1] и a[2] -> 4 3 2 1
шаг:    i=2 j=1   i < j ложно -> стоп
```

В обоих случаях каждый элемент переставлен ровно один раз, и мы делаем примерно `n / 2` обменов — вдвое меньше, чем длина массива. Быстро и без лишней памяти.

Полная программа `06-reverse.c`

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Нужно положительное количество.\n");
        return 1;
    }

    int a[100];
    if (n > 100) {
        n = 100;
    }
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        scanf("%d", &a[i]);
    }

    int i = 0;
    int j = n - 1;
    while (i < j) {
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
        i++;
        j--;
    }

    printf("Развёрнутый массив:");
    for (int k = 0; k < n; k++) {
        printf(" %d", a[k]);
    }
    printf("\n");
```

```
    return 0;
}
```

При печати мы взяли отдельный счётчик `k`, потому что `i` и `j` после цикла уже «израсходованы» (они встретились в центре). И обрати внимание на формат `" %d"` — пробел **перед** числом: так числа красиво разделяются пробелами, а лишнего пробела в начале строки нет (он идёт перед первым числом, сразу после двоеточия в `printf` выше).

Проверяем

```
Сколько чисел? 5
a[0] = 1
a[1] = 2
a[2] = 3
a[3] = 4
a[4] = 5
Развёрнутый массив: 5 4 3 2 1
```

Часть 3. Подсчёт значения и сдвиг влево

Две короткие, но полезные задачи в одной программе.

Задача A. Сколько раз встречается число

Идея простая: заводим счётчик `count = 0`, идём по массиву, и каждый раз, когда элемент равен искомому, прибавляем 1.

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (a[i] == target) {
        count++;
    }
}
```

- `target` — то значение, которое ищем (спросим у пользователя).
- `a[i] == target` — сравнение **на равенство**, двойное == (один = был бы присваиванием — классический баг из главы 2).
- `count++` — увеличить счётчик на 1 при каждом совпадении.

Всё. Один проход, одна переменная.

Задача Б. Циклический сдвиг влево на 1

«Сдвинуть влево на 1» значит: каждый элемент переезжает на одну позицию левее, а самый первый, которому уже некуда деваться слева, **уходит в самый конец**. Это и есть «циклический» (по кругу) сдвиг.

Руками на `10 20 30 40`:

```
было:   10 20 30 40
стало:  20 30 40 10
```

`20` встало на место `10`, `30` на место `20`, `40` на место `30`, а `10` уехало в хвост.

Как это запрограммировать? Первый элемент мы вот-вот затрём — значит, сначала **сохраним его** в отдельную переменную. Потом сдвинем все остальные: на позицию `i` положим то, что было на `i+1`. И в самый конец вернём сохранённый первый элемент.

Шаг 1. Наивно — и ловушка с границей

Напрашивается такой цикл:

```c
int first = a[0];
for (int i = 0; i < n; i++) {
    a[i] = a[i + 1];   // опасно на последнем шаге!
}
a[n - 1] = first;
```

Логика «двигаем каждый влево» вроде верная, но посмотри на последний оборот цикла: при `i = n - 1` строка обращается к `a[i + 1]`, то есть к `a[n]`. А элемента `a[n]` **не существует** — индексы только до `n - 1`. Это выход за границу массива: Си не проверяет границы и молча прочитает чужой кусок памяти. Программа может выдать мусор или упасть.

Шаг 2. Фикс — останавливаем цикл на один шаг раньше

Нам нужно сдвигать только до предпоследнего элемента включительно: последний (`a[n-1]`) мы всё равно перезапишем сохранённым `first` вручную. Поэтому условие — `i < n - 1`:

```c
int first = a[0];
for (int i = 0; i < n - 1; i++) {
    a[i] = a[i + 1];
}
a[n - 1] = first;
```

Теперь максимальное `i` равно `n - 2`, и `a[i + 1]` — это `a[n - 1]`, последний существующий элемент. За границу не вылезаем.

Проследим на `10 20 30 40` (n = 4, `first = 10`):

```
i=0:  a[0] = a[1]  ->  20 20 30 40
i=1:  a[1] = a[2]  ->  20 30 30 40
i=2:  a[2] = a[3]  ->  20 30 40 40
конец цикла, ставим a[3] = first(10) -> 20 30 40 10
```

Именно то, что мы насчитали руками.

Полная программа `06-count-rotate.c`

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Сколько чисел? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Нужно положительное количество.\n");
        return 1;
    }

    int a[100];
    if (n > 100) {
        n = 100;
    }
    for (int i = 0; i < n; i++) {
        printf("a[%d] = ", i);
        scanf("%d", &a[i]);
    }

    int target;
    printf("Какое значение посчитать? ");
    scanf("%d", &target);

    int count = 0;
    for (int i = 0; i < n; i++) {
```

```
        if (a[i] == target) {
            count++;
        }
    }
    printf("Значение %d встречается %d раз(а)\n", target, count);

    int first = a[0];
    for (int i = 0; i < n - 1; i++) {
        a[i] = a[i + 1];
    }
    a[n - 1] = first;

    printf("После сдвига влево:");
    for (int i = 0; i < n; i++) {
        printf(" %d", a[i]);
    }
    printf("\n");
    return 0;
}
```

Проверяем

```
Сколько чисел? 6
a[0] = 10
a[1] = 20
a[2] = 30
a[3] = 20
a[4] = 50
a[5] = 20
Какое значение посчитать? 20
Значение 20 встречается 3 раз(а)
После сдвига влево: 20 30 20 50 20 10
```

Число `20` встретилось 3 раза — верно. И массив сдвинулся: первый `10` уехал в хвост, остальные сместились влево.

Частые ошибки

Инициализация min/max нулём. Самая коварная: на положительных данных врёт минимум, на отрицательных — максимум, а на смешанных может случайно «повезти». Всегда стартуй с `a[0]`, а не с 0 (и не с «огромного числа наугад» — это тоже хрупко).

Минимум/максимум по пустому массиву. Если `n` равно 0, то `a[0]` не существует, и старт `min = a[0]` уже ошибка. Поэтому мы заранее отсекаем `n <= 0`.

Выход за границу при сдвиге. `a[i + 1]` в цикле до `i < n` лезет в `a[n]`. Останавливай цикл на `i < n - 1`. То же правило — при любом обращении к «соседу справа».

`a[n]` вместо `a[n - 1]`. Последний элемент — `n - 1`. Постоянный источник ошибок и в развороте (`j = n - 1`), и в сдвиге.

`=` вместо `==` при подсчёте. `if (a[i] = target)` присвоит элементу `target` (испортит массив!) и всегда сработает. Нужно `==`. Компилятор с `-Wall` предупреждает — не игнорируй предупреждения.

Среднее нацело. `double avg = sum / n;` посчитает нацело **до** записи в `double`. Приводи: `(double)sum / n`.

Обмен без временной переменной. `a[i] = a[j]; a[j] = a[i];` теряет одно из значений. Нужна `t`.

Упражнения

1. Добавь в `06-minmax.c` вывод **позиции** (индекса) минимума и максимума, а не только их значений. Подсказка: заведи `imin`, `imax` и обновляй их рядом с `min`, `max`.
2. Посчитай минимум и максимум за один проход, но **без отдельной переменной `sum`** — а среднее всё равно выведи. (Накапливай сумму в том же цикле — это и так делается, просто убедись, что понимаешь, что все три задачи живут в одном цикле.)
3. Сделай разворот **функцией** `void reverse(int *a, int n)` и вызови её из `main`. Вспомни из главы 3: имя массива — это адрес начала, поэтому функция изменит настоящий массив.
4. Циклический сдвиг **вправо** на 1: последний элемент уходит в начало, остальные сдвигаются вправо. Подумай, в какую сторону теперь должен идти цикл, чтобы не затереть нужное значение раньше времени.
5. Сдвиг влево не на 1, а на `k` позиций. Простой способ — повторить сдвиг на 1 ровно `k` раз в цикле.
6. Перепиши любую из программ так, чтобы массив выделялся через `malloc(n * sizeof(int))` (глава 3) под точное `n`, без потолка в 100 элементов. Не забудь `free` в конце.

Что мы получили

Теперь ты умеешь главные «однопроходные» приёмы над массивом: накопить сумму, удержать рекорд (мин/макс) с правильным стартом от `a[0]`, переставить элементы на месте двумя индексами навстречу и аккуратно сдвинуть массив, не вылезая за его границу. Это не просто упражнения — накопление и «бегунок навстречу» лежат в основе сортировок и поиска, которыми мы займёмся дальше.

В следующей главе возьмёмся за **поиск**: как найти нужный элемент в массиве. Сначала линейный поиск (тот же проход циклом, что и здесь), потом — двоичный, который находит число в отсортированном массиве за считанные шаги даже среди миллионов элементов.

Итоговые программы главы — `code/06-minmax.c`, `code/06-reverse.c` и `code/06-count-rotate.c`.

Глава 7. Поиск: линейный и бинарный

Самая частая работа с массивом — что-то в нём **найти**. Есть ли число 15 в списке? На каком оно месте? В этой главе мы решим эту задачу двумя способами. Сначала в лоб — пройдём весь массив подряд (линейный поиск). Потом — хитро, вдвое сужая зону поиска на каждом шаге (бинарный поиск). Заодно увидим, почему второй способ на больших данных в сотни раз быстрее, и какую цену за это приходится платить.

Что мы сделаем

Напишем две функции поиска. Обе получают массив, его длину и искомое число, а возвращают **индекс** найденного элемента или `-1`, если его нет. Линейный поиск работает на любом массиве. Бинарный — только на **отсортированном**, зато молниеносно. К концу главы ты будешь понимать, когда какой брать и почему.

Разбираем задачу руками

Представь список номеров шкафчиков, выданных на руки:

```
индекс:  0   1   2   3   4   5   6
число:  17   4  42   8  15  23  16
```

Вопрос: на каком месте стоит число 15? Глазами ты просто пробегаешь слева направо: 17 — не то, 4 — не то, 42 — не то, 8 — не то, 15 — есть! Это позиция (индекс) 4. Именно так и работает линейный поиск: проверяй элементы по очереди, пока не наткнёшься на нужный.

А если числа нет — например, ищем 99? Тогда придётся дойти до самого конца, проверить все семь штук и только тогда сказать «нету». Запомни этот момент: когда элемента нет, линейный поиск всегда проверяет **весь** массив.

Шаг 1. Наивный линейный поиск прямо в main

Сначала без функции, чтобы увидеть саму идею. Пройдём массив циклом и, как только встретим нужное число, напечатаем его позицию.

```c
#include <stdio.h>

int main(void)
{
    int a[] = {17, 4, 42, 8, 15, 23, 16};
    int n = (int)(sizeof a / sizeof a[0]);
    int target = 15;

    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            printf("Найдено на позиции %d\n", i);
        }
    }
    return 0;
}
```

Здесь почти всё знакомо, но есть одна новая полезная конструкция.

- `int a[] = {17, 4, ...};` — массив без явного размера в скобках. Компилятор сам посчитает, что элементов семь (см. главу 3).
- `sizeof a / sizeof a[0]` — так узнают **длину массива**. `sizeof a` — размер всего массива в байтах (7 чисел по 4 байта = 28). `sizeof a[0]` — размер одного элемента (4 байта). Поделили — получили **число элементов**: 28 / 4 = 7. Это надёжнее, чем писать 7 руками: поменяешь список — длина пересчитается сама.
- `(int)(...)` — `sizeof` выдаёт значение специального беззнакового типа `size_t`. Чтобы хранить длину в обычном `int` без предупреждений компилятора, мы приводим результат к `int` (приведение типа из главы 3).
- `target` — англ. «цель», искомое число.
- Внутри цикла сравниваем каждый `a[i]` с `target` через `==` и при совпадении печатаем индекс.

Собираем и запускаем:

```
Найдено на позиции 4
```

Работает. Но у этой наивной версии есть три проблемы, и сейчас мы их вытащим на свет.

Шаг 2. Что не так с наивной версией

Проблема 1: цикл не останавливается. Мы нашли число на позиции 4, но цикл всё равно добегает до конца массива. На семи элементах это пустяк, а на миллионе — впустую потраченное время. Нашёл — выходи.

Проблема 2: нет ответа «не найдено». Поищем число 99 — программа просто промолчит, ничего не напечатает. Это плохо: «ничего не вывели» и «не нашли» — разные вещи, но выглядят одинаково.

Проблема 3: поиск зашит в `main`. Захочешь искать в другом массиве — придётся копировать весь цикл. Логику поиска надо вынести в **функцию**, чтобы вызывать её сколько угодно раз.

Чинить будем все три разом: завернём поиск в функцию, которая **возвращает** индекс найденного элемента, а если не нашла — возвращает `-1`. Почему именно `-1`? Потому что настоящий индекс всегда `0` или больше. Значит, `-1` не спутать с настоящей позицией — это удобный знак «такого нет».

Шаг 3. Линейный поиск как функция

```c
int linear_search(const int a[], int n, int target)
{
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;       /* нашли — сразу отдаём индекс */
        }
    }
    return -1;              /* дошли до конца и не встретили target */
}
```

Разбор сигнатуры `int linear_search(const int a[], int n, int target)`:

- первый `int` — тип возвращаемого значения. Функция вернёт индекс (целое число) или `-1`, тоже целое.
- `linear_search` — имя, придумали сами. От англ. *linear* — «линейный», то есть идём по прямой, элемент за элементом.
- `const int a[]` — параметр-массив. Когда передаёшь массив в функцию, на самом деле передаётся **адрес его начала** (помнишь из главы 3: имя массива превращается в адрес). Запись `int a[]` и означает «сюда придёт массив целых». А слово **`const`** — обещание: «внутри функции я этот массив **менять не буду**, только читаю». Это новое для нас. `const` (от *constant* — постоянный) защищает данные: если ты случайно напишешь `a[i] = 5`,

компилятор остановит тебя с ошибкой. Для поиска это в самый раз — искать значит читать, а не портить.

- `int n` — сколько элементов в массиве. Массив сам по себе **не знает** свою длину (внутри функции трюк с `sizeof` уже не сработает — там `a` это адрес, а не весь массив), поэтому длину передают отдельным параметром. Это стандартный приём в Си.
- `int target` — что ищем.

Теперь самое важное — **`return` внутри цикла**:

- `return i;` — как только нашли совпадение, функция немедленно заканчивается и **отдаёт наружу** `i`. Цикл при этом обрывается на месте — `return` выходит сразу из всей функции, а не только из цикла. Это решает проблему 1: лишних шагов нет.
- `return -1;` — эта строка стоит **после** цикла. До неё дойдёт выполнение, только если цикл отработал целиком и ни разу не сделал `return i`. То есть мы проверили все элементы и не нашли. Значит — `-1`. Это решает проблему 2.

Полная программа: линейный поиск

Файл `code/07-linear.c`:

```c
#include <stdio.h>

/* Линейный поиск: ищем значение target в первых n элементах массива a.
   Возвращаем индекс первого совпадения или -1, если ничего не нашли. */
int linear_search(const int a[], int n, int target)
{
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;       /* нашли — сразу отдаём индекс */
        }
    }
    return -1;              /* дошли до конца и не встретили target */
}

int main(void)
{
    int a[] = {17, 4, 42, 8, 15, 23, 16};
    int n = (int)(sizeof a / sizeof a[0]);

    int target = 15;
```

```
    int index = linear_search(a, n, target);

    if (index != -1) {
        printf("Число %d найдено на позиции %d\n", target, index);
    } else {
        printf("Число %d в массиве не найдено\n", target);
    }

    int missing = 99;
    int index2 = linear_search(a, n, missing);
    if (index2 != -1) {
        printf("Число %d найдено на позиции %d\n", missing, index2);
    } else {
        printf("Число %d в массиве не найдено\n", missing);
    }

    return 0;
}
```

В `main` мы вызываем функцию и **сохраняем** её ответ в переменную `index`. Дальше одной проверкой `if (index != -1)` решаем, что печатать: позицию или «не найдено». Так результат функции используется по-взрослому — сначала получили число, потом разобрались, что оно значит.

Запуск:

```
Число 15 найдено на позиции 4
Число 99 в массиве не найдено
```

Оба случая обработаны: и когда число есть, и когда его нет.

Линейный поиск медленный — и вот почему

Линейный поиск прост и работает на **любом** массиве, хоть как перемешанном. Но у него есть слабое место. Прикинем, сколько сравнений он делает в **худшем случае** — когда искомого числа нет или оно стоит последним. Тогда придётся проверить **все** `n` элементов.

- массив из 10 элементов — до 10 проверок;
- из 1000 — до 1000 проверок;
- из 1 000 000 — до миллиона проверок.

Сколько элементов — столько и работы. Растёт массив в тысячу раз — растёт и время поиска в тысячу раз. Для небольших списков это незаметно. Но представь телефонный справочник на миллионы записей, где ищут постоянно — перебирать всё подряд каждый раз слишком расточительно.

Хорошая новость: если массив **отсортирован**, искать можно несравнимо быстрее. Этим и займёмся.

Шаг 4. Идея бинарного поиска — руками

Ты наверняка так искал слово в бумажном словаре. Не листаешь с первой страницы подряд! Открываешь примерно посередине, смотришь, в какую половину попадает твоё слово, и **отбрасываешь вторую половину целиком**. Потом повторяешь с оставшейся половиной. Каждый раз книга «худеет» вдвое — и нужная страница находится за считанные шаги.

Это и есть **бинарный поиск** (binary — «двоичный», «на две части»). Но у него жёсткое условие: он работает **только на отсортированном массиве**. Иначе «отбросить половину» нельзя — без порядка непонятно, в какой стороне искать.

Возьмём отсортированный по возрастанию массив и поищем в нём число 7:

```
индекс:  0   1   2   3   4   5
число:   1   3   5   7   9  11
```

Заведём две границы зоны поиска: `low` (левая, начало) и `high` (правая, конец). В начале зона — весь массив: `low = 0`, `high = 5`.

Шаг A. Середина: `mid = (0 + 5) / 2 = 2`. В середине стоит `a[2] = 5`. Это **меньше** семёрки. Раз массив отсортирован, всё, что левее середины, тоже меньше 7 — значит, левая половина и сама середина нам не нужны. Сдвигаем левую границу за середину: `low = mid + 1 = 3`. Зона сузилась до индексов 3..5.

Шаг B. Середина новой зоны: `mid = (3 + 5) / 2 = 4`. Там `a[4] = 9`. Это **больше** семёрки. Значит, всё, что правее (и сама девятка), нам не подходит. Сдвигаем правую границу: `high = mid − 1 = 3`. Зона — только индекс 3.

Шаг C. `mid = (3 + 3) / 2 = 3`. Там `a[3] = 7`. **Совпало!** Ответ — позиция 3.

Три шага вместо потенциальных шести. На маленьком массиве выигрыш скромный, но он растёт стремительно с размером — скоро увидим, насколько.

Шаг 5. Почему `mid = low + (high - low) / 2`, а не `(low + high) / 2`

Казалось бы, середину проще считать как `(low + high) / 2` — и математически это то же самое. Но в реальном коде есть ловушка.

`low` и `high` — переменные типа `int`. У `int` есть **потолок**: примерно 2 миллиарда (об этом упоминали в главе 2). Если массив огромный и обе границы заехали близко к этому потолку, их **сумма** `low + high` может его **перепрыгнуть** — это называется **переполнением** (overflow). При переполнении число молча портится: вместо большого положительного может получиться отрицательное, и `mid` укажет в никуда. Баг, который вылезет только на больших данных, — самый противный.

Запись `low + (high - low) / 2` даёт **то же самое** значение середины, но устроена безопасно:

- `high - low` — это ширина зоны, она никогда не больше самого массива, переполниться не может;
- делим её пополам — `(high - low) / 2`, это половина ширины;
- прибавляем к `low` — получаем точку ровно посередине между границами.

Сумма двух больших чисел нигде не возникает, потолок не пробивается. Привыкай писать середину именно так — это стандартная защита, её ждут от тебя в любом нормальном коде.

Полная программа: бинарный поиск

Файл `code/07-binary.c`:

```c
#include <stdio.h>

/* Бинарный поиск. ВАЖНО: массив a ДОЛЖЕН быть отсортирован по
   возрастанию. Ищем target, возвращаем его индекс или -1. */
int binary_search(const int a[], int n, int target)
{
    int low = 0;            /* левая граница диапазона поиска */
    int high = n - 1;       /* правая граница диапазона поиска */

    while (low <= high) {
        int mid = low + (high - low) / 2;   /* середина без переполнения */

        if (a[mid] == target) {
            return mid;                     /* попали точно в середину */
        } else if (a[mid] < target) {
```

```
            low = mid + 1;                  /* target правее середины */
        } else {
            high = mid - 1;                 /* target левее середины */
        }
    }
    return -1;                              /* диапазон схлопнулся — нет */
}

int main(void)
{
    /* Массив уже отсортирован по возрастанию. */
    int a[] = {1, 3, 5, 7, 9, 11};
    int n = (int)(sizeof a / sizeof a[0]);

    int target = 7;
    int index = binary_search(a, n, target);

    if (index != -1) {
        printf("Число %d найдено на позиции %d\n", target, index);
    } else {
        printf("Число %d в массиве не найдено\n", target);
    }

    int missing = 8;
    int index2 = binary_search(a, n, missing);
    if (index2 != -1) {
        printf("Число %d найдено на позиции %d\n", missing, index2);
    } else {
        printf("Число %d в массиве не найдено\n", missing);
    }

    return 0;
}
```

Разбор тела функции:

- `int low = 0;` и `int high = n - 1;` — границы зоны поиска. В начале она охватывает весь массив: от первого индекса `0` до последнего `n - 1`. (Последний индекс именно `n - 1`, а не `n` — нумерация с нуля, см. главу 3.)

- `while (low <= high)` — повторяем, **пока зона не пуста**. Условие `low <= high` означает «в зоне ещё есть хотя бы один элемент». Как только границы перехлестнутся (`low` станет больше `high`), искать негде — выходим. Знак `<=`, а не `<`, важен: при `low == high` в зоне остался ровно один элемент, и его тоже надо проверить.
- `int mid = low + (high - low) / 2;` — середина зоны, посчитанная безопасно (только что разобрали почему).
- `if (a[mid] == target) return mid;` — повезло, в середине ровно то, что ищем. Отдаём индекс `mid`.
- `else if (a[mid] < target)` — в середине число **меньше** искомого. Раз массив отсортирован по возрастанию, искомое (если оно есть) лежит **правее**. Отрезаем левую половину: `low = mid + 1`. Прибавляем 1, потому что саму середину уже проверили — повторять её незачем.
- `else` — остаётся случай «в середине число **больше** искомого». Тогда искомое лежит **левее**. Отрезаем правую половину: `high = mid - 1`.
- `return -1;` — если цикл завершился сам (границы сошлись), числа нет.

Обрати внимание: `mid + 1` и `mid - 1` — не прихоть. Если бы мы писали `low = mid` (без +1), то в зоне из двух элементов середина не двигалась бы, и цикл крутился бы **вечно**. Сдвиг через середину гарантирует, что зона на каждом шаге реально уменьшается.

Запуск:

```
Число 7 найдено на позиции 3
Число 8 в массиве не найдено
```

Семёрку нашли на позиции 3 — ровно как считали руками. А восьмёрки в массиве нет (там только нечётные), и функция честно вернула `-1`.

Как это работает: трассировка по шагам

Пройдём `binary_search(a, 6, 7)` по строчкам, выписывая значения границ. Массив: `1 3 5 7 9 11`, индексы `0..5`.

```
Старт:   low=0, high=5
Шаг 1:   mid = 0 + (5-0)/2 = 2   a[2]=5   5 < 7  -> low = 3
Шаг 2:   mid = 3 + (5-3)/2 = 4   a[4]=9   9 > 7  -> high = 3
Шаг 3:   mid = 3 + (3-3)/2 = 3   a[3]=7   7 == 7 -> return 3
```

Зона поиска ужималась так: сначала весь массив (6 элементов), потом индексы 3..5 (3 элемента), потом только индекс 3 (1 элемент) — и попадание. Каждый шаг выбрасывал примерно половину оставшихся вариантов.

А теперь трассировка для отсутствующего числа `8`:

```
Старт:   low=0, high=5
Шаг 1:   mid = 2   a[2]=5   5 < 8  -> low = 3
Шаг 2:   mid = 4   a[4]=9   9 > 8  -> high = 3
Шаг 3:   mid = 3   a[3]=7   7 < 8  -> low = 4
Теперь low=4, high=3  ->  low > high  ->  цикл кончился  ->  return -1
```

Границы перехлестнулись (`low` обогнал `high`) — это и есть сигнал «зона пуста, числа нет».

Во сколько раз бинарный быстрее

Линейный поиск в худшем случае делает `n` сравнений. Бинарный на каждом шаге **делит** зону пополам, поэтому число шагов — это «сколько раз нужно поделить `n` на 2, чтобы дойти до единицы». В математике это **двоичный логарифм**, `log2(n)`. Считать его не обязательно, важно почувствовать разницу на числах:

```
размер массива n   линейный (до n)   бинарный (~log2 n)
              10               10                   4
             100              100                   7
           1 000            1 000                  10
       1 000 000        1 000 000                  20
   1 000 000 000    1 000 000 000                  30
```

В массиве из тысячи элементов линейный поиск может сделать до 1000 проверок, а бинарный — всего около 10. Это в сотню раз меньше работы. А на миллиарде элементов разница уже в десятки миллионов раз: 30 шагов против миллиарда. Вот почему отсортированные данные так ценят: по ним можно искать почти мгновенно.

Удвой массив — линейному добавится столько же работы, сколько было. Бинарному добавится всего **один** шаг. В этом вся магия деления пополам.

Проверим на секундомере

Таблица — это теория. Замерим настоящим временем: возьмём массив на миллион отсортированных чисел и по 10 000 раз поищем в нём последний элемент (худший случай для линейного поиска). Секундомер — знакомый `clock()` из `<time.h>`:

```c
int n = 1000000;
int *a = malloc((size_t)n * sizeof(int));
for (int i = 0; i < n; i++) {
    a[i] = i * 2;                    /* уже отсортирован */
```

```
}
int queries = 10000;

clock_t t0 = clock();
for (int q = 0; q < queries; q++) {
    linear_search(a, n, (n - 1) * 2);
}
double lin = (double)(clock() - t0) / CLOCKS_PER_SEC;

t0 = clock();
for (int q = 0; q < queries; q++) {
    binary_search(a, n, (n - 1) * 2);
}
double bin = (double)(clock() - t0) / CLOCKS_PER_SEC;

printf("линейный: %.4f с\n", lin);
printf("бинарный: %.4f с\n", bin);
printf("разница:  в %.0f раз\n", lin / bin);
```

Полная программа лежит в `code/07-bench.c`. Результат на моей машине:

```
массив: 1000000 элементов, 10000 поисков последнего элемента
линейный: 3.6251 с
бинарный: 0.0002 с
разница:  в 16861 раз
(оба нашли индекс 999999 и 999999)
```

Три с половиной секунды против двух десятых миллисекунды. Абсолютные числа у тебя будут свои, и «во сколько раз» тоже прыгает от запуска к запуску (бинарный так быстр, что его время почти теряется в погрешности) — но порядок величины останется тем же: тысячи раз.

Обрати внимание: сам массив мы отсортированным **сделали** — просто заполнили по возрастанию. В реальной задаче за сортировку ещё нужно заплатить, и об этой цене — следующая глава.

Проверяем

Собери и запусти обе программы:

```
cc -std=c11 -Wall -Wextra -O2 07-linear.c -o 07-linear
./07-linear
Число 15 найдено на позиции 4
Число 99 в массиве не найдено

cc -std=c11 -Wall -Wextra -O2 07-binary.c -o 07-binary
./07-binary
Число 7 найдено на позиции 3
Число 8 в массиве не найдено
```

Ни одного предупреждения от компилятора, вывод совпал с ожидаемым — значит, обе функции работают.

Частые ошибки

Бинарный поиск по неотсортированному массиву. Это ошибка номер один. Если массив не отсортирован, «отбросить половину» бессмысленно — функция вернёт `-1` или вообще не тот индекс, хотя число в массиве есть. Бинарный поиск **молча** даёт неверный ответ на беспорядочных данных, не падает и не ругается. Прежде чем искать бинарно — убедись, что массив отсортирован.

Бесконечный цикл из-за неверного сдвига границ. Если написать `low = mid` вместо `low = mid + 1` (или `high = mid` вместо `high = mid - 1`), зона может перестать уменьшаться, и `while` закрутится навсегда. Всегда двигай границу **за** середину: `mid + 1` или `mid - 1`. Если программа зависла — Ctrl+C, и проверь эти две строки.

`<` вместо `<=` в условии цикла. С `while (low < high)` ты пропустишь случай, когда в зоне остался последний единственный элемент (`low == high`), и не найдёшь число, стоящее именно там. Нужно `while (low <= high)`.

Середина через `(low + high) / 2`. Работает на маленьких массивах, но на больших может переполнить `int`. Пиши `low + (high - low) / 2`.

Перепутаны направления. Если массив отсортирован по возрастанию и `a[mid] < target`, цель **правее** — двигаем `low`. Перепутаешь `low` и `high` местами — поиск будет уходить не туда и ничего не найдёт.

Линейный поиск без выхода (`return` в цикле). Если забыть `return i` и просто печатать совпадения, цикл прогонит весь массив зря, а функция не сможет вернуть индекс. Нашёл — возвращай сразу.

Упражнения

1. Добавь в линейный поиск **счётчик сравнений**: пусть функция через указатель-параметр (вспомни `&` и `*` из главы 3) сообщает в `main`, сколько раз сравнила элементы. Поищи число, стоящее последним, и убедись, что счётчик равен длине массива.
2. Сделай то же для бинарного поиска и сравни счётчики на одном и том же отсортированном массиве из 15 элементов. Во сколько раз меньше шагов?
3. Напиши `int count_occurrences(const int a[], int n, int target)` — функцию, которая считает, **сколько раз** число встречается в массиве (линейным проходом, без `return` на первом совпадении).
4. Измени бинарный поиск так, чтобы он искал в массиве, отсортированном **по убыванию** (например, `11 9 7 5 3 1`). Что нужно поменять в двух ветках со сдвигом границ?
5. Дай пользователю ввести искомое число с клавиатуры (`scanf`), а потом найди его бинарным поиском в захардкоженном отсортированном массиве. Не забудь проверить результат `scanf` (см. главу 6).
6. Подумай и проверь на бумаге: сколько шагов сделает бинарный поиск на массиве из 16 элементов в худшем случае? А из 17? Совпало ли с твоей догадкой про `log2`?

Что мы получили

Теперь у тебя два инструмента поиска. **Линейный** — простой и всеядный: работает на любом массиве, но в худшем случае проверяет все `n` элементов. **Бинарный** — стремительный (`log2 n` шагов вместо `n`), но требует **отсортированного** массива и аккуратной работы с границами `low`/`high`/`mid`. Ты увидел, как функция возвращает индекс или `-1`, почему середину считают через `low + (high - low) / 2`, и насколько бешеной становится разница в скорости на больших данных.

И тут всплывает честный вопрос: бинарный поиск великолепен, но он держится на одном условии — массив должен быть **отсортирован**. А что, если данные пришли вперемешку? Значит, перед поиском их надо упорядочить. Как расставить элементы по возрастанию — большая и интересная тема. Ею мы и займёмся в следующей главе: разберём **сортировки**, после чего бинарный поиск станет применим к любым данным, а не только к заранее аккуратным спискам.

Глава 8. Сортировки: пузырёк, выбором, вставками

Отсортировать — значит расставить числа по порядку: от меньшего к большему. Звучит просто, но именно на сортировках удобно почувствовать, что такое **алгоритм**: одну и ту же задачу можно решить по-разному, и способы отличаются скоростью и идеей.

В этой главе мы напишем три классические сортировки **с нуля**: пузырёк, выбором и вставками. Для каждой сначала разложим всё руками на маленьком массиве `[5, 2, 4, 1]`, потом превратим в код, потом запустим. В конце поговорим, почему эти три — медленные на больших данных, какие есть быстрые (быстрая сортировка и слияние) и как пользоваться готовой `qsort` из стандартной библиотеки.

Что мы сделаем

К концу главы у тебя будут три рабочие программы-сортировки и понимание, чем они отличаются. Ты научишься:

- расставлять элементы массива по возрастанию;
- видеть, как массив меняется проход за проходом;
- оценивать, быстрый алгоритм или медленный;
- вызывать стандартную `qsort` со своей функцией сравнения.

Всё, что нужно, ты уже знаешь из глав 1–7: массивы, циклы, обмен двух переменных через временную (swap), функции. Новым будет только устройство самих алгоритмов.

Общая заготовка: печать массива

Во всех трёх программах нам надо смотреть на массив до и после. Чтобы не повторять цикл печати, вынесем его в маленькую функцию:

```c
void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}
```

Разбор:

- `void` — функция ничего не возвращает, она просто печатает.
- `const int a[]` — принимаем массив целых. Слово `const` (от *constant*, «постоянный») — обещание: «внутри этой функции я массив **не меняю**, только читаю». Если случайно попробуешь записать в него, компилятор укажет на ошибку. Печати менять нечего — значит, `const` здесь к месту.
- `int n` — сколько элементов печатать. Массив сам не знает свою длину, её всегда передают отдельным числом (см. главу 3).
- внутри — обычный перебор и `printf("%d ", a[i])` с пробелом после каждого числа, в конце один перевод строки.

Эту функцию мы вставим в каждую из трёх программ.

Часть 1. Сортировка пузырьком

Идея руками

Пузырёк сравнивает **соседей** и меняет их местами, если они стоят не по порядку. Идём слева направо по всему массиву. За один такой проход самый большой элемент «всплывает» в самый конец — как пузырёк воздуха поднимается в воде. Отсюда название.

Возьмём `[5, 2, 4, 1]` и сделаем **первый проход**. Сравниваем пары соседей слева направо:

```
[5 2 4 1]  5 > 2 ? да  -> меняем -> [2 5 4 1]
[2 5 4 1]  5 > 4 ? да  -> меняем -> [2 4 5 1]
[2 4 5 1]  5 > 1 ? да  -> меняем -> [2 4 1 5]
```

После первого прохода: `[2 4 1 5]`. Заметь — **5 уехала в самый конец** и больше двигаться не будет. Это и есть всплывший максимум.

Второй проход (последний элемент уже на месте, его не трогаем):

```
[2 4 1 5]  2 > 4 ? нет
[2 4 1 5]  4 > 1 ? да  -> меняем -> [2 1 4 5]
```

После второго прохода: `[2 1 4 5]`. Теперь на местах два хвостовых элемента — `4` и `5`.

Третий проход:

```
[2 1 4 5]  2 > 1 ? да  -> меняем -> [1 2 4 5]
```

Получили `[1 2 4 5]` — отсортировано. Каждый следующий проход можно делать чуть короче: хвост уже разобран.

Шаг 1. Наивная версия

Запишем идею в лоб: внешний цикл повторяет проходы, внутренний идёт по соседям и меняет их местами.

```
void bubble_sort(int a[], int n)
{
    for (int pass = 0; pass < n - 1; pass++) {
        for (int i = 0; i < n - 1; i++) {
            if (a[i] > a[i + 1]) {
                int t = a[i];
                a[i] = a[i + 1];
                a[i + 1] = t;
            }
        }
    }
}
```

Разбор:

- `int a[]` — здесь **без** `const`: эту функцию массив именно меняет.
- внешний `for (pass ...)` — счётчик проходов. Проходов хватает `n - 1`: за каждый проход как минимум один элемент встаёт на место, а последний оставшийся окажется правильным сам собой.
- внутренний `for (i ...)` сравнивает соседей `a[i]` и `a[i + 1]`. Условие `i < n - 1`, а не `i < n`: иначе `a[i + 1]` при `i = n - 1` залезет за границу массива (см. главу 3, выход за границы — частый баг).
- `if (a[i] > a[i + 1])` — соседи стоят «неправильно» (левый больше правого), надо поменять.
- три строки с `t` — это **swap** через временную переменную, ровно как в главе 6: запомнили левый в `t`, на его место положили правый, а правому отдали `t`.

Эта версия уже сортирует правильно. Но в ней есть слабое место.

Шаг 2. Проблема: лишняя работа

Внутренний цикл всегда доходит до конца (`i < n - 1`), хотя хвост массива после каждого прохода уже отсортирован. И ещё хуже: даже если массив **уже** упорядочен, мы всё равно сделаем все `n - 1` проходов впустую.

Дай этой версии почти отсортированный `[1, 2, 3, 5, 4]` — она переставит `5` и `4` за первый же проход, а потом сделает ещё кучу проходов, ничего не меняя. Обидно.

Шаг 3. Фикс: укорачиваем хвост и ставим флаг

Две доработки:

1. Внутренний цикл доводим только до `n - 1 - pass`: всё, что правее, уже всплыло на свои места, проверять там нечего.
2. Заводим **флаг** `swapped`. Перед проходом ставим его в 0. Если за проход был хоть один обмен — ставим 1. Если после прохода флаг так и остался 0, значит, все соседи уже стояли по порядку — массив отсортирован, и можно выйти досрочно через `break`.

```c
void bubble_sort(int a[], int n)
{
    for (int pass = 0; pass < n - 1; pass++) {
        int swapped = 0;
        for (int i = 0; i < n - 1 - pass; i++) {
            if (a[i] > a[i + 1]) {
                int t = a[i];
                a[i] = a[i + 1];
                a[i + 1] = t;
                swapped = 1;
            }
        }
        if (swapped == 0) {
            break;
        }
    }
}
```

Разбор добавленного:

- `int swapped = 0;` — флаг «были ли обмены». 0 — «нет», 1 — «да». Это обычное использование числа как «да/нет»: помнишь из главы 2, ложь это 0, истина это любое ненулевое.

- `i < n - 1 - pass` — на проходе номер `pass` последние `pass` элементов уже на местах, до них не идём.
- `swapped = 1;` — отмечаем, что обмен случился.
- `if (swapped == 0) break;` — за весь проход ничего не поменяли, значит, дальше делать нечего. `break` немедленно прекращает внешний цикл.

На уже отсортированном массиве теперь хватит **одного** прохода: он пройдёт без обменов, флаг останется 0, и мы выйдем.

Полная программа

`code/08-bubble.c`:

```c
#include <stdio.h>

/* Печать массива в одну строку — пригодится во всех трёх программах. */
void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Сортировка пузырьком по возрастанию.
   За каждый проход самый большой из оставшихся «всплывает» вправо.
   Оптимизация: если за проход не было ни одного обмена — массив
   уже отсортирован, выходим досрочно. */
void bubble_sort(int a[], int n)
{
    for (int pass = 0; pass < n - 1; pass++) {
        int swapped = 0;              /* были ли обмены в этом проходе */

        /* На каждом следующем проходе хвост уже отсортирован,
           поэтому не доходим до конца: -1 за i+1, -pass за хвост. */
        for (int i = 0; i < n - 1 - pass; i++) {
            if (a[i] > a[i + 1]) {    /* соседи стоят не по порядку */
                int t = a[i];         /* классический swap через temp */
                a[i] = a[i + 1];
                a[i + 1] = t;
```

```
                swapped = 1;
            }
        }

        if (swapped == 0) {            /* за проход ничего не меняли */
            break;                     /* значит, всё уже на местах */
        }
    }
}

int main(void)
{
    int a[] = {5, 2, 4, 1};
    int n = (int)(sizeof a / sizeof a[0]);

    printf("До:    ");
    print_array(a, n);

    bubble_sort(a, n);

    printf("После: ");
    print_array(a, n);
    return 0;
}
```

Про `int n = (int)(sizeof a / sizeof a[0]);` — это привычный приём из главы 6: `sizeof a` — размер всего массива в байтах, `sizeof a[0]` — размер одного элемента; их частное даёт число элементов. Работает только там, где `a` — настоящий массив, а не пришедший в функцию указатель.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-bubble.c -o bubble
$ ./bubble
До:    5 2 4 1
После: 1 2 4 5
```

Совпадает с тем, что мы получили руками.

Часть 2. Сортировка выбором

Идея руками

Здесь логика другая. Мысленно делим массив на две части: слева — уже **отсортированная** (поначалу пустая), справа — ещё нет. На каждом шаге находим **минимум** во всей правой части и ставим его в самое её начало, меняя местами с тем, что там стояло. Отсортированная часть растёт на один элемент за шаг.

`[5, 2, 4, 1]`, отсортированная часть отмечена `|`:

```
| 5 2 4 1   минимум справа = 1 (поз. 3), меняем с поз. 0
1 | 2 4 5   минимум справа = 2 (поз. 1), уже на месте
1 2 | 4 5   минимум справа = 4 (поз. 2), уже на месте
1 2 4 | 5   остался один элемент — он на месте
```

Готово: `[1 2 4 5]`. Обрати внимание: обменов делается мало (максимум по одному на шаг), но сравнений много — каждый раз заново просматриваем весь остаток.

Код

Наивной и «починенной» версий тут не будет — алгоритм сразу пишется понятно. Важно лишь не менять элемент сам с собой, если минимум и так на месте.

`code/08-selection.c`:

```c
#include <stdio.h>

void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Сортировка выбором по возрастанию.
   Делим массив на две части: слева — уже отсортированная,
   справа — ещё нет. На каждом шаге находим минимум в правой части
   и ставим его в начало этой части. */
void selection_sort(int a[], int n)
{
```

```c
    for (int i = 0; i < n - 1; i++) {
        int min_pos = i;              /* пока считаем минимумом a[i] */

        /* ищем настоящий минимум среди a[i+1 .. n-1] */
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min_pos]) {
                min_pos = j;          /* запомнили позицию меньшего */
            }
        }

        if (min_pos != i) {           /* минимум не на своём месте */
            int t = a[i];             /* меняем его с началом части */
            a[i] = a[min_pos];
            a[min_pos] = t;
        }
    }
}

int main(void)
{
    int a[] = {5, 2, 4, 1};
    int n = (int)(sizeof a / sizeof a[0]);

    printf("До:    ");
    print_array(a, n);

    selection_sort(a, n);

    printf("После: ");
    print_array(a, n);
    return 0;
}
```

Разбор `selection_sort`:

- внешний `for (i ...)` — `i` это граница: позиция, куда сейчас поставим очередной минимум. Хватает до `n - 1`: последний элемент окажется правильным сам.
- `int min_pos = i;` — храним не само минимальное значение, а его **позицию** (индекс). Позиция нужна, чтобы потом сделать обмен.

- внутренний `for (j = i + 1; ...)` просматривает неотсортированный остаток, начиная сразу за `i`. Если нашли элемент меньше текущего минимума — обновляем `min_pos`.
- `if (min_pos != i)` — если минимум уже стоит на позиции `i`, обмен не нужен (это не баг, а просто экономия). Иначе — обычный swap через `t`.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-selection.c -o selection
$ ./selection
До:    5 2 4 1
После: 1 2 4 5
```

Часть 3. Сортировка вставками

Идея руками

Лучшая аналогия — как ты держишь карты в руке. Берёшь карты по одной из стопки и **вставляешь** каждую новую на правильное место среди уже разобранных, сдвигая остальные. Слева у тебя всегда упорядоченный веер, справа — то, что ещё не разобрал.

В массиве то же самое. Считаем, что первый элемент уже «отсортирован» (один элемент всегда упорядочен). Берём следующий и продвигаем его влево, пока слева есть элементы больше него.

`[5, 2, 4, 1]`. Отсортированную левую часть отметим `|`:

```
5 | 2 4 1   берём 2; 5 > 2 -> сдвигаем 5 вправо, 2 встаёт в начало
2 5 | 4 1   берём 4; 5 > 4 -> сдвигаем 5; 2 < 4 -> стоп, 4 за 2
2 4 5 | 1   берём 1; 5,4,2 все > 1 -> сдвигаем все, 1 в начало
1 2 4 5 |   разобрали все карты
```

Получили `[1 2 4 5]`. Главное действие — **сдвиг**: чтобы освободить место для вставляемого элемента, большие соседи едут на одну позицию вправо.

Шаг 1. Наивная мысль и сразу аккуратная версия

Новичок часто пытается «искать место, потом вставлять» в два прохода. Но сдвиг и поиск места удобно совместить в одном цикле `while`: пока слева элемент больше вставляемого — двигаем его вправо. Где цикл остановился, там и место.

Есть тонкость: вставляемый элемент надо **сначала запомнить** в отдельную переменную (`key`). Если этого не сделать, сдвиг затрёт его первым же шагом.

code/08-insertion.c:

```c
#include <stdio.h>

void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Сортировка вставками по возрастанию.
   Слева растёт отсортированная часть. Берём очередной элемент key
   и вставляем его в нужное место этой части, сдвигая большие
   элементы вправо. Так раскладывают карты в руке. */
void insertion_sort(int a[], int n)
{
    for (int i = 1; i < n; i++) {
        int key = a[i];                /* элемент, который вставляем */
        int j = i - 1;                 /* идём по отсортированной части */

        /* сдвигаем вправо всё, что больше key */
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;                /* нашлось место — кладём key */
    }
}

int main(void)
{
    int a[] = {5, 2, 4, 1};
    int n = (int)(sizeof a / sizeof a[0]);

    printf("До:    ");
    print_array(a, n);
```

```
    insertion_sort(a, n);

    printf("После: ");
    print_array(a, n);
    return 0;
}
```

Разбор `insertion_sort`:

- внешний `for (i = 1; ...)` начинается с **1**, а не с 0: элемент с индексом 0 считаем уже отсортированной частью из одного элемента.
- `int key = a[i];` — запоминаем вставляемый элемент. Дальше место `a[i]` будет затёрто сдвигом, поэтому значение нужно сохранить заранее.
- `int j = i - 1;` — `j` будет двигаться по отсортированной части справа налево.
- `while (j >= 0 && a[j] > key)` — два условия через `&&` (логическое «И», см. главу 2):
  - `j >= 0` — не вышли за левый край массива;
  - `a[j] > key` — текущий элемент больше вставляемого, значит, его надо подвинуть.

  Порядок важен! Сначала проверяется `j >= 0`. Если `j` стал −1, `&&` дальше не смотрит на `a[j]` (это называется «короткое замыкание»), и обращения к `a[-1]` не происходит. Поменяй условия местами — получишь выход за границу.
- `a[j + 1] = a[j];` — сдвигаем элемент на одну позицию вправо.
- `j--;` — шаг влево.
- после цикла `a[j + 1] = key;` — `j` указывает на первый элемент, не превышающий `key` (или на −1, если `key` самый маленький). Значит, место для `key` — следующая клетка, `j + 1`.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-insertion.c -o insertion
$ ./insertion
До:    5 2 4 1
После: 1 2 4 5
```

Маленький бонус вставок: на почти отсортированных данных она очень быстра — для каждого элемента `while` почти сразу останавливается.

Как это работает: чем три сортировки отличаются

Все три дают один результат, но «характер» у них разный:

- **Пузырёк** много **меняет** соседей и медленно «протаскивает» большие числа в конец. С флагом досрочно ловит уже готовый массив.
- **Выбор** делает мало обменов (по одному на шаг), но всегда честно просматривает весь остаток в поисках минимума — даже если массив уже отсортирован.
- **Вставки** аккуратно «дораскладывают» массив и особенно хороши, когда данные почти упорядочены.

Сложность: почему это медленно и что бывает быстрее

У всех трёх есть общая беда: **два вложенных цикла**. Для массива из `n` элементов внешний цикл идёт примерно `n` раз, и для каждого внешнего шага внутренний делает тоже порядка `n` действий. Итого порядка `n × n`, то есть $\mathbf{n^2}$ операций.

Это называется **сложность $\mathbf{O(n^2)}$** (читают «о большое от эн квадрат») — способ грубо описать, как растёт время работы с ростом `n`. Смысл такой: увеличишь массив в 10 раз — времени уйдёт примерно в 100 раз больше.

Прикинь масштаб:

```
n = 10        ->     ~100 операций      мгновенно
n = 1 000     -> ~1 000 000 операций    ещё терпимо
n = 1 000 000 -> ~10^12 операций        часами
```

На миллионе элементов $O(n^2)$ уже неприемлемо. К счастью, есть сортировки посложнее, но куда быстрее — со сложностью **O(n log n)**. Тут `log n` — это «сколько раз можно поделить `n` пополам, пока не дойдём до единицы» (для миллиона это около 20). То есть вместо `n × n` получается `n × 20` — огромная разница. Идею двух главных таких алгоритмов разберём словами, без полного кода (полностью их соберём позже, когда освоим рекурсию в главе 9).

Быстрая сортировка (quicksort)

Выбираем из массива один элемент — **опорный** (pivot). Переставляем остальные так, чтобы слева от опорного оказались все меньшие, а справа — все большие. Теперь опорный стоит на своём окончательном месте, а массив разбит на две части. Дальше **точно так же** сортируем левую часть и правую — каждую как отдельный массив поменьше. Деление продолжается, пока части не станут совсем крошечными.

За счёт того, что задача каждый раз дробится примерно пополам, в среднем выходит O(n log n). Именно идея «разбить относительно опорного и повторить для половин» делает quicksort одним из самых популярных алгоритмов.

Сортировка слиянием (merge sort)

Делим массив пополам, каждую половину тоже пополам, и так до кусочков по одному элементу (один элемент уже «отсортирован»). Потом начинаем **сливать** кусочки обратно попарно, и при слиянии сразу располагаем элементы по порядку: берём меньший из голов двух кусков, потом следующий меньший, и так пока оба не закончатся. Слияние упорядоченных кусков — дешёвая операция, а уровней деления всего около `log n`. Получаем тоже O(n log n), причём стабильно, без «неудачных случаев». Платим за это дополнительной памятью под временный массив для слияния.

Готовое решение: `qsort` из `<stdlib.h>`

В реальном коде свою сортировку обычно не пишут — берут библиотечную `qsort`. Она универсальная: сортирует массив **чего угодно** (числа, строки, структуры), потому что не знает заранее, что и как сравнивать. Эти знания ты даёшь ей сам — через **функцию сравнения** (компаратор).

`code/08-qsort.c`:

```c
#include <stdio.h>
#include <stdlib.h>     /* здесь живёт qsort */

void print_array(const int a[], int n)
{
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

/* Компаратор: говорит qsort, какой из двух элементов «раньше».
   Получает адреса двух элементов как const void *.
   Возвращает < 0, если первый меньше; 0, если равны; > 0, если больше. */
int cmp_int(const void *pa, const void *pb)
{
    int a = *(const int *)pa;   /* приводим адрес к int* и разыменовываем */
    int b = *(const int *)pb;
    if (a < b) return -1;
    if (a > b) return 1;
    return 0;
```

```c
}

int main(void)
{
    int a[] = {5, 2, 4, 1, 9, 3, 7};
    int n = (int)(sizeof a / sizeof a[0]);

    /* qsort(база, число_элементов, размер_элемента, компаратор) */
    qsort(a, (size_t)n, sizeof a[0], cmp_int);

    print_array(a, n);
    return 0;
}
```

Разбор — здесь несколько новых деталей:

- `qsort(a, (size_t)n, sizeof a[0], cmp_int)` — четыре аргумента:
  - `a` — массив (его адрес начала);
  - `(size_t)n` — сколько элементов сортировать. `size_t` — целочисленный тип «для размеров и количеств», именно его ждёт `qsort`; приводим `n` к нему, чтобы не было предупреждения о несовпадении типов;
  - `sizeof a[0]` — размер одного элемента в байтах. По нему `qsort` понимает, какими «шагами» ходить по массиву;
  - `cmp_int` — имя нашей функции сравнения. Передаём **саму функцию** (без скобок!) — `qsort` будет вызывать её, когда надо сравнить два элемента. Передача функции как аргумента — новая для нас вещь.
- `int cmp_int(const void *pa, const void *pb)` — компаратор обязан иметь именно такую форму, её требует `qsort`:
  - `const void *` — «адрес чего-то, тип заранее неизвестен». `void *` — «указатель на что угодно». `qsort` универсальна и не знает, что внутри, поэтому передаёт элементы безликими адресами;
  - `const` — обещаем не менять элементы при сравнении.
- `*(const int *)pa` — два действия сразу:
  - `(const int *)pa` — **приведение** безликого адреса к «адресу на `int`»: мы-то знаем, что сравниваем целые;
  - `*` перед этим — **разыменование** (см. главу 3): достаём само число по адресу.
- возврат: компаратор должен вернуть **отрицательное** число, если первый элемент должен идти раньше второго, **0** — если они равны, и **положительное** — если позже. По

этим знакам `qsort` и расставляет всё по порядку. Хочешь сортировку по убыванию — просто поменяй местами `a` и `b` в сравнениях.

Запуск:

```
$ cc -std=c11 -Wall -Wextra -O2 code/08-qsort.c -o qsort_demo
$ ./qsort_demo
1 2 3 4 5 7 9
```

Одна строка `qsort(...)` — и массив отсортирован быстрым алгоритмом. Понимать, что у неё внутри, всё равно полезно: иначе компаратор кажется магией.

Частые ошибки

Выход за границу в условии цикла. В пузырьке сравниваем `a[i]` и `a[i + 1]`, поэтому внутренний цикл идёт до `i < n - 1`, а не `i < n`. Иначе при `i = n - 1` обратишься к несуществующему `a[n]`.

Swap без временной переменной. Если написать `a[i] = a[j]; a[j] = a[i];`, ты сначала затрёшь `a[i]`, и оба элемента станут одинаковыми. Нужна третья переменная `t` (см. главу 6).

В выборе хранят значение вместо позиции. Для обмена нужен **индекс** минимума (`min_pos`), а не само минимальное число. По числу непонятно, какой элемент с каким менять.

Во вставках забыли сохранить `key`. Сдвиг `a[j + 1] = a[j]` затрёт вставляемый элемент, если ты не запомнил его заранее в `key`.

Перепутан порядок в `&&` у вставок. Условие должно быть `j >= 0 && a[j] > key`, а не наоборот. Сначала проверка границы, потом обращение к `a[j]`, иначе при `j == -1` залезешь за край.

Компаратор возвращает `a - b` для больших чисел. Частый «короткий» приём `return a - b;` может **переполнить** `int`, если числа очень большие (см. ловушку переполнения в главе 2). Надёжнее три сравнения, как у нас.

Упражнения

1. Добавь в `bubble_sort` счётчик обменов (`long swaps`) и печатай его в конце. Сравни число обменов для `[5, 2, 4, 1]` и для уже отсортированного `[1, 2, 3, 4]`.
2. Сделай так, чтобы все три сортировки печатали массив **после каждого прохода** (для пузырька/выбора) или **после каждой вставки**. Понаблюдай, как массив приходит в порядок.

3. Переделай `selection_sort` на поиск **максимума** и установку его в **конец** неотсортированной части. Результат должен быть тем же.
4. Напиши функцию `int is_sorted(const int a[], int n)`, возвращающую 1, если массив отсортирован по возрастанию, и 0 иначе. Проверяй ею массив до и после сортировки.
5. Заполни массив из 20 чисел вводом (или случайными, если уже умеешь `rand`) и отсортируй его любой из трёх сортировок.
6. Возьми `08-qsort.c` и сделай вторую версию компаратора `cmp_desc`, сортирующую по **убыванию**. Отсортируй один и тот же массив сначала по возрастанию, потом по убыванию.

Что мы получили

Ты написал три сортировки с нуля и понял, чем они отличаются: пузырёк «всплывает» обменами соседей (с флагом для досрочного выхода), выбор ищет минимум и ставит его в начало, вставки раскладывают элементы как карты в руке. Все три — $O(n^2)$, то есть медленные на больших данных. Ты узнал, что есть быстрые O(n log n) алгоритмы — быстрая сортировка и слияние, — и научился пользоваться готовой `qsort` со своим компаратором.

И quicksort, и merge sort устроены на одном приёме: **разбить задачу на части поменьше и решить их тем же способом**. Это называется **рекурсия** — функция, которая вызывает сама себя. Без неё эти быстрые сортировки не написать красиво. Поэтому следующая глава целиком про рекурсию: разберём её на простых примерах, а потом ты будешь готов собрать quicksort и merge sort самостоятельно.

Глава 9. Рекурсия: факториал, Фибоначчи, Ханойские башни

До сих пор, когда нужно было что-то повторить, мы брали цикл. Но есть второй способ повторения — когда функция **вызывает сама себя**. Это называется **рекурсия** (recursion). Звучит как фокус «зеркало в зеркале», но идея простая и очень мощная: большую задачу мы сводим к такой же задаче, только поменьше, и так — пока не упрёмся в самый маленький, очевидный случай.

В этой главе мы разберём рекурсию с нуля на трёх примерах:

- **факториал** — сравним рекурсию с циклом из главы 4;
- **числа Фибоначчи** — увидим, как наивная рекурсия становится чудовищно медленной, и починим это;
- **Ханойские башни** — задача, которую циклом писать больно, а рекурсией она решается в три строки.

Что такое рекурсия

Возьмём бытовой пример. Ты стоишь в очереди и хочешь узнать, какой ты по счёту. Можно посчитать всех впереди самому. А можно так: спроси у того, кто стоит прямо перед тобой, какой он по счёту, и прибавь 1. Но ему, чтобы ответить, надо спросить у того, кто перед ним. И так далее — **до самого первого человека**, который точно знает: «я первый». Его ответ покатится назад по очереди, и каждый прибавит свою единицу.

Вот это и есть рекурсия. У неё всегда две обязательные части:

1. **База** (base case) — самый простой случай, ответ на который мы знаем сразу, без всяких вызовов. В очереди это «я первый».
2. **Шаг** (recursive step) — сведение задачи к **меньшей** такой же задаче плюс немного работы. В очереди это «спроси соседа и прибавь 1».

Если есть и база, и шаг, и каждый шаг приближает нас к базе — рекурсия доберётся до базы и остановится.

Что будет без базы

Убери базу — и катастрофа. Функция будет вызывать себя, потом снова себя, и снова, и так **бесконечно**. Это **бесконечная рекурсия**. Закончится она не зависанием, как бесконечный цикл, а аварией: программа упадёт с ошибкой **stack overflow** («переполнение стека вызовов»). Сейчас разберём, что это за стек.

Стек вызовов простыми словами

Когда одна функция вызывает другую, компьютер должен **запомнить**, куда вернуться и какие у вызвавшей функции были локальные переменные. Для этого есть **стек вызовов** (call stack) — особая область памяти, устроенная как стопка тарелок.

- Вызвали функцию — на стопку **легла новая тарелка** (её локальные переменные, параметры, точка возврата).
- Функция доработала и сделала `return` — её **тарелку сняли**, и управление вернулось к той, что лежит ниже.

Снимать и класть можно только сверху — как с настоящей стопкой тарелок.

При рекурсии тарелки кладутся одна на другую: `factorial(4)` ещё не закончил — он ждёт `factorial(3)`, а тот ждёт `factorial(2)`, и так далее. Стопка растёт. Когда дойдём до базы, тарелки начнут сниматься сверху вниз, и ответы поедут обратно.

Стопка не бесконечная — память под неё ограничена (обычно несколько мегабайт). Если базы нет или до неё слишком далеко, тарелки переполнят стек, и программа рухнет. Это и есть **stack overflow**. Запомни как правило номер один: **у каждой рекурсии должна быть база, к которой шаги реально приближаются**.

Часть 1. Факториал рекурсивно

Факториал числа `n` (пишется `n!`) — это произведение всех целых от 1 до `n`. Например, `5! = 1·2·3·4·5 = 120`. И отдельно договорённость: `0! = 1`.

В главе 4 мы считали факториал циклом — просто перемножали числа от 2 до `n`:

```
long long fact = 1;
for (int i = 2; i <= n; i++) {
    fact = fact * i;
}
```

Теперь посмотрим на факториал **другими глазами**. Заметь:

```
5! = 5 · 4 · 3 · 2 · 1
4! =     4 · 3 · 2 · 1
```

То есть `5! = 5 · 4!`. А `4! = 4 · 3!`. В общем виде:

```
n! = n · (n−1)!
```

Это и есть **шаг**: факториал `n` выражается через факториал меньшего числа `n−1`. А **база** — `0! = 1` (и `1! = 1`), дальше уменьшать некуда.

Пишем код

```c
long long factorial(int n)
{
    if (n <= 1) {          /* база: дальше не спускаемся */
        return 1;
    }
    return n * factorial(n - 1);   /* шаг: сводим к меньшей задаче */
}
```

Разбор:

- `long long factorial(int n)` — функция принимает целое `n` и возвращает `long long`. Тип `long long` — это целое **увеличенной вместимости** (обычно 8 байт против 4 у `int`). Факториалы растут стремительно, и обычного `int` хватает лишь до `12!`; `long long` тянет до `20!`. Печатают его спецификатором `%lld` (на одну `l` больше, чем у `int`).
- `if (n <= 1) return 1;` — **база**. Для `0` и `1` ответ известен сразу: `1`. Никаких вызовов себя — функция просто возвращает число. Это та самая «остановка», без которой рекурсия не закончится.
- `return n * factorial(n - 1);` — **шаг**. Чтобы получить `n!`, мы умножаем `n` на результат **вызова себя же** с аргументом `n - 1`. Аргумент стал меньше — значит, рано или поздно дойдём до `1` и сработает база.

Вся собранная программа — `code/09-factorial.c`:

```c
#include <stdio.h>

/* Рекурсивный факториал.
   База: 0! = 1 и 1! = 1.
   Шаг: n! = n * (n-1)!. */
long long factorial(int n)
```

```
{
    if (n <= 1) {          /* база: дальше не спускаемся */
        return 1;
    }
    return n * factorial(n - 1);   /* шаг: сводим к меньшей задаче */
}

int main(void)
{
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }

    if (n > 20) {
        printf("Слишком большое: long long переполнится при n > 20.\n");
        return 1;
    }

    printf("%d! = %lld\n", n, factorial(n));
    return 0;
}
```

Проверки в `main` те же, что и в главе 4: `scanf` может вернуть не 1 (ввели не число), факториал не определён для отрицательных, а при `n > 20` даже `long long` переполнится. Переполнение — это когда результат не влезает в тип; число «заворачивается» и становится мусором без всякого предупреждения, поэтому мы отсекаем заранее.

Как это работает: раскрутка вызовов

Самое важное в рекурсии — увидеть, как вызовы **раскручиваются**. Посчитаем `factorial(4)`. Сначала вызовы идут вглубь, каждый ждёт младшего:

```
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1            <- база! вот она, остановка
```

Дошли до базы — `factorial(1)` вернул `1` без вызовов. Теперь ответы едут **обратно** (тарелки снимаются со стека сверху вниз):

```
factorial(1) = 1
factorial(2) = 2 * 1  = 2
factorial(3) = 3 * 2  = 6
factorial(4) = 4 * 6  = 24
```

Готово: `4! = 24`. Обрати внимание на форму — вниз мы **спускаемся**, дробя задачу, а вверх **поднимаемся**, собирая ответ. В этот момент на стеке одновременно лежали четыре «тарелки» `factorial`. Для `n = 20` их будет двадцать — пустяк для стека. А вот для миллиона рекурсия уже переполнила бы стек, и тут цикл надёжнее.

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/09-factorial.c -o factorial
$ ./factorial
Факториал какого числа? 5
5! = 120
$ ./factorial
Факториал какого числа? 0
0! = 1
$ ./factorial
Факториал какого числа? 20
20! = 2432902008176640000
```

Результат тот же, что у цикла из главы 4. Какой способ выбрать? Для факториала — дело вкуса: и цикл, и рекурсия одинаково эффективны. Рекурсия читается ближе к математическому определению `n! = n·(n-1)!`, цикл не тратит стек. Но дальше будет задача, где рекурсия — это ловушка.

Часть 2. Числа Фибоначчи и ловушка медленной рекурсии

Последовательность Фибоначчи: каждое число — сумма двух предыдущих. Начинается с 0 и 1:

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...
```

Определение прямо просится в рекурсию:

- база: `fib(0) = 0`, `fib(1) = 1`;
- шаг: `fib(n) = fib(n−1) + fib(n−2)`.

Наивная версия

```
long long fib_slow(int n)
{
    if (n < 2) {                /* база: fib(0)=0, fib(1)=1 */
        return n;
    }
    return fib_slow(n - 1) + fib_slow(n - 2);
}
```

Разбор:

- `if (n < 2) return n;` — хитрый, но честный приём для **базы**. При `n == 0` вернётся `0`, при `n == 1` вернётся `1` — ровно то, что нужно. Два базовых случая закрыты одной строкой.
- `return fib_slow(n - 1) + fib_slow(n - 2);` — **шаг**. Здесь функция вызывает себя **дважды**. Вот эта пара вызовов и будет источником беды.

Запустим — ответы правильные:

```
Какое число Фибоначчи посчитать? 10
Рекурсия:  fib(10) = 55
Цикл:      fib(10) = 55
```

В чём проблема: дерево вызовов

Посмотрим, что происходит при вычислении `fib(5)`. Каждый вызов порождает два новых, те — ещё по два, и получается **дерево**:

```
                    fib(5)
                  /        \
            fib(4)          fib(3)
            /     \         /     \
       fib(3)   fib(2)   fib(2)   fib(1)
       /    \    /   \    /   \
  fib(2) fib(1) f(1) f(0) f(1) f(0)
   /  \
 f(1) f(0)
```

Вглядись в дерево. `fib(3)` считается **дважды**. `fib(2)` — **трижды**. `fib(1)` и `fib(0)` — много-много раз. Рекурсия пересчитывает одни и те же значения снова и снова, не помня прошлых ответов. Чем больше `n`, тем чудовищнее размножаются повторы: число вызовов растёт **экспоненциально** — примерно удваивается на каждый шаг `n`.

«Экспоненциально» — это не фигура речи, это можно замерить. Секундомер живёт в `<time.h>` и называется `clock()`:

```
#include <time.h>

clock_t t0 = clock();                 /* засекли */
long long v = fib(n);
double sec = (double)(clock() - t0) / CLOCKS_PER_SEC;
printf("fib(%d) = %lld\t%.3f c\n", n, v, sec);
```

- `clock()` возвращает, сколько «тактов» процессорного времени программа израсходовала с начала работы. Тип результата — `clock_t`.
- Само по себе это число ни о чём не говорит: тактов в секунде на разных системах разное количество. Их число лежит в константе `CLOCKS_PER_SEC` — поэтому делим разницу на неё и получаем секунды.
- Приведение `(double)` обязательно: обе величины целые, и без него деление ушло бы нацело (глава 2) — получили бы 0 или 1.

Замеры наивной рекурсии на разных `n` (мой ноутбук, сборка с `-O2`):

```
fib(30) = 832040        0.002 c
fib(35) = 9227465       0.027 c
fib(40) = 102334155     0.300 c
fib(45) = 1134903170    3.245 c
```

Абсолютные секунды у тебя будут свои — они зависят от машины. А вот **отношение** между строками воспроизведётся у всех: каждые `+5` к `n` замедляют программу примерно в 11 раз. Уже `fib(45)` считается больше трёх секунд, а `fib(60)` ты не дождёшься никогда — уйдут годы. И всё это ради числа, которое можно получить мгновенно.

Фикс: один проход циклом

Беда наивной рекурсии — в повторном счёте. Лекарство простое: считать числа **по порядку, снизу вверх**, и помнить два последних. Никаких повторов, один проход:

```
long long fib_fast(int n)
{
    long long a = 0;              /* fib(0) */
```

```
    long long b = 1;              /* fib(1) */
    for (int i = 0; i < n; i++) {
        long long next = a + b;   /* следующее число */
        a = b;                    /* сдвигаем окно вправо */
        b = next;
    }
    return a;
}
```

Разбор:

- `a` и `b` — два соседних числа Фибоначчи. Стартуем с `a = fib(0) = 0` и `b = fib(1) = 1`.
- На каждом шаге считаем `next = a + b` — следующее число. Затем **сдвигаем окно**: то, что было `b`, становится новым `a`, а `next` — новым `b`. Так пара `(a, b)` ползёт по последовательности вправо.
- После `n` шагов в `a` лежит ровно `fib(n)`. Возвращаем его.

Здесь каждое число считается **ровно один раз**. Для `fib(90)` цикл сделает 90 сложений — доли микросекунды. Сравни: наивная рекурсия на таком `n` не закончилась бы и за всю твою жизнь.

Полная программа — `code/09-fib.c` — считает оба способа, чтобы видеть, что ответы совпадают:

```
#include <stdio.h>

/* Наивная рекурсия: красиво, но медленно.
   База: fib(0) = 0, fib(1) = 1.
   Шаг: fib(n) = fib(n-1) + fib(n-2). */
long long fib_slow(int n)
{
    if (n < 2) {                  /* база: fib(0)=0, fib(1)=1 */
        return n;
    }
    return fib_slow(n - 1) + fib_slow(n - 2);
}

/* Быстрый вариант: один проход циклом, две переменные. */
long long fib_fast(int n)
{
    long long a = 0;              /* fib(0) */
    long long b = 1;              /* fib(1) */
```

```
    for (int i = 0; i < n; i++) {
        long long next = a + b;  /* следующее число */
        a = b;                   /* сдвигаем окно вправо */
        b = next;
    }
    return a;
}

int main(void)
{
    int n;
    printf("Какое число Фибоначчи посчитать? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }
    if (n < 0) {
        printf("Нужно n >= 0.\n");
        return 1;
    }
    if (n > 90) {
        printf("Слишком большое: long long переполнится при n > 90.\n");
        return 1;
    }

    printf("Рекурсия:  fib(%d) = %lld\n", n, fib_slow(n));
    printf("Цикл:      fib(%d) = %lld\n", n, fib_fast(n));
    return 0;
}
```

Мы отсекаем `n > 90`, потому что `fib(93)` уже не влезает в `long long`. И учти: если ввести что-то вроде `50`, программа **зависнет на минуты** — виновата строка с `fib_slow`. Для больших `n` оставляй только `fib_fast`. Эта программа держит обе функции нарочно, чтобы ты сам прочувствовал разницу на небольших числах.

Проверим на маленьком `n`, где обе версии быстрые:

```
$ cc -std=c11 -Wall -Wextra -O2 code/09-fib.c -o fib
$ ./fib
Какое число Фибоначчи посчитать? 10
```

```
Рекурсия:  fib(10) = 55
Цикл:      fib(10) = 55
```

Вывод: рекурсия — красивый инструмент, но не всегда правильный. Если подзадачи **перекрываются** (как ветки дерева Фибоначчи), наивная рекурсия катастрофически медленна. Иногда её спасают, **запоминая** уже посчитанные ответы (это называется мемоизация), но часто, как здесь, проще и честнее обычный цикл.

Часть 3. Ханойские башни

Теперь задача, ради которой рекурсию и любят. Есть три стержня — `A`, `B`, `C`. На стержне `A` надета пирамидка из `n` дисков: внизу самый большой, выше — всё меньше. Нужно перенести всю башню на стержень `C`. Правила:

- за один ход переносим **только один** диск (верхний с какого-то стержня);
- **больший диск нельзя класть на меньший** — никогда.

Циклом это писать мучительно. А рекурсией — почти само собой.

Разбираем руками

Допустим, мы умеем переносить `n−1` дисков куда угодно (пока поверим в это на слово — это и есть рекурсивная вера). Тогда перенести `n` дисков с `A` на `C` можно так:

1. Перенести верхние `n−1` дисков с `A` на `B` (стержень `C` пока свободен — используем его как промежуточный).
2. Перенести самый большой, оставшийся диск с `A` на `C`. Один ход.
3. Перенести `n−1` дисков с `B` на `C` (теперь промежуточный — `A`).

Всё. Большая задача («перенести `n`») свелась к двум таким же, но меньшим («перенести `n−1`») плюс один ход в середине. Это **шаг**. А **база** — `n == 0`: переносить нечего, ничего не делаем.

Заметь, как ловко меняются роли стержней. В пункте 1 промежуточный — `C`, в пункте 3 — `A`. Поэтому функции удобно передавать три стержня: откуда (`from`), куда (`to`) и через какой (`via`).

Пишем код

```c
void hanoi(int n, char from, char to, char via)
{
    if (n == 0) {                /* база: переносить нечего */
        return;
    }
```

```
    hanoi(n - 1, from, via, to);  /* верхние n-1 уводим на промежуточный */
    printf("Переложить диск %d с %c на %c\n", n, from, to);
    hanoi(n - 1, via, to, from);  /* возвращаем n-1 на цель */
}
```

Разбор:

- `void hanoi(...)` — функция ничего не возвращает (`void`), она просто печатает ходы. Параметры — это **имена стержней** типа `char`: `from` (откуда), `to` (куда), `via` (промежуточный).
- `if (n == 0) return;` — **база**. Ноль дисков — пустой ход, выходим сразу. `return;` без значения, потому что тип `void`.
- `hanoi(n - 1, from, via, to);` — шаг 1. Уводим `n-1` дисков с `from` на `via`. Заметь: на месте «куда» теперь стоит `via`, а «через» — старая цель `to`. Роли поменялись.
- `printf("Переложить диск %d с %c на %c\n", n, from, to);` — шаг 2. Печатаем единственный ход для самого большого диска: с `from` на `to`. Спецификатор `%c` печатает один символ — имя стержня.
- `hanoi(n - 1, via, to, from);` — шаг 3. Переносим `n-1` дисков с `via` на `to`, используя `from` как промежуточный.

Полная программа — `code/09-hanoi.c`:

```
#include <stdio.h>

/* Перенести n дисков со стержня from на стержень to,
   используя via как промежуточный.
   База: 0 дисков — ничего не делаем.
   Шаг: n-1 на via, большой на to, n-1 с via на to. */
void hanoi(int n, char from, char to, char via)
{
    if (n == 0) {                /* база: переносить нечего */
        return;
    }
    hanoi(n - 1, from, via, to);  /* верхние n-1 уводим на промежуточный */
    printf("Переложить диск %d с %c на %c\n", n, from, to);
    hanoi(n - 1, via, to, from);  /* возвращаем n-1 на цель */
}

int main(void)
{
```

```
    int n;
    printf("Сколько дисков? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }
    if (n < 0) {
        printf("Нужно n >= 0.\n");
        return 1;
    }

    hanoi(n, 'A', 'C', 'B');      /* с A на C, B — запасной */
    return 0;
}
```

В `main` мы вызываем `hanoi(n, 'A', 'C', 'B')` — перенести всё с `A` на `C`, а `B` пусть будет запасным. Буквы стержней пишем в одинарных кавычках: `'A'` — это символьная константа типа `char` (см. главу 2).

Проверяем

```
$ cc -std=c11 -Wall -Wextra -O2 code/09-hanoi.c -o hanoi
$ ./hanoi
Сколько дисков? 3
Переложить диск 1 с A на C
Переложить диск 2 с A на B
Переложить диск 1 с C на B
Переложить диск 3 с A на C
Переложить диск 1 с B на A
Переложить диск 2 с B на C
Переложить диск 1 с A на C
```

Семь ходов — и башня из трёх дисков на месте. Пройдись по списку диском в руке (или монетками разного размера): ни разу больший не ляжет на меньший. Магия в том, что мы **не расписывали** эти семь ходов — мы лишь объяснили компьютеру правило «`n` = `n−1` + один + `n−1`», а рекурсия развернула его сама.

Число ходов для `n` дисков — это $2^n - 1$: для 3 дисков 7, для 4 — 15, для 10 — 1023. Растёт быстро, так что для проверки бери небольшие `n`.

Частые ошибки

Нет базы (или до неё не дойти). Если забыть условие остановки или сделать так, что аргумент не уменьшается, рекурсия будет вызывать себя вечно и упадёт со **stack overflow**. Всегда проверяй: есть ли база и **приближает** ли её каждый шаг? `factorial(n-1)` приближает (аргумент падает), а вот `factorial(n)` внутри `factorial(n)` — нет, это бесконечность.

Рекурсия там, где подзадачи повторяются. Наивный `fib` — учебный пример катастрофы: экспоненциальное число повторных вычислений. Прежде чем писать рекурсию «в лоб», спроси себя: не будут ли ветки считать одно и то же? Если да — цикл или мемоизация.

Неверный спецификатор для `long long`. Печатать `long long` нужно через `%lld`, а не `%d`. С `%d` получишь мусор или предупреждение компилятора.

Переполнение результата. Факториалы и числа Фибоначчи растут взрывообразно. `20!` — предел `long long` для факториала, `fib(92)` — для Фибоначчи. Дальше число молча «заворачивается». Мы отсекаем такие `n` заранее.

Путаница со стержнями в Ханое. Порядок аргументов важен: `hanoi(n-1, from, via, to)` и `hanoi(n-1, via, to, from)`. Перепутаешь `via` и `to` — и алгоритм начнёт класть большие диски на маленькие. Держи в голове: первый под-вызов освобождает цель, второй — её заполняет.

Упражнения

1. **Сумма цифр рекурсивно.** Напиши `int digit_sum(int n)`, которая возвращает сумму цифр числа. Подсказка: последняя цифра — это `n % 10`, а всё остальное число — `n / 10`. База — однозначное число (`n < 10`). Шаг: `n % 10 + digit_sum(n / 10)`.

2. **Степень рекурсивно.** Напиши `long long power(int base, int exp)`, возводящую `base` в степень `exp`. База: любое число в степени 0 равно 1. Шаг: `base * power(base, exp - 1)`.

3. **Обратный отсчёт.** Функция `void countdown(int n)` печатает числа от `n` до 1, а потом слово «Пуск!». Сделай это рекурсией без цикла. Где здесь база?

4. **Сумма от 1 до n.** Напиши `long long sum_to(int n)`, возвращающую `1 + 2 + ... + n` через рекурсию. Сравни с формулой `n*(n+1)/2`.

5. **Фибоначчи: посчитай вызовы.** Заведи глобальный счётчик и увеличивай его в начале `fib_slow`. Запусти для `n = 10`, `20`, `30` и посмотри, как стремительно растёт число вызовов. Убедись своими глазами в экспоненциальном взрыве.

6. **Ханой с подсчётом ходов.** Доработай `hanoi` так, чтобы она ещё и считала общее число ходов, и сравни результат с $2^n - 1$.

Что мы получили

Теперь ты владеешь рекурсией — вторым после цикла способом повторять действия. Главное, что нужно унести:

- у любой рекурсии есть **база** (где останавливаемся) и **шаг** (где сводим задачу к меньшей); без базы — `stack overflow`;
- вызовы складываются на **стек**, как стопка тарелок, и снимаются в обратном порядке — на этом строится «раскрутка» ответа;
- рекурсия бывает **изящной** (факториал, Ханой), а бывает **ловушкой** (наивный Фибоначчи с экспоненциальным повтором) — и тогда выручает цикл;
- некоторые задачи (Ханойские башни) рекурсией решаются почти даром, а циклом — мучительно.

В следующей главе мы вернёмся к строкам и займёмся **строковыми алгоритмами**: измерим длину без `strlen`, перевернём строку, проверим её на палиндром и научимся убирать тот самый `'\n'`, который оставляет `fgets`. Рекурсия там тоже пригодится — перевернуть строку можно и ею.

Глава 10. Строковые алгоритмы: длина, переворот, палиндром, слова, анаграммы

Строка — это просто массив символов с меткой конца `'\0'` (мы разобрали это в главе 3). А раз так, над строкой можно гонять циклы и считать что угодно: длину, число слов, проверять симметрию, сравнивать состав. В этой главе мы напишем четыре маленьких, но настоящих строковых алгоритма и поймём, как они устроены изнутри — не вызывая готовых функций, а собирая логику руками.

Важное предупреждение про русские буквы

Мы будем считать **символы** по одному. И тут есть ловушка. В кодировке UTF-8 (так хранят текст современные системы) латинская буква и цифра занимают **1 байт**, а русская буква — **2 байта**. Для программы на Си строка — это байты. Поэтому если ты введёшь «привет», программа увидит не 6 символов, а 12 байт, и посимвольные алгоритмы (переворот, палиндром) дадут кашу: половинки букв перемешаются.

Вывод простой: **в этой главе вводим только латиницу, цифры и пробелы**. Русский текст годится для сообщений (`printf` печатает байты как есть), но не для посимвольной обработки. Полноценная работа с Юникодом — отдельная большая тема, она за рамками книги.

Что мы сделаем

Четыре программы, каждая в своём файле:

- `10-reverse.c` — своя функция длины строки и переворот строки на месте.
- `10-palindrome.c` — проверка, читается ли слово одинаково с двух концов.
- `10-words.c` — подсчёт слов во фразе.

- `10-anagram.c` — проверка, состоят ли два слова из одних и тех же букв.

Ничего нового из синтаксиса тут почти нет — только массивы, циклы, `if` и `char`. Вся соль в **идеях алгоритмов**.

Часть 1. Длина и переворот строки

Разбираем задачу руками

Возьмём слово `hello`. Перевернуть его — значит получить `olleh`. Как это сделать «на бумаге»? Поставим палец слева (на `h`) и палец справа (на `o`) и **поменяем их местами**:

```
h e l l o      o e l l h      o l l e h
^       ^   ->     ^   ^    ->     ^^
```

Потом сдвигаем оба пальца к центру и снова меняем. Когда пальцы встретились (или перекрестились) — всё, строка перевёрнута. Этот приём — **два индекса с концов** — мы будем использовать постоянно.

Но прежде чем переворачивать, надо знать **длину** строки: где правый край. Длину мы могли бы взять из `strlen`, но давай напишем свою — так ты раз и навсегда поймёшь, как Си «видит» конец строки.

Шаг 1. Своя функция длины

Строка заканчивается невидимым символом `'\0'` (нулевой байт). Значит, длина — это сколько символов лежит **до** него. Идём циклом, пока не упрёмся в `'\0'`, и считаем шаги:

```c
int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}
```

Разбор:

- `const char *s` — параметр-строка. `char *` — указатель на символы, то есть адрес начала строки (вспомни главу 3: имя массива — это адрес его первого элемента). Слово `const` — обещание «я эту строку менять не буду», только читаю. Компилятор за этим проследит.

- `s[len]` — `len`-й символ строки. К указателю на символы можно обращаться по индексу точно как к массиву.
- `while (s[len] != '\0')` — крутимся, **пока** текущий символ не нулевой. `!=` — «не равно». `'\0'` в одинарных кавычках — это один символ с кодом 0, метка конца строки.
- На каждом шаге `len++` — продвигаемся дальше. Когда `s[len]` окажется `'\0'`, цикл встанет, а в `len` будет ровно число символов до него. Его и возвращаем.

Сам `'\0'` в длину **не входит** — мы останавливаемся на нём, не посчитав. Так же работает и настоящий `strlen` из `<string.h>`.

Шаг 2. Наивный переворот (и где он спотыкается)

Прочитаем строку через `fgets` и сразу попробуем перевернуть. Наивная мысль новичка: «пройду слева направо и буду переставлять символы». Запишем переворот двумя индексами с концов — это и есть правильный приём, но сначала запустим как есть и посмотрим на сюрприз:

```c
#include <stdio.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

int main(void)
{
    char line[100];
    printf("Введи строку (латиницей): ");
    fgets(line, sizeof line, stdin);

    int len = my_strlen(line);

    int i = 0, j = len - 1;
    while (i < j) {
        char tmp = line[i];
        line[i] = line[j];
```

```
        line[j] = tmp;
        i++;
        j--;
    }

    printf("Перевёрнутая: %s\n", line);
    return 0;
}
```

Вводим `hello`, запускаем:

```
Введи строку (латиницей): hello
Перевёрнутая:
olleh
```

Перевёрнутое слово на месте, но откуда **лишний перенос строки**? Почему `olleh` уехало на новую строку?

Шаг 3. Виноват `'\n'` от fgets — убираем его

Когда ты нажимаешь Enter, в строку попадает символ перевода строки `'\n'`, и `fgets` его **не выбрасывает**. То есть в `line` лежит не `hello`, а `hello\n` — шесть символов. `my_strlen` честно вернул 6, и наш переворот поставил `'\n'` в самое начало: получилось `\nolleh`. Вот откуда пустая строка перед результатом.

Лечится это всегда одинаково: после `fgets` смотрим на последний символ, и если это `'\n'` — затираем его на `'\0'`:

```
int len = my_strlen(line);
if (len > 0 && line[len - 1] == '\n') {
    line[len - 1] = '\0';   /* обрезали перевод строки */
    len--;                  /* и длина стала на 1 меньше */
}
```

- `len > 0` — защита от пустого ввода: если в строке вообще ничего нет, не лезем в `line[-1]`.
- `line[len - 1]` — последний реальный символ (индексы с нуля, поэтому последний — это `len - 1`).
- `line[len - 1] = '\0';` — ставим метку конца на место `'\n'`. Теперь для всех функций строка кончается раньше, а `'\n'` как бы исчез.
- `len--` — длина уменьшилась на единицу, дальше работаем с ней.

Этот кусочек (обрезать `'\n'`) тебе понадобится после **каждого** `fgets`. Запомни его как рефлекс.

Полная программа: `10-reverse.c`

```c
#include <stdio.h>

/* Своя длина строки: считаем символы до '\0'. */
int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

int main(void)
{
    char line[100];

    printf("Введи строку (латиницей): ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 0;                       /* ввода не было */
    }

    /* Убираем хвостовой '\n', который оставил fgets. */
    int len = my_strlen(line);
    if (len > 0 && line[len - 1] == '\n') {
        line[len - 1] = '\0';
        len--;
    }

    /* Переворот на месте: меняем местами края, идём к центру. */
    int i = 0;
    int j = len - 1;
    while (i < j) {
        char tmp = line[i];
        line[i] = line[j];
        line[j] = tmp;
        i++;
```

```
        j--;
    }

    printf("Длина: %d\n", len);
    printf("Перевёрнутая: %s\n", line);
    return 0;
}
```

Что добавилось по сравнению с наивной версией:

- `if (fgets(...) == NULL)` — `fgets` возвращает `NULL`, если читать было нечего (например, ввод закрыли). Тогда просто выходим. Полезная привычка — проверять результат ввода.
- Блок обрезки `'\n'` — разобрали выше.
- `char tmp` — временная коробка для обмена. Чтобы поменять два значения местами, одно надо где-то придержать, иначе затрёшь. Это тот же приём `swap`, что в главе 3, только для символов.
- `while (i < j)` — крутимся, пока левый индекс левее правого. Как только они встретились в центре (`i == j`) или разошлись — конец. Средний символ (если длина нечётная) остаётся на месте, и правильно: его двигать некуда.

Проверяем:

```
$ cc -std=c11 -Wall -Wextra -O2 10-reverse.c -o 10-reverse
$ ./10-reverse
Введи строку (латиницей): hello
Длина: 5
Перевёрнутая: olleh
```

Лишнего переноса больше нет — `'\n'` мы убрали.

Часть 2. Палиндром

Палиндром — слово, которое читается одинаково слева направо и справа налево: `level`, `radar`, `noon`. А `world` — не палиндром.

Идея

Тот же приём двух индексов, что и в перевороте, только теперь мы ничего не меняем, а **сравниваем**. Левый символ должен совпадать с правым, второй слева — со вторым справа, и так до центра. Нашли хоть одно несовпадение — это не палиндром, дальше можно не проверять.

Заведём флаг `is_palindrome`, поставим его в `1` («да, палиндром») и сбросим в `0`, если поймаем разные символы.

Программа: `10-palindrome.c`

```c
#include <stdio.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

int main(void)
{
    char line[100];

    printf("Введи слово (латиницей): ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 0;
    }

    int len = my_strlen(line);
    if (len > 0 && line[len - 1] == '\n') {
        line[len - 1] = '\0';
        len--;
    }

    /* Два индекса идут навстречу друг другу. */
    int i = 0;
    int j = len - 1;
    int is_palindrome = 1;              /* пока считаем, что да */
    while (i < j) {
        if (line[i] != line[j]) {
            is_palindrome = 0;          /* нашли несовпадение */
```

```
            break;
        }
        i++;
        j--;
    }

    if (is_palindrome) {
        printf("\"%s\" — палиндром\n", line);
    } else {
        printf("\"%s\" — не палиндром\n", line);
    }
    return 0;
}
```

Разбор новых мест:

- `int is_palindrome = 1;` — **флаг**: переменная-да/нет. `1` — «истина», `0` — «ложь» (в Си нет отдельного типа `bool` без подключения `<stdbool.h>`, поэтому просто берём `int`). Начинаем с оптимизма: считаем слово палиндромом, пока не доказано обратное.
- `if (line[i] != line[j])` — сравниваем края. Если символы разные — это уже не палиндром.
- `break;` — досрочно выходим из цикла. Раз нашли несовпадение, дальше проверять незачем — ответ уже известен. `break` обрывает ближайший цикл.
- `\"%s\"` — обратный слэш перед кавычкой говорит `printf`: «это обычная кавычка, печатай её», а не конец строки формата. Так слово в выводе берётся в кавычки.

Проверяем на двух словах:

```
$ cc -std=c11 -Wall -Wextra -O2 10-palindrome.c -o 10-palindrome
$ ./10-palindrome
Введи слово (латиницей): level
"level" — палиндром
$ ./10-palindrome
Введи слово (латиницей): world
"world" — не палиндром
```

Пустая строка и строка из одного символа — палиндромы по определению, и программа это учитывает: при `len <= 1` цикл `while (i < j)` ни разу не выполнится, флаг останется `1`.

Часть 3. Подсчёт слов

Посчитаем, сколько слов во фразе. Слова разделены пробелами, причём пробелов между словами может быть **несколько**, и в начале/конце они тоже бывают. Простое `считать пробелы и прибавить единицу` тут сломается: `the  quick` (три пробела) дало бы лишние слова.

Идея: ловим начало слова

Слово начинается там, где **за пробелом идёт буква**. Будем идти по строке символ за символом и держать флаг `in_word` — «мы сейчас внутри слова или нет». Логика:

- встретили пробел → мы вне слова (`in_word = 0`);
- встретили не-пробел и до этого были вне слова → это **начало нового слова**: прибавляем счётчик и помечаем `in_word = 1`;
- встретили не-пробел, но уже были внутри слова → ничего не делаем, просто продолжаем то же слово.

Так каждое слово считается ровно один раз — в момент его первой буквы. Сколько бы пробелов ни стояло вокруг, лишних единиц не набежит.

Программа: `10-words.c`

```c
#include <stdio.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
    }
    return len;
}

int main(void)
{
    char line[200];

    printf("Введи фразу (латиницей): ");
    if (fgets(line, sizeof line, stdin) == NULL) {
        return 0;
```

```
    }

    int len = my_strlen(line);
    if (len > 0 && line[len - 1] == '\n') {
        line[len - 1] = '\0';
        len--;
    }

    int words = 0;
    int in_word = 0;                    /* сейчас мы внутри слова? */
    for (int i = 0; i < len; i++) {
        char c = line[i];
        if (c == ' ' || c == '\t') {
            in_word = 0;                /* пробел — слово кончилось */
        } else {
            if (in_word == 0) {         /* переход «пробел -> буква» */
                words++;                /* начало нового слова */
                in_word = 1;
            }
        }
    }

    printf("Слов: %d\n", words);
    return 0;
}
```

Разбор:

- `char c = line[i];` — для удобства кладём текущий символ в `c`, чтобы не писать `line[i]` четыре раза.
- `if (c == ' ' || c == '\t')` — проверяем, разделитель ли это. `' '` — обычный пробел, `'\t'` — табуляция. `||` — «или»: условие верно, если это пробел **или** таб.
- В ветке-разделителе сбрасываем `in_word = 0` — «мы вышли из слова».
- В ветке-не-разделителе смотрим: `if (in_word == 0)` — если мы только что были вне слова, значит это его первая буква. Прибавляем `words++` и ставим `in_word = 1`. Пока буквы идут подряд, `in_word` остаётся `1`, и счётчик не растёт — слово считается один раз.

Проверяем на фразе с кучей лишних пробелов и на пустом вводе:

```
$ cc -std=c11 -Wall -Wextra -O2 10-words.c -o 10-words
$ ./10-words
Введи фразу (латиницей):   the   quick brown   fox
Слов: 4
$ ./10-words
Введи фразу (латиницей):
Слов: 0
```

Лишние пробелы по краям и между словами не помешали — флаг `in_word` сделал своё дело.

Часть 4. Анаграммы

Анаграммы — два слова из одних и тех же букв, переставленных в другом порядке: `listen` и `silent`, `anagram` и `nagaram`. А `hello` и `world` — нет.

Идея: считаем частоты букв

Как проверить руками? Можно отсортировать буквы каждого слова и сравнить. Но есть приём проще и быстрее — **подсчёт частот**.

Заведём на каждую возможную букву свой счётчик. Пройдём по первому слову и для каждой буквы **прибавим** её счётчик. Потом пройдём по второму слову и для каждой буквы **вычтем**. Если слова — анаграммы, то каждую букву мы столько же раз прибавили, сколько вычли, и все счётчики в конце окажутся **нулевыми**. Если где-то остался не ноль — буквы не совпали по составу.

Под счётчики возьмём массив `int count[256]`. Почему 256? Символ `char` в Си — это, по сути, маленькое целое число (код символа), и один байт принимает значения от 0 до 255. Сделав массив на 256 ячеек, мы заводим отдельный счётчик под **любой** возможный байт. Сам код символа служит индексом: для буквы `a` (код 97) растёт `count[97]`.

Программа: `10-anagram.c`

```c
#include <stdio.h>

int my_strlen(const char *s)
{
    int len = 0;
    while (s[len] != '\0') {
        len++;
```

```
    }
    return len;
}

/* Убирает хвостовой '\n' от fgets. */
void chomp(char *s)
{
    int len = my_strlen(s);
    if (len > 0 && s[len - 1] == '\n') {
        s[len - 1] = '\0';
    }
}

int main(void)
{
    char a[200];
    char b[200];

    printf("Первое слово (латиницей): ");
    if (fgets(a, sizeof a, stdin) == NULL) {
        return 0;
    }
    printf("Второе слово (латиницей): ");
    if (fgets(b, sizeof b, stdin) == NULL) {
        return 0;
    }

    chomp(a);
    chomp(b);

    /* По счётчику на каждый из 256 возможных байтов. */
    int count[256] = {0};

    for (int i = 0; a[i] != '\0'; i++) {
        count[(unsigned char)a[i]]++;  /* в первом слове — прибавляем */
    }
    for (int i = 0; b[i] != '\0'; i++) {
        count[(unsigned char)b[i]]--;  /* во втором — вычитаем */
```

```
    }

    /* Если все счётчики нулевые — буквы совпали по составу. */
    int anagram = 1;
    for (int k = 0; k < 256; k++) {
        if (count[k] != 0) {
            anagram = 0;
            break;
        }
    }

    if (anagram) {
        printf("\"%s\" и \"%s\" — анаграммы\n", a, b);
    } else {
        printf("\"%s\" и \"%s\" — не анаграммы\n", a, b);
    }
    return 0;
}
```

Разбор новых мест:

- `void chomp(char *s)` — мы вынесли обрезку `'\n'` в отдельную функцию, чтобы не дублировать её для `a` и `b`. `void` — функция ничего не возвращает, только меняет переданную строку (она получает её адрес, поэтому правки видны снаружи — см. главу 3).
- `int count[256] = {0};` — массив из 256 счётчиков. Запись `= {0}` заполняет **все** ячейки нулями. Это важно: без инициализации в массиве был бы мусор, и счёт сломался бы.
- `count[(unsigned char)a[i]]++` — берём код текущего символа и используем как индекс, увеличивая нужный счётчик.
- `(unsigned char)` — зачем приведение? Тип `char` может быть знаковым, и для байтов выше 127 (если вдруг затешется не-ASCII) код стал бы **отрицательным** — а это выход за границу массива (`count[-50]`). Приведение к `unsigned char` гарантирует диапазон 0..255. Привычка на будущее, чтобы не нарваться.
- Второй цикл с `--` — вычитаем частоты второго слова.
- Финальный цикл проверяет, что **все** счётчики обнулились. Нашли ненулевой — `anagram = 0` и `break`.

Проверяем:

```
$ cc -std=c11 -Wall -Wextra -O2 10-anagram.c -o 10-anagram
$ ./10-anagram
Первое слово (латиницей): listen
Второе слово (латиницей): silent
"listen" и "silent" — анаграммы
$ ./10-anagram
Первое слово (латиницей): hello
Второе слово (латиницей): world
"hello" и "world" — не анаграммы
```

Этот приём — «массив-счётчик, где индекс это сам символ» — очень мощный. Им же можно посчитать, сколько раз встречается каждая буква, найти первый неповторяющийся символ и многое другое.

Частые ошибки

Забыли обрезать `'\n'` после `fgets`. Самая частая беда этой главы. Перевод строки портит и переворот (улетает в начало), и сравнение (`"level\n"` уже не палиндром), и подсчёт. После каждого `fgets` — обрезай.

Сравнение строк через `==`. Если захочешь сравнить две строки целиком, `a == b` сравнит **адреса**, а не текст (см. главу 3). Для содержимого нужен `strcmp` из `<string.h>`. В наших алгоритмах мы сравниваем символы по одному (`line[i] != line[j]`) — вот так с отдельными `char` через `==`/`!=` всё в порядке, это числа.

Забыли `'\0'`. Если будешь собирать строку вручную (например, копировать символы в другой массив), не забудь поставить `'\0'` в конце сам — иначе `printf("%s", ...)` напечатает мусор до первого случайного нуля в памяти.

Не обнулили массив-счётчик. `int count[256];` без `= {0}` содержит мусор, и подсчёт частот соврёт. Всегда инициализируй счётчики нулём.

Кириллица в посимвольном алгоритме. Введёшь русское слово — UTF-8 разложит его на пары байтов, и переворот/палиндром дадут бессмыслицу. В этой главе — только латиница.

Упражнения

1. Добавь в `10-reverse.c` подсчёт **гласных** (`a, e, i, o, u`) в введённой строке и выведи их число.

2. Перепиши проверку палиндрома так, чтобы она **игнорировала регистр**: `Level` тоже считался палиндромом. Подсказка: перед сравнением приводи символы к одному регистру (`'A'..'Z'` отличаются от `'a'..'z'` на постоянную величину — 32).
3. Сделай вариант подсчёта слов, который ещё и печатает **длину самого длинного слова** во фразе.
4. Напиши программу, которая печатает, **сколько раз** встречается каждая латинская буква во введённой строке (используй массив-счётчик из части 4, выводи только буквы с ненулевым счётчиком).
5. Проверка анаграмм игнорирует пробелы: сделай так, чтобы `dirty room` и `dormitory` считались анаграммами (пробелы при подсчёте просто пропускай).
6. Напиши функцию `int my_strcmp(const char *a, const char *b)`, которая возвращает 0, если строки равны, и не 0 иначе — своими руками, сравнивая символы в цикле, без `<string.h>`.

Что мы получили

Теперь ты умеешь обрабатывать строки **изнутри**, не полагаясь на готовые функции: считать длину циклом до `'\0'`, ходить по строке двумя индексами с концов (переворот, палиндром), вести флаг состояния (`in_word` при подсчёте слов) и применять массив-счётчик по коду символа (анаграммы, частоты). И, что важно, ты знаешь главную ловушку — хвостовой `'\n'` от `fgets` — и убираешь его на автомате.

Пока все наши данные жили в массивах фиксированного размера: `char line[100]`, `int count[256]`. Но что, если заранее неизвестно, сколько элементов понадобится, и они должны то добавляться, то удаляться? Тогда массива мало. В следующей главе — **динамические структуры данных**: мы соберём список, который растёт сам, связывая элементы указателями. Тут и пригодятся `malloc`/`free` из главы 3.

Итоговые программы главы — `code/10-reverse.c`, `code/10-palindrome.c`, `code/10-words.c`, `code/10-anagram.c`.

Глава 11. Динамические структуры данных: растущий массив, стек, очередь, связный список

До сих пор мы работали с обычным массивом: завёл `int a[100]` — и живи в этих рамках. Но в настоящих программах данные приходят и уходят: игрок делает ходы, в очередь встают задачи, в историю добавляются записи. Заранее размер не угадать.

В этой главе соберём четыре **структуры данных** — четыре способа хранить набор значений, у каждого свой характер:

- **растущий массив** — обычный массив, который сам увеличивается, когда заполнится;
- **стек** — «последним пришёл, первым ушёл»;
- **очередь** — «первым пришёл, первым ушёл»;
- **связный список** — цепочка узлов, разбросанных по памяти.

А заодно познакомимся с `realloc` (расширить уже выделенную память) и с **структурами** (`struct`) — способом склеить несколько значений в один объект. Без `struct` связный список не построить, поэтому начнём с него.

Знакомимся: `struct` — свой составной тип

`int` хранит одно число. Массив — много чисел **одного** типа. А как описать узел списка, в котором лежит и число, и адрес следующего узла? Нужен тип, который объединяет **разные** поля под одним именем. Это и есть **структура** (`struct`, от *structure*).

Представь карточку игрока: имя, очки, уровень. Три разные вещи, но логически это один объект. Опишем его:

```
struct Player {
    char name[32];
```

```
    int score;
    int level;
};
```

Разбор:

- `struct Player { ... };` — мы **определили** новый тип с именем `struct Player`. Само по себе это не создаёт ни одной переменной — это как чертёж, описание «из чего состоит игрок».
- внутри фигурных скобок — **поля** (fields): `name`, `score`, `level`. Каждое со своим типом, как обычные переменные. Поля идут через `;`.
- после закрывающей `}` обязательно стоит `;` — про неё забывают чаще всего, и компилятор выдаёт странную ошибку. Не забудь точку с запятой.

Теперь заведём переменную этого типа и заполним поля:

```
struct Player hero;
hero.score = 0;
hero.level = 1;
```

- `struct Player hero;` — создали **переменную** `hero` типа `struct Player`. Вот теперь в памяти лежит реальный игрок: место под имя, под очки, под уровень.
- `hero.score = 0;` — обращение к полю через **точку** (`.`): «у объекта `hero` поле `score` сделай равным 0». Точка читается как «точка-у»: `hero.score` — «очки у hero».

Указатель на структуру и стрелка ->

Часто структуру создают в динамической памяти (`malloc`) или передают в функцию — и тогда у нас на руках не сама структура, а **указатель** на неё. К полям через точку уже не обратишься, нужен другой способ.

```
struct Player *p = &hero;   // p — адрес нашего игрока

(*p).score = 50;            // длинная запись
p->score = 50;              // короткая, то же самое
```

- `struct Player *p` — указатель на структуру (как `int *p`, только тип другой).
- `(*p).score` — сначала `*p` разыменовывает указатель («сама структура, на которую смотрит `p`»), потом `.score` берёт поле. Скобки нужны: без них приоритеты операторов поймут запись неправильно.
- `p->score` — **стрелка** `->`. Это просто **сокращение** для `(*p).score`. Запомни намертво:

`p->field` — это в точности `(*p).field`. Стрелка = «разыменуй и возьми поле».

Когда у тебя указатель на структуру — пиши через стрелку, так короче и так пишут все.

`typedef` — короткое имя для типа

Писать `struct Player` каждый раз длинно. `typedef` даёт типу псевдоним:

```
typedef struct Player Player;   // теперь "Player" = "struct Player"

Player hero;                    // можно без слова struct
```

Часто это совмещают с определением:

```
typedef struct {
    int value;
} Box;

Box b;          // удобно
```

В этой главе для связного списка я оставлю полную запись `struct Node` — так нагляднее видно, что это структура. Но знай: `typedef` ровно для того, чтобы избавиться от повторов слова `struct`.

Этого про структуры хватит, чтобы строить дальше. Поехали по четырём структурам данных.

Часть 1. Растущий массив

Разбираем руками

Проблема обычного массива: размер фиксирован. `int a[100]` — а если понадобится 101? А если выделишь сразу миллион «про запас» — впустую займёшь память.

Идея растущего массива простая, как у блокнота: начни с маленького, а когда страницы кончатся — **доклей** ещё. Конкретно:

1. Выделяем место под несколько элементов (пусть под 2).
2. Заводим счётчик `count` — сколько реально занято.
3. Кладём элемент — увеличиваем `count`.
4. Как только `count` упёрся в вместимость — **удваиваем** вместимость.

Почему удваиваем, а не добавляем по одному? Если расширять на 1 каждый раз, при каждой вставке придётся копировать всё заново — это медленно. Удвоение делает расширения редкими: 2 → 4 → 8 → 16. На большом числе вставок это окупается.

`realloc` — изменить размер уже выделенной памяти

Расширять память умеет `realloc` (*reallocate* — перевыделить), из `<stdlib.h>`:

```
int *bigger = realloc(data, new_size);
```

Что делает `realloc(старый_указатель, новый_размер_в_байтах)`:

- пытается расширить блок, на который указывает `data`, до нового размера;
- **старые данные сохраняются** (он сам их перенесёт, если придётся переехать на новое место);
- возвращает **адрес** блока нужного размера. Этот адрес может оказаться **другим** — поэтому результат обязательно присваиваем заново.

Ловушка, в которую падают почти все:

```
data = realloc(data, new_size);   // ОПАСНО
```

Если `realloc` вернёт `NULL` (памяти не дали), ты **затрёшь** `data` этим `NULL` и потеряешь адрес старого блока — это утечка. Поэтому пишем во временный указатель и проверяем:

```
int *bigger = realloc(data, new_size);
if (bigger == NULL) { /* старый data ещё цел, обработай ошибку */ }
data = bigger;
```

Наивно → запуск

Соберём программу: складываем в растущий массив числа от 1 до 10, начав всего с двух ячеек.

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int capacity = 2;                       // сколько влезает сейчас
    int count = 0;                          // сколько реально лежит
    int *data = malloc(capacity * sizeof(int));
    if (data == NULL) {
        printf("Не хватило памяти\n");
        return 1;
    }

    for (int value = 1; value <= 10; value++) {
        if (count == capacity) {            // заполнили — растём
```

```
            capacity = capacity * 2;
            int *bigger = realloc(data, capacity * sizeof(int));
            if (bigger == NULL) {           // не дали — старое цело
                printf("Не хватило памяти при росте\n");
                free(data);
                return 1;
            }
            data = bigger;                  // теперь работаем с новым
            printf("[рост: вместимость стала %d]\n", capacity);
        }
        data[count] = value;                // кладём в свободную ячейку
        count = count + 1;                  // занятых стало на одну больше
    }

    printf("Всего элементов: %d, вместимость: %d\n", count, capacity);
    for (int i = 0; i < count; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");

    free(data);
    return 0;
}
```

Разбор новых мест:

- `int capacity = 2;` и `int count = 0;` — два разных числа! `capacity` — сколько ячеек **выделено**, `count` — сколько из них **занято**. Путать их — типичная ошибка.
- `malloc(capacity * sizeof(int))` — просим память под 2 целых. `malloc` возвращает адрес, кладём его в `int *data` (см. главу 3).
- `if (count == capacity)` — проверка «свободного места не осталось». Делаем её **перед** записью, чтобы было куда писать.
- блок с `realloc` — расширение, разобрали выше. После него `data` указывает на блок вдвое больше, старые числа на месте.
- `data[count] = value;` — к памяти от `malloc`/`realloc` обращаемся как к обычному массиву по индексу.
- `free(data);` — один `free` на один блок, в конце.

Запуск (`cc -std=c11 -Wall -Wextra -O2 11-dynarray.c -o dyn && ./dyn`):

```
[рост: вместимость стала 4]
[рост: вместимость стала 8]
[рост: вместимость стала 16]
Всего элементов: 10, вместимость: 16
1 2 3 4 5 6 7 8 9 10
```

Видно, как массив рос: 2 → 4 → 8 → 16. На десятый элемент вместимости 8 не хватило, удвоились до 16. Все числа на месте — `realloc` сохранил их при каждом переезде. Это файл `code/11-dynarray.c`.

Часть 2. Стек (LIFO)

Аналогия: стопка тарелок

Представь стопку чистых тарелок. Новую кладёшь **сверху**. Берёшь — тоже **сверху**. До нижней тарелки так просто не добраться: сначала снимешь все, что лежат на ней.

Это и есть **стек** (stack — стопка). Правило работы: **LIFO** — *Last In, First Out*, «последним пришёл — первым ушёл». Кто лёг последним, того первым и снимут.

Три операции:

- **push** — положить элемент на вершину;
- **pop** — снять элемент с вершины (и вернуть его);
- **peek** — подсмотреть, что на вершине, **не снимая**.

Где встречается: кнопка «отменить» (Ctrl+Z) — последнее действие отменяется первым; вызовы функций в программе; проверка скобок в выражении.

Строим на массиве

Стек прекрасно ложится на массив плюс одно число — `top` (вершина). Договоримся: `top` — это **индекс первой свободной ячейки**, он же заодно равен количеству элементов в стеке.

- push: положить в `data[top]`, потом `top++`.
- pop: сначала `top--`, потом взять `data[top]`.
- стек пуст, если `top == 0`. Переполнен, если `top == CAPACITY`.

```c
#include <stdio.h>
#include <stdlib.h>

#define CAPACITY 5      // сколько тарелок влезает в стопку
```

```c
int main(void)
{
    int data[CAPACITY];     // сам стек
    int top = 0;            // индекс первой СВОБОДНОЙ ячейки = размер

    // push: кладём тарелку наверх
    for (int value = 10; value <= 60; value += 10) {
        if (top == CAPACITY) {                  // стопка полна
            printf("push %d: переполнение, не кладу\n", value);
            continue;
        }
        data[top] = value;                      // положили на вершину
        top = top + 1;                          // вершина поднялась
        printf("push %d (в стеке %d)\n", value, top);
    }

    // peek: смотрим на верхнюю тарелку, не снимая
    if (top > 0) {
        printf("peek: наверху %d\n", data[top - 1]);
    }

    // pop: снимаем тарелки сверху, пока не опустеет
    while (top > 0) {
        top = top - 1;                          // вершина опустилась
        int value = data[top];                  // взяли верхнюю
        printf("pop %d (осталось %d)\n", value, top);
    }

    // пытаемся снять из пустого
    if (top == 0) {
        printf("pop: стек пуст, снимать нечего\n");
    }

    return 0;
}
```

Разбор:

- `#define CAPACITY 5` — это **директива препроцессора**: перед компиляцией все слова `CAPACITY` в коде заменяются на `5`. Удобно задавать константы одним местом. Точки с запятой в конце `#define` **нет** — это не обычная инструкция.
- `int data[CAPACITY];` — массив на 5 элементов.
- `value += 10` в цикле — шаг 10, чтобы класть 10, 20, …, 60 (шесть значений, на одно больше вместимости — проверим переполнение).
- `if (top == CAPACITY) { ... continue; }` — если места нет, печатаем отказ и `continue` переходит к следующей итерации, не трогая память за границей. Без этой проверки `data[5]` — выход за границу массива.
- `peek` смотрит `data[top - 1]` — верхний элемент лежит на ячейку ниже свободной. Сам стек не меняем.
- в `pop` порядок строгий: **сначала** `top--`, потом читаем `data[top]`. Иначе прочитаешь пустую ячейку.

Запуск:

```
push 10 (в стеке 1)
push 20 (в стеке 2)
push 30 (в стеке 3)
push 40 (в стеке 4)
push 50 (в стеке 5)
push 60: переполнение, не кладу
peek: наверху 50
pop 50 (осталось 4)
pop 40 (осталось 3)
pop 30 (осталось 2)
pop 20 (осталось 1)
pop 10 (осталось 0)
pop: стек пуст, снимать нечего
```

Обрати внимание на порядок при pop: вышло **50, 40, 30, 20, 10** — ровно наоборот тому, как клали. Это и есть LIFO в действии. Файл — `code/11-stack.c`.

Часть 3. Очередь (FIFO)

Аналогия: очередь в магазине

Очередь у кассы работает честно: кто встал раньше, того обслужат раньше. Новый покупатель встаёт **в хвост**, кассир берёт из **головы**.

Это **очередь** (queue). Правило: **FIFO** — *First In, First Out*, «первым пришёл — первым ушёл». Полная противоположность стеку.

Операции:

- **enqueue** — встать в хвост;
- **dequeue** — забрать из головы.

Где встречается: задачи на печать (документы печатаются в порядке отправки), сообщения, обход в ширину по графу (об этом — в главах про алгоритмы посложнее).

Проблема наивной очереди и кольцевой буфер

Наивно: храним два индекса — `head` (откуда забираем) и `tail` (куда кладём). enqueue двигает `tail` вправо, dequeue двигает `head` вправо.

Беда: оба индекса ползут только **вперёд**. Заберём несколько элементов из начала — ячейки слева освободятся, но `tail`, дойдя до конца массива, встанет, хотя слева полно места. Массив «кончился», а он наполовину пуст.

Лечение — **кольцевой буфер**: считаем, что после последней ячейки снова идёт нулевая, будто массив свёрнут в кольцо. Чтобы индекс «перепрыгивал» с конца в начало, берём остаток от деления на размер:

```
tail = (tail + 1) % CAPACITY;
```

Когда `tail` дорастает до `CAPACITY`, остаток делает его снова 0 — мы вернулись в начало кольца и используем освободившиеся ячейки. Вот зачем оператор `%` (остаток), который мы разбирали в главе 2.

Будем хранить `head` и `count` (сколько элементов). Хвост вычисляется: `(head + count) % CAPACITY`. Так не запутаешься, пуста очередь или полна: пуста при `count == 0`, полна при `count == CAPACITY`.

```
#include <stdio.h>
#include <stdlib.h>

#define CAPACITY 5      // длина кольцевого буфера

int main(void)
{
    int data[CAPACITY];     // сама очередь (кольцо)
    int head = 0;           // откуда забираем (начало очереди)
    int count = 0;          // сколько человек сейчас стоит
```

```
    // enqueue: встаём в конец очереди
    for (int value = 1; value <= 6; value++) {
        if (count == CAPACITY) {                    // мест нет
            printf("enqueue %d: очередь полна\n", value);
            continue;
        }
        int tail = (head + count) % CAPACITY;       // первое свободное место
        data[tail] = value;                         // встали в хвост
        count = count + 1;                          // людей стало больше
        printf("enqueue %d (в очереди %d)\n", value, count);
    }

    // dequeue: обслуживаем тех, кто пришёл раньше
    while (count > 0) {
        int value = data[head];                     // берём первого
        head = (head + 1) % CAPACITY;               // голова сдвинулась
        count = count - 1;                          // людей стало меньше
        printf("dequeue %d (осталось %d)\n", value, count);
    }

    if (count == 0) {
        printf("dequeue: очередь пуста\n");
    }

    return 0;
}
```

Разбор:

- `int tail = (head + count) % CAPACITY;` — где хвост. Если голова на 0, а стоят трое, новый встанет на индекс 3. Когда сумма перевалит за `CAPACITY`, `%` завернёт её в начало кольца.
- `head = (head + 1) % CAPACITY;` — после выдачи голова сдвигается на одного и тоже умеет заворачиваться в начало.
- проверки `count == CAPACITY` (полна) и `count > 0` (есть кого выдавать) защищают от выхода за границы и от чтения пустоты.

Запуск:

```
enqueue 1 (в очереди 1)
enqueue 2 (в очереди 2)
enqueue 3 (в очереди 3)
enqueue 4 (в очереди 4)
enqueue 5 (в очереди 5)
enqueue 6: очередь полна
dequeue 1 (осталось 4)
dequeue 2 (осталось 3)
dequeue 3 (осталось 2)
dequeue 4 (осталось 1)
dequeue 5 (осталось 0)
dequeue: очередь пуста
```

Вышло **1, 2, 3, 4, 5** — в том же порядке, в каком вставали. Это FIFO. Сравни со стеком из прошлой части: там был обратный порядок. Файл — `code/11-queue.c`.

Часть 4. Связный список

Зачем он, если есть массив

У массива (даже растущего) есть слабое место: чтобы вставить элемент в **середину**, надо сдвинуть всё, что правее, — это копирование. Связный список вставляет в начало мгновенно и не требует непрерывного куска памяти: узлы могут лежать где угодно, лишь бы каждый знал, **где следующий**.

Связный список (linked list) — это цепочка **узлов** (nodes). Каждый узел хранит своё значение и адрес (указатель) на следующий узел. Последний узел указывает в `NULL` — «дальше никого».

Картинка цепочки из трёх узлов:

```
head
 |
 ▼
[3 | ·]──►[2 | ·]──►[1 | ·]──► NULL
```

В каждой клетке слева значение, справа стрелка `next`. `head` — наш единственный вход в список: указатель на первый узел.

Описываем узел через `struct`

Вот ради чего мы изучали структуры:

```c
struct Node {
    int value;              // полезные данные узла
    struct Node *next;      // адрес следующего узла (или NULL)
};
```

- `int value;` — то, что узел хранит.
- `struct Node *next;` — указатель на **такой же** узел. Заметь: внутри определения `struct Node` мы уже ссылаемся на `struct Node *`. Так можно — это указатель (адрес), его размер компилятору известен, даже пока тип описывается. Узел, ссылающийся на узел того же типа, — и делает цепочку цепочкой.

Добавление в начало — по шагам

Добавлять в начало проще всего. Пусть в списке уже `2 -> 1 -> NULL`, и мы хотим добавить `3`. Три шага:

1. Создаём новый узел через `malloc`, кладём в него `value = 3`.
2. Говорим новому узлу смотреть туда, куда сейчас смотрит `head`: `node->next = head;` — то есть на старый первый узел (`2`).
3. Делаем новый узел головой: `head = node;`.

Покажу стрелками, как меняются указатели. До вставки:

```
head ──▶ [2|·] ──▶ [1|·] ──▶ NULL
```

Шаг 1–2: создали `[3|·]` и направили его `next` на текущую голову:

```
        [3|·]
          │ next
          ▼
head ──▶ [2|·] ──▶ [1|·] ──▶ NULL
```

Шаг 3: переставили `head` на новый узел:

```
head ──▶ [3|·] ──▶ [2|·] ──▶ [1|·] ──▶ NULL
```

Порядок шагов 2 и 3 **нельзя менять**! Если сначала сделать `head = node`, то старую голову мы потеряем — на неё больше никто не указывает, а `node->next` ещё не выставлен. Это называется **потеря головы списка**: вся остальная цепочка станет недостижимой (и утечёт). Сначала `node->next = head`, потом `head = node`.

Обход и освобождение

Обход: заводим «бегунок» `cur`, ставим на `head`, и пока он не `NULL`, печатаем `cur->value` и прыгаем дальше: `cur = cur->next`.

Освобождение хитрее. Нельзя просто `free(cur); cur = cur->next;` — после `free` узел уже мёртв, читать из него `next` нельзя (это разыменование освобождённой памяти). Поэтому **сначала** запоминаем адрес следующего во временную переменную, **потом** освобождаем текущий.

Полная программа

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int value;              // полезные данные узла
    struct Node *next;      // адрес следующего узла (или NULL)
};

int main(void)
{
    struct Node *head = NULL;   // пустой список: головы ещё нет

    // добавляем 1, 2, 3 в начало списка
    for (int value = 1; value <= 3; value++) {
        struct Node *node = malloc(sizeof(struct Node));
        if (node == NULL) {
            printf("Не хватило памяти\n");
            return 1;
        }
        node->value = value;    // заполнили данные
        node->next = head;      // новый смотрит на старую голову
        head = node;            // новый узел становится головой
        printf("добавили %d\n", value);
    }

    // обход: идём от головы по стрелкам next, пока не упрёмся в NULL
    printf("список: ");
    for (struct Node *cur = head; cur != NULL; cur = cur->next) {
```

```
        printf("%d -> ", cur->value);
    }
    printf("NULL\n");

    // освобождение: идём по списку и удаляем каждый узел
    struct Node *cur = head;
    while (cur != NULL) {
        struct Node *next = cur->next;  // СНАЧАЛА запомнили следующий
        free(cur);                      // потом удалили текущий
        cur = next;                     // перешли к запомненному
    }
    head = NULL;                        // список пуст

    return 0;
}
```

Разбор ключевых строк:

- `struct Node *head = NULL;` — пустой список. `NULL` здесь значит «узлов ещё нет». Всегда инициализируй голову в `NULL`, иначе в ней мусор и обход улетит в никуда.
- `malloc(sizeof(struct Node))` — просим память под **один** узел. `sizeof(struct Node)` сам посчитает нужный размер (поле `int` плюс поле-указатель). Возвращённый адрес кладём в `struct Node *node`.
- `node->value = value;` — стрелка: записали данные. Помни, это то же, что `(*node).value = value;`.
- `node->next = head; head = node;` — те самые два шага вставки в правильном порядке.
- цикл обхода `for (cur = head; cur != NULL; cur = cur->next)` — пока бегунок указывает на узел, печатаем и сдвигаемся по `next`. Когда дойдём до последнего, `cur->next` будет `NULL`, условие `cur != NULL` станет ложным — цикл закончится.
- цикл освобождения — на каждый `malloc` свой `free`. Мы выделили три узла, значит, делаем три `free`. Временная `next` спасает от чтения уже освобождённой памяти.

Запуск:

```
добавили 1
добавили 2
добавили 3
список: 3 -> 2 -> 1 -> NULL
```

Заметь: добавляли 1, 2, 3, а список вышел `3 -> 2 -> 1`. Логично — каждый новый узел вставал **в начало**, оттесняя предыдущие. По сути добавление в начало превращает список в стек. Файл — `code/11-list.c`.

Частые ошибки

Забытая `;` после `struct`. После закрывающей `}` определения структуры точка с запятой обязательна: `struct Node { ... };`. Без неё посыплются непонятные ошибки на следующих строках.

Точка вместо стрелки (и наоборот). Если на руках указатель — пиши `p->field`. Если сама структура — `s.field`. `p.field` для указателя компилятор не пропустит.

`data = realloc(data, ...)` напрямую. При неудаче `realloc` вернёт `NULL`, и ты затрёшь единственный адрес старого блока — утечка плюс потеря данных. Пиши во временный указатель и проверяй на `NULL`.

Потеря головы списка. Сначала `node->next = head;`, только потом `head = node;`. Перепутаешь порядок — вся цепочка станет недостижимой.

`free` перед чтением `next`. В цикле освобождения сначала запомни `cur->next` во временную переменную, потом `free(cur)`. Иначе читаешь освобождённую память.

Разыменование `NULL`. Обращение `cur->value`, когда `cur == NULL`, — падение. Перед работой с указателем проверяй, что он не `NULL` (в обходе это делает условие цикла).

Утечки памяти. На каждый `malloc`/`realloc` однажды должен прийтись `free`. У растущего массива — один `free` в конце; у списка — по одному `free` на каждый узел. Стек и очередь у нас на статическом массиве, там `free` не нужен.

Упражнения

1. В растущий массив добавь функцию `print` и пополни его не десятью, а ста числами. Проследи по выводу `[рост: ...]`, до какой вместимости он дорастёт.
2. Сделай стек строковым: храни не `int`, а символы (`char`). Положи в него слово по буквам, затем снимай по одной — на экране слово развернётся задом наперёд.
3. Добавь в очередь подсчёт: после всех операций напечатай, сколько элементов прошло через неё всего. (Заведи отдельный счётчик, который только растёт при enqueue.)
4. В связном списке напиши функцию `int length(struct Node *head)`, возвращающую число узлов (пройди обходом, считай).
5. Напиши функцию `int contains(struct Node *head, int x)`, которая возвращает 1, если значение `x` есть в списке, и 0, если нет.

6. (Посложнее.) Добавь в связный список добавление **в конец**, а не в начало, чтобы порядок сохранялся: `1 -> 2 -> 3 -> NULL`. Подсказка: дойди обходом до узла, у которого `next == NULL`, и подвесь новый к нему.

Что мы получили

Теперь в твоём арсенале четыре структуры данных и понимание, когда какая нужна. **Растущий массив** — когда нужен быстрый доступ по индексу, а размер заранее неизвестен (`realloc` с удвоением). **Стек** — когда важен обратный порядок, LIFO. **Очередь** — когда важен прямой порядок, FIFO, с кольцевым буфером поверх массива. **Связный список** — гибкая цепочка узлов на `struct` и указателях, с дешёвой вставкой и ручным управлением памятью. Плюс мы наконец освоили `struct`, точку, стрелку `->` и `typedef` — без них в Си дальше никуда.

На этом часть про базовые алгоритмы и структуры данных закончена. Дальше начинается самое весёлое — **игры**. В следующей главе напишем первую: классическую **«Угадай число»**. Компьютер загадает число, ты будешь гадать, а программа — подсказывать «больше» или «меньше». Там пригодится почти всё, что мы прошли: ввод, цикл, ветвления, случайные числа. Идём играть.

Глава 12. Игра «Угадай число»

Часть про алгоритмы позади. Теперь начинается самое весёлое — **игры**. Мы напишем настоящие маленькие программы, в которые можно играть, и по дороге выучим то, чего ещё не касались.

Первая игра — классика всех времён. Компьютер загадывает число от 1 до 100, а ты пытаешься его угадать. После каждой попытки программа подсказывает: «больше» или «меньше». В конце она поздравляет тебя и сообщает, за сколько попыток ты справился.

Звучит просто, но по пути мы разберём важную новую тему — **случайные числа**. Без них почти ни одна игра не обходится: бросок кубика, перемешивание колоды, появление врага в неожиданном месте — всё это случайность.

Что мы сделаем

К концу главы у тебя будет программа, которая:

- загадывает **случайное** число от 1 до 100 (и каждый запуск — новое);
- в цикле спрашивает твою догадку;
- отвечает «Больше» или «Меньше», подводя тебя к ответу;
- считает попытки;
- поздравляет, когда ты угадал, и не падает, если ввести ерунду.

Новое в этой главе:

- генератор случайных чисел: `rand`, `srand`, `time`;
- как из `rand()` получить число в нужном диапазоне (`% 100 + 1`);
- игровой цикл `while`, который крутится, **пока не угадал**;
- защита от неправильного ввода.

Разбираем задачу руками

Сыграем партию на бумаге. Пусть компьютер загадал **45** (мы-то этого не знаем, диапазон 1..100).

Самая глупая стратегия — называть числа подряд: 1, 2, 3… В худшем случае это 100 попыток. Долго.

Умнее — каждый раз бить в **середину** оставшегося диапазона. Это та самая идея, что и в бинарном поиске из главы 7, только теперь её выполняешь ты, а «массив» — это числа от 1 до 100.

```
диапазон 1..100, говорю 50  → «Меньше» (значит, секрет < 50) → 1..49
диапазон 1..49,  говорю 25  → «Больше» (значит, секрет > 25) → 26..49
диапазон 26..49, говорю 37  → «Больше» → 38..49
диапазон 38..49, говорю 43  → «Больше» → 44..49
диапазон 44..49, говорю 46  → «Меньше» → 44..45
диапазон 44..45, говорю 44  → «Больше» → 45..45
говорю 45 → «Угадал!»
```

Семь попыток вместо сотни. Каждый вопрос делит диапазон пополам, поэтому даже для 100 чисел хватает максимум семи догадок. Эту стратегию мы потом подарим компьютеру в упражнениях — пусть он сам угадывает твоё число.

А пока загадывать будет машина. Значит, сперва надо научить её придумывать случайное число.

Шаг 1. Откуда брать случайность

В Си генератор случайных чисел живёт в заголовке `<stdlib.h>` (*standard library*). Главная функция — `rand` (от *random* — «случайный»). Она не принимает аргументов и возвращает случайное целое от 0 до некоторого большого предела `RAND_MAX`.

Попробуем напечатать три «случайных» числа:

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    printf("%d\n", rand());
    printf("%d\n", rand());
```

```
    printf("%d\n", rand());
    return 0;
}
```

Новое:

- `#include <stdlib.h>` — подключаем библиотеку, где лежит `rand`.
- `rand()` — вызов функции без аргументов (скобки пустые). Возвращает целое число, которое мы тут же печатаем через `%d`.

Запустим:

```
1804289383
846930886
1681692777
```

Числа и правда выглядят случайными. Но запусти программу **ещё раз**:

```
1804289383
846930886
1681692777
```

Те же самые! И в третий раз будут те же. Это и есть проблема.

Шаг 2. Почему `rand()` повторяется и что такое «засев»

`rand()` не достаёт числа из воздуха. Внутри он считает их по формуле из предыдущего числа. Самое первое число он берёт от стартового значения — **зерна** (англ. *seed*). По умолчанию зерно всегда одно и то же, поэтому и вся цепочка чисел каждый запуск одинаковая.

Для игры это никуда не годится: компьютер будет загадывать одно и то же число при каждом старте, и угадать его станет скучно.

Решение — **засеять** генератор разным зерном перед игрой. За это отвечает функция `srand` (*seed random* — «засеять случайность»), тоже из `<stdlib.h>`. Она принимает одно целое — зерно:

```
srand(42);   // засеяли зерном 42
```

Но если всегда писать `srand(42)`, зерно опять постоянное — толку ноль. Нам нужно зерно, которое **само меняется** от запуска к запуску. Идеальный кандидат — текущее время: оно не стоит на месте.

Шаг 3. Зерно из текущего времени — `time`

Узнать текущее время умеет функция `time` из заголовка `<time.h>` (*time* — «время»). Вызов `time(NULL)` возвращает число секунд, прошедших с 1 января 1970 года. Каждую секунду это число другое — отличный источник свежего зерна.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    printf("%d\n", rand());
    printf("%d\n", rand());
    printf("%d\n", rand());
    return 0;
}
```

Разбор новой строки `srand((unsigned int)time(NULL));`:

- `#include <time.h>` — подключили заголовок ради `time`.
- `time(NULL)` — спрашиваем текущее время. Аргумент `NULL` означает «результат никуда дополнительно не записывай, просто верни его» (`time` умеет ещё писать ответ по переданному адресу, но нам это не нужно).
- `(unsigned int)` — приведение типа (мы видели приведения в главе 3). `time` возвращает специальный тип `time_t`, а `srand` ждёт `unsigned int` («беззнаковое целое» — целое без минуса). Приводим одно к другому, чтобы компилятор не ворчал.
- `srand(...)` — засеваем генератор полученным зерном. **Эту строку пишут один раз, в самом начале программы.** Не нужно звать `srand` перед каждым `rand()` — наоборот, так станет хуже.

Теперь каждый запуск даёт новую тройку чисел. Случайность настоящая.

Важное правило: **`srand` вызывают ровно один раз за всю программу.** Дальше сколько угодно раз дёргают `rand()` — он сам продвигается по цепочке.

Шаг 4. Сужаем диапазон до 1..100

`rand()` возвращает огромное число (до миллиардов), а нам нужно от 1 до 100. Тут на помощь приходит **остаток от деления** `%`, который мы разбирали в главе 2.

Вспомни: `a % b` — это остаток от деления `a` на `b`. Каким бы большим ни было `a`, остаток от деления на 100 всегда попадает в диапазон **0..99** (остаток не может быть равен делителю или больше него).

```
rand() % 100        // число от 0 до 99
```

Нам нужно 1..100, а не 0..99. Просто прибавим 1 — весь диапазон сдвинется на единицу вверх:

```
rand() % 100 + 1    // число от 1 до 100
```

Разберём по шагам на примере. Пусть `rand()` вернул 1804289383.

- `1804289383 % 100` = 83 (последние две цифры — это и есть остаток от деления на 100).
- `83 + 1` = 84.

Получили 84 — годное число из диапазона 1..100. Если бы остаток вышел 0, после `+ 1` стало бы 1 (нижняя граница); если остаток 99 — после `+ 1` станет 100 (верхняя). Концы диапазона на месте.

Общая формула на будущее: чтобы получить число от `min` до `max` включительно, пиши `rand() % (max - min + 1) + min`. Для 1..100 это `rand() % 100 + 1`.

Проверим, что числа правда попадают в диапазон:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));
    for (int i = 0; i < 5; i++) {
        printf("%d\n", rand() % 100 + 1);
    }
    return 0;
}
```

```
84
7
100
```

```
56
23
```

Все пять — в пределах 1..100. Генератор загадок готов.

Шаг 5. Одна попытка угадать

Сложим первую наивную версию игры: загадываем число, спрашиваем **одну** догадку, сравниваем.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));
    int secret = rand() % 100 + 1;

    int guess;
    printf("Угадай число от 1 до 100: ");
    scanf("%d", &guess);

    if (guess < secret) {
        printf("Больше.\n");
    } else if (guess > secret) {
        printf("Меньше.\n");
    } else {
        printf("Угадал!\n");
    }
    return 0;
}
```

Разбор:

- `int secret = rand() % 100 + 1;` — загадали число и положили в переменную `secret` («секрет»). Игрок его не видит.
- `scanf("%d", &guess);` — читаем догадку. Не забываем `&` перед `guess` (см. главу 2): `scanf` нужен **адрес**, куда записать ввод.
- сравнение в три ветки:

- `guess < secret` — игрок назвал **меньше**, чем загадано, значит, нужное число **больше**. Печатаем «Больше».
- `guess > secret` — наоборот, «Меньше».
- иначе числа равны — «Угадал!».

Запуск:

```
Угадай число от 1 до 100: 50
Меньше.
```

Работает, но играть невозможно: программа задаёт **один** вопрос и завершается. Чтобы реально угадать, придётся запускать её снова и снова — и каждый раз будет новое число. Нужен **цикл**, который повторяет вопрос, пока не угадаешь.

Шаг 6. Игровой цикл — спрашиваем, пока не угадал

Обернём вопрос и сравнение в цикл `while`. Условие цикла — «пока догадка не равна секрету»:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));
    int secret = rand() % 100 + 1;
    int guess = 0;

    printf("Я загадал число от 1 до 100. Угадай!\n");

    while (guess != secret) {
        printf("Твой вариант: ");
        scanf("%d", &guess);

        if (guess < secret) {
            printf("Больше.\n");
        } else if (guess > secret) {
            printf("Меньше.\n");
        } else {
            printf("Угадал!\n");
```

```
        }
    }
    return 0;
}
```

Что изменилось:

- `int guess = 0;` — теперь догадку **обязательно** задаём заранее значением 0. Почему именно 0? Потому что условие `while` проверяется **до** первого ввода, а секрет всегда в диапазоне 1..100. Значит, `0 != secret` гарантированно истинно, и цикл точно зайдёт внутрь хотя бы раз. Если бы мы оставили `guess` без значения, там лежал бы мусор — могло бы случайно совпасть с секретом (маловероятно, но оставлять переменную без значения нельзя).
- `while (guess != secret)` — «пока `guess` **не равно** `secret`, повторяй тело». Оператор `!=` — «не равно» (см. главу 2). Как только игрок введёт число, равное секрету, условие станет ложным, и цикл закончится.
- внутри — тот же вопрос и то же сравнение, что и раньше, только теперь они повторяются.

Запуск:

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: 50
Меньше.
Твой вариант: 25
Больше.
Твой вариант: 37
Больше.
Твой вариант: 43
Больше.
Твой вариант: 45
Угадал!
```

Вот это уже игра! Цикл крутится, подсказки ведут к ответу, на верной догадке всё завершается.

Чего не хватает? Игра не считает попытки. Добавим счётчик.

Шаг 7. Считаем попытки

Заведём переменную-счётчик и будем увеличивать её на каждой догадке. Приём знакомый — так мы считали в циклах в главе 2.

```
int attempts = 0;
```

Внутри цикла, сразу после ввода, добавим:

```
attempts++;
```

`attempts++` — увеличить `attempts` на 1 (краткая запись `attempts = attempts + 1`). Ставим её после `scanf`, чтобы каждая введённая догадка засчитывалась как попытка.

А в ветке «Угадал!» напечатаем итог:

```
printf("Угадал! Это было число %d.\n", secret);
printf("Тебе понадобилось попыток: %d.\n", attempts);
```

Теперь игра поздравляет и сообщает результат. Осталась одна неприятность — что будет, если вместо числа ввести буквы?

Шаг 8. Защита от неправильного ввода

Проверим версию из шага 6 на прочность: вместо числа введём `abc`.

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: abc
Больше.
Твой вариант: Больше.
Твой вариант: Больше.
Твой вариант: Больше.
Твой вариант: Больше.
```

Здесь три точки в конце были бы честнее любого числа строк: это не обрезанный вывод, а **бесконечный**. Программа сыплет «Больше» до тех пор, пока ты не остановишь её Ctrl+C. Почему?

`scanf("%d", &guess)` ждёт **целое число**. Когда на входе буквы, он не может ничего прочитать: переменную `guess` не меняет, а буквы **оставляет во входном буфере** — они никуда не делись. На следующем витке `scanf` снова натыкается на те же буквы, снова ничего не читает, и так бесконечно. Это и есть зацикливание.

Спасает то, что `scanf` **возвращает значение** — сколько значений ему удалось прочитать. Если он успешно прочёл наше число, вернёт 1. Если не смог (буквы вместо цифр) — вернёт 0. А если ввод вообще закончился (пользователь нажал Ctrl+D или ввод идёт из файла, который кончился) — вернёт особое значение `EOF` (*end of file* — «конец ввода»).

Проверяем результат `scanf`:

```c
int rc = scanf("%d", &guess);
if (rc == EOF) {
    printf("\nИгра прервана. Я загадывал %d.\n", secret);
    return 0;
}
if (rc != 1) {
    printf("Это не число. Введи целое от 1 до 100.\n");
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        /* выбрасываем мусорный ввод до конца строки */
    }
    continue;
}
```

Разбираем по кускам:

- `int rc = scanf(...)` — сохраняем результат `scanf` в переменную `rc` (от *return code* — «код возврата»), чтобы проверить его.
- `if (rc == EOF)` — ввод иссяк. Дальше спрашивать некого: вежливо прощаемся и выходим через `return 0`. Без этой проверки на конце ввода программа тоже зациклилась бы.
- `if (rc != 1)` — `scanf` не смог прочитать число (попались буквы). Сообщаем об ошибке и **чистим буфер** от застрявшего мусора:
  - `getchar()` — функция из `<stdio.h>`, читает **один символ** из ввода и возвращает его (или `EOF`, если ввод кончился).
  - `while ((c = getchar()) != '\n' && c != EOF)` — крутим, пока не дочитаем до конца строки (символ `'\n'` — перевод строки от Enter) или до конца ввода. Так мы выбрасываем всё, что осталось от неудачной попытки, и следующий `scanf` начнёт с чистого листа. Само тело цикла пустое — вся работа в условии.
  - `continue;` — оператор, который **прерывает текущий виток цикла** и переходит к проверке условия `while` заново. Мы не считаем такой ввод за попытку и не сравниваем мусор с секретом — просто спрашиваем заново.

Маленькая деталь: счётчик `attempts++` теперь должен стоять **после** этих проверок — чтобы неудачный ввод не засчитывался как попытка.

Полная программа

Собираем всё вместе. Файл — `code/12-guess.c`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    int secret = rand() % 100 + 1;
    int guess = 0;
    int attempts = 0;

    printf("Я загадал число от 1 до 100. Угадай!\n");

    while (guess != secret) {
        printf("Твой вариант: ");

        int rc = scanf("%d", &guess);
        if (rc == EOF) {           /* ввод закончился (Ctrl+D) */
            printf("\nИгра прервана. Я загадывал %d.\n", secret);
            return 0;
        }
        if (rc != 1) {             /* ввели не число */
            printf("Это не число. Введи целое от 1 до 100.\n");
            int c;
            while ((c = getchar()) != '\n' && c != EOF) {
                /* выбрасываем мусорный ввод до конца строки */
            }
            continue;
        }

        attempts++;

        if (guess < 1 || guess > 100) {
            printf("Число должно быть от 1 до 100.\n");
        } else if (guess < secret) {
            printf("Больше.\n");
        } else if (guess > secret) {
```

```
            printf("Меньше.\n");
        } else {
            printf("Угадал! Это было число %d.\n", secret);
            printf("Тебе понадобилось попыток: %d.\n", attempts);
        }
    }

    return 0;
}
```

Единственное, чего мы ещё не обсудили, — проверка `if (guess < 1 || guess > 100)`. Это вежливость к игроку: если он ввёл число вне диапазона (например, 200 или −5), мы напоминаем правила, а не говорим «Меньше». Оператор `||` — «ИЛИ» (см. главу 2): условие истинно, если число меньше 1 **или** больше 100.

Как это работает

Пройдём по программе сверху вниз.

1. `srand((unsigned int)time(NULL))` засевает генератор текущим временем — один раз на старте. Благодаря этому каждый запуск даёт новую загадку.
2. `rand() % 100 + 1` берёт случайное число и ужимает его до диапазона 1..100. Результат прячем в `secret`.
3. `guess = 0` гарантирует, что цикл зайдёт внутрь хотя бы раз (0 не равно ни одному числу из 1..100).
4. Цикл `while (guess != secret)` крутится, пока игрок не назовёт секрет. Каждый виток:
   - печатаем приглашение;
   - читаем ввод и проверяем `scanf`: конец ввода → выходим; не число → ругаемся, чистим буфер, начинаем виток заново;
   - засчитываем попытку;
   - сравниваем догадку с секретом и печатаем подсказку;
   - на совпадении печатаем поздравление со счётчиком. Тут же условие `guess != secret` становится ложным, и цикл завершается.
5. `return 0` — программа отработала без ошибок.

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 code/12-guess.c -o guess
./guess
```

Пример партии (играем серединой диапазона, как разбирали руками):

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: 50
Меньше.
Твой вариант: 25
Больше.
Твой вариант: 37
Больше.
Твой вариант: 43
Больше.
Твой вариант: 46
Меньше.
Твой вариант: 44
Больше.
Твой вариант: 45
Угадал! Это было число 45.
Тебе понадобилось попыток: 7.
```

Проверим устойчивость к неверному вводу:

```
Я загадал число от 1 до 100. Угадай!
Твой вариант: abc
Это не число. Введи целое от 1 до 100.
Твой вариант: 200
Число должно быть от 1 до 100.
Твой вариант: ^D
Игра прервана. Я загадывал 15.
```

Буквы отвергнуты, число вне диапазона отвергнуто, а Ctrl+D (`^D`) аккуратно завершает игру вместо зависания.

Частые ошибки

Забыли `srand`. Если не засеять генератор, `rand()` каждый запуск выдаёт одну и ту же последовательность — компьютер будет загадывать одно и то же число. Всегда зови `srand((unsigned int)time(NULL))` один раз в начале.

Зовут `srand` перед каждым `rand()`. Частая «починка наоборот». Если засевать генератор в цикле, да ещё временем, которое за быстрый цикл не успевает измениться, числа начнут повторяться. `srand` — ровно один раз за программу.

Используют `rand()` без диапазона. `rand()` сам по себе возвращает гигантское число. Без `% 100 + 1` секрет окажется где-то в миллиардах, и угадать его нельзя. Не забывай ужимать диапазон.

Промахнулись с `% 100` против `% 101`. `rand() % 100` даёт 0..99. Чтобы получить 1..100, нужно `+ 1`. Если хочешь 0..100 (101 число), тогда `% 101`. Считай границы внимательно.

Забыли `&` в `scanf`. `scanf("%d", guess)` без `&` — почти гарантированное падение (см. главу 2). Нужно `scanf("%d", &guess)`.

Не проверяют результат `scanf`. Если не ловить случай «введены не цифры», программа зациклится на мусоре во входном буфере, как мы видели в шаге 8.

Упражнения

1. **Другой диапазон.** Сделай, чтобы компьютер загадывал число от 1 до 1000. Подсказка: меняется только формула `rand() % ... + 1`. Сколько попыток теперь нужно при стратегии «бей в середину»?

2. **Ограничь число попыток.** Дай игроку не больше 7 попыток. Если он не угадал за 7 ходов — выведи «Ты проиграл, я загадывал N» и заверши игру. Подсказка: добавь в условие цикла проверку `attempts < 7` через `&&`.

3. **«Горячо/холодно».** Кроме «Больше»/«Меньше» подсказывай, насколько игрок близок. Если разница между догадкой и секретом меньше 5 — печатай «Горячо!», если меньше 15 — «Тепло», иначе — «Холодно». Подсказка: пригодится модуль разности. Своей функции `abs` писать не обязательно — можно сравнить `guess - secret` и `secret - guess`.

4. **Сыграй ещё раз.** После завершения партии спроси «Сыграем ещё? (1 — да, 0 — нет)» и, если игрок ввёл 1, загадай новое число и начни сначала. Подсказка: оберни всю игру во внешний цикл.

5. **Компьютер угадывает.** Поменяй роли: теперь **ты** загадываешь число (держишь его в голове), а компьютер угадывает по стратегии «бей в середину». Программа называет число, ты отвечаешь буквой: `b` — «больше», `m` — «меньше», `t` — «точно, угадал». Компьютер ведёт границы `low` и `high`, каждый раз называя `(low + high) / 2`, и сужает диапазон по твоему ответу. Это тот самый бинарный поиск из главы 7, только вживую. Проверь: за сколько ходов компьютер загоняет любое число от 1 до 100?

6. **Счётчик рекордов.** Запоминай лучший результат (наименьшее число попыток) за несколько партий и показывай его перед каждой новой игрой.

Что мы получили

Мы написали первую полноценную игру и заодно открыли тему, без которой дальше никуда, — **случайные числа**. Теперь ты знаешь, как засеять генератор временем (`srand((unsigned int)time(NULL))`), как получить случайное число (`rand()`) и как загнать его в нужный диапазон (`rand() % 100 + 1`). Ты собрал игровой цикл `while`, который крутится, пока не выполнено условие победы, научился считать попытки и не дал программе сломаться от неправильного ввода — проверяя результат `scanf` и подчищая входной буфер.

В следующей главе сделаем ещё одну классику — **«камень, ножницы, бумага»**. Компьютер будет выбирать ход случайно (`rand` уже в кармане), а нам предстоит разобрать чуть более хитрую логику сравнения ходов и аккуратно посчитать счёт за несколько раундов.

Глава 13. Игра «Камень, ножницы, бумага»

В прошлой главе компьютер загадывал число, а ты угадывал. Теперь поменяем роли местами: ты и компьютер играете на равных в «Камень, ножницы, бумага». Компьютер делает случайный выбор, ты — свой, и кто-то выигрывает раунд. Сыграем несколько раундов, посчитаем очки и в конце объявим победителя матча.

По дороге познакомимся с новой полезной штукой — **перечислением** (`enum`), научимся аккуратно превращать выбор игрока в число, разберём красивый приём с остатком от деления для определения победителя и сделаем ввод устойчивым к ерунде вроде `hello` или `5`.

Что мы сделаем

К концу главы у нас будет программа `code/13-rps.c`, которая:

- предлагает выбрать камень, ножницы или бумагу (или выйти);
- сама случайно выбирает свой ход;
- говорит, кто выиграл раунд;
- ведёт счёт игрока и компьютера;
- играет столько раундов, сколько захочешь, а по команде выхода печатает итог матча.

Всё, что нужно, у тебя уже есть из прошлых глав: переменные, `if`, циклы, функции (главы 2), массивы и строки (глава 3), а также `rand`/`srand`/`time` для случайных чисел (глава 12). Случайность мы заново разбирать не будем — просто пользуемся.

Разбираем задачу руками

Сначала договоримся о правилах. Их три:

- камень бьёт ножницы (тупит лезвие);
- ножницы бьют бумагу (режут);

- бумага бьёт камень (заворачивает).

Если оба выбрали одно и то же — ничья.

Чтобы компьютер мог «думать», выбор нужно превратить в число. Закодируем ходы так:

- `0` — камень;
- `1` — ножницы;
- `2` — бумага.

Кодировка не случайна. Посмотри, что бьёт что: камень (0) бьёт ножницы (1), ножницы (1) бьют бумагу (2), бумага (2) бьёт камень (0). Каждый ход бьёт **следующий по кругу**: 0→1→2→0. Эта закономерность скоро очень нам пригодится.

Выпишем все девять исходов в таблицу. Слева — ход игрока, сверху — ход компьютера, в клетке — кто выиграл (И = игрок, К = компьютер, Н = ничья):

```
                компьютер
              камень ножницы бумага
игрок    ┌──────────────────────────
камень   │   Н       И        К
ножницы  │   К       Н        И
бумага   │   И       К        Н
```

По диагонали — ничьи (выбрали одинаково). Остальные клетки — кто кого побил по правилам выше. Эту таблицу мы и научим программу заполнять.

Шаг 1. Один раунд, выбор компьютера

Начнём с самого маленького: компьютер делает случайный ход, мы его печатаем. Случайное число от 0 до 2 даёт `rand() % 3` — остаток от деления на 3 всегда попадает в диапазон 0..2 (см. главу 12).

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void)
{
    srand((unsigned int)time(NULL));

    int computer = rand() % 3;
    printf("Компьютер выбрал: %d\n", computer);
```

```
    return 0;
}
```

Здесь всё знакомо по главе 12:

- `#include <stdlib.h>` — оттуда `rand` и `srand`.
- `#include <time.h>` — оттуда `time`.
- `srand((unsigned int)time(NULL));` — «встряхиваем» генератор текущим временем, чтобы при каждом запуске числа были разными.
- `rand() % 3` — случайное число, остаток от деления на 3, то есть 0, 1 или 2.

Запуск печатает что-то вроде `Компьютер выбрал: 2`. Работает, но человеку «2» ни о чём не говорит. Нужно показывать слово, а не число.

Шаг 2. Превращаем число в слово

Можно было бы каждый раз писать `if (computer == 0) printf("камень")` и так далее. Но это придётся повторять для игрока и для компьютера — дублирование. Лучше сделать **функцию**, которая по числу возвращает название хода.

Функция должна вернуть текст. Текст в Си — это массив символов, а доступ к нему мы передаём через адрес первого символа, то есть через **указатель на `char`** (см. главу 3). Поэтому тип результата — `const char *`:

```
const char *move_name(int m)
{
    if (m == 0) return "камень";
    if (m == 1) return "ножницы";
    return "бумага";
}
```

Разбор того, что здесь нового:

- `const char *` — тип возвращаемого значения: «указатель на символы», то есть строка. Слово `const` означает «только для чтения»: вызывающий не должен менять эти буквы. Это разумно, ведь мы возвращаем **строковый литерал** — текст `"камень"`, зашитый в программу.
- `return "камень";` — возвращаем адрес начала этой строки. Строковый литерал живёт в памяти всё время работы программы, так что возвращать на него указатель безопасно.
- Третий вариант (`return "бумага";`) стоит без `if`: если число не 0 и не 1, остаётся только 2 — бумага.

Теперь вывод понятен человеку:

```
printf("Компьютер выбрал: %s\n", move_name(computer));
```

`%s` печатает строку по указателю, который вернула функция (см. главу 3).

Шаг 3. Даём имена числам — `enum`

Числа 0, 1, 2 работают, но в коде `if (m == 1)` непонятно, что такое «1». Через месяц ты и сам забудешь. В Си для таких случаев есть **перечисление** — `enum` (от англ. *enumeration*, «перечисление»).

`enum` позволяет дать числам понятные имена:

```
enum Move { ROCK, SCISSORS, PAPER };
```

Что здесь происходит:

- `enum Move { ... }` — объявляем новый набор именованных целых констант под общим названием `Move`.
- `ROCK, SCISSORS, PAPER` — имена. По умолчанию `enum` раздаёт значения с нуля по порядку: `ROCK` это 0, `SCISSORS` это 1, `PAPER` это 2. Ровно наша кодировка! Можно задать и явно (`enum Move { ROCK = 0, SCISSORS = 1, PAPER = 2 };`), но и так сходится.

Имена `enum` — это обычные целые числа, просто с человеческим лицом. Можно писать `ROCK` вместо `0` где угодно: в сравнениях, в `switch`, в арифметике. Компилятор подставит число. Перепишем `move_name` понятнее:

```
const char *move_name(int m)
{
    if (m == ROCK)     return "камень";
    if (m == SCISSORS) return "ножницы";
    return "бумага";
}
```

Логика та же, но теперь код читается как фраза: «если ход — камень, верни “камень”». Имена-константы делают программу понятной без комментариев.

Шаг 4. Кто победил — наивный способ

Теперь главное: по двум ходам определить победителя. Самый честный путь — просто перечислить все случаи, как в таблице. Сначала ничья, потом три случая победы игрока, остальное — победа компьютера.

```
int player = ROCK;        /* пока зашьём вручную, ввод будет позже */
int computer = rand() % 3;

if (player == computer) {
    printf("Ничья.\n");
} else if ((player == ROCK     && computer == SCISSORS) ||
           (player == SCISSORS  && computer == PAPER)    ||
           (player == PAPER     && computer == ROCK)) {
    printf("Ты выиграл раунд!\n");
} else {
    printf("Компьютер выиграл раунд.\n");
}
```

Разбор:

- `player == computer` — выбрали одинаково, значит ничья.
- большое условие через `||` («ИЛИ», см. главу 2) перечисляет три выигрышных для игрока расклада прямо по правилам: камень против ножниц, ножницы против бумаги, бумага против камня. Если хоть один верен — победа игрока.
- `else` ловит всё остальное: раз не ничья и не победа игрока, значит выиграл компьютер.

Это абсолютно правильно и понятно. Единственный минус — громоздко: три строки условий, которые легко набрать с опечаткой. Покажу способ короче.

Шаг 5. Красивый приём с остатком

Вернёмся к замеченной закономерности: каждый ход бьёт **следующий по кругу** (0→1→2→0). Посчитаем такое выражение:

```
int result = (computer - player + 3) % 3;
```

Идея: насколько ход компьютера «опережает» ход игрока по кругу. Разберём по кусочкам.

- `computer - player` — разница ходов. Может быть −2, −1, 0, 1 или 2.
- `+ 3` — добавляем 3, чтобы убрать минус. Остаток от деления отрицательного числа в Си ведёт себя неудобно, а после +3 число точно неотрицательное.
- `% 3` — берём остаток от деления на 3. Прибавленная тройка на остаток не влияет (3 делится на 3 без остатка), зато минусы исчезли. Результат всегда 0, 1 или 2.

Что означают эти три значения? Проверим на каждой паре ходов:

- **0** — ходы равны (`computer == player`): ничья.

- **1** — компьютер на один шаг «впереди» игрока по кругу. Например, игрок камень (0), компьютер ножницы (1): `(1 - 0 + 3) % 3 = 1`. Камень бьёт ножницы — выиграл **игрок**.
- **2** — компьютер на два шага впереди (то же самое, что игрок на один шаг впереди компьютера). Например, игрок ножницы (1), компьютер камень (0): `(0 - 1 + 3) % 3 = 2`. Камень бьёт ножницы — выиграл **компьютер**.

Коротко: `0` — ничья, `1` — победа игрока, `2` — победа компьютера. Одна строка заменила всю таблицу. Чтобы это не было «магией», вот честная проверка всех девяти исходов — она совпадает с таблицей из начала главы:

```
игрок=камень  комп=камень  -> ничья
игрок=камень  комп=ножницы -> игрок
игрок=камень  комп=бумага  -> комп
игрок=ножницы комп=камень  -> комп
игрок=ножницы комп=ножницы -> ничья
игрок=ножницы комп=бумага  -> игрок
игрок=бумага  комп=камень  -> игрок
игрок=бумага  комп=ножницы -> комп
игрок=бумага  комп=бумага  -> ничья
```

Приём работает только потому, что мы заранее **удачно занумеровали** ходы (каждый бьёт следующего по кругу). Поменяй кодировку — и формула сломается. Если такой трюк кажется слишком хитрым, оставь явные `if` из шага 4: они длиннее, но понятнее, и это нормальный выбор. Мы возьмём короткую формулу, но теперь ты знаешь, почему она верна.

Шаг 6. Ввод игрока с проверкой

Игрок должен вводить свой ход. Хочется принимать число (0, 1 или 2) и букву `q` для выхода, и при этом не падать от мусора вроде `hello` или `7`.

Прочитаем ввод как одно «слово» в буфер, а потом разберёмся, что это. Так мы не запутаемся, цифру ввели или букву.

```c
char buf[16];

if (scanf("%15s", buf) != 1) {
    break;              /* ввод кончился (Ctrl+D) — выходим */
}

if (buf[0] == 'q' || buf[0] == 'Q') {
    break;
```

```
}

if (buf[0] < '0' || buf[0] > '2' || buf[1] != '\0') {
    printf("Не понял. Введи 0, 1, 2 или q.\n");
    continue;
}

int player = buf[0] - '0';
```

Разбираем построчно:

- `char buf[16];` — массив символов на 16 байт под введённое слово (см. главу 3). С большим запасом.
- `scanf("%15s", buf)` — читаем слово (до пробела) в `buf`. Перед `buf` нет `&`: имя массива само по себе — адрес начала (см. главу 3). `%15s` ограничивает чтение 15 символами, чтобы не переполнить буфер из 16 (последний байт — под `'\0'`). `scanf` возвращает, сколько значений он успешно прочитал; `!= 1` означает «прочитать не удалось» — обычно это конец ввода (Ctrl+D). Тогда `break` выходит из цикла. Проверять результат `scanf` мы научились в главе 12.
- `buf[0] == 'q' || buf[0] == 'Q'` — первый символ буквы выхода, в любом регистре. Если это `q` — выходим из цикла через `break`.
- большое `if` — проверка корректности. Условие верно (значит ввод плохой), если выполнено хоть что-то:
  - `buf[0] < '0' || buf[0] > '2'` — первый символ не цифра из набора 0,1,2. Символы сравниваем как числа (у `char` есть числовой код, см. главу 2), а цифры в таблице символов идут подряд, поэтому такое сравнение работает.
  - `buf[1] != '\0'` — после первого символа сразу должен стоять конец строки `'\0'`. Если там что-то есть, значит ввели больше одного символа (например `12` или `5x`) — это не наш одиночный ход. Про `'\0'` как конец строки — глава 3.
  - при плохом вводе печатаем подсказку и `continue` — переходим к следующему витку цикла, заново спрашиваем ход.
- `int player = buf[0] - '0';` — превращаем символ-цифру в число. Хитрость в том, что коды символов `'0'`, `'1'`, `'2'` идут подряд. Значит `'0' - '0'` это 0, `'1' - '0'` это 1, `'2' - '0'` это 2. Вычитание кода `'0'` переводит символ цифры в само число. Это стандартный приём, запомни его.

Полная программа

Собираем всё вместе: цикл раундов, счёт, итог матча. Это и есть файл `code/13-rps.c`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

enum Move { ROCK, SCISSORS, PAPER };

const char *move_name(int m)
{
    if (m == ROCK)     return "камень";
    if (m == SCISSORS) return "ножницы";
    return "бумага";
}

int main(void)
{
    srand((unsigned int)time(NULL));

    int player_score = 0;
    int computer_score = 0;
    int rounds = 0;
    char buf[16];

    printf("Камень, ножницы, бумага!\n");
    printf("Выбор: 0 – камень, 1 – ножницы, 2 – бумага, q – выход.\n");

    while (1) {
        printf("\nТвой ход: ");

        if (scanf("%15s", buf) != 1) {
            break;              /* ввод кончился (Ctrl+D) – выходим */
        }

        if (buf[0] == 'q' || buf[0] == 'Q') {
            break;
        }

        if (buf[0] < '0' || buf[0] > '2' || buf[1] != '\0') {
            printf("Не понял. Введи 0, 1, 2 или q.\n");
```

```
            continue;
        }

        int player = buf[0] - '0';
        int computer = rand() % 3;

        printf("Ты: %s. Компьютер: %s. ",
               move_name(player), move_name(computer));

        int result = (computer - player + 3) % 3;

        if (result == 0) {
            printf("Ничья.\n");
        } else if (result == 1) {
            printf("Ты выиграл раунд!\n");
            player_score++;
        } else {
            printf("Компьютер выиграл раунд.\n");
            computer_score++;
        }

        rounds++;
    }

    printf("\n=== Итог ===\n");
    printf("Сыграно раундов: %d\n", rounds);
    printf("Ты: %d, компьютер: %d.\n", player_score, computer_score);

    if (player_score > computer_score) {
        printf("Поздравляю, ты победил в матче!\n");
    } else if (player_score < computer_score) {
        printf("Компьютер оказался сильнее. В другой раз!\n");
    } else {
        printf("Ничья по матчу.\n");
    }

    return 0;
}
```

Как это работает

- В начале `main` заводим три счётчика: `player_score`, `computer_score` и `rounds` (сколько раундов сыграно). Все по нулям.
- `while (1)` — **бесконечный цикл**. Условие `1` — это всегда «истина» (ложь — 0, истина — любое ненулевое, см. главу 2). Сам по себе такой цикл не кончится никогда; выходим из него вручную через `break` — когда игрок ввёл `q` или ввод закончился. `break` мгновенно прекращает цикл и передаёт управление строкам после него.
- Внутри: спрашиваем ход, проверяем ввод (шаг 6), получаем `player`, бросаем кубик за компьютер (`rand() % 3`), печатаем оба хода.
- `result = (computer - player + 3) % 3` — определяем победителя одной формулой (шаг 5). По её значению печатаем итог раунда и при победе увеличиваем нужный счётчик через `++` (см. главу 2).
- `rounds++` в конце витка считает сыгранные раунды (плохой ввод сюда не доходит — там был `continue`).
- После цикла печатаем итог: сколько раундов, какой счёт, и кто победил в матче — сравнением `player_score` и `computer_score`.

Функция `move_name` стоит **выше** `main`, потому что `main` её вызывает, а функцию нужно объявить до вызова (см. главу 2).

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 code/13-rps.c -o rps
./rps
```

Можно играть руками, а можно «скормить» ходы через конвейер (pipe), чтобы не печатать вручную. Команда `printf` в терминале отправит строки во ввод программы:

```
printf '0\n1\n2\nq\n' | ./rps
```

Пример игры (ходы компьютера у тебя будут свои — они случайные):

```
Камень, ножницы, бумага!
Выбор: 0 - камень, 1 - ножницы, 2 - бумага, q - выход.

Твой ход: Ты: камень. Компьютер: камень. Ничья.

Твой ход: Ты: ножницы. Компьютер: ножницы. Ничья.
```

```
Твой ход: Ты: бумага. Компьютер: камень. Ты выиграл раунд!

Твой ход:
=== Итог ===
Сыграно раундов: 3
Ты: 1, компьютер: 0.
Поздравляю, ты победил в матче!
```

Проверим и защиту ввода — подсунем мусор перед нормальным ходом:

```
printf '5\nhello\n1\nq\n' | ./rps
```

```
Камень, ножницы, бумага!
Выбор: 0 — камень, 1 — ножницы, 2 — бумага, q — выход.

Твой ход: Не понял. Введи 0, 1, 2 или q.

Твой ход: Не понял. Введи 0, 1, 2 или q.

Твой ход: Ты: ножницы. Компьютер: ножницы. Ничья.

Твой ход:
=== Итог ===
Сыграно раундов: 1
Ты: 0, компьютер: 0.
Ничья по матчу.
```

`5` и `hello` программа отвергла и переспросила, а `1` приняла и сыграла раунд. Падения нет, зацикливания тоже.

Обрати внимание на строчки «Твой ход:» без ответа рядом. Через конвейер ввод не печатается на экран — эхо набранных символов рисует терминал, а тут никто ничего не набирает. Играя руками, ты увидишь свои `5`, `hello` и `1` сразу после приглашения.

Частые ошибки

Перепутанная кодировка ходов. Формула `(computer - player + 3) % 3` работает только при нумерации 0=камень, 1=ножницы, 2=бумага (каждый бьёт следующего по кругу). Если занумеруешь иначе, формула начнёт врать. Тогда честнее вернуться к явным `if` из шага 4.

Минус в остатке. Без `+ 3` выражение `(computer – player) % 3` для отрицательной разницы даст отрицательный остаток, и сравнения с 1 и 2 сломаются. Тройка гарантирует неотрицательное число перед `%`.

`'1'` это не 1. Символ цифры и само число — разные вещи: код символа `'1'` равен 49, а не 1. Чтобы получить число, вычитаем `'0'`: `buf[0] – '0'`. Забудешь — получишь чушь.

Ввод одним символом. Если читать ход через `scanf("%d", ...)`, то на `hello` программа зациклится: `scanf` не сможет прочитать число и не сдвинет ввод. Поэтому мы читаем слово через `%s` и разбираем его сами — так проще не запутаться.

Бесконечный цикл без выхода. В `while (1)` обязательно должен быть `break` (или другой выход), иначе программа повиснет. У нас выход — по `q` и по концу ввода.

Упражнения

1. **Best of 5.** Останавливай матч автоматически, как только кто-то набрал 3 победы (или сыграно 5 раундов). Подскажу: добавь условие выхода из цикла по `player_score` и `computer_score`.
2. **Колодец (расширенный вариант).** Сделай игру на пять ходов: добавь «колодец» и «ящерицу» (или «огонь» и «воду»). Продумай новую таблицу побед. Подумай, можно ли обобщить формулу с остатком — или проще задать таблицу исходов массивом.
3. **Статистика.** В конце матча печатай, сколько было ничьих и какой ход ты выбирал чаще всего. Заведи счётчики ничьих и трёх ходов (можно массивом из трёх элементов, индекс — номер хода).
4. **Усиленная защита ввода.** Принимай не только цифры, но и первые буквы: `к` — камень, `н` — ножницы, `б` — бумага. (Помни: русская буква в UTF-8 занимает несколько байт, см. главу 3 — проще принимать латинские `r`, `s`, `p`.)
5. **Человеческая подсказка.** Если игрок проиграл раунд, печатай, чем его можно было побить ход компьютера: «бумага побила бы камень».
6. **Серия побед.** Считай и показывай самую длинную серию подряд выигранных тобой раундов за матч.

Что мы получили

Мы собрали полноценную интерактивную игру: случайный соперник, проверка ввода, цикл раундов, счёт и итог матча. Попутно появилось перечисление `enum` — способ давать числам понятные имена, разобрали перевод символа-цифры в число через `– '0'` и красивый трюк с остатком от деления, который заменил целую таблицу одной строкой (и поняли, **почему** он работает, а не просто поверили).

В следующей главе возьмёмся за игру посерьёзнее — **«Виселицу»**. Там понадобится работать со строкой загаданного слова, отмечать угаданные и неугаданные буквы, считать ошибки и рисовать состояние игры. Массивы, строки и `'\0'` из главы 3 наконец развернутся в полную силу.

Глава 14. Игра «Виселица»

Угадал число — пора угадывать слова. «Виселица» (hangman) — это та самая игра, где компьютер загадывает слово, показывает его пустыми прочерками `_ _ _ _`, а ты называешь буквы по одной. Угадал букву — она открывается во всех своих местах. Промахнулся — теряешь жизнь и к виселице пририсовывается ещё одна часть тела. Открыл всё слово — победа; кончились жизни — проигрыш.

Игра простая на вид, но в ней соберётся всё, что ты уже знаешь: массив строк, случайный выбор, посимвольная работа со строкой, цикл, ветвления, флаги. И добавится пара новых приёмов: тип `const char *` (указатель на неизменяемую строку) и хитрый пробел в формате `scanf`, который чистит ввод от лишних переводов строки.

Что мы сделаем

Программу `code/14-hangman.c`, которая:

- держит список слов и выбирает одно случайно;
- показывает его как ряд прочерков, открывая угаданные буквы;
- читает букву с клавиатуры, приводит к нижнему регистру;
- проверяет, есть ли буква в слове; за промах снимает жизнь;
- рисует стадии виселицы ASCII-картинкой;
- объявляет победу, когда открыты все буквы, и поражение, когда кончились жизни.

Сразу важное предупреждение, как и в главе 10: мы считаем **символы**, а в UTF-8 русская буква занимает 2 байта, латинская — 1. Поэтому слова берём **английские** (латиница). Русский текст оставляем только для сообщений — `printf` печатает байты как есть, и они выглядят верно.

Разбираем задачу руками

Возьмём слово `apple`. Что происходит за одну партию?

```
секрет:  a p p l e
экран:   _ _ _ _ _      (пять прочерков — по числу букв)

игрок называет 'p'  -> 'p' есть в слове, открываем ОБА p:
экран:   _ p p _ _

игрок называет 'z'  -> 'z' в слове нет: минус жизнь
игрок называет 'a'  -> открываем: a p p _ _
игрок называет 'l'  -> a p p l _
игрок называет 'e'  -> a p p l e   -> все буквы открыты -> ПОБЕДА
```

Что нам нужно хранить, чтобы это повторить машиной:

1. **само слово** (`secret`);
2. **какие буквы уже названы** — чтобы знать, что открывать и не засчитывать повтор;
3. **сколько ошибок** сделано — для жизней и для картинки виселицы.

Заведём под пункт 2 массив на 26 английских букв: `guessed['a'-'a']`, `guessed['b'-'a']`, … По букве легко получить её номер: `c - 'a'`. Помнишь из главы 2, что `char` — это маленькое целое (код символа)? Значит `'c' - 'a'` равно 2, `'a' - 'a'` равно 0 — готовый индекс.

Дальше строим программу по кусочкам.

Шаг 1. Список слов и случайный выбор

Слова — это набор строк. Удобнее всего хранить их так:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

int main(void)
{
    const char *words[] = {
        "apple", "banana", "orange", "computer", "program",
        "keyboard", "function", "pointer", "rainbow", "elephant"
    };
    int word_count = (int)(sizeof words / sizeof words[0]);

    srand((unsigned int)time(NULL));
```

```
    const char *secret = words[rand() % word_count];

    printf("Загадано слово из %zu букв.\n", strlen(secret));
    printf("(для проверки: %s)\n", secret);
    return 0;
}
```

Здесь новое — строка `const char *words[]`. Разберём её по косточкам, это важный тип.

- Из главы 3 ты знаешь: строка `"apple"` живёт в памяти как массив символов, а её имя превращается в **адрес первого символа**. То есть каждый литерал в списке — это адрес строки.
- `const char *` — это «**указатель на символ**», то есть переменная, хранящая адрес начала строки. Слово `const` (константа) добавляет обещание: «через этот указатель я строку **менять не буду**». Строковые литералы в Си менять нельзя (попытка — это падение), поэтому `const` тут не каприз, а защита: компилятор не даст случайно написать в такую строку.
- `const char *words[]` — массив таких указателей. Получается список адресов: `words[0]` — адрес `"apple"`, `words[1]` — адрес `"banana"`, и так далее. Размер `[]` пустой — компилятор сам посчитает по числу элементов в фигурных скобках.

Дальше:

- `sizeof words / sizeof words[0]` — стандартный приём «сколько элементов в массиве». `sizeof words` — размер всего массива в байтах, `sizeof words[0]` — размер одного элемента (одного указателя). Их частное и есть число слов. Так не придётся вручную писать `10` и ошибаться при добавлении слов. Результат `sizeof` — беззнаковый тип `size_t`, поэтому приводим к `int` через `(int)`, чтобы не было предупреждений при сравнении со счётчиком.
- `srand((unsigned int)time(NULL));` — заводим генератор случайных чисел текущим временем (разобрали в главе про угадайку): без этого `rand()` каждый запуск выдавал бы одну и ту же «случайность».
- `words[rand() % word_count]` — `rand() % word_count` даёт случайный индекс от 0 до `word_count - 1`, по нему берём слово. Кладём его адрес в `const char *secret`.
- `%zu` — спецификатор для `size_t` (то, что возвращает `strlen`). Печатать `size_t` через обычный `%d` — неверный тип; для него есть отдельный `%zu`.

Строку `(для проверки: %s)` мы потом уберём — она просто показывает загаданное слово, чтобы убедиться, что выбор работает.

```
Загадано слово из 6 букв.
(для проверки: orange)
```

Слово выбирается, длину знаем. Едем дальше.

Шаг 2. Показываем слово прочерками

Игрок не должен видеть слово — он видит ряд _. Напишем это отдельной функцией: пройдём по символам и для каждого напечатаем либо саму букву (если уже угадана), либо _.

Пока «угаданных» букв у нас нет, поэтому заведём массив-флаги `guessed` на 26 букв и передадим его в функцию. Все нули — значит, открытых букв нет, и функция нарисует одни прочерки.

```c
int draw_word(const char *word, const int *guessed)
{
    int revealed = 0;
    for (int i = 0; word[i] != '\0'; i++) {
        char ch = word[i];
        if (guessed[ch - 'a']) {
            printf("%c ", ch);
            revealed++;
        } else {
            printf("_ ");
        }
    }
    printf("\n");
    return revealed;
}
```

Разбор функции:

- `int draw_word(const char *word, const int *guessed)` — принимает слово (указатель на строку) и массив флагов. Массив в Си передаётся как указатель на первый элемент (глава 3), поэтому параметр — `const int *guessed`. `const` снова обещает «менять не буду» — рисуем по флагам, но не трогаем их.
- `for (int i = 0; word[i] != '\0'; i++)` — идём по строке, пока не упрёмся в метку конца `'\0'`. Это привычный по главе 10 способ перебрать строку, не зная заранее её длины.
- `char ch = word[i];` — текущий символ.

- `guessed[ch - 'a']` — флаг для этой буквы. `ch - 'a'` — номер буквы (0 для `a`, 25 для `z`). Если флаг ненулевой — букву уже называли, печатаем её через `%c` и пробел, и считаем открытой (`revealed++`).
- иначе печатаем _ и пробел — буква спрятана.
- функция возвращает `revealed` — сколько букв открыто. Это нам пригодится для проверки победы: если открыто столько же, сколько всех букв, — слово отгадано.

Проверим в `main` (заведём пустой `guessed` и нарисуем):

```
int guessed[26] = {0};       /* все нули — ничего не открыто */
draw_word(secret, guessed);
```

`int guessed[26] = {0};` — массив из 26 целых, и `= {0}` заполняет **весь** массив нулями (хватает одного нуля в фигурных скобках — остальные элементы тоже станут нулями). Вывод для `orange`:

```
_ _ _ _ _ _
```

Шесть прочерков. То, что нужно.

Шаг 3. Читаем букву — и ловим коварный перевод строки

Теперь спросим у игрока букву. Букву читают через `%c` в `scanf`. Напишем наивно:

```
char c;
printf("Твоя буква: ");
scanf("%c", &c);
printf("Ты ввёл: [%c]\n", c);

printf("Ещё буква: ");
scanf("%c", &c);
printf("Ты ввёл: [%c]\n", c);
```

`%c` читает **ровно один символ** (в отличие от `%d`, который пропускает пробелы и читает число). `&c` — адрес переменной, куда записать символ (как в главе 2).

Запускаем, вводим `a`, жмём Enter, потом `b`, Enter:

```
Твоя буква: a
Ты ввёл: [a]
Ещё буква: Ты ввёл: [
]
```

Что за ерунда? Второй раз мы и спросить толком не успели, а в `c` оказался перевод строки. Вот в чём дело: когда ты набрал `a` и нажал Enter, в буфер ввода попало **два** символа — сама буква `a` и невидимый символ перевода строки `'\n'` (от Enter). Первый `scanf` забрал `a`, а `'\n'` остался ждать. Второй `scanf("%c")` тут же схватил этот `'\n'` — вот он и «ввёлся».

Фикс — пробел перед `%c`: `scanf(" %c", &c)`. Пробел в строке формата означает «пропусти сначала любое количество пробельных символов» — пробелы, табы и переводы строки. То есть он съест застрявший `'\n'` (и любые лишние пробелы) и дойдёт до настоящей буквы.

```
char c;
printf("Твоя буква: ");
scanf(" %c", &c);          /* пробел перед %c чистит мусор */
```

Теперь оба ввода читаются как буквы. Запомни этот пробел — он спасает почти всегда, когда читаешь символы по одному.

Приводим к нижнему регистру

Игрок может ввести `A` или `a` — для нас это одна буква. Приведём заглавную к строчной:

```
if (c >= 'A' && c <= 'Z') {
    c = (char)(c - 'A' + 'a');
}
```

- `c >= 'A' && c <= 'Z'` — проверяем, что символ в диапазоне заглавных латинских букв. Сравнивать символы можно как числа — это и есть их коды.
- `c - 'A' + 'a'` — арифметика над кодами: вычли «начало заглавных», прибавили «начало строчных» — получили ту же букву в нижнем регистре (`'B'` → `'b'`). Приведение `(char)` нужно, потому что в Си результат таких выражений имеет тип `int`, и без `(char)` компилятор с `-Wall -Wextra` выдаст предупреждение о сужении при записи обратно в `char`.

И отсечём всё, что не английская буква (цифры, знаки, кириллица):

```
if (c < 'a' || c > 'z') {
    printf("Это не английская буква. Попробуй ещё.\n");
    continue;
}
```

`continue` — пропустить остаток текущего витка цикла и сразу перейти к следующему. То есть «плохой» ввод не тратит жизнь, мы просто переспрашиваем.

Шаг 4. Запоминаем букву и открываем её

Букву прочитали. Что с ней делать?

Сначала — не повторялась ли она:

```
if (guessed[c - 'a']) {
    printf("Букву '%c' ты уже называл.\n", c);
    continue;
}
guessed[c - 'a'] = 1;        /* отмечаем букву как названную */
```

Если флаг уже стоит — буква была, переспрашиваем (тоже без штрафа). Иначе ставим флаг в 1. С этого момента `draw_word` будет открывать эту букву во всех её местах автоматически — ведь функция рисует по флагам.

Теперь проверим, есть ли буква в слове:

```
int found = 0;
for (int i = 0; i < len; i++) {
    if (secret[i] == c) {
        found = 1;
        break;
    }
}
```

- `found` — флаг «нашли». Заводим как 0.
- идём по слову; как только нашли совпадение, ставим `found = 1` и `break` — выходим из цикла досрочно, дальше искать незачем.
- `len` — длина слова, посчитаем один раз через `strlen` (см. ниже).

Реакция:

```
if (found) {
    printf("Есть такая буква!\n");
} else {
    wrong++;
    printf("Мимо.\n");
}
```

Промах — `wrong++` (на одну ошибку больше). Попадание — ничего не снимаем.

Шаг 5. Жизни, победа и поражение

Соберём цикл партии. Ошибок разрешим, скажем, шесть — это и будет число стадий виселицы. Заведём константу:

```
#define MAX_WRONG 6
```

`#define` — это **директива препроцессора**: перед компиляцией текст `MAX_WRONG` везде заменится на `6`. Удобно: захочешь сделать игру легче — поменяешь число в одном месте. Имена таких констант по традиции пишут заглавными.

Главный цикл крутится, пока ошибок меньше предела:

```
int len = (int)strlen(secret);
int wrong = 0;

while (wrong < MAX_WRONG) {
    int revealed = draw_word(secret, guessed);
    if (revealed == len) {
        printf("Победа! Слово: %s\n", secret);
        return 0;
    }
    /* ... спросить букву, обработать (шаги 3–4) ... */
}

printf("Ты проиграл. Слово было: %s\n", secret);
```

Как это работает:

- сначала рисуем слово и узнаём `revealed` — сколько букв открыто;
- если открыто столько же, сколько всего (`revealed == len`), — слово отгадано, печатаем победу и `return 0` (выходим из программы);
- иначе спрашиваем букву и обрабатываем её;
- промахи копят `wrong`; как только `wrong` дорастёт до `MAX_WRONG`, условие `while` станет ложным, цикл закончится — и сработает строка про проигрыш.

Заметь порядок: победу проверяем **в начале** витка, после отрисовки. Поэтому когда ты последней буквой открываешь всё слово, на следующем витке `draw_word` покажет полное слово и тут же объявит победу.

Шаг 6. Рисуем виселицу

Осталось украшение — ASCII-картинка, которая дорастает с каждой ошибкой. Сделаем массив из семи строк: на 0 ошибок, на 1, …, на 6. Индекс — число ошибок.

```
const char *gallows[MAX_WRONG + 1] = {
    "  +---+\n  |   |\n      |\n      |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  0   |\n      |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  0   |\n  |   |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  0   |\n /|   |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  0   |\n /|\\  |\n      |\n      |\n=========",
    "  +---+\n  |   |\n  0   |\n /|\\  |\n /    |\n      |\n=========",
    "  +---+\n  |   |\n  0   |\n /|\\  |\n / \\  |\n      |\n========="
};
```

- это снова массив `const char *` — список строк, как наш список слов.
- `[MAX_WRONG + 1]` — нужно 7 ячеек: от 0 до 6 ошибок включительно.
- внутри строк `\n` — перевод строки: одна строковая константа рисует сразу несколько строк картинки.
- `\\` — это **один** символ обратной косой черты `\`. В строке `\` начинает спецсимвол (`\n`), поэтому сам бэкслеш записывают удвоенным — `\\`. Так рисуются руки и нога человечка.

Печатаем нужную стадию по числу ошибок: `printf("%s\n", gallows[wrong])`. А при проигрыше показываем последнюю, полную: `gallows[MAX_WRONG]`.

Всё, деталей хватает. Складываем программу целиком.

Полная программа

`code/14-hangman.c`:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_WRONG 6   /* сколько ошибок позволяем до проигрыша */

/* Рисует слово: открытые буквы — как есть, закрытые — подчёркиванием.
   Возвращает, сколько букв уже открыто. */
int draw_word(const char *word, const int *guessed)
```

```
{
    int revealed = 0;
    for (int i = 0; word[i] != '\0'; i++) {
        char ch = word[i];
        if (guessed[ch - 'a']) {
            printf("%c ", ch);
            revealed++;
        } else {
            printf("_ ");
        }
    }
    printf("\n");
    return revealed;
}

int main(void)
{
    const char *words[] = {
        "apple", "banana", "orange", "computer", "program",
        "keyboard", "function", "pointer", "rainbow", "elephant"
    };
    int word_count = (int)(sizeof words / sizeof words[0]);

    /* Стадии виселицы: индекс = число ошибок (0..MAX_WRONG). */
    const char *gallows[MAX_WRONG + 1] = {
        "  +---+\n  |   |\n      |\n      |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n      |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n  |   |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|   |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|\\  |\n      |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|\\  |\n /    |\n      |\n=========",
        "  +---+\n  |   |\n  O   |\n /|\\  |\n / \\  |\n      |\n========="
    };

    srand((unsigned int)time(NULL));

    const char *secret = words[rand() % word_count];
    int len = (int)strlen(secret);
```

```
    int guessed[26] = {0};   /* guessed[c - 'a'] == 1, если букву уже называли */
    int wrong = 0;

    printf("Игра \"Виселица\". Угадай слово по буквам!\n");

    while (wrong < MAX_WRONG) {
        printf("\n%s\n\n", gallows[wrong]);

        int revealed = draw_word(secret, guessed);
        if (revealed == len) {
            printf("Победа! Слово: %s\n", secret);
            return 0;
        }

        printf("Ошибок: %d из %d\n", wrong, MAX_WRONG);
        printf("Твоя буква: ");

        char c;
        if (scanf(" %c", &c) != 1) {
            /* ввод кончился (Ctrl+D) — это не проигрыш, а выход */
            printf("\nИгра прервана. Слово было: %s\n", secret);
            return 0;
        }

        if (c >= 'A' && c <= 'Z') {
            c = (char)(c - 'A' + 'a');   /* в нижний регистр */
        }
        if (c < 'a' || c > 'z') {
            printf("Это не английская буква. Попробуй ещё.\n");
            continue;
        }
        if (guessed[c - 'a']) {
            printf("Букву '%c' ты уже называл.\n", c);
            continue;
        }
        guessed[c - 'a'] = 1;
```

```
        int found = 0;
        for (int i = 0; i < len; i++) {
            if (secret[i] == c) {
                found = 1;
                break;
            }
        }
        if (found) {
            printf("Есть такая буква!\n");
        } else {
            wrong++;
            printf("Мимо.\n");
        }
    }

    printf("\n%s\n\n", gallows[MAX_WRONG]);
    printf("Ты проиграл. Слово было: %s\n", secret);
    return 0;
}
```

Пара мест, которые мы ещё не отметили:

- `if (scanf(" %c", &c) != 1) { ... }` — `scanf` возвращает, сколько значений он успешно прочитал. Здесь ждём одно. Если вернулось не 1 — ввод кончился (нажали Ctrl+D или скормили файл, который закончился). Проверять результат `scanf` — хорошая привычка (об этом был разговор ещё в главе 2).

  Внутри — `return 0`, а не `break`, и вот почему. Цикл заканчивается ровно в одном случае: игрок исчерпал все шесть ошибок. Поэтому сразу за циклом стоит «Ты проиграл» с полной виселицей. Если по Ctrl+D мы просто выйдем из цикла через `break`, программа объявит проигрыш человеку, у которого могло быть пять открытых букв и ни одной ошибки, и дорисует ему петлю. Это враньё. Прерванная игра — не проигрыш, поэтому выходим отдельным путём и говорим об этом честно.

- `printf("Игра \"Виселица\"...")` — внутри строки кавычку экранируют через `\"`, иначе компилятор решит, что строка тут и закончилась.

Как это работает

Один виток главного цикла:

1. печатаем виселицу по числу ошибок: `gallows[wrong]`;
2. рисуем слово прочерками и узнаём, сколько букв открыто;
3. если открыты все — победа, выходим;
4. показываем счётчик ошибок и просим букву;
5. читаем символ (`" %c"` чистит лишний `'\n'`), приводим к нижнему регистру, отсекаем не-буквы и повторы;
6. ищем букву в слове: попала — радуемся, промах — `wrong++`;
7. возвращаемся к шагу 1.

Когда `wrong` дорастает до `MAX_WRONG`, цикл заканчивается, и печатается проигрыш. Когда `revealed == len` — победа в шаге 3. Массив `guessed` связывает всё вместе: достаточно поставить один флаг, и буква сама «проявляется» при следующей отрисовке во всех своих позициях.

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 code/14-hangman.c -o hangman
./hangman
```

Играть руками удобно, но для проверки можно скормить буквы через конвейер (pipe) — программа прочитает их вместо клавиатуры:

```
printf "a\np\nl\ne\n" | ./hangman
```

Удачная партия (загадано `apple`, называем `a p l e`). Слово выбирается случайно, так что с первого раза может выпасть другое — запускай, пока не попадётся нужное:

```
Игра "Виселица". Угадай слово по буквам!


  +---+
  |   |
      |
      |
      |
      |
=========


_ _ _ _ _
Ошибок: 0 из 6
Твоя буква: Есть такая буква!
```

```
  +---+
  |   |
      |
      |
      |
      |
=========

a _ _ _ _
Ошибок: 0 из 6
Твоя буква: Есть такая буква!

  +---+
  |   |
      |
      |
      |
      |
=========

a p p _ _
Ошибок: 0 из 6
Твоя буква: Есть такая буква!

  +---+
  |   |
      |
      |
      |
      |
=========

a p p l _
Ошибок: 0 из 6
Твоя буква: Есть такая буква!

  +---+
```

```
  |   |
      |
      |
      |
      |
=========

a p p l e
Победа! Слово: apple
```

Обрати внимание: мы назвали `p` один раз, а открылись **обе** буквы `p` — это и есть работа массива `guessed`.

А вот проигрыш — называем буквы, которых в слове нет, шесть раз:

```
  +---+
  |   |
  O   |
 /|\  |
 / \  |
      |
=========

Ты проиграл. Слово было: function
```

Человечек дорисован полностью (6 ошибок), игра честно объявила слово.

Частые ошибки

`scanf("%c")` без пробела. Главная ловушка главы. После ввода в буфере остаётся `'\n'` от Enter, и следующий `%c` читает его вместо буквы. Лечится пробелом: `scanf(" %c", &c)`. Для `%d` и `%s` пробел не нужен — они сами пропускают пробельные символы, а вот `%c` — нет.

Забыл привести регистр. Если не переводить `A` в `a`, то заглавная буква не совпадёт ни с одной строчной буквой слова, и игрок будет честно угадывать, но «промахиваться». Приводи ввод к нижнему регистру.

Повторная буква снимает жизнь. Если не проверять `guessed` до поиска, игрок назовёт ту же неудачную букву дважды и потеряет две жизни за одну ошибку (или будет бесконечно «угадывать» уже открытую). Проверка `if (guessed[c - 'a']) continue;` это закрывает.

Индекс за границей `guessed`. `guessed[c - 'a']` верен только для `c` от `'a'` до `'z'`. Если сначала не отсечь не-буквы, то, скажем, цифра `'5'` даст `'5' - 'a'` — отрицательный индекс и выход за границы массива (глава 3 предупреждала, чем это грозит). Поэтому проверку диапазона делаем **до** обращения к `guessed`.

Победу проверяем не там. Если сверять `revealed == len` только после ввода буквы, а не в начале витка, легко получить лишний запрос буквы уже у отгаданного слова. Рисуем — проверяем — потом спрашиваем.

Упражнения

1. Добавь подсчёт **открытых букв** в подсказке: печатай не только «Ошибок: 2 из 6», но и «Открыто: 3 из 5».
2. Показывай игроку, какие буквы он уже называл (пройди по `guessed` и напечатай все буквы с флагом 1). Это убережёт от повторов.
3. Сделай уровни сложности: спроси перед игрой число жизней (например, от 4 до 8) и используй его вместо `MAX_WRONG`. Не забудь, что число стадий виселицы должно совпадать с числом жизней.
4. Разреши слова с **пробелами** (загадки из двух слов, например `red apple`). Подсказка: пробел не угадывают — рисуй его как пробел, а не как `_`, и не учитывай в проверке победы.
5. После партии спроси «Сыграем ещё? (y/n)» и, если `y`, начни заново (вынеси одну партию в функцию `int play_round(void)` и вызывай её в цикле в `main`).
6. Подсчёт очков: за победу начисляй очки, зависящие от числа оставшихся жизней, и веди счёт между партиями.

Что мы получили

Ты собрал полноценную текстовую игру: случайный выбор из списка строк (`const char *words[]`), посимвольная маскировка слова через массив флагов `guessed`, аккуратный ввод одной буквы с чисткой буфера (`scanf(" %c", &c)`), приведение регистра, подсчёт жизней и ASCII-арт, который растёт с ошибками. Попутно разобрали `const char *` (указатель на неизменяемую строку), приём `sizeof массив / sizeof массив[0]` и директиву `#define`.

Главный новый навык — **состояние игры в нескольких переменных** (слово, флаги, счётчик ошибок) и цикл, который это состояние двигает. В следующей главе мы сделаем шаг в сторону игрового **поля**: напишем «Крестики-нолики» с двумерным массивом, проверкой выигрышных линий и, по желанию, простым компьютерным противником.

Глава 15. Игра «Крестики-нолики»

Пора собрать первую настоящую игру с полем. Крестики-нолики знают все: поле 3×3, двое игроков по очереди ставят X и O, кто первым выстроит три в ряд — выиграл. Если поле заполнилось, а ряда нет — ничья.

Игра простая на вид, но в ней есть всё, из чего состоят игры посложнее: **двумерное поле** в памяти, **печать** этого поля на экран, **ввод хода** с проверкой, **смена игрока** и **проверка условия победы**. Это ровно тот набор, который мы переиспользуем в сапёре, морском бое и дальше.

Главное новое в этой главе — **двумерный массив**. С ним и начнём.

Что мы сделаем

Игру на двоих за одним компьютером. Программа:

- рисует поле 3×3 с номерами клеток-подсказок;
- спрашивает у игрока номер клетки (1–9);
- проверяет, что клетка существует и свободна;
- ставит фигуру, меняет игрока;
- после каждого хода проверяет победу (три в ряд) и ничью;
- объявляет результат и заканчивает партию.

Никаких новых библиотек не понадобится — только `stdio.h`, который ты уже знаешь. Зато разберём двумерный массив, вложенные циклы и аккуратную проверку ввода.

Разбираем задачу руками

Возьми лист бумаги и нарисуй поле. Девять клеток. Пронумеруем их так:

```
 1 | 2 | 3
---+---+---
 4 | 5 | 6
```

```
---+---+---
 7 | 8 | 9
```

Игрок X говорит «5» — ставим X в центр. Игрок O говорит «1» — ставим O в левый верхний угол. И так далее. После каждого хода смотрим: не собрал ли кто-то три своих знака в одну линию?

Линий всего восемь:

- три горизонтальные: {1,2,3}, {4,5,6}, {7,8,9};
- три вертикальные: {1,4,7}, {2,5,8}, {3,6,9};
- две диагонали: {1,5,9} и {3,5,7}.

Если в какой-то линии стоят три одинаковых знака — этот игрок победил. Если все девять клеток заняты, а победителя нет — ничья.

Теперь главный вопрос: как хранить поле в памяти?

Двумерный массив — таблица в памяти

Обычный массив — это ряд коробок:

```
int a[5];        // a[0], a[1], a[2], a[3], a[4]
```

Но поле — не ряд, а **таблица**: 3 строки и 3 столбца. Для таблиц в Си есть **двумерный массив** — массив, у которого **два** индекса:

```
char board[3][3];
```

Читается так: «`board` — это 3 строки, в каждой по 3 ячейки типа `char`». Всего 3 × 3 = 9 ячеек. Чтобы добраться до одной, нужны **два** номера:

```
board[row][col]
```

- первый индекс `row` (англ. *row* — строка) — **номер строки**, 0, 1 или 2;
- второй индекс `col` (англ. *column* — столбец) — **номер столбца**, 0, 1 или 2.

Как и у обычного массива, **нумерация с нуля**. Левая верхняя клетка — это `board[0][0]`, правая нижняя — `board[2][2]`. Центр — `board[1][1]`.

Вот вся карта индексов:

```
 board[0][0] | board[0][1] | board[0][2]
-------------+-------------+-------------
 board[1][0] | board[1][1] | board[1][2]
-------------+-------------+-------------
 board[2][0] | board[2][1] | board[2][2]
```

Как это лежит в памяти

В памяти нет никаких «строк» и «столбцов» — там всё подряд, одной лентой. Двумерный массив раскладывается **построчно**: сначала вся строка 0, потом вся строка 1, потом строка 2.

```
[0][0] [0][1] [0][2] [1][0] [1][1] [1][2] [2][0] [2][1] [2][2]
```

Девять ячеек `char` подряд — это 9 байт. Запись `board[row][col]` — просто удобный способ сказать компьютеру «отсчитай нужную ячейку в этой ленте». Тебе про ленту думать не нужно: пиши два индекса, и всё.

Перебор двумерного массива — цикл в цикле

Один индекс перебирают одним циклом `for`. Два индекса — **двумя вложенными** циклами: внешний идёт по строкам, внутренний по столбцам.

```
for (int row = 0; row < 3; row++) {
    for (int col = 0; col < 3; col++) {
        // здесь доступна клетка board[row][col]
    }
}
```

Как это работает по шагам: внешний цикл ставит `row = 0` и запускает внутренний цикл целиком — `col` пробегает 0, 1, 2. Это вся первая строка. Потом внешний цикл ставит `row = 1`, и внутренний снова крутится 0, 1, 2 — вторая строка. И так до `row = 2`. В сумме внутреннее тело выполнится 3 × 3 = 9 раз, ровно по одному на каждую клетку. Этот приём — основа работы с любым полем.

Шаг 1. Рисуем пустое поле

Начнём с малого: завести поле, заполнить его цифрами-подсказками 1–9 и напечатать. Цифры в пустых клетках удобны — игрок сразу видит, каким числом назвать клетку.

```
#include <stdio.h>

int main(void)
{
    char board[3][3];

    char digit = '1';
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
```

```
            board[row][col] = digit;
            digit++;
        }
    }

    for (int row = 0; row < 3; row++) {
        printf(" %c | %c | %c \n",
               board[row][0], board[row][1], board[row][2]);
    }
    return 0;
}
```

Разбор нового:

- `char board[3][3];` — наш двумерный массив на 9 символов. Тип `char` потому, что в клетке будет либо цифра-подсказка (`'1'`…`'9'`), либо фигура (`'X'` или `'O'`) — а это всё одиночные символы.
- `char digit = '1';` — заводим символ-счётчик и ставим его в `'1'`. Обрати внимание: это **символ** `'1'` в одинарных кавычках, а не число 1. Внутри `char` хранится код символа, и у соседних цифр коды идут подряд: за `'1'` следует `'2'`, за `'2'` — `'3'`. Поэтому…
- `digit++;` — увеличивает символ до следующего. После `'1'` получим `'2'`, и так до `'9'`. Тот же `++`, что и для чисел.
- `board[row][col] = digit;` — кладём текущую цифру в клетку. На первом обороте это `board[0][0] = '1'`, дальше `'2'`, `'3'` и т. д.
- В `printf(" %c | %c | %c \n", ...)` спецификатор `%c` печатает один символ (мы знаем его с главы 2). Три `%c` — три клетки одной строки, между ними рисуем разделитель `|`.

Запуск:

```
 1 | 2 | 3
 4 | 5 | 6
 7 | 8 | 9
```

Поле есть. Но между строками нет горизонтальных линий — выглядит бедно.

Шаг 2. Добавляем разделители и выносим печать в функцию

Печатать поле придётся после **каждого** хода. Копировать цикл печати по всему коду — плохо: длинно и легко ошибиться. Вынесем печать в отдельную функцию `print_board`, а заодно дорисуем горизонтальные линии.

```c
void print_board(char board[3][3])
{
    printf("\n");
    for (int row = 0; row < 3; row++) {
        printf(" %c | %c | %c \n",
               board[row][0], board[row][1], board[row][2]);
        if (row < 2) {
            printf("---+---+---\n");
        }
    }
    printf("\n");
}
```

Разбор:

- `void print_board(char board[3][3])` — функция, которая ничего не возвращает (`void`, см. главу 2), а принимает наше поле. Как передать двумерный массив в функцию? Указываем тип ровно так, как объявляли: `char board[3][3]`. Размеры в скобках важны — по ним функция знает, как раскладывать ленту памяти на строки.
  - Тонкость: массив (в том числе двумерный) передаётся в функцию **по адресу**, а не копией (см. главу 3 про указатели). Это значит, что функция работает с **тем же самым** полем, что и `main`. Для печати это неважно, а вот позже для проверки победы — очень удобно: не надо копировать поле туда-сюда.
- `if (row < 2)` — линию-разделитель рисуем после строк 0 и 1, но **не** после последней строки 2. Иначе под полем повисла бы лишняя черта.
- `printf("\n")` в начале и в конце — пустые строки сверху и снизу, чтобы поле не липло к остальному тексту.

Теперь `main` короткий: завести поле, заполнить, позвать `print_board`. Запуск даёт аккуратную картинку:

```
 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9
```

Шаг 3. Один ход игрока

Теперь научимся ставить фигуру. Игрок вводит число 1–9, а нам нужны два индекса `[row][col]`. Как перевести одно в другое?

Клетки пронумерованы построчно: 1, 2, 3 — первая строка; 4, 5, 6 — вторая; 7, 8, 9 — третья. Вычтем из номера 1, чтобы считать с нуля (`cell - 1` даёт 0…8), и дальше:

- **номер строки** = `(cell - 1) / 3` — целочисленное деление на 3 (помнишь из главы 2: `/` для целых отбрасывает дробь). Для клеток 1–3 даст 0, для 4–6 даст 1, для 7–9 даст 2.
- **номер столбца** = `(cell - 1) % 3` — остаток от деления на 3. Даёт 0, 1, 2 внутри строки.

Проверь руками. Клетка 5: `(5-1)/3 = 4/3 = 1` (строка 1), `(5-1)%3 = 4%3 = 1` (столбец 1) — центр `board[1][1]`. Верно. Клетка 7: `(7-1)/3 = 2`, `(7-1)%3 = 0` — `board[2][0]`, левый нижний угол. Тоже верно.

Наивный ход (пока без проверок):

```
int cell;
scanf("%d", &cell);

int row = (cell - 1) / 3;
int col = (cell - 1) % 3;

board[row][col] = player;     // player — это 'X' или 'O'
```

Работает, но доверять вводу нельзя. Что если игрок наберёт 42? Тогда `row = 41/3 = 13` — и мы запишем фигуру **за границы** массива, в чужую память. Си границы не проверяет (см. главу 3), поэтому проверяем сами.

Шаг 4. Проверяем ввод

Хороший ход обязан пройти три проверки:

1. Введено вообще число? (`scanf` мог не справиться — игрок набрал буквы).
2. Число в диапазоне 1–9?
3. Клетка свободна (там ещё цифра, а не `'X'`/`'O'`)?

```
int cell;
int rc = scanf("%d", &cell);

if (rc == EOF) {                /* ввод закончился (Ctrl+D) */
    printf("\nИгра прервана.\n");
```

```
    return 0;
}
if (rc != 1) {                  /* ввели не число */
    printf("Это не число. Введи цифру от 1 до 9.\n");
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        /* выбрасываем мусорный ввод до конца строки */
    }
    continue;
}
if (cell < 1 || cell > 9) {
    printf("Нет такой клетки. Нужно число от 1 до 9.\n");
    continue;
}

int row = (cell - 1) / 3;
int col = (cell - 1) % 3;

if (board[row][col] == 'X' || board[row][col] == 'O') {
    printf("Клетка %d уже занята. Выбери другую.\n", cell);
    continue;
}
```

Разбор (часть уже знакома по игре «угадай число» из главы 12):

- `int rc = scanf("%d", &cell);` — `scanf` **возвращает**, сколько значений ему удалось прочитать. Мы просили одно число, значит, успех — это `rc == 1`. Раньше мы возвращаемое значение `scanf` игнорировали; здесь оно важно, потому что ввод может быть «грязным».
- `rc == EOF` — *end of file*, «конец ввода». Бывает, если ввод закончился (например, нажали Ctrl+D или партию подали из файла). Просто выходим.
- `rc != 1` — `scanf` не смог прочитать число (игрок набрал `abc`). Беда в том, что эти буквы **остались** во входном буфере, и следующий `scanf` снова на них споткнётся — получится бесконечный поток ошибок. Поэтому чистим буфер циклом `getchar()` до конца строки. `getchar` читает по одному символу; крутим, пока не съедим перевод строки `'\n'` или не упрёмся в конец ввода.
- `continue;` — пропустить остаток тела цикла и сразу пойти на новую попытку ввода. Игрок ошибся — даём ему повторить ход, не меняя поле и не передавая ход сопернику.

- `board[row][col] == 'X' || board[row][col] == 'O'` — клетка занята, если в ней стоит любая из фигур. Сравниваем символы через == (для одиночных `char` это можно — это не строки, см. главу 3, где строки нельзя сравнивать через ==).

Только пройдя все проверки, ставим фигуру: `board[row][col] = player;`.

Шаг 5. Проверка победы — выносим в функцию

После хода нужно понять, не выиграл ли текущий игрок. Проверять восемь линий прямо в `main` — это каша. Вынесем в функцию `has_won`, которая отвечает на вопрос «занял ли игрок `player` три клетки в ряд?».

```
int has_won(char board[3][3], char player)
{
    for (int row = 0; row < 3; row++) {
        if (board[row][0] == player &&
            board[row][1] == player &&
            board[row][2] == player) {
            return 1;
        }
    }
    for (int col = 0; col < 3; col++) {
        if (board[0][col] == player &&
            board[1][col] == player &&
            board[2][col] == player) {
            return 1;
        }
    }
    if (board[0][0] == player &&
        board[1][1] == player &&
        board[2][2] == player) {
        return 1;
    }
    if (board[0][2] == player &&
        board[1][1] == player &&
        board[2][0] == player) {
        return 1;
    }
```

```
    return 0;
}
```

Разбор:

- `int has_won(...)` — функция возвращает `int`: 1, если игрок выиграл, 0, если нет. Мы помним из главы 2: в Си истина — это любое ненулевое число (обычно 1), ложь — 0. Так что результат можно прямо подставлять в `if`.
- **Три строки.** Цикл по `row`: внутри строки фиксируем номер строки и проверяем все три столбца — `[row][0]`, `[row][1]`, `[row][2]`. Если все три равны `player`, строка собрана — `return 1`.
- **Три столбца.** Зеркально: цикл по `col`, фиксируем столбец и проверяем три строки — `[0][col]`, `[1][col]`, `[2][col]`.
- `&&` — логическое «И» (см. главу 2): победа в линии, только если **все три** клетки совпали с `player`.
- **Главная диагональ** — клетки `[0][0]`, `[1][1]`, `[2][2]` (из левого верхнего угла в правый нижний). **Побочная** — `[0][2]`, `[1][1]`, `[2][0]` (из правого верхнего в левый нижний). У обеих центр общий — `[1][1]`.
- `return 1;` внутри прерывает функцию сразу, как только нашли линию. Если ни одна не сложилась, доходим до `return 0;` в конце.

`return` где угодно завершает функцию и отдаёт значение. Поэтому первое же совпадение даёт ответ, остальное не проверяется.

Шаг 6. Проверка ничьей

Ничья — это когда ходов больше нет, а победителя нет. «Ходов нет» значит, что все девять клеток заняты фигурами. Проверим это функцией:

```
int board_is_full(char board[3][3])
{
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            if (board[row][col] != 'X' && board[row][col] != '0') {
                return 0;
            }
        }
    }
    return 1;
}
```

Логика «от противного»: ищем хотя бы одну **не**занятую клетку. Если в клетке не `'X'` и не `'O'` (там ещё цифра-подсказка) — поле не заполнено, сразу `return 0`. Если оба вложенных цикла отработали и ни одной свободной клетки не нашлось — `return 1`, поле полное.

Порядок проверок в игре важен: **сначала** победа, **потом** ничья. Последним, девятым, ходом игрок может одновременно и заполнить поле, и выстроить линию — это победа, а не ничья. Поэтому в `main` сначала зовём `has_won`, и только если победы нет — `board_is_full`.

Полная программа

Собираем всё вместе. Это `code/15-tic-tac-toe.c`:

```c
#include <stdio.h>

/* Печать поля 3x3 с разделителями. */
void print_board(char board[3][3])
{
    printf("\n");
    for (int row = 0; row < 3; row++) {
        printf(" %c | %c | %c \n",
               board[row][0], board[row][1], board[row][2]);
        if (row < 2) {
            printf("---+---+---\n");
        }
    }
    printf("\n");
}

/* Проверка: занял ли игрок player три клетки в ряд?
   Возвращает 1 (победа) или 0. */
int has_won(char board[3][3], char player)
{
    /* три строки */
    for (int row = 0; row < 3; row++) {
        if (board[row][0] == player &&
            board[row][1] == player &&
            board[row][2] == player) {
            return 1;
        }
```

```
    }
    /* три столбца */
    for (int col = 0; col < 3; col++) {
        if (board[0][col] == player &&
            board[1][col] == player &&
            board[2][col] == player) {
            return 1;
        }
    }
    /* главная диагональ */
    if (board[0][0] == player &&
        board[1][1] == player &&
        board[2][2] == player) {
        return 1;
    }
    /* побочная диагональ */
    if (board[0][2] == player &&
        board[1][1] == player &&
        board[2][0] == player) {
        return 1;
    }
    return 0;
}

/* Все девять клеток заняты фигурами? */
int board_is_full(char board[3][3])
{
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            if (board[row][col] != 'X' && board[row][col] != '0') {
                return 0;
            }
        }
    }
    return 1;
}

int main(void)
```

```
{
    char board[3][3];

    /* Заполняем поле цифрами '1'..'9' — это подсказка номеров клеток. */
    char digit = '1';
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 3; col++) {
            board[row][col] = digit;
            digit++;
        }
    }

    char player = 'X';   /* первым ходит X */
    int moves = 0;       /* сколько ходов сделано */

    printf("Крестики-нолики. Игроки X и O ходят по очереди.\n");
    printf("Выбирай клетку числом от 1 до 9.\n");

    while (1) {
        print_board(board);
        printf("Ход игрока %c. Клетка (1-9): ", player);

        int cell;
        int rc = scanf("%d", &cell);

        if (rc == EOF) {                /* ввод закончился (Ctrl+D) */
            printf("\nИгра прервана.\n");
            return 0;
        }
        if (rc != 1) {                  /* ввели не число */
            printf("Это не число. Введи цифру от 1 до 9.\n");
            int c;
            while ((c = getchar()) != '\n' && c != EOF) {
                /* выбрасываем мусорный ввод до конца строки */
            }
            continue;
        }
        if (cell < 1 || cell > 9) {
```

```
            printf("Нет такой клетки. Нужно число от 1 до 9.\n");
            continue;
        }

        /* Переводим номер 1..9 в индексы строки и столбца. */
        int row = (cell - 1) / 3;
        int col = (cell - 1) % 3;

        if (board[row][col] == 'X' || board[row][col] == 'O') {
            printf("Клетка %d уже занята. Выбери другую.\n", cell);
            continue;
        }

        board[row][col] = player;        /* ставим фигуру */
        moves++;

        if (has_won(board, player)) {
            print_board(board);
            printf("Победил игрок %c! Ходов: %d.\n", player, moves);
            return 0;
        }
        if (board_is_full(board)) {
            print_board(board);
            printf("Ничья! Поле заполнено.\n");
            return 0;
        }

        /* Меняем игрока: X -> O, O -> X. */
        if (player == 'X') {
            player = 'O';
        } else {
            player = 'X';
        }
    }
}
```

Как это работает

- Три функции стоят **выше** `main` — чтобы к моменту их вызова компилятор их уже знал (про это правило мы говорили в главе 2).
- `while (1)` — **бесконечный** цикл. `1` — это всегда истина, значит, условие никогда не станет ложным само. Это удобно для игрового цикла: один оборот = один ход. Выходим из него не по условию, а изнутри — через `return 0;` (победа, ничья или прерывание ввода). Так часто оформляют игры, где заранее неизвестно, сколько будет ходов.
- Один оборот цикла: показали поле → попросили ход → проверили ввод → поставили фигуру → проверили победу → проверили ничью → сменили игрока → и заново.
- Смена игрока в конце оборота: `'X'` становится `'O'` и наоборот. А если ход был отбракован (`continue`), мы до смены игрока не доходим — ход остаётся за тем же игроком. Это правильно: за чужую ошибку очередь не переходит.
- `moves` считает только **состоявшиеся** ходы (мы увеличиваем его после всех проверок). В конце печатаем, за сколько ходов выиграли.

Проверяем

Скомпилируй и запусти. Компиляция должна пройти без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 15-tic-tac-toe.c -o ttt
./ttt
```

Сыграть можно вручную, набирая клетки и нажимая Enter. Но проверить, что победа и ничья распознаются верно, удобнее «прокруткой» готовой партии через канал ввода. Подадим серию ходов сразу, командой `printf`:

```
printf "1\n4\n2\n5\n3\n" | ./ttt
```

Здесь `\n` — перевод строки (как нажатие Enter после каждого числа), а `|` направляет этот текст программе на вход вместо клавиатуры. Партия: X берёт 1, O берёт 4, X берёт 2, O берёт 5, X берёт 3 — и X собирает всю верхнюю строку. Вывод (конец партии):

```
Ход игрока X. Клетка (1-9):
 X | X | X
---+---+---
 O | O | 6
---+---+---
 7 | 8 | 9

Победил игрок X! Ходов: 5.
```

Победа по строке работает. Проверим **ничью** — заполним всё поле так, чтобы линии ни у кого не сложилось:

```
printf "1\n2\n3\n5\n4\n6\n8\n7\n9\n" | ./ttt
```

Конец партии:

```
 X | O | X
---+---+---
 X | O | O
---+---+---
 O | X | X

Ничья! Поле заполнено.
```

А вот победа по **диагонали** (`printf "5\n1\n3\n2\n7\n"` — X занимает 5, 3 и 7, это побочная диагональ):

```
 O | O | X
---+---+---
 4 | X | 6
---+---+---
 X | 8 | 9

Победил игрок X! Ходов: 5.
```

Читается так: X стоит в клетках 3, 5 и 7 — это побочная диагональ, идущая из правого верхнего угла в левый нижний. O успел занять 1 и 2, но линию не собрал. Клетки 4, 6, 8, 9 остались с цифрами-подсказками — до них партия не дошла.

Все три исхода — строка, диагональ и ничья — определяются верно.

Частые ошибки

Путаница «номер клетки» и «индекс». Игрок думает клетками 1–9, а массив — индексами 0–8 по каждому измерению. Если забыть `- 1` в `(cell - 1) / 3`, всё поле «съедет», а клетка 9 вообще улетит за границы. Перевод номера в индексы делай в одном месте и проверяй руками на паре примеров.

Проверка занятости через цифру. Можно подумать: «клетка свободна, если в ней цифра». Но тогда условие пришлось бы писать длинно (`>= '1' && <= '9'`). Надёжнее наоборот: клетка **занята**, если в ней `'X'` или `'O'`. Свободна — во всех остальных случаях. Так не зависишь от того, чем именно помечены пустые клетки.

Грязный буфер ввода. Если игрок вместо числа набрал буквы, `scanf` их не прочитает и **оставит** во входном потоке. Без чистки буфера (`getchar` до конца строки) следующий `scanf` снова на них наткнётся — и игра уйдёт в бесконечную печать «Это не число». Всегда подчищай за неудачным `scanf`.

Сначала ничья, потом победа. Если проверять `board_is_full` раньше `has_won`, то выигрышный девятый ход засчитается как ничья. Победа проверяется первой.

Выход за границы при невалидной клетке. Вычислять `row`/`col` и лезть в `board[row][col]` нужно **только после** проверки `cell` на диапазон 1–9. Иначе число 42 уведёт индекс за пределы массива, а Си это не остановит (см. главу 3).

Упражнения

1. **Счёт серий.** Сейчас после партии программа завершается. Оберни игру во внешний цикл: после результата спроси «Сыграем ещё? (y/n)» и, если да, начни новую партию. Веди счёт побед X, побед O и ничьих, печатай его перед каждой партией.
2. **Поле N×N.** Замени `board[3][3]` на `board[N][N]`, где `N` — константа (`#define N 3` вверху файла). Перепиши печать и заполнение через циклы до `N`. Победу пока можешь оставить на 3 в ряд только для `N == 3` — но печать и ввод должны работать при любом `N`.
3. **Ввод строкой и столбцом.** Вместо номера 1–9 разреши вводить два числа — номер строки и номер столбца (1–3 каждый). Переведи их в индексы (`row - 1`, `col - 1`) и проверь диапазон обоих.
4. **Простой бот.** Сделай так, чтобы за O играл компьютер. Самый наивный бот: пройди поле и поставь O в **первую попавшуюся** свободную клетку. Подключи `rand` (см. главу 12), чтобы бот выбирал свободную клетку случайно — так интереснее.
5. **Подсветка победной линии.** После победы не просто объяви победителя, а определи, какая именно линия собрана, и напечатай её (например, «верхняя строка» или «главная диагональ»). Подсказка: пусть `has_won` возвращает не просто 1, а номер линии.
6. **Защита от дурака в боте.** Усложни бота из задачи 4: перед своим ходом пусть он проверит, не может ли соперник выиграть следующим ходом, и закроет эту клетку.

Что мы получили

Ты собрал полноценную игру с полем — и по дороге освоил **двумерный массив**: два индекса `[строка][столбец]`, построчную раскладку в памяти, обход вложенными циклами. Научился переводить номер клетки в пару индексов через `/` и `%`, аккуратно проверять ввод, выносить

логику в функции (`print_board`, `has_won`, `board_is_full`) и проверять победу по строкам, столбцам и диагоналям.

Двумерное поле плюс вложенные циклы — это каркас почти любой настольной игры. В следующей главе мы возьмём поле побольше и поинтереснее: напишем **сапёр**. Там тоже сетка клеток, но добавятся спрятанные мины, подсчёт мин-соседей вокруг каждой клетки (привет, обход восьми соседей) и рекурсивное «открытие» пустых областей.

Глава 16. Игра «Сапёр»

Помнишь «Сапёра» из старой Windows? Поле из клеток, под некоторыми спрятаны мины, и ты по одной открываешь клетки. В открытой клетке — число: сколько мин стоит вокруг неё. Открыл пустую — рядом раскрывается целый кусок поля. Открыл мину — проиграл.

Это идеальная игра, чтобы собрать вместе всё, что мы уже умеем: **двумерные массивы** (поле — это сетка, см. главу 15), **случайные числа** (мины ставим наугад, `rand`/`srand` — глава 12) и **рекурсию** (автоматическое раскрытие пустой области — глава 9). Ничего нового из языка не понадобится — только новая комбинация знакомых деталей.

Что мы сделаем

Правила нашего «Сапёра»:

- поле фиксированного размера **9×9** клеток;
- на нём случайно расставлены **10 мин**;
- игрок вводит координаты клетки (строку и столбец), чтобы её открыть;
- в открытой клетке показывается **число соседних мин** (от 1 до 8);
- если соседних мин **ноль** — клетка пустая, и вокруг неё автоматически открываются соседи (и так дальше, волной);
- открыл клетку с миной — **проигрыш**, показываем все мины;
- открыл все клетки без мин — **победа**.

Поле в терминале будет выглядеть так (точка — закрытая клетка):

```
  1 2 3 4 5 6 7 8 9
1 . . . . . . . . .
2 . . . . . . . . .
3 . . . . . . . . .
4 . . . . . . . . .
5 . . . . . . . . .
6 . . . . . . . . .
```

```
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
```

К концу главы у тебя будет полноценная играбельная консольная версия.

Разбираем задачу руками

Прежде чем писать код, разложим игру на части на бумаге.

Две карты, наложенные друг на друга

Самое важное решение — как хранить поле. У каждой клетки есть **две независимые** характеристики:

1. **Есть ли тут мина?** Это знает только программа, игрок не видит.
2. **Открыта ли клетка игроком?** Это видно на экране.

Эти два факта не связаны: клетка может быть с миной и закрытой, без мины и открытой, и так далее. Поэтому удобнее всего завести **два двумерных массива** одинакового размера, как две прозрачные плёнки, наложенные друг на друга:

- `mines[H][W]` — где `1`, там мина; где `0`, там пусто;
- `opened[H][W]` — где `1`, там игрок уже открыл клетку; где `0`, клетка ещё закрыта.

Можно было бы обойтись и одним массивом символов с разными состояниями (`'.'`, `'*'`, цифра), но тогда пришлось бы кодировать в одной клетке сразу всё, и логика запуталась бы. Две раздельные карты читаются куда проще: «стоит ли тут мина» и «открыта ли клетка» — два отдельных вопроса, два отдельных массива.

Числа в клетках мы **нигде не храним** — посчитаем их на лету, когда понадобится. Так меньше шансов рассинхронизировать данные.

Сколько мин вокруг клетки

У клетки в середине поля **восемь** соседей: сверху, снизу, слева, справа и четыре по диагонали. Чтобы узнать число в клетке, надо обойти всех восьмерых и посчитать, у скольких стоит мина.

Возьмём кусочек поля. Буква `M` — мина, мы смотрим на центральную клетку `?`:

```
. M .
. ? M
M . .
```

Обходим восемь соседей центральной клетки: сверху-середина — мина, справа — мина, снизу-слева — мина. Остальные пять пустые. Значит, в центральной клетке будет число **3**.

У клетки с краю или в углу соседей меньше — часть «вылезает» за поле. За эти несуществующие клетки нельзя заглядывать: там нет нашего массива, там чужая память. Проверку границ разберём отдельно, это важный момент.

Заливка пустой области

Когда игрок открывает клетку, рядом с которой **ноль** мин, в настоящем «Сапёре» одним кликом раскрывается целая поляна. Почему? Потому что у пустой клетки все соседи безопасны (раз вокруг ноль мин, ни в одном соседе мины нет). Их можно сразу открыть. А если какой-то сосед тоже пустой — открыть и его соседей. И так волной, пока не упрёмся в клетки с числами (они стоят на границе минного поля).

Это в точности **рекурсия** из главы 9: «открыть клетку» сводится к «открыть эту + открыть соседей такой же процедурой». Разберём заливку на маленьком поле 3×3 с одной миной в углу. Цифры — это реальное число соседних мин, точки — пока закрыто:

```
. . .
. . .
. . M
```

Игрок открывает верхний левый угол (клетка 1,1). У неё ноль соседних мин — значит, открываем её и зовём заливку для всех её соседей. Те, что пустые, тоже зовут своих соседей. Доходит до клеток рядом с миной — у них сосед-мина есть, число не ноль, дальше заливка не идёт. В итоге раскроется почти всё поле, а у клеток возле мины проступят числа:

```
        ← пусто (0)
1 1 1   ← по соседству с миной
1 . 1   ← точка = сама мина (её мы не открываем)
```

Ключевой вопрос рекурсии: **где база, на которой она остановится?** У заливки баз целых три:

1. клетка **за пределами поля** — стоп (туда нельзя);
2. клетка **уже открыта** — стоп (иначе закружимся: А зовёт В, В зовёт А, А зовёт В… бесконечно);
3. клетка **не пустая** (рядом есть мины) — открыли её и стоп, дальше волну не пускаем.

Без второй базы заливка зациклится насмерть. Это мы специально увидим ниже на наивной версии.

Теперь, когда план ясен, собираем по кусочкам.

Шаг 1. Поле и его печать

Начнём с самого простого: объявим поле и научимся его рисовать. Размеры и число мин вынесем в именованные константы — так их легко поменять в одном месте.

```c
#include <stdio.h>

#define H 9         /* высота поля (строк)   */
#define W 9         /* ширина поля (столбцов) */
#define MINES 10    /* сколько мин на поле    */

int mines[H][W];   /* 1 — здесь мина, 0 — пусто           */
int opened[H][W];  /* 1 — клетка открыта игроком, 0 — нет */
```

Разбор:

- `#define H 9` — это **макрос-константа**. `#define ИМЯ значение` говорит препроцессору: «перед компиляцией замени везде в тексте `H` на `9`». Удобно: размер поля задан один раз и читается как имя, а не как магическое число `9` посреди кода. Точки с запятой в конце `#define` **не ставят** — это не инструкция Си, а команда препроцессору.
- `int mines[H][W];` — двумерный массив `9×9` целых чисел (см. главу 15). Первый индекс — строка, второй — столбец: `mines[r][c]`.
- Оба массива объявлены **снаружи** функций — это **глобальные** переменные. Глобальные массивы автоматически заполняются нулями при старте программы (локальные — нет, в них мусор). Нам это на руку: поле сразу «чистое» — ни мин, ни открытых клеток. И ещё: к глобальным массивам имеют доступ все функции, не нужно таскать их параметрами.

Теперь функция печати. Сверху нарисуем номера столбцов, слева — номера строк, чтобы игроку было куда целиться:

```c
void print_board(void)
{
    printf("   ");
    for (int c = 0; c < W; c++) {
        printf("%d ", c + 1);        /* шапка со столбцами 1..W */
    }
    printf("\n");

    for (int r = 0; r < H; r++) {
        printf("%2d ", r + 1);       /* номер строки слева */
        for (int c = 0; c < W; c++) {
```

```
            if (opened[r][c] == 0) {
                printf(". ");        /* закрытая клетка */
            } else {
                printf("X ");        /* пока заглушка */
            }
        }
        printf("\n");
    }
}
```

Разбор:

- внешний `for` по `r` идёт по строкам, внутренний по `c` — по столбцам. Это стандартный обход двумерного массива из главы 15: для каждой строки печатаем всю её слева направо, потом `\n`.
- `printf("%2d ", r + 1)` — печатаем номер строки. `%2d` — это `%d` с минимальной шириной 2 (помнишь из главы 2), чтобы однозначные и двузначные числа выровнялись в столбик.
- `c + 1` и `r + 1` — **внутри** массива нумерация с нуля (0..8), а игроку показываем по-человечески с единицы (1..9). Про этот сдвиг — отдельный разговор в главном цикле, это частый источник путаницы.

Пока вместо настоящего содержимого клетки печатаем заглушку `X`. Но поле уже рисуется, и каркас готов.

Шаг 2. Расставляем мины случайно

Нужно поставить ровно `MINES` мин в разные клетки. Напрашивается: выбрать случайную клетку и поставить мину. Сделаем наивно:

```
void place_mines(void)
{
    for (int i = 0; i < MINES; i++) {
        int r = rand() % H;          /* случайная строка  0..H-1 */
        int c = rand() % W;          /* случайный столбец 0..W-1 */
        mines[r][c] = 1;
    }
}
```

- `rand()` возвращает случайное целое (см. главу 12). `rand() % H` даёт остаток от деления на `H`, то есть число в диапазоне `0..H-1` — как раз допустимый индекс строки. Так же для столбца.

- ставим 1 в выбранную клетку — это мина.

Проблема: `rand()` может **дважды выбрать одну и ту же клетку**. Тогда второй `mines[r][c] = 1` ничего не меняет (там уже 1), и мин окажется **меньше десяти**. Цикл прокрутится 10 раз, но мин будет, скажем, 9.

Чинить надо так: ставить мину только в **свободную** клетку, а счётчик увеличивать **только когда мину реально поставили**. Поэтому вместо `for` на фиксированное число повторов берём `while` и считаем поставленные мины сами:

```
void place_mines(void)
{
    int placed = 0;
    while (placed < MINES) {
        int r = rand() % H;
        int c = rand() % W;
        if (mines[r][c] == 0) {      /* клетка ещё свободна */
            mines[r][c] = 1;
            placed++;                /* только теперь засчитываем */
        }
    }
}
```

Теперь, если случайно выпала занятая клетка, `if` не сработает, `placed` не вырастет, и `while` просто попробует ещё раз. Цикл крутится, пока не наберётся ровно `MINES` разных мин.

Первый ход не должен убивать

Осталась ещё одна беда, и она не про код, а про честность игры. Мины стоят до того, как игрок сделал первый ход. Поле закрыто целиком, подсказок нет — ткнуть можно только наугад. А мин десять на 81 клетку: примерно каждая восьмая партия заканчивается словом «Бабах» на первом же ходу. Проиграть, не успев ничего решить, — обидно, и игрок тут ни при чём.

В настоящем «Сапёре» этой проблемы нет: первый клик безопасен всегда. Сделаем так же. Приём простой: **не ставить мины заранее**. Ждём первого хода, узнаём, куда игрок ткнул, и уже потом раскидываем мины — в обход этой клетки.

`place_mines` принимает координаты безопасной клетки и пропускает её:

```
void place_mines(int safe_r, int safe_c)
{
    int placed = 0;
    while (placed < MINES) {
```

```
        int r = rand() % H;
        int c = rand() % W;
        if (r == safe_r && c == safe_c) {
            continue;               /* сюда игрок сейчас ткнёт */
        }
        if (mines[r][c] == 0) {
            mines[r][c] = 1;
            placed++;
        }
    }
}
```

`continue` внутри `while` работает так же, как в главе 12: бросает текущий виток и уходит на новую проверку условия. Выпала запретная клетка — просто крутим дальше, `placed` не растёт.

А в `main` заводим флажок и ставим мины ровно один раз, между проверкой координат и проверкой «мина ли тут»:

Флаг объявляем в начале `main`, рядом с остальными переменными:

```
int mines_placed = 0;           /* мины ставим только после первого хода */
```

А сама постановка мин — уже внутри игрового цикла:

```
if (!mines_placed) {            /* самый первый ход задаёт поле */
    place_mines(row, col);
    mines_placed = 1;
}
```

- `int mines_placed = 0;` — обычная переменная-флаг: 0 значит «ещё не ставили», 1 — «уже».
- `if (!mines_placed)` — `!` переворачивает: условие истинно, пока флаг равен нулю. Внутри ставим мины и тут же поднимаем флаг, чтобы на втором ходу поле не перегенерировалось.
- Порядок важен: сначала убеждаемся, что координаты нормальные (иначе `safe_r`/`safe_c` окажутся мусором), и только потом минируем — до строчки, которая проверяет `mines[row][col] == 1`.

Теперь первый ход гарантированно открывает пустое место, и партия всегда начинается с какой-то информации о поле.

Шаг 3. Считаем соседей

Самая аккуратная функция в игре — подсчёт мин вокруг клетки. Соседей восемь, их смещения относительно клетки `(r, c)` такие: строка меняется на −1, 0 или +1, и столбец тоже на −1, 0 или +1. Все сочетания этих сдвигов и дают соседей — плюс саму клетку (сдвиг 0,0), которую считать не надо.

Перебрать все сочетания — это **два вложенных цикла по смещениям** от −1 до 1:

```
int count_neighbors(int r, int c)
{
    int total = 0;
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;            /* саму клетку не считаем */
            }
            int nr = r + dr;         /* строка соседа  */
            int nc = c + dc;         /* столбец соседа */
            if (nr < 0 || nr >= H || nc < 0 || nc >= W) {
                continue;            /* сосед за краем поля */
            }
            if (mines[nr][nc] == 1) {
                total++;
            }
        }
    }
    return total;
}
```

Разберём по частям, тут два тонких места.

- `dr` и `dc` (delta row, delta column) пробегают `-1, 0, 1` каждый. Вместе они дают **девять** комбинаций — восемь соседей плюс саму клетку.
- `if (dr == 0 && dc == 0) continue;` — когда оба смещения нулевые, мы смотрим на саму клетку `(r, c)`, а не на соседа. `continue` — «пропусти оставшееся тело цикла и переходи к следующей итерации» (см. главу 6). Так центральная клетка не учитывается.
- `nr = r + dr`, `nc = c + dc` — координаты соседа.

Теперь **проверка границ** — разберём подробно, потому что без неё программа лезет в чужую память. У клетки на краю поля часть соседей оказывается за пределами массива. Например,

для клетки в строке 0 сосед сверху имеет `nr = -1` — такой строки нет. Для клетки в последнем столбце `W-1` сосед справа имеет `nc = W`, а допустимые индексы только `0..W-1`.

Условие

```
if (nr < 0 || nr >= H || nc < 0 || nc >= W)
```

ловит все четыре способа вылезти за поле:

- `nr < 0` — выше верхнего края;
- `nr >= H` — ниже нижнего края (индексы строк только до `H-1`);
- `nc < 0` — левее левого края;
- `nc >= W` — правее правого края.

Если хоть одно верно (`||` — «или»), сосед за полем — `continue`, не трогаем его. Помнишь из главы 3: Си **не проверяет** границы массива сам. Обратись к `mines[-1][5]` — и ты читаешь случайный кусок памяти, получишь мусор или падение. Эта проверка — наша собственная защита, и пропускать её нельзя.

Шаг 4. Открываем клетку и запускаем заливку

Теперь сердце игры — рекурсивная заливка. Сначала напишем её **наивно и неправильно**, чтобы своими глазами увидеть, что бывает без базы.

Наивная идея: открыть клетку, и если она пустая — позвать себя для всех восьми соседей.

```
/* НАИВНАЯ ВЕРСИЯ — С БАГОМ, не используй */
void reveal(int r, int c)
{
    opened[r][c] = 1;                /* открыли клетку */
    if (count_neighbors(r, c) == 0) {
        for (int dr = -1; dr <= 1; dr++) {
            for (int dc = -1; dc <= 1; dc++) {
                reveal(r + dr, c + dc);   /* зовём соседей */
            }
        }
    }
}
```

Что тут не так? Сразу несколько бед:

1. Мы зовём `reveal(r + dr, c + dc)` **без проверки границ**. На краю поля это `reveal(-1, ...)`, и первая же строка `opened[-1][c]` пишет в чужую память.

2. Среди девяти вызовов есть `reveal(r, c)` — клетка зовёт **сама себя**. И вообще: открыли A, пошли к соседу B, B пустой — зовёт соседей, среди них снова A. А раз мы нигде не проверяем «уже открыто», A откроется опять, снова позовёт B, B — A… **бесконечная рекурсия**. Из главы 9 ты помнишь, чем это кончается: стек вызовов переполняется, программа падает с **stack overflow**.

Если такую версию запустить, в лучшем случае получишь аварийный вылет вроде:

```
Segmentation fault
```

Чиним добавлением **тех самых трёх баз** из разбора руками. База рекурсии — это условие, при котором функция сразу выходит, не углубляясь дальше:

```
void reveal(int r, int c)
{
    if (r < 0 || r >= H || c < 0 || c >= W) {
        return;                     /* база: вышли за поле */
    }
    if (opened[r][c] == 1) {
        return;                     /* база: уже открыто */
    }

    opened[r][c] = 1;               /* открываем эту клетку */

    if (count_neighbors(r, c) != 0) {
        return;                     /* база: рядом есть мины — стоп */
    }

    /* Клетка пустая (0 соседей) — продолжаем заливку на 8 соседей. */
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;
            }
            reveal(r + dr, c + dc);
        }
    }
}
```

Что изменилось и почему теперь всё хорошо:

- **Первая база** — проверка границ. Зашли за поле — сразу `return`, никакого доступа к `opened[-1]...`. Это позволяет звать соседей не глядя: вылетевшие за край вызовы тихо вернутся.
- **Вторая база** — `if (opened[r][c] == 1) return;`. Уже открытую клетку второй раз не трогаем. Вот она и разрывает порочный круг A→B→A: когда заливка вернётся к уже открытой клетке, она остановится. **Это главная база против зацикливания.**
- Только пройдя обе проверки, ставим `opened[r][c] = 1`.
- **Третья база** — `if (count_neighbors(r, c) != 0) return;`. Клетку с числом мы открыли (число станет видно), но волну дальше не пускаем: раз рядом мина, соседей раскрывать опасно.
- И лишь для **пустой** клетки идём по восьми соседям и зовём `reveal` для каждого. Каждый такой вызов либо упрётся в базу, либо откроет ещё кусок. Поле конечное, открытых клеток всё больше, повторно их не трогаем — значит, рано или поздно все вызовы упрутся в базы и заливка закончится. Ровно как очередь из главы 9 докатывается до «я первый».

Сравни с главой 9: там у факториала была одна база (`n == 0`), здесь их три, но принцип тот же — **каждый вызов либо отвечает сразу (база), либо сводит задачу к меньшей** (открыли клетку — непройденных стало меньше).

Шаг 5. Главный цикл: ход, проигрыш, победа

Осталось связать всё в игру. Нам ещё нужно:

- показывать мины при проигрыше — добавим в печать параметр;
- считать, сколько клеток открыто, чтобы поймать победу.

Допишем `print_board` параметром `show_mines` и заменим заглушку `X` на настоящее содержимое. Меняется только тело внутреннего цикла:

```
if (show_mines && mines[r][c] == 1) {
    printf("* ");                /* при проигрыше видны мины */
} else if (opened[r][c] == 0) {
    printf(". ");                /* закрытая клетка */
} else {
    int n = count_neighbors(r, c);
    if (n == 0) {
        printf("  ");            /* открыто и пусто */
    } else {
        printf("%d ", n);        /* открыто, рядом n мин */
```

```
    }
}
```

- `show_mines && mines[r][c] == 1` — мину рисуем `*`, но только когда `show_mines` включён (мы делаем это лишь при проигрыше);
- иначе закрытую клетку рисуем точкой;
- иначе клетка открыта: ноль соседей — два пробела (пусто), иначе само число.

Победу ловим так: считаем открытые клетки и сравниваем с числом безопасных (`H*W - MINES`). Открыли все безопасные — победа.

```
int count_opened(void)
{
    int total = 0;
    for (int r = 0; r < H; r++)
        for (int c = 0; c < W; c++)
            if (opened[r][c] == 1) total++;
    return total;
}
```

Осталось собрать игровой цикл `main` (полный текст — в листинге ниже). Его логика: «нарисовали поле → проверили победу → взяли ход → обработали». Ключевые места:

- `srand((unsigned int)time(NULL))` — встряхиваем генератор от текущего времени, чтобы мины каждый раз ложились по-новому (глава 12). Без `srand` поле было бы одинаковым при каждом запуске.
- `while (1)` — бесконечный цикл (`1` всегда истина); выходим через `break` при победе, проигрыше или конце ввода.
- `scanf("%d %d", &row, &col)` — читаем **два** числа за раз. `scanf` возвращает, сколько значений прочитал. Проверка как в главе 12: `EOF` — ввод кончился (Ctrl+D); `rc != 2` — ввели не два числа, чистим строку через `getchar` и просим заново.
- `row--; col--;` — **перевод координат из человеческих в машинные**. Игрок вводит с **1**, массив индексируется с **0**. Забыть этот сдвиг — классическая ошибка: либо первая строка поля станет недоступна, либо обращение к `mines[9][...]` вылезет за границу. Вычли единицу — и `row`, `col` стали правильными индексами.
- три проверки по порядку: вне поля? уже открыто? мина (тогда `print_board(1)` и `break`)? Если всё чисто — `reveal` открывает клетку и запускает заливку.

Полная программа

Файл `code/16-minesweeper.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define H 9         /* высота поля (строк)   */
#define W 9         /* ширина поля (столбцов) */
#define MINES 10    /* сколько мин на поле    */

int mines[H][W];   /* 1 — здесь мина, 0 — пусто           */
int opened[H][W];  /* 1 — клетка открыта игроком, 0 — нет */

/* Сколько мин стоит вокруг клетки (r, c) — среди 8 соседей. */
int count_neighbors(int r, int c)
{
    int total = 0;
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;            /* саму клетку не считаем */
            }
            int nr = r + dr;         /* строка соседа  */
            int nc = c + dc;         /* столбец соседа */
            if (nr < 0 || nr >= H || nc < 0 || nc >= W) {
                continue;            /* сосед за краем поля */
            }
            if (mines[nr][nc] == 1) {
                total++;
            }
        }
    }
    return total;
}

/* Рекурсивная заливка: открыть клетку (r, c) и, если она пустая,
   открыть всех её соседей, и так дальше. */
void reveal(int r, int c)
{
    if (r < 0 || r >= H || c < 0 || c >= W) {
```

```
        return;                     /* база: вышли за поле */
    }
    if (opened[r][c] == 1) {
        return;                     /* база: уже открыто */
    }

    opened[r][c] = 1;               /* открываем эту клетку */

    if (count_neighbors(r, c) != 0) {
        return;                     /* база: рядом есть мины — стоп */
    }

    /* Клетка пустая (0 соседей) — продолжаем заливку на 8 соседей. */
    for (int dr = -1; dr <= 1; dr++) {
        for (int dc = -1; dc <= 1; dc++) {
            if (dr == 0 && dc == 0) {
                continue;
            }
            reveal(r + dr, c + dc);
        }
    }
}

/* Расставить MINES мин по случайным клеткам без повторов.
   Клетку (safe_r, safe_c) — первый ход игрока — не минируем,
   иначе можно проиграть на первом же ходу, ничего не зная о поле. */
void place_mines(int safe_r, int safe_c)
{
    int placed = 0;
    while (placed < MINES) {
        int r = rand() % H;
        int c = rand() % W;
        if (r == safe_r && c == safe_c) {
            continue;               /* сюда игрок сейчас ткнёт */
        }
        if (mines[r][c] == 0) {     /* клетка ещё свободна */
            mines[r][c] = 1;
            placed++;
```

```
        }
    }
}

/* Напечатать поле. Если show_mines != 0 — показать и мины (при проигрыше). */
void print_board(int show_mines)
{
    printf("   ");
    for (int c = 0; c < W; c++) {
        printf("%d ", c + 1);        /* шапка со столбцами 1..W */
    }
    printf("\n");

    for (int r = 0; r < H; r++) {
        printf("%2d ", r + 1);       /* номер строки слева */
        for (int c = 0; c < W; c++) {
            if (show_mines && mines[r][c] == 1) {
                printf("* ");
            } else if (opened[r][c] == 0) {
                printf(". ");        /* закрытая клетка */
            } else {
                int n = count_neighbors(r, c);
                if (n == 0) {
                    printf("  ");    /* открыто и пусто */
                } else {
                    printf("%d ", n);
                }
            }
        }
        printf("\n");
    }
}

/* Сколько клеток уже открыто. */
int count_opened(void)
{
    int total = 0;
    for (int r = 0; r < H; r++) {
```

```
        for (int c = 0; c < W; c++) {
            if (opened[r][c] == 1) {
                total++;
            }
        }
    }
    return total;
}

int main(void)
{
    srand((unsigned int)time(NULL));

    int mines_placed = 0;            /* мины ставим только после первого хода */
    int safe_cells = H * W - MINES;  /* столько клеток без мин */

    printf("Сапёр %dx%d, мин: %d\n", H, W, MINES);
    printf("Вводи координаты: строка 1..%d и столбец 1..%d.\n", H, W);

    while (1) {
        print_board(0);

        if (count_opened() == safe_cells) {
            printf("Победа! Все безопасные клетки открыты.\n");
            break;
        }

        printf("Ход (строка столбец): ");
        int row, col;
        int rc = scanf("%d %d", &row, &col);
        if (rc == EOF) {
            printf("\nИгра прервана.\n");
            break;
        }
        if (rc != 2) {
            printf("Нужно ввести два числа. Попробуй ещё раз.\n");
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {
```

```
                /* выбрасываем мусорный ввод до конца строки */
            }
            continue;
        }

        row--;                      /* игрок считает с 1, массив — с 0 */
        col--;

        if (row < 0 || row >= H || col < 0 || col >= W) {
            printf("Координаты вне поля: строка 1..%d, столбец 1..%d.\n",
                   H, W);
            continue;
        }
        if (opened[row][col] == 1) {
            printf("Эта клетка уже открыта.\n");
            continue;
        }

        if (!mines_placed) {        /* самый первый ход задаёт поле */
            place_mines(row, col);
            mines_placed = 1;
        }

        if (mines[row][col] == 1) {
            print_board(1);         /* показываем все мины */
            printf("Бабах! Ты открыл мину. Игра окончена.\n");
            break;
        }

        reveal(row, col);           /* безопасный ход — заливаем */
    }

    return 0;
}
```

Как это работает

Пройдём один ход целиком.

1. Оба массива, `mines` и `opened`, пока в нулях: мин ещё нет (их поставим после первого хода), не открыто ничего.
2. `print_board(0)` рисует поле из точек — игрок видит закрытую сетку.
3. `count_opened()` вернёт 0, до `safe_cells` (это 81 − 10 = 71) далеко, победы нет.
4. Игрок вводит, скажем, `1 1`. `scanf` прочитал два числа, `rc == 2`. `row--`, `col--` превращают их в индексы `0, 0`.
5. Клетка `(0,0)` не за полем и не открыта. Ход первый, значит `place_mines(0, 0)` раскидывает 10 мин по всему полю, кроме этой клетки. Мины тут гарантированно нет — зовём `reveal(0, 0)`.
6. Если вокруг `(0,0)` нет мин, заливка раскрывает целую область: `reveal` зовёт соседей, те — своих, пока волна не упрётся в клетки с числами и в края поля. На экране появится пустая поляна, окаймлённая цифрами.
7. Цикл повторяется. Когда `count_opened()` дорастёт до 71, печатается «Победа». Если же игрок ткнёт в мину — печатаем поле с `*` и «Бабах».

Проверяем

Скомпилируй без предупреждений:

```
cc -std=c11 -Wall -Wextra -O2 code/16-minesweeper.c -o minesweeper
```

Запусти `./minesweeper` и поиграй вживую. А чтобы показать заливку наглядно, вот реальный прогон (поле случайное, у тебя будет своё). Игрок открыл угол `1 1`, и раскрылась большая область, обрамлённая числами:

```
Сапёр 9x9, мин: 10
Вводи координаты: строка 1..9 и столбец 1..9.
   1 2 3 4 5 6 7 8 9
 1 . . . . . . . . .
 2 . . . . . . . . .
 3 . . . . . . . . .
 4 . . . . . . . . .
 5 . . . . . . . . .
 6 . . . . . . . . .
 7 . . . . . . . . .
 8 . . . . . . . . .
 9 . . . . . . . . .
Ход (строка столбец): 1 1
   1 2 3 4 5 6 7 8 9
 1       1 . . . . .
```

```
 2     1 3 . . . . .
 3     1 . . . . . .
 4     1 3 . 2 1 3 .
 5       1 . 1   2 .
 6       1 1 1   2 .
 7               1 1
 8
 9
Ход (строка столбец):
```

Видишь: одним ходом открылась целая пустая область (пробелы) — левый край и весь низ поля, — а по её границе проступили числа. Закрытыми (`.`) остались клетки за этой границей, куда заливка не пошла.

А вот проигрыш. Первый ход, как мы помним, безопасен всегда, поэтому на мину игрок нарвался вторым — ткнул наугад в дальний угол `9 9`, и поле показало все мины звёздочками:

```
Ход (строка столбец): 9 9
   1 2 3 4 5 6 7 8 9
 1     1 * . . * . .
 2 1 1 2 . . . . . .
 3 . * . . . . . . .
 4 . . * . . . . . .
 5 . . . * . . . . .
 6 . . . . . . . . .
 7 * . . . . . . . .
 8 . . . . . . . . .
 9 . * . * . . . * *
Бабах! Ты открыл мину. Игра окончена.
```

Частые ошибки

Выход за границы при подсчёте соседей. Самая частая беда этой игры — забыть проверку `nr < 0 || nr >= H || nc < 0 || nc >= W` в `count_neighbors` (и аналогичную базу в `reveal`). Для клеток у края соседи вылезают за массив, и без проверки ты читаешь чужую память: числа врут, а то и падение. Си границы не сторожит (глава 3) — это твоя забота.

Заливка без базы → зацикливание. Если убрать `if (opened[r][c] == 1) return;`, соседние пустые клетки начнут открывать друг друга по кругу — A зовёт B, B зовёт A, бесконечно. Стек

вызовов переполнится, и программа рухнет с `stack overflow` (глава 9). База «уже открыто» обязательна.

Путаница в нумерации координат. Игрок считает с 1, массив — с 0. Забудешь `row--; col--;` — и либо потеряешь первую строку/столбец, либо вылезешь за поле на `mines[9][...]`. Решай это **в одном месте**, сразу после ввода, чтобы дальше везде были чистые индексы 0..8.

Мины ставятся с повторами. Если расставлять мины простым `for` на `MINES` повторов без проверки `if (mines[r][c] == 0)`, в одну клетку может попасть две «мины», и их окажется меньше задуманного. Считай **реально поставленные**, как в `place_mines`.

`[H][W]` против `[r][c]`. Лёгко перепутать порядок и написать `mines[c][r]`. Договорись раз и навсегда: первый индекс — строка, второй — столбец, и держись этого во всех функциях.

Упражнения

1. **Счётчик ходов.** Заведи переменную `moves` и при победе печатай, за сколько ходов игрок прошёл поле.
2. **Флажки на мины.** Добавь третий массив `flags[H][W]` и команду пометить клетку флажком (например, ввод `0 r c` — пометить, `1 r c` — открыть). Флажок печатай как `F`. Это помогает игроку помнить, где он подозревает мину.
3. **Размер и число мин с ввода.** Спроси у игрока `H`, `W` и `MINES` в начале. Подсказка: с фиксированными `#define` так нельзя — понадобятся либо глобальные массивы максимального размера, либо `malloc` (глава 3). Не забудь проверить, что мин меньше, чем клеток.
4. **Первый ход открывает поляну.** Мы уже сделали первую клетку безопасной. В настоящем «Сапёре» идут дальше: первый ход почти всегда раскрывает целую пустую область. Добейся того же — запрети мины не только в самой клетке, но и во всех восьми её соседях (тогда число соседних мин там будет ноль, и заливка пойдёт). Подсказка: условие в `place_mines` станет проверкой «разница по строке и по столбцу не больше единицы». Не забудь: мин должно по-прежнему хватить на оставшиеся клетки.
5. **Показ числа открытых.** После каждого хода печатай «открыто N из 71» — пусть видно, сколько осталось до победы.
6. **Авто-проигрыш-разбор.** При проигрыше дополнительно покажи, в скольких клетках игрок успел открыть пустые поляны, и где он ошибся.

Что мы получили

Мы собрали настоящую игру из готовых кирпичей: **два двумерных массива** как наложенные карты (где мины и что открыто), **случайную расстановку без повторов и без мины под**

первым ходом, **подсчёт восьми соседей с проверкой границ** и **рекурсивную заливку с тремя базами**. Ни одной новой конструкции языка — только знакомое из прошлых глав, сложенное в новую форму. Так и пишутся программы посложнее: не из экзотики, а из комбинаций простых, надёжных деталей.

Заодно мы ещё раз прочувствовали рекурсию (глава 9) на живой задаче и увидели, почему база — это не формальность, а единственное, что не даёт программе рухнуть.

В следующей главе возьмёмся за игру в реальном времени — **«Змейку»**. Там поле тоже будет двумерным, но появится новое: считывать клавиши **без нажатия Enter** и перерисовывать экран в цикле, пока змейка ползёт. Для этого познакомимся с режимом терминала через `termios`. Готовь стрелки.

Глава 17. Игра «Змейка»

Вот мы и добрались до финала. «Змейка» — та самая игра из старых телефонов: по полю ползёт змейка, ест еду, растёт и не должна врезаться ни в стену, ни в саму себя. Игра простая на вид, но в ней сходится всё, что мы прошли: двумерное поле, массивы координат, структура состояния, случайная еда, цикл и проверки столкновений.

И тут появляется по-настоящему новое. Раньше наши программы ждали, пока ты нажмёшь клавишу и Enter. Змейка так не умеет: она ползёт **сама**, а клавишу должна ловить **мгновенно**, без Enter и без эха на экране. За это отвечает «сырой» режим терминала через `termios` — главная новая тема главы. Разберём её спокойно и по частям.

Что мы сделаем

К концу главы у тебя будет настоящая игра в терминале:

- поле с рамкой из символов `#`;
- змейка (`0` — голова, `o` — тело), которая ползёт сама;
- управление клавишами W A S D **без нажатия Enter**;
- еда `*`, которая появляется в случайном месте; съел — змейка растёт и счёт увеличивается;
- проигрыш при столкновении со стеной или с собственным телом;
- выход по клавише `q`.

Строить будем как всегда: от простого к сложному. Сначала просто нарисуем поле и неподвижную змейку. Потом заставим её двигаться. Потом научимся ловить клавиши. Потом добавим еду и рост. И в конце — столкновения и проигрыш.

Разбираем задачу руками

Прежде чем писать код, поймём две вещи: как хранить змейку и почему обычного ввода тут не хватит.

Как хранить змейку

Змейка — это цепочка клеток на поле. Каждая клетка — пара чисел: номер столбца `x` и номер строки `y`. Заведём два массива:

```
snake_x:  9  8  7
snake_y:  5  5  5
```

Это змейка из трёх сегментов на одной строке. Элемент с индексом `0` — **голова** (`x=9, y=5`), дальше идёт тело. Длину храним отдельно в переменной `snake_len`.

Как змейка делает шаг? Представь гусеницу: каждый сегмент переползает на место предыдущего, а голова делает шаг в новую клетку. Сдвинули тело — поставили новую голову. Всё.

```
было:   голова(9,5)  тело(8,5)  тело(7,5)
сдвиг:  ?(9,5)        (9,5)      (8,5)      <- каждый занял место соседа
голова: (10,5)        (9,5)      (8,5)      <- голова шагнула вправо
```

Направление движения тоже храним парой чисел `dir_x, dir_y`: вправо — это `(1, 0)`, влево — `(-1, 0)`, вверх — `(0, -1)`, вниз — `(0, 1)`. Обрати внимание: вверх — это **минус** по `y`, потому что строки нумеруются сверху вниз (нулевая строка — самая верхняя).

Почему обычный ввод не подходит

По умолчанию терминал работает в **каноническом режиме** (canonical mode, его ещё зовут «строчным»). Это значит две вещи:

1. Терминал **ждёт Enter**. Что бы ты ни печатал, программа ничего не получит, пока ты не нажмёшь Enter. Вся строка копится в буфере.
2. Терминал **печатает эхо** — показывает на экране каждый набранный символ.

Для обычной программы это удобно: можно стереть опечатку до Enter. Но для игры — катастрофа. Змейка должна реагировать на `d` сразу, а не ждать Enter. И буквы `wasd` не должны замусоривать наше поле.

Значит, нам нужно временно выключить два этих поведения: ожидание Enter и эхо. А ещё сделать чтение **неблокирующим** — чтобы программа не зависала, ожидая клавишу, когда её не нажали. Змейка ведь ползёт и без ввода.

Всё это делается через `termios` — набор настроек терминала в POSIX (стандарт для macOS и Linux). На Windows механизм другой (там для той же задачи берут `_getch` из `<conio.h>`), но мы пишем под macOS/Linux.

Шаг 1. Поле и неподвижная змейка

Начнём с самого простого: нарисуем рамку, змейку и еду — пока без всякого движения. Цель шага — увидеть картинку.

```c
#include <stdio.h>

#define WIDTH   20
#define HEIGHT  10

int main(void)
{
    int snake_x[3] = {9, 8, 7};
    int snake_y[3] = {5, 5, 5};
    int snake_len = 3;

    /* верхняя рамка */
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');

    for (int y = 0; y < HEIGHT; y++) {
        putchar('#');                       /* левая стенка */
        for (int x = 0; x < WIDTH; x++) {
            char ch = ' ';
            for (int i = 0; i < snake_len; i++) {
                if (snake_x[i] == x && snake_y[i] == y) {
                    ch = (i == 0) ? 'O' : 'o';
                    break;
                }
            }
            putchar(ch);
        }
        putchar('#');                       /* правая стенка */
        putchar('\n');
    }

    /* нижняя рамка */
```

```
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');
    return 0;
}
```

Разбор нового:

- `#define WIDTH 20` — это **макрос-константа**. `#define ИМЯ значение` говорит компилятору: «перед сборкой замени везде в тексте слово `WIDTH` на `20`». Удобно: размер поля задан в одном месте, захочешь поле побольше — поменяешь одну строку. Точки с запятой в конце `#define` **не ставят**.
- `WIDTH` и `HEIGHT` — ширина и высота **внутренней** части поля, без рамки. Рамка добавляет по одному символу слева и справа, поэтому в цикле рамки идёт `WIDTH + 2`.
- `putchar('#')` — функция из `stdio.h`, печатает **один символ**. Это как `printf("%c", '#')`, только короче и быстрее. Аргумент — символ в одинарных кавычках.
- Внешний цикл по `y` идёт по строкам поля (0..HEIGHT−1), внутренний по `x` — по столбцам (0..WIDTH−1). Для каждой клетки `(x, y)` решаем, что в ней напечатать.
- `char ch = ' ';` — по умолчанию клетка пустая (пробел).
- Внутренний цикл `for (int i = 0; ...)` проверяет: не стоит ли в этой клетке сегмент змейки? Если координаты сегмента `i` совпали с `(x, y)`, рисуем символ.
- `ch = (i == 0) ? 'O' : 'o';` — это **тернарный оператор** (условное выражение). Читается так: «если `i == 0`, то значение `'O'`, иначе `'o'`». Короткая запись `if/else` прямо внутри выражения. Голова (индекс 0) — заглавная `O`, тело — строчная `o`.
- `break;` — как только нашли сегмент в клетке, выходим из внутреннего цикла: дальше проверять нет смысла.

Скомпилируй и запусти:

```
######################
#                    #
#                    #
#                    #
#                    #
#                    #
#       ooO          #
#                    #
#                    #
#                    #
```

```
#                    #
######################
```

Поле есть, змейка на месте (голова `0` справа, тело `oo` слева). Но она не двигается. Чиним.

Шаг 2. Заставляем змейку ползти

Чтобы змейка двигалась, нужен **игровой цикл**: бесконечно повторяем «сдвинули змейку → перерисовали поле → подождали чуть-чуть». А чтобы картинка не уезжала вниз простынёй, перед каждой отрисовкой будем **очищать экран**.

Очистка делается специальной командой для терминала — **управляющей последовательностью** (escape sequence):

```
printf("\033[H\033[2J");
```

Разберём по кускам:

- `\033` — это символ Escape (его код 27). Запись `\033` — код в восьмеричной системе. С него начинаются почти все команды управления терминалом. Терминал, увидев Escape и `[`, понимает: дальше не текст для печати, а команда.
- `\033[H` — команда «переместить курсор в левый верхний угол» (Home). Следующая печать пойдёт с самого верха.
- `\033[2J` — команда «стереть весь экран» (J — erase, 2 — целиком).

Вместе `\033[H\033[2J` — «очисти экран и поставь курсор в угол». Это работает в любом нормальном терминале macOS/Linux.

Теперь сдвиг змейки. Как договорились, тело переползает по цепочке, а голова шагает в направлении `dir`:

```
/* сдвигаем тело: каждый сегмент встаёт на место предыдущего */
for (int i = snake_len - 1; i > 0; i--) {
    snake_x[i] = snake_x[i - 1];
    snake_y[i] = snake_y[i - 1];
}
/* голова делает шаг */
snake_x[0] += dir_x;
snake_y[0] += dir_y;
```

Важно идти **с хвоста к голове** (`i` от конца к 1). Если бы мы шли с головы, то затёрли бы координаты, которые ещё нужны соседнему сегменту.

Чтобы между кадрами была видимая пауза, нужна функция `usleep` (*microsecond sleep* — «спать микросекунды»). Она из POSIX (`<unistd.h>`):

```
usleep(150000);   /* 150 000 микросекунд = 0,15 секунды */
```

В одной секунде миллион микросекунд, так что `150000` — это примерно 0,15 секунды между кадрами. Чем меньше число, тем быстрее змейка.

Маленькая, но важная деталь сборки. Мы пишем со стандартом `-std=c11`, а `usleep` — функция POSIX, не из чистого C11. Чтобы компилятор её «увидел», в самом начале файла добавим строку:

```
#define _DEFAULT_SOURCE
```

Это «фича-макрос»: он включает доступ к POSIX-функциям (`usleep`, `read`, `termios`) при строгом стандарте C11. Поставить его нужно **до** всех `#include`.

Соберём шаг 2 целиком — змейка ползёт вправо сама:

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <unistd.h>

#define WIDTH   20
#define HEIGHT  10

int main(void)
{
    int snake_x[3] = {9, 8, 7};
    int snake_y[3] = {5, 5, 5};
    int snake_len = 3;
    int dir_x = 1, dir_y = 0;   /* ползём вправо */

    for (int step = 0; step < 8; step++) {   /* пока 8 шагов для пробы */
        printf("\033[H\033[2J");

        /* рисуем поле */
        for (int x = 0; x < WIDTH + 2; x++) putchar('#');
        putchar('\n');
        for (int y = 0; y < HEIGHT; y++) {
            putchar('#');
            for (int x = 0; x < WIDTH; x++) {
```

```
                char ch = ' ';
                for (int i = 0; i < snake_len; i++) {
                    if (snake_x[i] == x && snake_y[i] == y) {
                        ch = (i == 0) ? 'O' : 'o';
                        break;
                    }
                }
                putchar(ch);
            }
            putchar('#');
            putchar('\n');
        }
        for (int x = 0; x < WIDTH + 2; x++) putchar('#');
        putchar('\n');

        /* двигаем змейку */
        for (int i = snake_len - 1; i > 0; i--) {
            snake_x[i] = snake_x[i - 1];
            snake_y[i] = snake_y[i - 1];
        }
        snake_x[0] += dir_x;
        snake_y[0] += dir_y;

        usleep(150000);
    }
    return 0;
}
```

Запусти — увидишь, как змейка проползает вправо восемь шагов и программа завершается. Уже живая картинка! Но управлять ей мы пока не можем, а на стену она просто уедет. Добавим управление.

Шаг 3. Ловим клавиши без Enter — сырой режим

Это сердце главы. Нам нужно:

1. выключить ожидание Enter и эхо (сырой режим);
2. читать клавишу **не блокируясь** — нет нажатия, идём дальше;
3. в конце игры **вернуть** терминал в обычное состояние.

Сначала подключаем заголовки:

```
#include <termios.h>   /* настройки терминала */
#include <unistd.h>    /* read, STDIN_FILENO, usleep, _exit */
#include <signal.h>    /* signal, SIGINT — перехват Ctrl+C */
```

`<termios.h>` даёт структуру `struct termios` (набор флагов терминала) и функции `tcgetattr`/`tcsetattr` (прочитать и записать эти настройки). `STDIN_FILENO` из `<unistd.h>` — это номер стандартного ввода (клавиатуры), равен 0. `<signal.h>` понадобится чуть ниже, когда будем страховаться от Ctrl+C.

Включаем сырой режим

```
static struct termios orig_termios;   /* сюда сохраним исходные настройки */

static void disable_raw_mode(void)
{
    tcsetattr(STDIN_FILENO, TCSANOW, &orig_termios);
}

static void enable_raw_mode(void)
{
    tcgetattr(STDIN_FILENO, &orig_termios);   /* запомнили как было */
    struct termios raw = orig_termios;        /* копия для правки */

    raw.c_lflag &= ~(ICANON | ECHO);          /* снять канонику и эхо */
    raw.c_cc[VMIN]  = 0;                       /* read не ждёт символов */
    raw.c_cc[VTIME] = 0;                       /* и не ждёт по таймеру */

    tcsetattr(STDIN_FILENO, TCSANOW, &raw);    /* применить сейчас же */
}
```

Разбираем по строчкам:

- `static struct termios orig_termios;` — глобальная переменная под исходные настройки. Слово `static` тут значит «видна только в этом файле». `struct termios` — структура с настройками терминала (про структуры мы говорили в главе 11).
- `tcgetattr(STDIN_FILENO, &orig_termios)` — **прочитать** текущие настройки терминала и сложить в `orig_termios`. `&` — адрес: функция запишет настройки в нашу переменную (как `scanf` пишет по адресу).

- `struct termios raw = orig_termios;` — делаем **копию**. Исходник `orig_termios` бережём нетронутым, чтобы в конце восстановить.
- `raw.c_lflag &= ~(ICANON | ECHO);` — самая важная строка. `c_lflag` — поле «локальных флагов» терминала; в нём набор включённых режимов.
  - `ICANON` — флаг канонического режима (ожидание Enter).
  - `ECHO` — флаг эха (печать набранного на экран).
  - `ICANON | ECHO` — операцией `|` (побитовое ИЛИ) объединяем оба флага в один набор битов.
  - `~(...)` — побитовое НЕ: переворачивает биты, получая «всё, кроме этих двух флагов».
  - `&=` — присвоить результат побитового И. Вместе строка означает: «оставить все флаги как есть, но **снять** `ICANON` и `ECHO`». Эта связка `flag &= ~(БИТЫ)` — стандартный способ выключить биты.
- `raw.c_cc[VMIN] = 0; raw.c_cc[VTIME] = 0;` — массив `c_cc` хранит «спецсимволы» и параметры чтения. `VMIN = 0` и `VTIME = 0` вместе означают: «чтение возвращает управление **сразу**, даже если ничего не нажато». Это и есть неблокирующее чтение — без `fcntl`, штатными средствами termios.
- `tcsetattr(STDIN_FILENO, TCSANOW, &raw)` — **записать** новые настройки. `TCSANOW` — «применить немедленно».
- `disable_raw_mode` делает обратное: возвращает сохранённый `orig_termios`. Её **обязательно** вызвать в конце, иначе после игры терминал останется без эха и будет вести себя странно.

Страховка: терминал надо вернуть в любом случае

«Вызвать в конце» звучит просто, но конец бывает разный. Игра может закончиться нормально — тогда мы дойдём до `disable_raw_mode` сами. А может и не дойти: ты нажмёшь Ctrl+C, программа упадёт, ты закроешь её из другого окна. Терминал при этом останется в сыром режиме, и дальше в нём невозможно работать: печатаешь — букв не видно, жмёшь Enter — ничего.

Проверь сам: запусти змейку и нажми Ctrl+C. Пока в программе нет страховки, ты получишь ровно это (спасает команда `reset`, набранная вслепую). Поставим две подпорки:

```
atexit(disable_raw_mode);      /* вернуть режим при любом обычном выходе */
signal(SIGINT, on_sigint);     /* ...и при Ctrl+C тоже */
```

- `atexit(функция)` из `<stdlib.h>` регистрирует функцию, которую система вызовет **при нормальном завершении программы** — хоть через `return` из `main`, хоть через `exit()`. Аргумент — имя функции без скобок: мы передаём саму функцию, а не результат её

вызова. Теперь, даже если мы добавим где-то в середине `exit(1)`, терминал всё равно починится.

- `atexit` не спасает от сигналов: Ctrl+C посылает программе сигнал `SIGINT`, и по умолчанию тот убивает процесс **мимо** всех зарегистрированных функций. Поэтому сигнал перехватываем: `signal(SIGINT, on_sigint)` из `<signal.h>` говорит «когда придёт SIGINT, вызови вот эту функцию».

Сам обработчик короткий:

```
static void on_sigint(int sig)
{
    (void)sig;
    disable_raw_mode();
    _exit(130);          /* 128 + номер SIGINT — так принято в шеллах */
}
```

- `int sig` — номер пришедшего сигнала; такова обязательная подпись обработчика. Нам он не нужен, но параметр убрать нельзя, а `-Wextra` ругнётся на неиспользованный. Строка `(void)sig;` — общепринятый способ сказать компилятору «я знаю про этот параметр и намеренно его не трогаю».
- `_exit(130)` из `<unistd.h>` завершает процесс **немедленно**. Почему не обычный `exit`? Внутри обработчика сигнала безопасно вызывать далеко не всё: сигнал может прилететь в любой момент, хоть посреди работы `printf`, и повторный заход в ту же функцию всё сломает. `_exit` — из короткого списка того, что вызывать можно. Число 130 — принятый код выхода «убит по Ctrl+C» (128 плюс номер SIGINT, равный 2).

Строго говоря, `tcsetattr` в этом списке безопасных функций тоже есть — иначе так починить терминал не вышло бы.

Читаем клавишу

```
static int read_key(void)
{
    char c;
    ssize_t n = read(STDIN_FILENO, &c, 1);
    if (n == 1) {
        return (unsigned char)c;
    }
    return -1;
}
```

- `read(STDIN_FILENO, &c, 1)` — POSIX-функция чтения: «прочитать из стандартного ввода в переменную `c` максимум 1 байт». Возвращает, сколько байт реально прочла.
- `ssize_t` — целочисленный тип для результата `read` (может быть и отрицательным при ошибке). Считай его «специальным `int` для размеров».
- Если `n == 1` — клавишу нажали, возвращаем её код. Приведение `(unsigned char)` нужно, чтобы коды не ушли в отрицательные.
- Если нажатия не было, `read` вернёт 0 (мы же сделали неблокирующее чтение), и функция вернёт `-1` — «клавиши нет».

Теперь в `main` обернём всё: включим режим в начале, выключим в конце, а в цикле будем спрашивать `read_key`. Реверс (резкий разворот на 180°) запрещаем — иначе змейка мгновенно врежется в себя:

```
static int dir_x, dir_y;

static void set_dir(int nx, int ny)
{
    if (nx == -dir_x && ny == -dir_y) {
        return;   /* противоположное текущему — игнорируем */
    }
    dir_x = nx;
    dir_y = ny;
}
```

`set_dir` меняет направление, но если новое — точная противоположность текущему (`nx == -dir_x` и `ny == -dir_y`), просто выходит, ничего не меняя.

Выглядит убедительно. И всё-таки эта версия ломается — сейчас увидим, как. Но сначала посмотрим, откуда берётся поломка.

В игровом цикле читаем **все** накопившиеся нажатия за кадр (вдруг ты нажал несколько раз) и применяем их:

```
int key;
while ((key = read_key()) != -1) {
    if (key == 'w' || key == 'W') set_dir(0, -1);
    else if (key == 's' || key == 'S') set_dir(0, 1);
    else if (key == 'a' || key == 'A') set_dir(-1, 0);
    else if (key == 'd' || key == 'D') set_dir(1, 0);
    else if (key == 'q' || key == 'Q') game_over = 1;
}
```

- `while ((key = read_key()) != -1)` — присваиваем результат `read_key()` переменной `key` **и тут же** сравниваем с `-1`. Пока клавиши есть, крутимся. Опустошили буфер — `read_key` вернёт `-1`, цикл закончится. Скобки вокруг `key = read_key()` обязательны.
- Дальше обычный разбор: WASD меняют направление (учитываем и заглавные на случай Caps Lock), `q` — выход.

Где ломается защита от разворота

Вот здесь два куска кода и подставляют друг друга. Цикл выгребает за кадр **все** нажатия подряд, а `set_dir` сравнивает новое направление с `dir_x/dir_y` — то есть с результатом предыдущего нажатия, а не с тем, куда змейка на самом деле ползёт.

Змейка едет вправо, `dir = (1, 0)`. Ты нервно тыкаешь W, а следом A (обе клавиши успевают попасть в один кадр):

1. `set_dir(0, -1)` — вверх. Проверка: `(0,-1)` против `-(1,0) = (-1,0)`. Не совпало, разрешено. Теперь `dir = (0, -1)`.
2. `set_dir(-1, 0)` — влево. Проверка: `(-1,0)` против `-(0,-1) = (0,1)`. Не совпало — тоже разрешено! `dir = (-1, 0)`.

Итог: за один кадр направление перевернулось с «вправо» на «влево» через промежуточное «вверх», и `update` вгонит голову ровно в тот сегмент, который идёт сразу за ней. Мгновенный проигрыш, причём из-за нажатия, которое игрок считает законным.

Проверить это можно, не играя. Соберём голый огрызок игры — только координаты, без терминала:

```
/* два нажатия за один кадр: W, потом A, пока ползём вправо */
set_dir(0, -1);
set_dir(-1, 0);
printf("направление после двух нажатий: (%d,%d)\n", dir_x, dir_y);
update();
printf("после кадра: голова (%d,%d), game_over=%d\n",
       snake_x[0], snake_y[0], game_over);
```

Запуск со старым `set_dir`:

```
направление после двух нажатий: (-1,0)
после кадра: голова (9,5), game_over=1
```

Голова стояла в `(10,5)`, тело тянулось влево — и она шагнула прямо в себя.

Чинится это одной идеей: сравнивать надо не с «накопленным» направлением, а с тем, **куда змейка реально поехала в прошлом кадре**. Заводим для этого вторую пару переменных:

```
static int dir_x, dir_y;       /* куда голова поедет в следующем кадре */
static int moved_x, moved_y;   /* куда она реально поехала в прошлом */

static void set_dir(int nx, int ny)
{
    if (nx == -moved_x && ny == -moved_y) {
        return;   /* противоположное реальному движению — игнорируем */
    }
    dir_x = nx;
    dir_y = ny;
}
```

А `update` в самом начале запоминает, с каким направлением работает этот кадр:

```
static void update(void)
{
    moved_x = dir_x;
    moved_y = dir_y;
    /* ... дальше сдвиг тела и шаг головы, как было ... */
}
```

Теперь второе нажатие в том же кадре меряется от `moved = (1,0)`, и `A` честно отсекается. Тот же огрызок после правки:

```
направление после двух нажатий: (0,-1)
после кадра: голова (10,4), game_over=0
```

Змейка ушла вверх — ровно то, чего игрок и хотел. Не забудь в `main` задать `moved_x = dir_x; moved_y = dir_y;` вместе с начальным направлением, иначе первый же `set_dir` сравнится с парой нулей и пропустит что угодно.

Теперь змейкой можно рулить. Осталась еда, рост и проигрыш.

Шаг 4. Еда, рост и счёт

Еда `*` лежит в случайной клетке. Съел — змейка прибавляет сегмент, а счёт растёт. Заводим переменные `food_x, food_y, score`.

Случайные числа — из главы про `rand` (`<stdlib.h>`). Кладём еду в свободную клетку:

```
static void place_food(void)
{
```

```
    int ok = 0;
    while (!ok) {
        food_x = rand() % WIDTH;
        food_y = rand() % HEIGHT;
        ok = 1;
        for (int i = 0; i < snake_len; i++) {
            if (snake_x[i] == food_x && snake_y[i] == food_y) {
                ok = 0;   /* попали в тело — выбираем заново */
                break;
            }
        }
    }
}
```

- `rand() % WIDTH` — остаток от деления случайного числа на ширину даёт число в диапазоне 0..WIDTH−1, то есть допустимый столбец. Аналогично для строки.
- Цикл `while (!ok)` повторяет выбор, пока еда не окажется в пустой клетке: если случайно попали в тело змейки, ставим `ok = 0` и бросаем кубик заново.

Теперь рост. Хитрость в том, что при сдвиге тела последний сегмент (хвост) «теряется» — его координаты затираются. Если змейка съела еду, мы вернём этот хвост обратно, и она станет на сегмент длиннее. Поэтому **перед** сдвигом запоминаем хвост:

```
int tail_x = snake_x[snake_len - 1];
int tail_y = snake_y[snake_len - 1];

/* ...сдвиг тела и шаг головы как раньше... */

if (snake_x[0] == food_x && snake_y[0] == food_y) {
    if (snake_len < MAX_LEN) {
        snake_x[snake_len] = tail_x;   /* отрастили хвост обратно */
        snake_y[snake_len] = tail_y;
        snake_len++;
    }
    score += 10;
    place_food();
}
```

- `snake_len - 1` — индекс последнего сегмента (помни: нумерация с 0, поэтому последний — не `snake_len`, а на единицу меньше).

- Если голова попала на еду, добавляем новый сегмент в позицию бывшего хвоста: `snake_x[snake_len] = tail_x`, и увеличиваем длину.
- `if (snake_len < MAX_LEN)` — защита от выхода за границы массива: не даём длине превысить размер. `MAX_LEN` — максимум сегментов (все клетки поля).
- `score += 10` — за каждую еду 10 очков. После — кладём новую еду.

В отрисовку добавляем еду перед проверкой змейки:

```
if (x == food_x && y == food_y) {
    ch = '*';
} else {
    /* ...поиск сегмента змейки, как раньше... */
}
```

Шаг 5. Столкновения и конец игры

Последний штрих — проигрыш. Игра кончается, если голова вышла за стену или наехала на собственное тело. Проверяем **после** шага головы:

```
/* столкновение со стеной */
if (snake_x[0] < 0 || snake_x[0] >= WIDTH ||
    snake_y[0] < 0 || snake_y[0] >= HEIGHT) {
    game_over = 1;
    return;
}

/* столкновение с собственным телом (со 2-го сегмента) */
for (int i = 1; i < snake_len; i++) {
    if (snake_x[i] == snake_x[0] && snake_y[i] == snake_y[0]) {
        game_over = 1;
        return;
    }
}
```

- Первое условие: голова ушла левее 0, правее поля, выше 0 или ниже поля — это удар о стену.
- Второй цикл начинается с `i = 1` (нулевой сегмент — сама голова, сравнивать её с собой бессмысленно). Если координаты головы совпали с любым сегментом тела — змейка укусила себя.
- `game_over = 1` поднимает флаг, `return` обрывает шаг: дальше еду проверять незачем.

Главный цикл крутится, пока `!game_over`. Как только флаг поднят (или нажали `q`), выходим, **возвращаем терминал** и печатаем счёт. Собираем всё вместе.

Полная программа

Сохрани это как `code/17-snake.c`.

```c
/* Глава 17. Игра «Змейка» в терминале.
 *
 * Управление: W A S D — вверх/влево/вниз/вправо, q — выход.
 * Компиляция (macOS/Linux):
 *   cc -std=c11 -Wall -Wextra -O2 17-snake.c -o snake
 *
 * Программа интерактивная: запускать в обычном терминале, не в пайпе.
 */

#define _DEFAULT_SOURCE   /* открыть доступ к usleep, read, termios в C11 */

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <time.h>
#include <termios.h>
#include <unistd.h>

/* ---- Размеры игрового поля (без учёта рамки) ---- */
#define WIDTH   20
#define HEIGHT  10
#define MAX_LEN (WIDTH * HEIGHT)   /* змейка не длиннее всех клеток поля */

/* ---- Состояние игры ---- */
static int snake_x[MAX_LEN];   /* координаты сегментов: [0] — голова */
static int snake_y[MAX_LEN];
static int snake_len;          /* сколько сегментов сейчас */
static int dir_x, dir_y;       /* куда голова поедет в следующем кадре */
static int moved_x, moved_y;   /* куда она реально поехала в прошлом кадре */
static int food_x, food_y;     /* где лежит еда */
static int score;              /* счёт */
static int game_over;          /* 0 — играем, 1 — конец */
```

```c
/* ---- Сырой режим терминала ---- */
static struct termios orig_termios;   /* сюда сохраним исходные настройки */

/* Вернуть терминал в нормальный режим. */
static void disable_raw_mode(void)
{
    tcsetattr(STDIN_FILENO, TCSANOW, &orig_termios);
}

/* Ctrl+C во время игры: чиним терминал и только потом умираем.
   Сам по себе сигнал убивает процесс мимо atexit, поэтому перехватываем. */
static void on_sigint(int sig)
{
    (void)sig;
    disable_raw_mode();
    _exit(130);          /* 128 + номер SIGINT — так принято в шеллах */
}

/* Включить «сырой» неблокирующий ввод: без ожидания Enter и без эха. */
static void enable_raw_mode(void)
{
    tcgetattr(STDIN_FILENO, &orig_termios);   /* запомнили как было */
    struct termios raw = orig_termios;        /* копия для правки */

    raw.c_lflag &= ~(ICANON | ECHO);          /* снять канонический режим и эхо */
    raw.c_cc[VMIN]  = 0;                       /* read не ждёт символов */
    raw.c_cc[VTIME] = 0;                       /* и не ждёт по таймеру */

    tcsetattr(STDIN_FILENO, TCSANOW, &raw);    /* применить сейчас же */
}

/* Прочитать одну клавишу. Если её нет — вернуть -1 (не ждём ввода). */
static int read_key(void)
{
    char c;
    ssize_t n = read(STDIN_FILENO, &c, 1);
    if (n == 1) {
```

```
        return (unsigned char)c;
    }
    return -1;
}

/* Положить еду в случайную свободную клетку. */
static void place_food(void)
{
    int ok = 0;
    while (!ok) {
        food_x = rand() % WIDTH;
        food_y = rand() % HEIGHT;
        ok = 1;
        for (int i = 0; i < snake_len; i++) {
            if (snake_x[i] == food_x && snake_y[i] == food_y) {
                ok = 0;   /* попали в тело — выбираем заново */
                break;
            }
        }
    }
}

/* Нарисовать поле, змейку, еду и счёт. */
static void draw(void)
{
    printf("\033[H\033[2J");   /* курсор в угол + очистка экрана */

    /* верхняя рамка */
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');

    for (int y = 0; y < HEIGHT; y++) {
        putchar('#');                       /* левая стенка */
        for (int x = 0; x < WIDTH; x++) {
            char ch = ' ';
            if (x == food_x && y == food_y) {
```

```
                ch = '*';                   /* еда */
            } else {
                for (int i = 0; i < snake_len; i++) {
                    if (snake_x[i] == x && snake_y[i] == y) {
                        ch = (i == 0) ? 'O' : 'o';   /* голова / тело */
                        break;
                    }
                }
            }
            putchar(ch);
        }
        putchar('#');                       /* правая стенка */
        putchar('\n');
    }

    /* нижняя рамка */
    for (int x = 0; x < WIDTH + 2; x++) {
        putchar('#');
    }
    putchar('\n');

    printf("Счёт: %d   (WASD — движение, q — выход)\n", score);
}

/* Сменить направление, не разрешая разворот на 180 градусов.
   Сравниваем с тем, куда змейка ПОЕХАЛА в прошлом кадре (moved_x/moved_y),
   а не с уже накопленным dir_x/dir_y: иначе два нажатия за один кадр
   (вверх, потом влево, пока ползём вправо) сложились бы в разворот. */
static void set_dir(int nx, int ny)
{
    if (nx == -moved_x && ny == -moved_y) {
        return;   /* противоположное реальному движению — игнорируем */
    }
    dir_x = nx;
    dir_y = ny;
}

/* Один шаг змейки: сдвиг тела, новая голова, столкновения, еда. */
```

```
static void update(void)
{
    /* фиксируем направление этого кадра — на него будет смотреть set_dir */
    moved_x = dir_x;
    moved_y = dir_y;

    /* запомним хвост — он пригодится, если змейка вырастет */
    int tail_x = snake_x[snake_len - 1];
    int tail_y = snake_y[snake_len - 1];

    /* сдвигаем тело: каждый сегмент встаёт на место предыдущего */
    for (int i = snake_len - 1; i > 0; i--) {
        snake_x[i] = snake_x[i - 1];
        snake_y[i] = snake_y[i - 1];
    }

    /* голова делает шаг в текущем направлении */
    snake_x[0] += dir_x;
    snake_y[0] += dir_y;

    /* столкновение со стеной */
    if (snake_x[0] < 0 || snake_x[0] >= WIDTH ||
        snake_y[0] < 0 || snake_y[0] >= HEIGHT) {
        game_over = 1;
        return;
    }

    /* столкновение с собственным телом (со 2-го сегмента) */
    for (int i = 1; i < snake_len; i++) {
        if (snake_x[i] == snake_x[0] && snake_y[i] == snake_y[0]) {
            game_over = 1;
            return;
        }
    }

    /* съели еду? */
    if (snake_x[0] == food_x && snake_y[0] == food_y) {
        if (snake_len < MAX_LEN) {
```

```
            snake_x[snake_len] = tail_x;   /* отрастили хвост обратно */
            snake_y[snake_len] = tail_y;
            snake_len++;
        }
        score += 10;
        place_food();
    }
}

int main(void)
{
    srand((unsigned)time(NULL));   /* разный «случай» при каждом запуске */

    enable_raw_mode();             /* переключаем терминал в игровой режим */
    atexit(disable_raw_mode);      /* вернуть режим при любом обычном выходе */
    signal(SIGINT, on_sigint);     /* ...и при Ctrl+C тоже */

    /* начальная змейка из 3 сегментов в центре, ползёт вправо */
    snake_len = 3;
    int cx = WIDTH / 2;
    int cy = HEIGHT / 2;
    snake_x[0] = cx;     snake_y[0] = cy;
    snake_x[1] = cx - 1; snake_y[1] = cy;
    snake_x[2] = cx - 2; snake_y[2] = cy;
    dir_x = 1;
    dir_y = 0;
    moved_x = dir_x;
    moved_y = dir_y;
    score = 0;
    game_over = 0;

    place_food();
    draw();                        /* показываем стартовое поле сразу */

    while (!game_over) {
        /* считываем все накопившиеся нажатия */
        int key;
        while ((key = read_key()) != -1) {
```

```
            if (key == 'w' || key == 'W') set_dir(0, -1);
            else if (key == 's' || key == 'S') set_dir(0, 1);
            else if (key == 'a' || key == 'A') set_dir(-1, 0);
            else if (key == 'd' || key == 'D') set_dir(1, 0);
            else if (key == 'q' || key == 'Q') game_over = 1;
        }

        if (game_over) {
            break;              /* нажали q */
        }

        update();
        draw();

        usleep(150000);         /* пауза 0,15 с между кадрами */
    }

    disable_raw_mode();         /* ОБЯЗАТЕЛЬНО вернуть терминал как было */

    printf("\nИгра окончена! Итоговый счёт: %d\n", score);
    return 0;
}
```

Как это работает

Проследим один оборот игрового цикла, пока `game_over` равен 0:

1. **Ввод.** Внутренний `while` вытягивает из буфера все нажатые клавиши. WASD меняют `dir_x/dir_y` через `set_dir` (он сверяется с `moved_x/moved_y` — направлением прошлого кадра — и не пускает разворот), `q` поднимает `game_over`. Нет нажатий — `read_key` сразу отдаёт `-1`, и мы не зависаем.
2. **Шаг.** `update` фиксирует `moved_x/moved_y`, запоминает хвост, сдвигает тело по цепочке, двигает голову на `(dir_x, dir_y)`. Затем проверяет стену и тело — при ударе ставит `game_over` и выходит. Если голова на еде — отращивает хвост обратно, плюсует счёт и кладёт новую еду.
3. **Отрисовка.** `draw` чистит экран `\033[H\033[2J` и заново рисует рамку, еду `*`, голову `O`, тело `o` и строку счёта.

4. **Пауза.** `usleep(150000)` ждёт 0,15 секунды — иначе кадры мелькали бы слишком быстро, чтобы что-то разглядеть.

Цикл повторяется, и из быстрой смены кадров рождается плавное движение. Как закончили — `disable_raw_mode` возвращает терминал в нормальный вид, и печатается итоговый счёт.

Глобальные переменные (`snake_x`, `dir_x` и прочие) вынесены наружу, чтобы все функции (`update`, `draw`, `place_food`) работали с одним и тем же состоянием, не передавая кучу аргументов. Для маленькой игры в один файл это нормально.

Проверяем

Скомпилируй и запусти:

```
cc -std=c11 -Wall -Wextra -O2 17-snake.c -o snake
./snake
```

Команда `-std=c11` задаёт стандарт языка, `-Wall -Wextra` включают все предупреждения (их быть не должно), `-O2` — оптимизацию. Благодаря `#define _DEFAULT_SOURCE` в начале файла никакие лишние флаги для POSIX не нужны — собирается чисто, без предупреждений.

Запусти `./snake` **в обычном окне терминала** (не через пайп и не в режиме, где нет клавиатуры). Увидишь поле, змейку в центре и еду. Управляй клавишами W, A, S, D — Enter нажимать **не надо**, змейка поворачивает сразу. Наводи голову на `*`: змейка растёт, счёт растёт на 10. Врежешься в стену или в себя — игра закончится и покажет счёт. Выйти в любой момент — клавиша `q`.

Картинка во время игры выглядит примерно так:

```
######################
#                    #
#          *         #
#                    #
#                    #
#       oooO         #
#                    #
#                    #
#                    #
#                    #
#                    #
######################
Счёт: 20   (WASD — движение, q — выход)
```

Частые ошибки

Забыл вернуть режим терминала. Если программа завершится (или упадёт) до `disable_raw_mode`, терминал останется в сыром режиме: ты печатаешь, а букв не видно, Enter не срабатывает как обычно. Это не поломка навсегда — набери вслепую команду `reset` и нажми Enter, терминал починится. Чтобы такого не было, зови `disable_raw_mode` перед выходом и подстрахуйся `atexit` плюс обработчиком `SIGINT`: одного явного вызова в конце `main` мало, потому что до конца `main` можно и не добраться.

Картинка мерцает или «лесенкой» уезжает вниз. Значит, ты не очищаешь экран перед отрисовкой. Каждый кадр должен начинаться с `printf("\033[H\033[2J")`. Если мерцание сильное, проверь, что не печатаешь лишних пустых строк между кадрами.

Выход за границу массива сегментов. Если растить змейку без проверки `if (snake_len < MAX_LEN)`, при заполнении всего поля длина вылезет за размер массивов `snake_x/snake_y` — порча памяти. Граница обязательна.

Змейка мгновенно проигрывает при развороте. Без защиты в `set_dir` нажатие в обратную сторону вгоняет голову в свой второй сегмент — и сразу «конец игры». Поэтому развороты на 180° запрещены. И сверяться при этом надо с `moved_x/moved_y` — направлением, в котором змейка реально сделала прошлый шаг. Если сравнивать с `dir_x/dir_y`, два быстрых нажатия за один кадр (W, потом A на движении вправо) проскочат проверку по очереди и всё равно сложатся в разворот.

Цикл сдвига тела «съедает» змейку. Если сдвигать сегменты от головы к хвосту (а не от хвоста к голове), координаты затрутся раньше времени, и тело схлопнется. Иди в цикле от `snake_len - 1` к 1.

`usleep` не компилируется при `-std=c11`. Это и значит, что не хватает `#define _DEFAULT_SOURCE` в самом начале файла (до `#include`). Либо добавь его, либо собирай с `-D_DEFAULT_SOURCE`.

Упражнения

1. **Ускорение.** Сделай так, чтобы с каждой съеденной едой игра шла быстрее: заведи переменную задержки (например, начать с `150000`) и уменьшай её на немного при росте, но не ниже какого-то минимума, чтобы змейка не стала неуправляемой. Передавай это значение в `usleep`.
2. **Пауза.** Добавь клавишу `p` — пауза. По нажатию игра замирает (цикл крутится, но `update` не вызывается и рисуется надпись «ПАУЗА»), повторное нажатие `p` продолжает.

3. **Стены-телепорты.** Сделай режим, где змейка не разбивается о стену, а появляется с противоположной стороны. Подсказка: вместо проверки на проигрыш заверни координаты по модулю — например, `snake_x[0] = (snake_x[0] + WIDTH) % WIDTH`.
4. **Рекорд.** Запоминай лучший счёт за сессию и показывай его рядом с текущим. Усложнение: сохраняй рекорд в файл (`fopen`, `fprintf`, `fscanf`) и читай при старте, чтобы он жил между запусками.
5. **Две еды.** Добавь второй вид еды, которая даёт больше очков, но живёт на поле ограниченное число кадров и потом исчезает.
6. **Размер поля.** Вынеси `WIDTH` и `HEIGHT` в аргументы командной строки (`int main(int argc, char *argv[])`), чтобы запускать `./snake 30 15`. Подумай, как тогда задать размеры массивов (подсказка: `malloc`).

Что мы получили

Ты собрал полноценную аркадную игру: поле, самодвижущаяся змейка, мгновенное управление без Enter, случайная еда, рост, счёт и честный проигрыш. По дороге разобрал по-настоящему важную тему — **управление терминалом**: сырой и неблокирующий ввод через `termios`, очистку экрана управляющими последовательностями и игровой цикл с паузой `usleep`. Это тот самый набор, на котором держатся все текстовые игры — от «Тетриса» до рогаликов.

И на этом наша часть про игры завершается. Оглянись: ты прошёл путь от первого `printf("Привет, мир!")` до интерактивной игры в реальном времени. Между ними — переменные и циклы, массивы и указатели, строки и структуры, рекурсия, сортировки и поиск, двумерные поля, случайные числа и работа с терминалом. Этого фундамента хватит, чтобы браться за свои проекты: придумай игру или инструмент, разбей на маленькие шаги — ровно как мы делали в каждой главе — и собирай по кусочку. Дальше дорогу выбираешь ты.

Глава 18. Что дальше

Если ты дошёл до этой страницы, написав и запустив программы по дороге, — ты уже умеешь программировать на Си. Не «слышал про», а именно умеешь: объявить переменную и понять, где она лежит в памяти, перебрать массив, написать функцию, передать в неё адрес, выделить память и вернуть её, реализовать сортировку, поиск, рекурсию и собрать полноценную игру с вводом без Enter. Это солидный фундамент.

Давай коротко оглянёмся, а потом наметим, куда расти.

Что мы прошли

Основы. Компилятор, первая программа, переменные и типы, ввод-вывод, ветвления, циклы, функции, массивы, строки, указатели и динамическая память. Это азбука, на которой держится всё остальное.

Алгоритмы. Числовые задачи и FizzBuzz, НОД Евклида, простые числа и решето Эратосфена, операции над массивами, линейный и бинарный поиск, три сортировки, рекурсия (факториал, Фибоначчи, Ханойские башни), строковые алгоритмы, динамические структуры данных — растущий массив, стек, очередь, связный список.

Игры. «Угадай число», камень-ножницы-бумага, виселица, крестики-нолики, сапёр и змейка. Каждая — это сборка изученного в одно работающее целое: случайность, состояние на двумерной сетке, рекурсивная заливка, управление терминалом в реальном времени.

Главное, что ты унёс, — не список тем, а **способ мышления**: разбить задачу на шаги, написать наивную версию, запустить, найти проблему, починить. Так пишут код все, от новичка до сеньора.

Привычки, которые стоит закрепить

- **Компилируй с предупреждениями.** Всегда `-Wall -Wextra` и держи код «чистым» — без единого warning. Это бесплатно ловит половину багов.

- **Проверяй краевые случаи.** Пустой ввод, ноль, отрицательное число, массив из одного элемента, самое большое и самое маленькое значение. Ломается обычно именно там.
- **Следи за памятью.** На каждый `malloc` — однажды `free`. Не выходи за границы массивов. Не разыменовывай `NULL`.
- **Читай ошибки компилятора до конца.** Они кажутся пугающими, но почти всегда точно указывают строку и причину.

Инструменты, которые сделают тебя сильнее

- **Отладчик** `lldb` (на macOS) или `gdb` (на Linux). Позволяет останавливать программу в любой строке и смотреть значения переменных. Один вечер с отладчиком экономит недели «печати через `printf`».
- **Санитайзеры.** Скомпилируй с `-fsanitize=address,undefined` — и программа сама закричит, если ты вышел за границу массива, использовал освобождённую память или допустил переполнение. Бесценно для Си.
- **`valgrind`** (на Linux) — находит утечки памяти и обращения к неинициализированным данным.
- **Система сборки.** Когда файлов станет много, познакомься с `make` и простым `Makefile`, чтобы не набирать команды компиляции руками.
- **Git** — система контроля версий. Сохраняй историю своего кода с первого дня; однажды это спасёт тебя от потери дня работы.

Куда двигаться дальше

Углубить Си. Мы намеренно не трогали часть языка, чтобы не перегружать. `struct` мы лишь начали; `enum` и указатели на функции попадались мельком (перечисление ходов в камень-ножницы-бумаге, функция сравнения для `qsort`), но не разбирались всерьёз. Совсем за бортом остались объединения (`union`), многомерная динамическая память, работа с файлами (`fopen`/`fread`/`fwrite`), битовые операции и препроцессор глубже макро-констант. Классическая книга для следующего шага — «Язык программирования C» Кернигана и Ритчи (K&R): тонкая, строгая, написана авторами языка.

Углубить алгоритмы. Следующий уровень: быстрая сортировка и сортировка слиянием с полной реализацией, бинарные деревья поиска, хеш-таблицы, графы и обход в ширину/глубину, динамическое программирование. Отличная практика — решать задачи на онлайн-площадках (Codeforces, LeetCode, e-olymp): там тысячи задач с автопроверкой.

Сделать игры красивее. Загляни в библиотеку **ncurses** — она берёт на себя управление терминалом (то, что мы делали руками через `termios` в змейке) и даёт цвета, окна, рамки. А

если захочется графики, звука и спрайтов — библиотека **SDL2** или **raylib**: на чистом Си можно написать настоящую 2D-игру с окном и мышкой.

Применить Си по-настоящему. Микроконтроллеры (Arduino, STM32), системное программирование под Linux, написание расширений для Python, парсеры, интерпретаторы. Си открывает дверь туда, где важна скорость и контроль над «железом».

Маленький совет напоследок

Не пытайся выучить всё сразу. Возьми одну вещь из списка выше, которая зажгла интерес, и сделай на ней небольшой проект — пусть кривой, пусть маленький. Доведённый до конца крошечный проект учит больше, чем десять прочитанных, но не написанных глав.

Ты начинал эту книгу, возможно, не написав ни строчки кода. Теперь у тебя за плечами почти четыре десятка работающих программ — от «Привет, мир!» до змейки, — и это не считая решений задач с собеседований из приложений. Дальше — только интереснее. Удачи, и пусть компилятор будет к тебе благосклонен.

Как готовиться к собеседованию по Си

Следующие три приложения — это задачи с собеседований на C-разработчика, разбитые по уровням: Junior, Middle, Senior. Прежде чем браться за них, полезно понять, **что вообще проверяют** на таких собеседованиях и как готовиться эффективно. Этот короткий раздел — карта местности.

Что проверяют на каждом уровне

Уровни условны (в разных компаниях планка своя), но общая логика такая.

Junior. Тебя проверяют на «ты вообще умеешь писать код». Базовый синтаксис, циклы, массивы, функции, простые алгоритмы (развернуть строку, найти максимум, проверить палиндром). Любят каверзные вопросы на понимание основ: чему равно `5 / 2`, что напечатает `sizeof('a')`, в чём разница `=` и `==`. Здесь не ждут блестящих алгоритмов — ждут аккуратности и понимания того, что ты пишешь.

Middle. Главная тема — **указатели и память**. Своя `strlen`, операции со связными списками, работа с битами, понимание, чем `char *s = "abc"` отличается от `char s[] = "abc"`, что такое «распад» массива в указатель, где живут утечки и use-after-free. Алгоритмы уже посерьёзнее: бинарный поиск без ошибок на единицу, цикл Флойда, слияние массивов. Проверяют, что ты понимаешь, **что происходит в памяти**, а не просто заучил рецепты.

Senior. Тонкости языка и инженерное мышление. Неопределённое поведение (UB) и его источники, `volatile`, две роли `static`, выравнивание и padding в структурах, опасности макросов, порядок вычислений, endianness. Из алгоритмов — быстрая сортировка и сортировка слиянием, обобщённые структуры через `void *`, хеш-таблицы, кэш LRU. Здесь важно не только решить, но и обсудить компромиссы: сложность, память, краевые случаи, что сломается в продакшене.

Как пользоваться задачами

1. **Сначала реши сам.** Прочитай условие, закрой решение, возьми бумагу или редактор. Даже неудачная попытка включает голову так, как чтение готового ответа никогда не включит.
2. **Проговаривай вслух.** На собеседовании ценят не только ответ, но и ход мысли. Тренируйся объяснять, что и почему ты делаешь.
3. **Скомпилируй и запусти.** У каждой задачи на код есть файл в `code/`. Собери его с `-Wall -Wextra`, поиграй с входными данными, сломай нарочно — так понимание закрепляется.
4. **Разбери краевые случаи.** Пустой ввод, ноль, отрицательные числа, массив из одного элемента, переполнение. Интервьюер почти всегда спросит «а что, если…».

Частые ошибки на собеседовании (и как их избежать)

- **Молчать.** Тишина пугает интервьюера сильнее неверного ответа. Думай вслух: «так, мне нужно… я попробую вот так… здесь может быть проблема с границей».
- **Бросаться в код сразу.** Сначала уточни условие и проговори план. «Правильно ли я понял: массив отсортирован? могут быть дубликаты?»
- **Забывать про краевые случаи.** Пустой массив и переполнение `int` — любимые ловушки. Проверяй их сам, не дожидаясь вопроса.
- **Путать `=` и `==`, забывать `&` в `scanf`, выходить за границы массива.** Базовые ошибки на собеседовании читаются как «невнимательный».
- **Игнорировать память.** Выделил через `malloc` — скажи, где сделаешь `free`. Это сигнал, что ты понимаешь Си, а не просто пишешь на нём.
- **Спорить, когда не прав.** Если интервьюер показал ошибку — признай, разберись, исправь. Умение принимать обратную связь ценят высоко.

Что повторить перед собеседованием

- **Указатели**: `&`, `*`, указатель на указатель, указатель и массив, передача в функцию по адресу (глава 3).
- **Память**: stack против heap, `malloc`/`free`, утечки, use-after-free, double free (главы 3 и 11).
- **Строки**: устройство через `'\0'`, почему `==` не сравнивает строки (главы 3 и 10).
- **Биты**: `&`, `|`, `^`, `<<`, `>>`, приёмы вроде `n & (n - 1)`.
- **Сложность**: O(n), O(n log n), O(n^2) — уметь оценить свой код.
- **UB**: знать, что переполнение знакового `int`, выход за границы и запись в строковый литерал — неопределённое поведение.

Не пытайся выучить всё за ночь. Пройди задачи по уровням, начиная с Junior, даже если метишь выше: простые задачи быстро освежают базу, а каверзы из них всплывают на собеседованиях любого уровня. Поехали.

Приложение А. Задачи с собеседований: Junior

Книга закончилась, а собеседования только начинаются. Это приложение — сборник задач, которые реально дают на собеседовании С-разработчику уровня **Junior** (младший, начинающий). Здесь нет ничего, чего ты не проходил: всё опирается на главы 1–18 — числа, массивы, строки, указатели, циклы, функции. Новой теории не будет. Будет другое — взгляд на знакомые приёмы глазами интервьюера.

Зачем это нужно. На собесе мало написать работающий код. Спрашивающий смотрит, **как ты думаешь**: учёл ли краевые случаи, помнишь ли про переполнение, понимаешь ли, что делает каждая строка. Поэтому к каждой задаче я добавил блок «что спросят дальше» — те самые уточняющие вопросы, которыми тебя будут «дожимать».

Как пользоваться

Каждая задача оформлена так:

- **Условие** — как её формулируют вслух на собесе.
- **Идея** — как к ней подступиться, без кода.
- **Решение** — рабочий код.
- **Разбор** — почему так, а не иначе.
- **Что спросят дальше** — продолжения и подвохи.

Главный совет: **сначала реши сам**, на бумаге или в редакторе, и только потом смотри решение. Подсмотренное решение забывается к утру. Решённое самому — остаётся. Даже если решил криво — сравни с моим вариантом, и разница тебе многое объяснит.

Раздел делится на две части. Сначала **задачи на код** — где надо что-то написать. Потом блок **каверз** «что выведет / в чём ошибка» — короткие фрагменты, на которых проверяют, насколько ты внимателен к деталям языка.

Рабочие решения лежат в `code/19-XX-*.c` — их можно собрать и запустить. Каверзы намеренно содержат ошибки или неопределённое поведение, поэтому в файлы они не вынесены: разбираем их прямо здесь.

Часть 1. Задачи на код

Задача 1. Поменять местами две переменные

Условие. Есть две переменные `a` и `b`. Поменяй их значения местами. После обмена в `a` должно лежать то, что было в `b`, и наоборот.

Идея. Наивная попытка «в лоб» не работает:

```
a = b;   /* старое значение a потеряно навсегда */
b = a;   /* сюда попадёт b, а не старое a */
```

Первая же строка затирает `a`. Нужна **временная переменная**, чтобы запомнить значение перед затиранием.

Решение (`code/19-01-swap.c`):

```c
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 9;

    printf("До:    a = %d, b = %d\n", a, b);

    int temp = a;   /* запомнили a */
    a = b;          /* в a положили b */
    b = temp;       /* в b положили старое a */

    printf("После: a = %d, b = %d\n", a, b);
    return 0;
}
```

Запуск:

```
До:    a = 5, b = 9
После: a = 9, b = 5
```

Разбор. `temp` хранит копию `a`, пока мы перезаписываем `a` значением `b`. Потом достаём из `temp` старое `a` и кладём в `b`. Три строки, всё прозрачно.

Что спросят дальше. «А можно без временной переменной?» Да, два способа.

Через арифметику:

```c
a = a + b;
b = a - b;   /* = старое a */
a = a - b;   /* = старое b */
```

Через XOR (исключающее «или», оператор `^`):

```c
a = a ^ b;
b = a ^ b;
a = a ^ b;
```

Оба выглядят эффектно, и интервьюер любит про них спросить. Но честный ответ — **в реальном коде так делать не надо**. Почему:

- Арифметический способ **переполняется**: если `a + b` вылезет за пределы `int`, получишь мусор (см. главу 4 про переполнение).
- XOR-своп ломается, если `a` и `b` — **одна и та же переменная** (один адрес): `x ^ x` даёт 0, и значение теряется. Для `swap(&x, &x)` это реальная ловушка.
- Оба варианта **нечитаемы** и не быстрее: современный компилятор и так оптимально реализует обмен через `temp`.

Правильный вывод, который хочет услышать грамотный интервьюер: «знаю про XOR-трюк, но в продакшене пишу через `temp` — понятнее и без подвохов».

Задача 2. Проверить чётность числа

Условие. Дано целое число. Определи, чётное оно или нечётное.

Идея. Школьный способ — остаток от деления: `n % 2 == 0` (см. главу 4). Но на собесе часто хотят увидеть **битовый** приём: у чётного числа младший бит равен 0, у нечётного — 1. Проверить младший бит можно операцией `&` (побитовое «И») с единицей.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int n = 7;

    if (n & 1) {
        printf("%d — нечётное\n", n);
    } else {
        printf("%d — чётное\n", n);
    }
    return 0;
}
```

Разбор. `n & 1` сравнивает биты `n` и числа 1 по позициям. У единицы выставлен только самый младший бит. Поэтому `n & 1` оставляет от `n` ровно этот младший бит: 1 у нечётных, 0 у чётных. В `if` ненулевое — это истина (см. главу 2), так что `if (n & 1)` срабатывает на нечётных.

Что спросят дальше. «Чем `n & 1` лучше `n % 2`?» Для положительных чисел — ничем существенным, оба корректны и одинаково быстры. Но есть тонкость с **отрицательными**: в C `-7 % 2` даёт `-1` (мы это проверим в каверзах), поэтому проверка `n % 2 == 1` для отрицательных нечётных **не сработает** — там получится `-1`, а не `1`. Безопаснее писать `n % 2 != 0`. А `n & 1` на типичной машине даёт правильный младший бит и для отрицательных. Это хороший момент показать, что ты знаешь про знак.

Задача 3. Максимум и минимум массива

Условие. Дан массив целых. Найди наибольший и наименьший элементы за один проход.

Идея. Заводим `max` и `min`, инициализируем их **первым элементом** массива (не нулём! см. подвох ниже) и идём по остальным, обновляя.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int a[] = {4, -2, 9, 1, 7, -8, 3};
    int n = (int)(sizeof(a) / sizeof(a[0]));
```

```
    int max = a[0];
    int min = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
        if (a[i] < min) {
            min = a[i];
        }
    }

    printf("min = %d, max = %d\n", min, max);
    return 0;
}
```

Разбор. `sizeof(a) / sizeof(a[0])` — это размер всего массива в байтах, делённый на размер одного элемента, то есть **число элементов**. Приём работает только пока `a` — настоящий массив, а не указатель (см. ниже). Цикл стартует с `i = 1`, потому что нулевой элемент уже лежит в `max` и `min`.

Что спросят дальше.

- «Почему инициализируешь первым элементом, а не нулём?» Если поставить `max = 0`, а все числа в массиве отрицательные — максимумом ошибочно останется 0, которого в массиве нет. То же с `min` и положительными. Безопасно стартовать значением **из самого массива**.
- «А если передать массив в функцию и там взять `sizeof`?» Не сработает! В функции параметр-массив превращается в **указатель**, и `sizeof` вернёт размер указателя (8 байт), а не массива. Поэтому длину всегда передают **отдельным параметром**: `void f(int *a, int n)`. Это классический подвох (см. главу 3 про указатели и каверзу про `sizeof` ниже).

Задача 4. Развернуть строку на месте

Условие. Дана строка. Переверни её **на месте** — без создания второго массива.

Идея. Два указателя-индекса: один на начало (`left`), другой на конец (`right`). Меняем символы местами и идём навстречу, пока они не встретятся посередине.

Решение (`code/19-02-reverse-string.c`):

```c
#include <stdio.h>
#include <string.h>

/* Переворачивает строку на месте, без второго массива. */
void reverse(char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        char tmp = s[left];
        s[left] = s[right];
        s[right] = tmp;
        left++;
        right--;
    }
}

int main(void)
{
    char text[] = "Hello, world";
    reverse(text);
    printf("%s\n", text);
    return 0;
}
```

Запуск:

```
dlrow ,olleH
```

Разбор. `strlen(s)` — длина строки без учёта завершающего нуля `'\0'` (см. главу 10). Последний символ имеет индекс `strlen(s) - 1`. Внутри цикла — обычный обмен через `tmp` (как в задаче 1). Условие `left < right` останавливает нас на середине: средний символ при нечётной длине трогать не нужно.

Что спросят дальше.

- «Почему `char text[] = "..."`, а не `char *text = "..."`?» Строковый литерал через указатель (`char *`) — это, как правило, **read-only** память; попытка её менять — неопределённое поведение и падение. Массив `char text[]` — это твоя изменяемая копия. Менять можно только её. Это важная деталь (см. каверзу про литералы ниже).

- «Сложность?» O(n) по времени, O(1) по дополнительной памяти — мы не выделяли новый буфер, переставляли на месте.

Задача 5. Проверка палиндрома

Условие. Палиндром читается одинаково слева направо и справа налево (`level`, `radar`). Проверь, палиндром ли данная строка.

Идея. Та же пара указателей с краёв, но теперь не меняем символы, а **сравниваем**. Нашли несовпадение — сразу не палиндром.

Решение (`code/19-03-palindrome.c`):

```c
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если строка — палиндром, иначе 0. */
int is_palindrome(const char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        if (s[left] != s[right]) {
            return 0;   /* нашли несовпадение — это не палиндром */
        }
        left++;
        right--;
    }
    return 1;
}

int main(void)
{
    const char *words[] = {"level", "radar", "hello", "noon"};
    for (int i = 0; i < 4; i++) {
        if (is_palindrome(words[i])) {
            printf("%s — палиндром\n", words[i]);
        } else {
            printf("%s — нет\n", words[i]);
        }
```

```
    }
    return 0;
}
```

Запуск:

```
level — палиндром
radar — палиндром
hello — нет
noon — палиндром
```

Разбор. Параметр `const char *s` — обещание «эту строку я менять не буду», только читаю. Поэтому функция спокойно принимает строковые литералы. Выход `return 0` при первом же несовпадении — это **ранний выход**: нет смысла проверять дальше, ответ уже ясен.

Что спросят дальше.

- «Сделай регистронезависимо: `Level` тоже палиндром.» Сравнивай не `s[left]` и `s[right]`, а их версии в одном регистре (приём приведения к нижнему регистру — в задаче 8 про гласные).
- «А если игнорировать пробелы и знаки: `A man a plan a canal Panama`?» Тогда указатели нужно «перепрыгивать» через не-буквы. Часто это и есть настоящая задача за вывеской простого палиндрома.

Задача 6. Сколько цифр в числе

Условие. Дано целое число. Посчитай, сколько в нём цифр (123 → 3).

Идея. Делим число на 10 нацело, пока оно не станет нулём, и считаем число шагов (см. главу 4 — «откусывание» цифр).

Решение. Подсчёт цифр объединён с суммой цифр в одном файле `code/19-08-digits.c` — см. задачу 7. Ядро подсчёта:

```
int count = 0;
if (n == 0) {
    count = 1;   /* у нуля одна цифра */
}
while (n > 0) {
    count++;
    n /= 10;     /* то же, что n = n / 10 */
}
```

Разбор. Каждое деление на 10 убирает одну цифру справа, и так до нуля. Отдельный случай — само число **0**: цикл `while (n > 0)` для него не выполнится ни разу, а ответ должен быть 1 (одна цифра). Поэтому ноль обрабатываем вручную.

Что спросят дальше. «А отрицательные?» У −123 три цифры, минус не считается. Возьми модуль в начале: `if (n < 0) n = -n;`. И сразу подвох: `-n` для самого маленького `int` (`INT_MIN`) переполнится — у него нет положительной пары. На Junior это редко спрашивают, но упомянуть — большой плюс.

Задача 7. Сумма цифр числа

Условие. Посчитай сумму цифр числа (456 → 4+5+6 = 15).

Идея. То же «откусывание», но вместо счётчика копим сумму остатков `n % 10`.

Решение (`code/19-08-digits.c` — считает и количество, и сумму):

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        n = -n;   /* знак на количество и сумму цифр не влияет */
    }

    int count = 0;   /* сколько цифр */
    int sum = 0;     /* сумма цифр */

    if (n == 0) {
        count = 1;   /* у нуля одна цифра */
    }
    while (n > 0) {
        int digit = n % 10;
        sum += digit;
```

```
        count++;
        n /= 10;
    }

    printf("Цифр: %d, сумма цифр: %d\n", count, sum);
    return 0;
}
```

Запуск:

```
Введи целое число: 90834
Цифр: 5, сумма цифр: 24
```

Разбор. `n % 10` — последняя цифра, `n /= 10` — отбрасываем её. `digit` вычисляем **один раз** и используем для суммы, чтобы не повторять `n % 10` (см. главу 4, шаг 3). `count` и `sum` стартуют с 0: сумма копится с нуля, как и положено накопителю.

Что спросят дальше. «Цифровой корень: складывай сумму цифр, пока не останется одна цифра (9875 → 29 → 11 → 2).» Это сумма цифр в цикле поверх цикла — частое продолжение.

Задача 8. Подсчёт гласных в строке

Условие. Посчитай, сколько гласных букв в строке (английские a, e, i, o, u в любом регистре).

Идея. Идём по строке посимвольно до `'\0'`. Для каждого символа проверяем, не гласная ли он. Чтобы не плодить проверки для верхнего и нижнего регистра, приводим символ к нижнему.

Решение (`code/19-06-vowels.c`):

```
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если c — английская гласная (любого регистра). */
int is_vowel(char c)
{
    char lower = c;
    if (c >= 'A' && c <= 'Z') {
        lower = (char)(c - 'A' + 'a');   /* привели к нижнему регистру */
    }
    return lower == 'a' || lower == 'e' || lower == 'i'
        || lower == 'o' || lower == 'u';
}
```

```c
int main(void)
{
    const char *s = "Programming In C";
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (is_vowel(s[i])) {
            count++;
        }
    }
    printf("В строке \"%s\" гласных: %d\n", s, count);
    return 0;
}
```

Запуск:

```
В строке "Programming In C" гласных: 4
```

Разбор. Трюк `c - 'A' + 'a'` переводит заглавную букву в строчную. Символы в C — это коды (см. главу 2): буквы `A..Z` идут подряд, и `a..z` тоже. Разница между заглавной и строчной одной буквы постоянна (`'a' - 'A'`), поэтому сдвиг работает. Условие цикла `s[i] != '\0'` — канонический способ пройти строку до завершающего нуля.

Что спросят дальше.

- «А `<ctype.h>`?» В стандартной библиотеке есть `tolower()` — он делает ровно наш сдвиг и работает с любой локалью. На собесе ценят, когда ты знаешь про готовое, но умеешь и руками.
- «Учитывай ‘y’ / русские гласные» — расширение списка, проверка внимательности.

Задача 9. Факториал итеративно

Условие. Посчитай факториал числа `n` циклом (без рекурсии). `n! = 1·2·…·n`, при этом `0! = 1`.

Идея. Накопитель `fact = 1`, домножаем на 2, 3, …, n. Стартовое значение — **единица**, а не ноль (умножение на 0 всё обнулит). И сразу помним про переполнение из главы 4.

Решение (`code/19-04-factorial.c`):

```c
#include <stdio.h>

int main(void)
{
```

```
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }
    if (n > 20) {
        printf("Слишком большое: long long переполнится при n > 20.\n");
        return 1;
    }

    long long fact = 1;   /* 0! = 1 и 1! = 1 — стартуем с единицы */
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %lld\n", n, fact);
    return 0;
}
```

Запуск:

```
Факториал какого числа? 13
13! = 6227020800
```

Разбор. Тип накопителя — `long long` (8 байт), потому что `int` лопается уже на 13! (см. главу 4). Печатаем через `%lld`. Цикл идёт с 2 (умножать на 1 бессмысленно); для `n = 0` и `n = 1` он не выполнится, и `fact` останется 1 — верный ответ.

Что спросят дальше.

- «До какого `n` это корректно?» До 20: 20! ≈ 2.4·10^18 ещё помещается в `long long`, а 21! — уже нет. Поэтому мы заранее отсекаем `n > 20`. Грамотный кандидат **сам** проговаривает границу типа.
- «А рекурсией?» Можно (см. главу 9), но итеративно — без риска переполнить стек вызовов на больших `n` и обычно быстрее.

Задача 10. FizzBuzz

Условие. Числа от 1 до 100. Кратные 3 заменяй на `Fizz`, кратные 5 — на `Buzz`, кратные и 3, и 5 — на `FizzBuzz`, остальные печатай как есть.

Идея. Вся соль — в **порядке** проверок. Самое строгое условие (кратно 15) должно идти первым, иначе число 15 поймается веткой «кратно 3» и выдаст `Fizz` вместо `FizzBuzz` (полный разбор — в главе 4).

Решение (готовый файл — `code/04-fizzbuzz.c` из главы 4):

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 100; i++) {
        if (i % 15 == 0) {
            printf("FizzBuzz\n");
        } else if (i % 3 == 0) {
            printf("Fizz\n");
        } else if (i % 5 == 0) {
            printf("Buzz\n");
        } else {
            printf("%d\n", i);
        }
    }
    return 0;
}
```

Разбор. В цепочке `if / else if` срабатывает **первая** подходящая ветка. Поэтому «кратно 15» (то же, что «кратно и 3, и 5») ставим раньше «кратно 3» и «кратно 5».

Что спросят дальше. «Сделай без `% 15`, только через `% 3` и `% 5`.» Собери строку из флагов:

```c
int by3 = (i % 3 == 0);
int by5 = (i % 5 == 0);
if (by3) printf("Fizz");
if (by5) printf("Buzz");
if (!by3 && !by5) printf("%d", i);
printf("\n");
```

Этот вариант легко расширяется (добавить «кратно 7 → Bazz»), и интервьюер часто просит именно его — проверить, что ты не зубрил, а понимаешь структуру.

Задача 11. Второй по величине элемент массива

Условие. Найди второй по величине элемент массива за **один** проход (не сортируя).

Идея. Держим две переменные: `max` (наибольший) и `second` (второй). Для каждого числа: если оно больше `max` — старый `max` становится `second`, а само число — новым `max`. Иначе, если оно меньше `max`, но больше `second` — обновляем `second`. Дубликаты максимума вторым считать не должны.

Решение (`code/19-05-secondmax.c`):

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int a[] = {3, 9, 9, 1, 7, 9, 4};
    int n = (int)(sizeof(a) / sizeof(a[0]));

    int max = INT_MIN;     /* самое большое */
    int second = INT_MIN;  /* второе по величине */

    for (int i = 0; i < n; i++) {
        if (a[i] > max) {
            second = max;  /* бывший максимум становится вторым */
            max = a[i];
        } else if (a[i] > second && a[i] < max) {
            second = a[i]; /* не максимум, но больше второго */
        }
    }

    if (second == INT_MIN) {
        printf("Второго по величине нет.\n");
    } else {
        printf("Максимум: %d, второй: %d\n", max, second);
    }
    return 0;
}
```

Запуск:

```
Максимум: 9, второй: 7
```

Разбор. `INT_MIN` из `<limits.h>` — самое маленькое значение `int`. Стартуем с него, чтобы любой реальный элемент оказался больше. Массив `{3, 9, 9, 1, 7, 9, 4}` содержит максимум 9 трижды, но вторым выводится 7, а не 9 — условие `a[i] < max` отсекает дубликаты максимума. Это именно тот случай, который любят подсунуть.

Что спросят дальше.

- «Что если в массиве все элементы одинаковы (`{5,5,5}`)?» Тогда второго по величине нет — наш код напечатает «Второго нет», потому что `second` так и остался `INT_MIN`. Уточни у интервьюера, считать ли дубликат вторым: трактовки бывают разные, и важно проговорить выбор.
- «А k-й по величине?» Обобщение — там уже нужна частичная сортировка или куча (heap). Это уже уровень Middle.

Задача 12. Проверка на простое число

Условие. Дано число `n`. Определи, простое ли оно (делится только на 1 и на себя; 0, 1 и отрицательные — не простые).

Идея. Перебираем возможные делители от 2. Нашли хоть один — число составное. Ключевая оптимизация: проверять делители только **до корня** из `n`, потому что делители идут парами (`n = d · (n/d)`), и меньший из пары всегда не больше √n.

Решение (`code/19-07-prime.c`):

```c
#include <stdio.h>

/* Возвращает 1, если n — простое число, иначе 0. */
int is_prime(int n)
{
    if (n < 2) {
        return 0;             /* 0, 1 и отрицательные — не простые */
    }
    for (int d = 2; d * d <= n; d++) {
        if (n % d == 0) {
            return 0;         /* нашли делитель — составное */
        }
    }
    return 1;
```

```c
}

int main(void)
{
    int n;
    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%d — простое\n", n);
    } else {
        printf("%d — не простое\n", n);
    }
    return 0;
}
```

Запуск:

```
Введи число: 17
17 — простое
```

Разбор. Условие цикла `d * d <= n` вместо `d <= n` — это и есть «до корня»: вместо `sqrt()` сравниваем квадрат делителя с числом, без дробной арифметики. Для n = 1 000 000 это ~1000 проверок вместо миллиона. Случаи `n < 2` отсекаем сразу — иначе единица ошибочно сошла бы за простую.

Что спросят дальше.

- «Почему `d * d <= n`, а не `d <= sqrt(n)`?» Чтобы не звать `sqrt` (дробная функция, возможны погрешности на границе) и не подключать `<math.h>`. Целочисленное сравнение точнее и быстрее. Осторожно: для очень больших `n` `d * d` само может переполниться — тогда берут `long long` или сравнивают `d <= n / d`.
- «Ускорь ещё.» Проверять 2 отдельно, а дальше только нечётные делители (`d += 2`) — вдвое меньше итераций.
- «Все простые до N?» Это уже **решето Эратосфена** (см. главу 5) — частый следующий шаг.

Часть 2. Каверзы: «что выведет / в чём ошибка»

Здесь короткие фрагменты. На собесе их показывают и спрашивают: «что напечатает?» или «найди ошибку». Цель — не код написать, а проверить, насколько хорошо ты знаешь язык в деталях. Сначала подумай сам, потом читай ответ.

Каверза 1. `sizeof('a')` — сколько это?

```c
printf("%zu %zu\n", sizeof(int), sizeof('a'));
```

Вопрос. Что выведет? Подсказка-ловушка: `'a'` — это же один символ, значит 1 байт?

Ответ. Выведет `4 4` (на типичной системе). Сюрприз в том, что в языке **C** символьный литерал `'a'` имеет тип не `char`, а **`int`**! Поэтому `sizeof('a')` равен `sizeof(int)`, то есть 4, а не 1.

Важно: это особенность именно C. В C++ `'a'` — это `char`, и там `sizeof('a')` равно 1. На собесе по C правильный ответ — 4.

А вот `sizeof(char)` по стандарту **всегда** равен 1 — это и есть тот самый «единица измерения» для `sizeof`.

`%zu` — спецификатор для типа `size_t`, который возвращает `sizeof` (беззнаковый размер). Печатать его через `%d` формально неверно.

Каверза 2. Целочисленное деление `5 / 2`

```c
printf("%d\n", 5 / 2);
double x = 5 / 2;
printf("%f\n", x);
```

Вопрос. Что в обеих строках?

Ответ. Первая печатает `2` — деление двух **целых** идёт нацело, дробная часть отбрасывается (см. главу 2), не округляется. `5 / 2` это 2, а не 2.5 и не 3.

Вторая печатает `2.000000`, а **не** 2.5, и это главный подвох. Хотя `x` объявлен как `double`, выражение `5 / 2` считается **до** записи в `x`: оба операнда целые → целочисленное деление → 2 → и уже 2 превращается в `2.0`. Тип переменной слева не влияет на то, как вычисляется правая часть.

Чтобы получить 2.5, дробным должен быть хотя бы один операнд: `5.0 / 2`, или `(double)5 / 2`, или `5 / 2.0`.

Каверза 3. `printf("%d", c)` для `char`

```
char c = 'A';
printf("%d\n", c);
```

Вопрос. Что выведет?

Ответ. Выведет `65`. Символ в C — это маленькое целое, его **код**. Код буквы `A` в таблице ASCII равен 65. Спецификатор `%d` просит напечатать число, поэтому печатается не сама буква, а её код. Если хочешь увидеть букву `A` — используй `%c`.

И наоборот полезно помнить: `printf("%c\n", 66)` напечатает `B` — число истолкуется как код символа. На этой двойственности (символ = число) держатся трюки вроде перевода регистра из задачи 8.

Каверза 4. `=` вместо `==` в условии

```
int n = 5;
if (n = 0) {
    printf("ноль\n");
} else {
    printf("не ноль\n");
}
```

Вопрос. Что выведет?

Ответ. Выведет `не ноль`, и при этом **испортит** `n`. Здесь стоит один `=` (присваивание), а не `==` (сравнение). Поэтому `n = 0` сначала **кладёт** 0 в `n`, а потом результатом всего выражения становится само присвоенное значение — 0. Ноль в условии — это «ложь», поэтому срабатывает ветка `else`. Заодно `n` теперь равно 0, хотя мы вроде бы только «проверяли».

Если бы было `if (n = 5)`, условие стало бы истинным (5 — ненулевое), и такие баги особенно коварны: код «как будто работает». Спасает компилятор с `-Wall` — он предупреждает о присваивании внутри `if`. Защитный приём — «йода-условие»: писать `if (0 == n)`; тогда опечатку `0 = n` компилятор отвергнет как ошибку (нельзя присвоить константе).

Каверза 5. Лишняя `;` после `for(...)`

```c
int sum = 0;
for (int i = 1; i <= 5; i++);
{
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Найди ошибку. Что не так?

Ответ. Лишняя точка с запятой сразу после `for(...)`. Эта `;` — **пустое тело цикла**. То есть цикл крутится 5 раз и ничего не делает. А блок в фигурных скобках ниже — это уже **обычный блок**, не связанный с циклом; он выполнится **один раз**.

Хуже того: этот фрагмент вообще не скомпилируется — `i` объявлена внутри `for` и за его пределами не видна, так что `sum += i;` обратится к неизвестному имени. Но даже если бы `i` была видна, логика была бы сломана. Мораль: точка с запятой сразу после `for`, `while` или `if` — почти всегда ошибка. Компилятор с `-Wall` предупреждает об «empty body».

Каверза 6. `printf("%d %d", i++, ++i)`

```c
int i = 1;
printf("%d %d\n", i++, ++i);
```

Вопрос. Что выведет?

Ответ. Честный ответ: **неизвестно**. Это **неопределённое поведение** (undefined behavior, UB). В одном выражении переменная `i` и читается, и дважды меняется (`i++` и `++i`) без гарантированного порядка между этими изменениями. Стандарт C не определяет, в каком порядке вычисляются аргументы `printf` и когда применяются `++`. Разные компиляторы выдадут разные числа, и **полагаться нельзя ни на одно из них**.

Если на собесе спрашивают «что выведет» — правильный ответ не «2 3» и не «1 3», а именно: «это UB, результат не определён, так писать нельзя». Интервьюер проверяет, узнаёшь ли ты опасную конструкцию, а не считаешь ли в уме «как у меня на ноутбуке».

Лечится разнесением по строкам:

```
int a = i++;   /* теперь порядок однозначен */
int b = ++i;
printf("%d %d\n", a, b);
```

Каверза 7. Неинициализированная переменная

```
int sum;
for (int i = 1; i <= 5; i++) {
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Что выведет?

Ответ. Заранее **неизвестно**. `sum` объявлена, но не инициализирована — в локальной переменной лежит **мусор** (что осталось в этой ячейке памяти). К мусору прибавляется 1+2+3+4+5 = 15, и печатается «мусор + 15». Может вывестись и 15 (если повезло и там был 0), и любое другое число, и значение может меняться от запуска к запуску.

Чтение неинициализированной локальной переменной — тоже неопределённое поведение. Фикс — один символ: `int sum = 0;`. Любой накопитель перед циклом обнуляй (сумма — в 0, произведение — в 1). Компилятор с `-Wall` часто предупреждает «may be used uninitialized» — не игнорируй.

Каверза 8. Остаток от отрицательного числа

```
printf("%d %d\n", -7 / 2, -7 % 2);
```

Вопрос. Что выведет?

Ответ. Выведет `-3 -1`. В современном C (стандарт C99 и новее) деление округляется **к нулю**: `-7 / 2` это −3 (а не −4). А остаток подчиняется правилу `(a / b) * b + a % b == a`, поэтому `-7 % 2` равен −1, а не +1.

Практический вывод: проверять нечётность через `n % 2 == 1` **опасно** — для отрицательных нечётных получится −1, и проверка провалится. Пиши `n % 2 != 0`. Это прямая связь с задачей 2 про чётность.

Мини-резюме

Что стоит унести из этого приложения — не сами решения (их в интернете тысячи), а привычки, которые отличают подготовленного кандидата:

- **Краевые случаи проговаривай вслух**: пустой массив, один элемент, ноль, отрицательное, дубликаты, переполнение. Интервьюер часто ставит плюс именно за то, что ты сам про них вспомнил.
- **Помни про типы и переполнение.** `int` лопается на ~2 млрд; символ — это `int` в C; деление целых идёт нацело. Эти три факта спрашивают чаще всего.
- **Знай про опасные приёмы, но не используй их без нужды.** XOR-своп, арифметический обмен — рассказать можешь, в код кладёшь `temp`.
- **UB называй своим именем.** Увидел `i++` и `++i` в одном выражении, неинициализированную переменную, изменение строкового литерала — отвечай «это неопределённое поведение», а не угадывай результат.
- **Компилируй с `-Wall -Wextra`.** Половину каверз из второй части компилятор ловит сам.

Все рабочие решения собираются и запускаются:

```
cc -std=c11 -Wall -Wextra -O2 code/19-07-prime.c -o prime
./prime
```

Куда дальше

Это был уровень Junior — задачи, где главное показать, что основы языка сидят крепко, а голова замечает краевые случаи. В **Приложении Б (Middle)** мы поднимем планку: связные списки и их развороты, работа с указателями всерьёз, своя реализация `qsort`-подобной сортировки и функций сравнения, хеш-таблицы, аккуратная работа с памятью под присмотром санитайзеров, и каверзы потяжелее — про указатели, выравнивание и время жизни объектов. Если задачи отсюда даются легко и без подглядывания — ты готов двигаться дальше.

Приложение Б. Задачи с собеседований: Middle

В Приложении А мы размялись на простых задачах уровня Junior. Теперь — ступень выше. Это вопросы, которые задают на собеседовании, когда хотят проверить, что ты понимаешь Си **по-настоящему**, а не просто умеешь печатать `printf`. Здесь всё крутится вокруг четырёх тем, на которых ломается большинство:

- **указатели** — кто на что смотрит и кто кого меняет;
- **память** — где живёт строка, когда `malloc`, когда `free`;
- **строки** — массив `char` с `'\0'` на конце (см. главу 3);
- **биты** — числа как набор нулей и единиц;
- **связные списки** — узлы и стрелки `next` (см. главу 11).

Как пользоваться этим приложением

Не читай решение сразу. Прочитай условие, закрой книгу и попробуй сам — хоть на бумаге, хоть в редакторе. На собеседовании ценят не зазубренный код, а ход мысли. Поэтому у каждой задачи есть блок «идея» (как рассуждать) и «доп. вопросы» — то, что интервьюер спросит следом, чтобы понять, не списал ли ты.

Задачи идут от простого к сложному. Сначала — **задачи на код** (их решения лежат отдельными файлами в `code/`, каждое собирается без предупреждений командой `cc -std=c11 -Wall -Wextra -O2`). Потом — **каверзы**: коварные вопросы про язык, где правильный ответ не «что выведет», а «почему так, и можно ли на это полагаться».

Поехали.

Часть 1. Задачи на код

Задача 1. Своя `strlen`, `strcpy`, `strcmp`

Условие. Реализуй три функции стандартной библиотеки `<string.h>` своими руками, без её вызова: длину строки, копирование, сравнение.

Идея. Строка в Си — это массив `char`, заканчивающийся нулевым символом `'\0'` (см. главу 3). Значит, любой обход строки — это «иди по символам, пока не упрёшься в `'\0'`». На этом держатся все три функции.

Решение (`code/20-01-mystr.c`):

```c
#include <stdio.h>

/* Своя длина строки: считаем символы до '\0'. */
size_t my_strlen(const char *s)
{
    size_t n = 0;
    while (s[n] != '\0') {  /* пока не дошли до конца строки */
        n++;
    }
    return n;
}

/* Своё копирование: льём src в dst посимвольно, вместе с '\0'. */
char *my_strcpy(char *dst, const char *src)
{
    char *start = dst;          /* запомнили начало, чтобы вернуть */
    while ((*dst = *src) != '\0') {
        dst++;
        src++;
    }
    return start;
}

/* Своё сравнение: 0 если равны, иначе разница первых отличий. */
int my_strcmp(const char *a, const char *b)
{
    while (*a != '\0' && *a == *b) {
        a++;
```

```
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}
```

Объяснение.

- `size_t` — это специальный беззнаковый тип для размеров и длин. Именно его возвращает настоящая `strlen`. Печатают `size_t` через `%zu`.
- `const char *s` — указатель на строку, которую мы **только читаем**. Слово `const` обещает «я не буду менять то, на что смотрит `s`». Это хороший тон: компилятор подстрахует, и тому, кто вызывает функцию, видно, что аргумент в безопасности.
- В `my_strcpy` трюк в строке `while ((*dst = *src) != '\0')`. Сначала `*dst = *src` копирует один символ **и** возвращает его значение. Это значение тут же сравниваем с `'\0'`. Получается: скопировали символ; если это был конец строки — выходим (но `'\0'` уже скопирован, что нам и нужно). Скобки вокруг `*dst = *src` обязательны: без них `!=` сработает раньше `=`.
- В `my_strcmp` идём, пока символы совпадают и строка не кончилась. На первом различии (или на общем конце) выходим и возвращаем разность кодов символов. Если строки равны, оба `*a` и `*b` — это `'\0'`, разность 0. Приведение к `unsigned char` нужно, чтобы символы с кодом больше 127 не «ушли в минус» (на некоторых системах `char` знаковый) — стандартная `strcmp` сравнивает именно как `unsigned char`.

Запуск:

```
my_strlen("Hello") = 5
my_strcpy -> skopiruem
my_strcmp(abc, abc) = 0
my_strcmp(abc, abd) = -1
my_strcmp(abd, abc) = 1
```

Доп. вопросы интервьюера.

- *Чем опасна `strcpy`?* Она не знает размер приёмника `dst` и пишет, пока в `src` не кончатся символы. Если `dst` короче — затрёт чужую память (переполнение буфера). Безопаснее `strncpy` или `snprintf`.
- *Почему `strcmp` возвращает не просто 1/-1, а разность?* Стандарт гарантирует только знак результата (меньше/равно/больше нуля), не само число. Полагайся на знак, не на конкретную величину.

Задача 2. Развернуть односвязный список

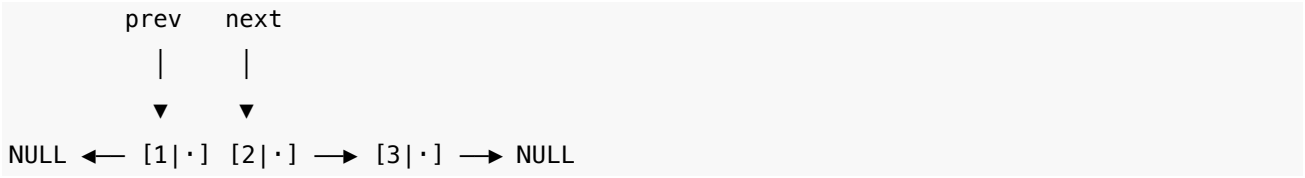
Условие. Дан односвязный список `1 -> 2 -> 3 -> ... -> NULL`. Разверни его, чтобы стало `... -> 3 -> 2 -> 1 -> NULL`. На месте, без выделения нового списка.

Идея. У каждого узла есть стрелка `next`, смотрящая вперёд. Развернуть список — значит у каждого узла **перенаправить стрелку назад**. Идём по списку и на каждом шаге разворачиваем одну стрелку. Но если просто написать `cur->next = prev`, мы потеряем доступ к хвосту — поэтому хвост надо запомнить **до** разворота. Нужны три указателя: `prev` (узел позади), `cur` (текущий), `next` (запасённый хвост).

Нарисуем. Было:



Запомнили `next = cur->next` (это `[2]`), развернули стрелку `cur` на `prev`:



Сдвинули `prev` и `cur` на шаг вперёд (`prev = cur; cur = next`) и повторяем. Когда `cur` станет `NULL`, `prev` будет указывать на бывший последний узел — это новая голова.

Решение (`code/20-02-reverse-list.c`, привожу ядро):

```c
struct Node {
    int value;
    struct Node *next;
};

/* Разворот списка через три указателя: prev, cur, next. */
struct Node *reverse(struct Node *head)
{
    struct Node *prev = NULL;       /* позади ничего нет */
    struct Node *cur = head;        /* бегунок по списку */
    while (cur != NULL) {
        struct Node *next = cur->next;  /* запомнили хвост */
        cur->next = prev;               /* развернули стрелку назад */
```

```
        prev = cur;                     /* prev сдвинулся вперёд */
        cur = next;                     /* cur сдвинулся вперёд */
    }
    return prev;                    /* prev — новая голова */
}
```

Объяснение. Порядок четырёх строк внутри цикла — священен.

1. `next = cur->next` — сохранили, куда идти дальше, пока стрелка цела.
2. `cur->next = prev` — развернули стрелку текущего узла назад.
3. `prev = cur` — `prev` догнал `cur`.
4. `cur = next` — `cur` шагнул на сохранённый хвост.

Поменяешь местами хоть две — либо потеряешь хвост, либо зациклишь узел сам на себя. Функция возвращает `prev`, потому что после цикла `cur` уже `NULL`, а `prev` стоит на последнем обработанном узле — он и есть новая голова. В `main` обязательно перезаписываем голову: `head = reverse(head)`.

Запуск:

```
До:    5 -> 4 -> 3 -> 2 -> 1 -> NULL
После: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
```

Доп. вопросы.

- *Сложность?* Один проход — O(n) по времени, O(1) по памяти (три указателя, новый список не выделяем).
- *Как развернуть рекурсивно?* Можно (см. главу 9 про рекурсию), но тогда расход памяти O(n) на стек вызовов, и на длинном списке возможно переполнение стека. Итеративная версия лучше.

Задача 3. Середина списка (быстрый и медленный указатель)

Условие. Найди средний узел односвязного списка за один проход, не зная заранее его длину.

Идея. Самый прямой способ — пройти список дважды: сначала посчитать длину, потом дойти до середины. Но можно за **один** проход трюком «два указателя разной скорости». Медленный `slow` шагает по одному узлу, быстрый `fast` — по два. Когда `fast` добежит до конца, `slow` будет ровно посередине: он прошёл вдвое меньше.

Решение (`code/20-03-floyd.c`, ядро):

```
/* Середина списка: медленный шагает на 1, быстрый на 2. */
struct Node *middle(struct Node *head)
```

```
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          /* +1 */
        fast = fast->next->next;    /* +2 */
    }
    return slow;
}
```

Объяснение. Главное здесь — условие цикла `fast != NULL && fast->next != NULL`. Быстрый прыгает через два узла, и перед прыжком надо убедиться, что **оба** узла существуют: и сам `fast`, и `fast->next`. Иначе `fast->next->next` разыменует `NULL` — падение. Порядок проверок важен: `&&` вычисляет левую часть первой, и если `fast` уже `NULL`, до `fast->next` дело не дойдёт (это называется ленивым, или укороченным, вычислением — см. главу про условия). Для списка из 7 узлов середина — четвёртый, значение 4.

Запуск (первая строка):

```
Середина списка из 7: 4
```

Доп. вопросы.

- *А если узлов чётное число?* Тогда `slow` остановится на «правом из двух центральных». Хочешь левый — начни `fast` с `head->next` или останавливайся, когда `fast->next->next == NULL`.
- *Где ещё нужен этот приём?* В обнаружении цикла — следующая задача.

Задача 4. Цикл в списке (черепаха и заяц, алгоритм Флойда)

Условие. Список может быть «закольцован»: последний узел указывает не на `NULL`, а на один из предыдущих. Обычный обход по такому списку зациклится навечно. Определи, есть ли в списке цикл.

Идея. Та же пара указателей. Заяц (`fast`) бежит вдвое быстрее черепахи (`slow`). Если список заканчивается `NULL` — заяц добежит до конца, цикла нет. Если есть кольцо — заяц никогда не выйдет, он начнёт бегать по кругу и рано или поздно **догонит** черепаху сзади. Как на круговой беговой дорожке: быстрый бегун обязательно настигнет медленного. Момент `slow == fast` и означает цикл.

Решение (`code/20-03-floyd.c`, ядро):

```c
/* Обнаружение цикла: черепаха и заяц (Флойд). 1 если цикл есть. */
int has_cycle(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {         /* заяц догнал черепаху */
            return 1;
        }
    }
    return 0;                       /* быстрый упёрся в NULL — цикла нет */
}
```

Объяснение. Сравниваем именно **указатели** (`slow == fast`), то есть адреса узлов, а не их значения: важно, что оба бегунка попали в один и тот же узел. Если цикла нет, `fast` выйдет на `NULL` и цикл `while` завершится. Почему заяц гарантированно догонит черепаху, а не «перепрыгнет»? Потому что за каждый шаг расстояние между ними внутри кольца сокращается ровно на 1, значит, оно непременно станет 0.

В программе мы сначала проверяем нормальный список (цикла нет), потом вручную закольцовываем его (последний узел направляем на третий) и проверяем снова:

```
Цикл есть? 0
После закольцовки цикл есть? 1
```

Важная деталь из кода: перед `free` цикл надо **разорвать** (`last->next = NULL`), иначе освобождение зациклится и мы дважды освободим одни и те же узлы.

Доп. вопросы.

- *Сложность?* O(n) по времени, O(1) по памяти. Альтернатива — хранить посещённые узлы в хеш-таблице: тоже O(n) времени, но O(n) памяти.
- *Как найти, с какого узла начинается цикл?* Это вторая часть алгоритма Флойда: после встречи поставить один указатель в голову и двигать оба по одному шагу — встретятся как раз в начале кольца.

Задача 5. Подсчёт единичных битов (popcount)

Условие. Дано целое число. Посчитай, сколько единиц в его двоичной записи. Например, 7 это `111` — три единицы; 8 это `1000` — одна.

Идея. Наивно: смотрим младший бит (`n & 1`), прибавляем его к счётчику, сдвигаем число вправо (`n >> 1`), повторяем, пока число не обнулится. Это делает столько шагов, сколько всего битов в числе (до 32). Красивее — **приём Брайана Кернигана**: операция `n &= n - 1` за один раз **гасит самый младший единичный бит**. Тогда число итераций равно числу единиц, а не общему числу битов.

Почему `n & (n - 1)` убирает младшую единицу? Вычитание единицы переворачивает младший единичный бит в 0, а все нули правее него — в единицы. Например, `n = 1100`, `n - 1 = 1011`. Их `&` даёт `1000` — младшая единица исчезла, остальное цело.

Решение (`code/20-04-popcount.c`, ядро):

```c
/* Наивный popcount: проверяем младший бит и сдвигаем вправо. */
int popcount_naive(unsigned int n)
{
    int count = 0;
    while (n != 0) {
        count += n & 1u;    /* младший бит: 0 или 1 */
        n >>= 1;            /* сдвинули число на один бит вправо */
    }
    return count;
}

/* Приём Брайана Кернигана: n &= n - 1 гасит младший единичный бит. */
int popcount_kernighan(unsigned int n)
{
    int count = 0;
    while (n != 0) {
        n &= n - 1;         /* убрали одну единицу */
        count++;            /* итераций ровно столько, сколько единиц */
    }
    return count;
}
```

Объяснение.

- Аргумент — `unsigned int` (беззнаковое). Битовые операции и сдвиги делают **только** на беззнаковых типах. Сдвиг вправо отрицательного знакового числа ведёт себя зависимо от компилятора — этого избегают.
- `n & 1u` — побитовое И с единицей оставляет только младший бит: результат 0 или 1. Суффикс `u` у литерала `1u` подчёркивает, что работаем с беззнаковым.

- `n >>= 1` — то же, что `n = n >> 1`: сдвиг всех битов на одну позицию вправо, как деление на 2.
- `n &= n - 1` — то же, что `n = n & (n - 1)`: гасим младшую единицу.

Запуск:

```
    0: бит=0 (Кернига=0), степень_двойки=0
    1: бит=1 (Кернига=1), степень_двойки=1
    2: бит=1 (Кернига=1), степень_двойки=1
    7: бит=3 (Кернига=3), степень_двойки=0
    8: бит=1 (Кернига=1), степень_двойки=1
  255: бит=8 (Кернига=8), степень_двойки=0
 1024: бит=1 (Кернига=1), степень_двойки=1
```

Обе функции дают одно число, но версия Кернигана для числа `1024` (это `10000000000`) сделает **один** виток цикла, а наивная — одиннадцать.

Доп. вопросы.

- *Есть ли это в железе?* Да, у многих процессоров есть инструкция popcount; в GCC/Clang к ней даёт доступ `__builtin_popcount`.
- *Как посчитать единицы в `unsigned long long`?* Тот же код, тип шире; следи, чтобы литералы и тип счётчика были достаточны.

Задача 6. Степень ли двойки

Условие. Проверь, является ли положительное число степенью двойки (1, 2, 4, 8, 16, …).

Идея. У степени двойки в двоичной записи **ровно один** единичный бит: `1`, `10`, `100`, `1000`. А мы только что видели, что `n & (n - 1)` гасит младшую единицу. Если единица была одна-единственная — после гашения останется 0. Значит, проверка: `n != 0 && (n & (n - 1)) == 0`.

Решение (`code/20-04-popcount.c`, ядро):

```c
/* Степень двойки: ровно один единичный бит, и число положительное. */
int is_power_of_two(unsigned int n)
{
    return n != 0 && (n & (n - 1)) == 0;
}
```

Объяснение. Условие `n != 0` обязательно. Без него ноль прошёл бы проверку: `0 & (0 - 1)` для беззнакового — это `0 & 0xFFFFFFFF`, что равно 0, и `is_power_of_two(0)` ошибочно вернуло

бы «истина». Ноль не степень двойки, поэтому отсекаем его явно. Результат — это значение логического выражения: 1 (истина) или 0 (ложь).

Доп. вопросы.

- *Почему не считать popcount и не сравнивать с 1?* Можно, но это лишний цикл. Трюк с `n & (n-1)` — за одну операцию, без ветвлений и циклов.
- *Как округлить до ближайшей степени двойки вверх?* Серия сдвигов и `|`, либо инструкция подсчёта ведущих нулей (`__builtin_clz`). Это уже уровень повыше.

Задача 7. Удалить дубликаты из отсортированного массива на месте

Условие. Дан отсортированный массив, например `[1, 1, 2, 2, 2, 3, 5, 5, 8]`. Убери повторы **на месте**, не выделяя второй массив. Верни новую длину; уникальные элементы должны оказаться в начале.

Идея. Раз массив отсортирован, все одинаковые элементы лежат рядом. Заведём два индекса (приём «два указателя» в одном массиве): `r` (reader) бежит по всем элементам, `w` (writer) показывает, куда положить следующий уникальный. Если `a[r]` отличается от последнего записанного `a[w-1]` — это новый уникальный, копируем его в `a[w]` и двигаем `w`.

Решение (`code/20-07-dedup.c`, ядро):

```c
/* Удаление дубликатов из ОТСОРТИРОВАННОГО массива на месте.
   Возвращает новую длину; уникальные элементы — в начале a. */
int dedup_sorted(int *a, int n)
{
    if (n == 0) {
        return 0;
    }
    int w = 1;                  /* w — куда писать следующий уникальный */
    for (int r = 1; r < n; r++) {   /* r — что читаем */
        if (a[r] != a[w - 1]) {     /* отличается от последнего записанного */
            a[w] = a[r];
            w++;
        }
    }
    return w;
}
```

Объяснение. Первый элемент `a[0]` уникален всегда, поэтому стартуем с `w = 1` и `r = 1`. Сравниваем именно с `a[w - 1]` — последним, что мы уже оставили, а не с соседом `a[r - 1]`:

так корректно пропускаем длинные серии повторов. Проверка `n == 0` нужна, иначе `a[w - 1]` при пустом массиве полез бы в `a[-1]` — выход за границу.

Запуск:

```
Уникальных: 5
1 2 3 5 8
```

Доп. вопросы.

- *Почему важно, что массив отсортирован?* Только тогда дубликаты рядом и хватает сравнения с предыдущим уникальным. Для неотсортированного понадобится множество (хеш) — O(n) памяти.
- *Что в хвосте массива после удаления?* Старый мусор за позицией `w`. Мы его не трогаем — вызывающий пользуется только первыми `w` элементами.

Задача 8. Число, встречающееся нечётное число раз (XOR)

Условие. В массиве все числа встречаются **парами**, и только одно — нечётное число раз. Найди его. Желательно за один проход и без лишней памяти.

Идея. Спасает операция XOR (исключающее ИЛИ, `^`). У неё два золотых свойства: `x ^ x == 0` (число, сложенное само с собой, гасится) и `x ^ 0 == x` (ноль ничего не меняет). Проксорим **все** элементы подряд. Парные погасят друг друга и дадут 0, а одиночка останется — это и есть ответ. Порядок неважен, XOR коммутативен.

Решение (`code/20-08-oddtimes.c`, ядро):

```c
/* XOR всех элементов: парные гасят друг друга, остаётся одиночка. */
int find_odd(const int *a, int n)
{
    int x = 0;
    for (int i = 0; i < n; i++) {
        x ^= a[i];      /* a ^ a == 0, x ^ 0 == x */
    }
    return x;
}
```

Объяснение. `x ^= a[i]` — то же, что `x = x ^ a[i]`. Стартуем с `x = 0`, потому что 0 — нейтральный для XOR. После прохода все, кто шёл парами, взаимно уничтожились, в `x` лежит число с непарным количеством. Память — одна переменная, время — один проход.

Запуск (4 встречается 3 раза):

```
Нечётное число раз встречается: 4
```

Доп. вопросы.

- *А если два числа непарные?* Тогда общий XOR даст их «смесь». Их можно разделить, выбрав бит, в котором они различаются, и разбив массив на две группы — это классическое усложнение задачи.
- *Где XOR ещё полезен?* Обмен двух чисел без временной переменной, простое шифрование, контрольные суммы.

Задача 9. Корректный бинарный поиск

Условие. Реализуй бинарный поиск в отсортированном массиве. Главная ловушка — ошибки «на единицу» (off-by-one): неверная граница цикла или неверный сдвиг `lo`/`hi` ведут к зацикливанию или пропуску элемента.

Идея. Держим отрезок `[lo, hi]`, где может находиться искомое. Смотрим середину. Угадали — готово. Если в середине меньше искомого — всё левее бесполезно, двигаем `lo`. Если больше — двигаем `hi`. Отрезок сжимается, пока не найдём или пока он не опустеет.

Решение (`code/20-05-binsearch.c`, ядро):

```c
/* Бинарный поиск в отсортированном массиве.
   Возвращает индекс target или -1, если не найдено. */
int binary_search(const int *a, int n, int target)
{
    int lo = 0;
    int hi = n - 1;             /* оба конца ВКЛЮЧИТЕЛЬНО */
    while (lo <= hi) {          /* пока отрезок непустой */
        int mid = lo + (hi - lo) / 2;   /* середина без переполнения */
        if (a[mid] == target) {
            return mid;
        } else if (a[mid] < target) {
            lo = mid + 1;       /* искомое правее */
        } else {
            hi = mid - 1;       /* искомое левее */
        }
    }
    return -1;
}
```

Объяснение. Три места, где обычно ошибаются, и как мы их обходим:

- **Границы включительные.** `hi = n - 1`, и цикл идёт, пока `lo <= hi` (не <). Когда `lo == hi`, отрезок ещё содержит один элемент — его нужно проверить. Напишешь `lo < hi` — пропустишь этот последний элемент.
- **Сдвиг с +1/–1.** После проверки середины сам `mid` уже исключён, поэтому новые границы — `mid + 1` и `mid - 1`, а не `mid`. Если сдвигать на `mid`, отрезок может перестать сжиматься и цикл зависнет.
- **Середина без переполнения.** Пишем `lo + (hi - lo) / 2`, а не `(lo + hi) / 2`. На больших индексах сумма `lo + hi` может переполнить `int`; разность `hi - lo` безопасна. Классический баг, который годами жил даже в стандартных библиотеках.

Запуск:

```
  5 найдено на индексе 4
 42 найдено на индексе 8
 -7 найдено на индексе 0
100 не найдено
  0 найдено на индексе 2
```

Доп. вопросы.

- *Сложность?* O(log n): каждый шаг вдвое урезает отрезок. Для миллиона элементов — около 20 шагов.
- *Что вернуть, если в массиве есть дубликаты искомого?* Базовая версия вернёт «какой-то» из них. Чтобы найти первый или последний, цикл меняют так, чтобы он не останавливался на совпадении, а продолжал сужать сторону.

Задача 10. Слияние двух отсортированных массивов

Условие. Даны два отсортированных массива. Слей их в один отсортированный. Это сердце сортировки слиянием (merge sort).

Идея. Идём по обоим массивам сразу, двумя индексами. Сравниваем текущие элементы и в результат кладём **меньший**, продвигая его индекс. Так — пока один из массивов не кончится. Остаток второго дописываем как есть: он уже отсортирован.

Решение (`code/20-06-merge.c`, ядро):

```c
/* Слияние двух отсортированных массивов a и b в результат out.
   out должен вмещать na + nb элементов. */
void merge(const int *a, int na, const int *b, int nb, int *out)
{
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) {       /* пока есть в обоих */
```

```
        if (a[i] <= b[j]) {
            out[k++] = a[i++];       /* меньший идёт первым */
        } else {
            out[k++] = b[j++];
        }
    }
    while (i < na) {                 /* хвост a, если остался */
        out[k++] = a[i++];
    }
    while (j < nb) {                 /* хвост b, если остался */
        out[k++] = b[j++];
    }
}
```

Объяснение. Запись `out[k++] = a[i++]` делает две вещи: кладёт `a[i]` в `out[k]`, а потом увеличивает оба индекса (постфиксный `++` сначала даёт старое значение, потом увеличивает — разбор `++` будет в каверзах ниже). Сравнение `a[i] <= b[j]` именно «меньше или равно» — это делает слияние **устойчивым** (равные элементы сохраняют исходный относительный порядок; для чисел незаметно, для записей с полями важно). Два хвостовых цикла дописывают то, что осталось от длинного массива.

Ответственность за размер `out` — на вызывающем: там должно быть место под `na + nb` элементов. В `main` массив `out[11]` именно такой.

Запуск:

```
1 2 3 4 5 6 7 8 9 10 12
```

Доп. вопросы.

- *Сложность?* O(na + nb) по времени — один проход по каждому. Память — под результат, O(na + nb).
- *Как слить «на месте» в конец первого массива (как в задачах про merge sorted array)?* Идти **с конца** обоих массивов, заполняя `out` справа налево — тогда не затрёшь ещё не прочитанные элементы.

Задача 11. Поменять местами два числа через указатели

Условие. Напиши функцию `swap`, которая меняет местами значения двух переменных. Так, чтобы изменения были видны **снаружи** функции.

Идея. В Си аргументы передаются по значению — функция получает копии (см. главу 3). Поменяв копии, оригиналы не тронешь. Чтобы дотянуться до самих переменных, передаём их **адреса** и работаем через указатели.

Решение (показываю целиком, это короткая классика):

```c
#include <stdio.h>

void swap(int *a, int *b)   /* принимаем адреса двух переменных */
{
    int t = *a;             /* запомнили значение по a */
    *a = *b;                /* в a положили значение по b */
    *b = t;                 /* в b положили запомненное */
}

int main(void)
{
    int x = 5, y = 9;
    swap(&x, &y);           /* передаём АДРЕСА x и y */
    printf("x=%d y=%d\n", x, y);   /* x=9 y=5 */
    return 0;
}
```

Объяснение. `int *a` — указатель: `a` хранит адрес чужой переменной. `*a` — значение по этому адресу, то есть сама переменная вызывающего. Меняя `*a` и `*b`, мы меняем `x` и `y` в `main`. При вызове пишем `&x` и `&y` — берём адреса. Именно поэтому `scanf` тоже хочет `&`: ему нужен адрес, чтобы записать в твою переменную.

Доп. вопросы.

- *Почему версия `void swap(int a, int b)` не работает?* Она меняет местами копии; снаружи всё остаётся как было.
- *Можно ли без временной переменной, через XOR?* Можно: `*a ^= *b; *b ^= *a; *a ^= *b;`. Но есть подвох: если `a` и `b` указывают на **одну** переменную, она обнулится. Версия с временной `t` надёжнее и читается лучше — на собеседовании предпочитают её.

Часть 2. Каверзы

Эти вопросы не про «реши задачу», а про «понимаешь ли ты язык». Любимая территория собеседующих: ответ короткий, но за ним стоит понимание памяти, указателей и неопределённого

поведения. Каждую каверзу разбираем, поэтому решений в `code/` для них нет — некоторые из них специально содержат ошибку, и компилировать их как «образец» нельзя.

Каверза 1. `char *s = "abc"` против `char s[] = "abc"`

```c
char *s1 = "abc";       /* указатель на строковый ЛИТЕРАЛ */
char  s2[] = "abc";     /* МАССИВ-копия, лежит у тебя */

s2[0] = 'A';            /* ок: меняем свой массив, выйдет "Abc" */
s1[0] = 'A';            /* НЕЛЬЗЯ: запись в литерал — UB */
```

В чём разница. `s2` — это **массив**: компилятор выделил тебе личные 4 байта (`a`, `b`, `c`, `\0`) и скопировал туда содержимое литерала. Твой массив — меняй на здоровье.

`s1` — это **указатель** на строковый литерал `"abc"`. Литерал лежит в области памяти только для чтения. Попытка `s1[0] = 'A'` — это **неопределённое поведение** (undefined behavior, UB): стандарт не обещает про исход вообще ничего.

И вот это стоит увидеть своими глазами, потому что «не обещает ничего» звучит абстрактно. Соберём один и тот же файл двумя способами.

Без оптимизации (`-O0`):

```
s2 = Abc, sizeof s2 = 4
сейчас попробуем записать в литерал...
[код выхода: 138]
```

Программа умерла на строке `s1[0] = 'A'`. Код 138 — это 128 плюс номер сигнала 10, то есть `SIGBUS`: обращение к памяти, которую трогать нельзя. Ожидаемое падение.

Тот же файл с `-O2`:

```
s2 = Abc, sizeof s2 = 4
сейчас попробуем записать в литерал...
s1 = abc
[код выхода: 0]
```

Ничего не упало. И — приглядись — `s1` остался `abc`, хотя мы вроде бы записали туда `'A'`. Оптимизатор рассуждает так: «запись в литерал — UB, значит, по стандарту такого случиться не может, значит, эту строку можно считать несуществующей». Он её просто выбросил.

Вот это и есть самое опасное лицо UB. Не падение — падение как раз подарок, ты сразу видишь проблему. Опасно, когда программа с UB **молча делает не то**, и поведение меняется от флагов

сборки: отладил на `-O0`, выкатил на `-O2` — и получил другой продукт. Полагаться нельзя ни на один из двух исходов.

Правило. Хочешь менять строку — заводи массив (`char s[]`). Указатель на литерал годится только для чтения; чтобы не забыть, объявляй его `const char *s1 = "abc";` — тогда компилятор сам запретит запись.

Каверза 2. `sizeof` массива внутри функции

```c
void f(int arr[])              /* на самом деле int *arr ! */
{
    printf("%zu\n", sizeof arr);    /* размер УКАЗАТЕЛЯ: 8 (на 64-бит) */
}

int main(void)
{
    int a[10];
    printf("%zu\n", sizeof a);      /* размер МАССИВА: 40 (10 * 4) */
    f(a);
    return 0;
}
```

В чём подвох. В месте объявления `sizeof a` знает настоящий размер массива — 40 байт (10 чисел по 4). Но при передаче в функцию массив **распадается** (decays) в указатель на первый элемент. Параметр `int arr[]` — это синтаксическая маскировка под `int *arr`. Поэтому внутри функции `sizeof arr` — это размер **указателя** (обычно 8 байт на 64-битной системе), а не массива.

Проверка на живой программе даёт ровно это:

```
в main sizeof a = 40
внутри функции sizeof arr = 8
```

Причём компилятор про такую ошибку знает и сам предупреждает — ещё один довод всегда собирать с `-Wall`:

```
warning: sizeof on array function parameter will return size of 'int *'
instead of 'int[]' [-Wsizeof-array-argument]
```

Следствие. Внутри функции по самому массиву его длину не узнать. Поэтому длину **всегда передают отдельным параметром**: `f(int *arr, int n)`. Так устроены все наши функции

в этой главе. Это же объясняет, почему `strlen` нужна строкам: длину массива `char` иначе не вычислить, спасает только метка `'\0'`.

Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)`

```
int arr[] = {10, 20, 30};
printf("%d\n", arr[1]);     /* 20 */
printf("%d\n", *(arr + 1)); /* 20 — то же самое */
printf("%d\n", 1[arr]);     /* 20 — и это тоже!? */
```

Объяснение. По определению языка `arr[i]` — это **ровно** `*(arr + i)`: «возьми адрес начала, прибавь `i`, разыменуй». Сложение коммутативно: `arr + i` и `i + arr` — одно и то же. Значит, `*(arr + i)` равно `*(i + arr)`, а это по тому же правилу записывается как `i[arr]`. Поэтому `1[arr]` — легальный (хоть и дикий) способ написать `arr[1]`.

`arr + 1` — это **арифметика указателей**: «следующий элемент», то есть адрес сдвигается не на 1 байт, а на размер одного элемента (`sizeof(int)`). Компилятор сам умножает на размер типа.

Зачем это знать. На собеседовании этим проверяют, понимаешь ли ты, что индексация — синтаксический сахар над арифметикой указателей. Писать `1[arr]` в реальном коде, разумеется, нельзя — это запутывает.

Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++`

Четыре похожих записи, которые делают **разное**. Разберём по очереди. Пусть:

```
int a[] = {10, 20, 30};
int *p = a;             /* p смотрит на a[0] (значение 10) */
```

- **`++*p`** — это `++(*p)`. Разыменуй `p` (получим `a[0]`) и сделай префиксный инкремент значения. Результат: `a[0]` стало 11, `p` не двигался. Выражение даёт 11.
- **`(*p)++`** — постфиксный инкремент **значения**, на которое смотрит `p`. `a[0]` станет 11, но выражение вернёт **старое** значение 10. `p` на месте.
- **`*p++`** — здесь приоритет у постфиксного `++`: это `*(p++)`. Сначала берётся текущий `p` (адрес `a[0]`), потом `p` сдвигается на следующий элемент. Звёздочка разыменует **старый** адрес. Итог: выражение даёт 10 (значение `a[0]`), а `p` теперь смотрит на `a[1]`. Значения в массиве не менялись. Эта запись — рабочая лошадка при проходе по строке: `while (*p++)`.
- **`*++p`** — сначала префиксный `++p` (сдвигаем `p` на `a[1]`), потом разыменование. Выражение даёт 20, `p` смотрит на `a[1]`.

Все четыре строки, запущенные подряд (каждый раз с чистым массивом `{10, 20, 30}`):

```
++*p  -> выражение 11, b[0]=11, p смотрит на b[0]
(*p)++ -> выражение 10, b[0]=11, p смотрит на b[0]
*p++  -> выражение 10, b[0]=10, p смотрит на b[1]
*++p  -> выражение 20, b[0]=10, p смотрит на b[1]
```

Сравни первые две строки: обе меняют `b[0]` на 11, но `++*p` отдаёт наружу новое значение, а `(*p)++` — старое. И сравни вторую пару: там массив не тронут вовсе, двигается только указатель.

Как не путаться. Запомни приоритеты: постфиксный `++` цепляется к `p` сильнее, чем `*`. Поэтому `*p++` — это «разыменуй, а указатель сдвинь», а не «сдвинь значение». Если хочешь менять **значение** — ставь скобки: `(*p)++`. На собеседовании именно скобки показывают, что ты понимаешь, что делаешь.

Каверза 5. `const char *p` против `char * const p`

```c
char a = 'a', b = 'b';

const char *p1 = &a;    /* нельзя менять *p1, можно менять p1 */
/* *p1 = 'x';  -- ошибка компиляции */
p1 = &b;                /* ок: указатель переставили */

char * const p2 = &a;   /* можно менять *p2, нельзя менять p2 */
*p2 = 'x';              /* ок: значение поменяли (а стало 'x') */
/* p2 = &b;  -- ошибка компиляции */
```

Как читать. Правило простое: читай объявление **справа налево от имени**.

- `const char *p1` → «`p1` — указатель на `const char`». Под запретом **то, на что смотрим** (`*p1`). Сам указатель свободно переставляется.
- `char * const p2` → «`p2` — `const`-указатель на `char`». Под запретом **сам указатель** (его нельзя перенаправить). А значение по нему менять можно.

Зачем. `const char *` — самая частая форма для параметров-строк, которые функция только читает (как в нашей `my_strlen`). А бывает и `const char * const p` — нельзя ни то, ни другое.

Каверза 6. Найди баг: утечка памяти

```c
char *make_greeting(const char *name)
{
    char *buf = malloc(64);
```

```
    if (buf == NULL) return NULL;
    snprintf(buf, 64, "Привет, %s!", name);
    return buf;
}

int main(void)
{
    printf("%s\n", make_greeting("Аня"));   /* БАГ: утечка */
    return 0;
}
```

Где баг. `make_greeting` выделяет память через `malloc` и возвращает указатель. Но в `main` мы передаём этот указатель сразу в `printf` и **теряем** его — переменной, которая хранила бы адрес, нет. Освободить такой блок уже нечем: `free` вызвать не от чего. Это **утечка памяти**: блок занят до конца программы.

Как чинить. Сохрани адрес, попользуйся, освободи:

```
char *msg = make_greeting("Аня");
if (msg != NULL) {
    printf("%s\n", msg);
    free(msg);              /* на каждый malloc — свой free */
}
```

Правило из главы 3 в полный рост: на каждый `malloc` однажды должен прийтись ровно один `free`. Если функция **возвращает** выделенную память, в её документации/имени обычно намекают, что освобождать — обязанность вызывающего.

Каверза 7. Найди баг: use-after-free и двойной free

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);                /* память возвращена системе */

printf("%d\n", *p);     /* БАГ 1: use-after-free — читаем мёртвое */
free(p);                /* БАГ 2: double free — освобождаем дважды */
```

Что не так. После `free(p)` блок больше не наш. Указатель `p` всё ещё хранит старый адрес (он стал «висячим», dangling), но трогать память по нему нельзя:

- `*p` после `free` — **use-after-free**: читаем (или пишем) освобождённую память. Неопределённое поведение: может показать старое значение, мусор или уронить программу.
- второй `free(p)` — **double free**: освобождение уже освобождённого блока. Тоже UB, частая причина падений и уязвимостей.

Как чинить. Не обращайся к памяти после `free`, а указатель сразу обнуляй:

```
free(p);
p = NULL;       /* теперь p честно «никуда»; free(NULL) безопасен */
```

Привычка `p = NULL` после `free` лечит оба бага разом: повторный `free(NULL)` по стандарту ничего не делает, а случайное `*p` по `NULL` упадёт сразу и заметно, а не испортит данные втихую.

Каверза 8. Возврат адреса локальной переменной

```
int *broken(void)
{
    int x = 42;
    return &x;          /* БАГ: x умрёт по выходе из функции */
}

int main(void)
{
    int *p = broken();
    printf("%d\n", *p); /* читаем мёртвую память — UB */
    return 0;
}
```

Где баг. `x` — локальная переменная, она живёт на стеке только пока работает `broken`. После `return` её память освобождается. Адрес `&x` становится висячим: данные по нему уже не наши. Обращение `*p` в `main` — неопределённое поведение. Иногда «случайно работает» (значение ещё не затёрто) — и это самое коварное, потому что баг прячется.

Хорошая новость: этот баг компилятор ловит сам, если собирать с `-Wall`:

```
warning: address of stack memory associated with local variable 'x'
returned [-Wreturn-stack-address]
```

А вот чем оборачивается игнор предупреждения — сборка с `-O2` напечатала:

```
-1270017712
```

Не 42 и даже не ноль, а то, что успело оказаться в этой ячейке стека. Число будет разным от запуска к запуску и от компилятора к компилятору. Забавная деталь: под адресным

санитайзером (`-fsanitize=address`) та же программа печатает честные `42` — стек там устроен иначе, и значение уцелело. Это ровно та ловушка, о которой речь: UB иногда «работает», и именно поэтому на него нельзя опираться.

Как чинить. Либо вернуть **значение**, а не адрес (`int broken(void)` с `return x;`), либо выделить память динамически (`malloc`) и отдать ответственность за `free` вызывающему, либо принять буфер от вызывающего параметром. Возвращать адрес `static`-переменной можно, но это уже другая история (общая память на все вызовы).

Мини-резюме

Что должно остаться в голове после этого приложения:

- **Строки** — это `char`-массивы с `'\0'`; любая своя строковая функция идёт по символам до нуля. Литерал `"abc"` менять нельзя.
- **Списки** — три указателя разворачивают, два указателя разной скорости находят середину и ловят цикл (Флойд). Следи за порядком присваиваний и не разыменовывай `NULL`.
- **Биты** — `n & (n-1)` гасит младшую единицу (popcount Кернигана, проверка степени двойки); XOR гасит пары (одиночка за O(1) памяти).
- **Поиск и слияние** — бинарный поиск держи на включительных границах `[lo, hi]` с `lo + (hi - lo) / 2`; слияние — два индекса плюс хвосты.
- **Память** — на каждый `malloc` свой `free`; после `free` ставь `p = NULL`; не возвращай адрес локальной переменной; помни про decay массива в указатель и `const`-квалификаторы.

И главное, что проверяют на Middle: ты не зубришь алгоритмы, а понимаешь, **что происходит в памяти**. Поэтому на собеседовании проговаривай вслух, кто на что указывает и кто кого освобождает, — это ценится больше готового ответа.

Что дальше

Дальше — **Приложение В: Senior**. Там задачи злее: обобщённые структуры через `void *` и указатели на функции, своя мини-куча на `malloc`, выравнивание и упаковка `struct`, разбор хитрых объявлений вроде «указатель на функцию, возвращающую указатель на массив», устранение рекурсии, и каверзы про неопределённое поведение, переполнение знакового `int` и порядок вычисления аргументов. Если этот раздел дался без больших мучений — ты готов идти выше.

Приложение B. Задачи с собеседований: Senior

Это третий, самый трудный сборник задач с собеседований. Если в «Junior» проверяли, помнишь ли ты синтаксис, а в «Middle» — умеешь ли собрать алгоритм, то здесь смотрят глубже: понимаешь ли ты, **как Си устроен внутри**. Память, указатели на указатели, `void *`, выравнивание структур, неопределённое поведение (UB), тонкости препроцессора — то, на чём спотыкаются даже люди с опытом.

Задачи идут от простого к сложному и делятся на два блока:

- **на код** — написать рабочую программу: быстрые сортировки, обобщённые структуры через `void *`, хеш-таблица, кольцевой буфер, LRU-кэш и другие классические задачи;
- **каверзы и теория** — короткие вопросы, где надо не написать код, а объяснить, **почему так**: что выведет хитрый `sizeof`, чем опасен макрос `SQR(x)`, что такое strict aliasing.

Всё, что нужно, ты уже прошёл в главах 1–18: указатели и память (главы 3, 11), рекурсия (глава 9), сортировки и `qsort` (глава 8), структуры и связные списки (глава 11). Здесь мы доведём это до уровня, который ждут от senior-разработчика на Си.

Совет тот же, что и раньше: сначала реши задачу сам на бумаге и в редакторе, и только потом сверяйся с разбором. Все программы лежат в `code/21-*.c` и собираются без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 code/21-01-quicksort.c -o q && ./q
```

Часть 1. Задачи на код

Задача 1. Быстрая сортировка (quicksort)

Условие. Реализуй быструю сортировку массива `int` на месте (без дополнительного массива). Объясни выбор опорного элемента и сложность.

Идея. В главе 8 мы разобрали идею словами: выбираем **опорный** (pivot) элемент, переставляем массив так, чтобы слева оказались все меньшие опорного, а справа — большие. После такой перестановки опорный стоит на своём окончательном месте. Дальше рекурсивно сортируем левую и правую части.

Главная деталь — **разбиение** (partition). Возьмём схему Ломуто: опорным берём последний элемент `a[hi]`. Ведём границу `i` — индекс последнего «малого» элемента. Идём `j` от начала: как только встретили элемент меньше опорного, сдвигаем границу и меняем его туда. В конце ставим опорный сразу за границей.

Решение (`code/21-01-quicksort.c`):

```c
#include <stdio.h>

/* Обмен двух int через указатели (см. главу 3). */
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

/* Разбиение Ломуто: опорный — последний элемент a[hi]. */
int partition(int a[], int lo, int hi)
{
    int pivot = a[hi];          /* опорный (pivot) */
```

```
    int i = lo - 1;             /* граница «меньших опорного» */

    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {     /* нашли элемент меньше опорного */
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);    /* опорный встаёт на своё место */
    return i + 1;
}

void quicksort(int a[], int lo, int hi)
{
    if (lo >= hi) {             /* 0 или 1 элемент — уже отсортировано */
        return;
    }
    int p = partition(a, lo, hi);
    quicksort(a, lo, p - 1);
    quicksort(a, p + 1, hi);
}
```

В `main` зовём `quicksort(a, 0, n - 1)` — границы **включительно**, поэтому `hi = n - 1`, а не `n`.

Объяснение.

- `swap(int *a, int *b)` принимает **адреса** переменных и меняет их содержимое через разыменование `*a`, `*b` (см. главу 3). Передай мы числа по значению — обмен случился бы только внутри функции.
- `int i = lo - 1;` — граница начинается «до» отрезка: малых элементов пока нет. Инвариант разбиения: всё в `a[lo..i]` меньше опорного, всё в `a[i+1..j-1]` — больше либо равно.
- цикл `for (j = lo; j < hi; j++)` доходит только до `hi - 1`, сам опорный `a[hi]` не трогаем — его поставим в конце.
- `swap(&a[i + 1], &a[hi])` — после цикла `i+1` это первая «большая» ячейка; меняем её с опорным, и тот оказывается ровно между меньшими и большими. Возвращаем `i + 1` — его финальную позицию.

Выбор опорного и сложность. В среднем разбиение делит массив пополам, уровней рекурсии около `log n`, на каждом уровне суммарно `n` сравнений — итого **O(n log n)**. Но беда в **худшем случае**: если опорный каждый раз оказывается минимумом или максимумом (а

на уже отсортированном массиве взятие последнего элемента именно это и даёт!), части получаются 1 и n−1, глубина рекурсии вырастает до n, и сложность скатывается в **O(n²)** — плюс риск переполнить стек.

Лечат это выбором опорного поумнее:

- **случайный опорный** — `swap(&a[hi], &a[lo + rand() % (hi - lo + 1)])` перед разбиением; отсортированный вход больше не «убивает» сортировку;
- **медиана трёх** — берут медиану из `a[lo]`, `a[mid]`, `a[hi]`; почти бесплатно и хорошо защищает от худшего случая.

Память: quicksort работает на месте, дополнительной памяти — только под стек рекурсии (O(log n) при хорошем опорном).

Доп. вопросы.

- Чем схема Хоара (два бегунка навстречу) лучше Ломуто? (Меньше обменов.)
- Почему quicksort **нестабильна** (равные элементы могут поменять порядок)?
- Как сделать quicksort нерекурсивной, на явном стеке?

Задача 2. Сортировка слиянием (merge sort)

Условие. Реализуй сортировку слиянием. В чём её плюс перед quicksort?

Идея. Делим массив пополам, рекурсивно сортируем каждую половину, потом **сливаем** две отсортированные половины в одну. Слияние — главный шаг: идём двумя бегунками по головам половин и каждый раз забираем меньший. В отличие от quicksort, merge sort даёт **гарантированные** O(n log n) даже в худшем случае и **стабильна**, но требует доп. памяти под временный буфер.

Решение (`code/21-02-mergesort.c`):

```c
void merge(int a[], int tmp[], int lo, int mid, int hi)
{
    int i = lo;          /* бегунок по левой половине */
    int j = mid + 1;     /* бегунок по правой половине */
    int k = lo;          /* куда пишем в tmp */

    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) {     /* <= даёт стабильность */
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
```

```
        }
    }
    while (i <= mid) { tmp[k++] = a[i++]; }   /* хвост левой */
    while (j <= hi)  { tmp[k++] = a[j++]; }   /* хвост правой */
    for (int t = lo; t <= hi; t++) {          /* копируем обратно */
        a[t] = tmp[t];
    }
}

void merge_sort(int a[], int tmp[], int lo, int hi)
{
    if (lo >= hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;   /* без переполнения lo+hi */
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid + 1, hi);
    merge(a, tmp, lo, mid, hi);
}
```

Объяснение.

- `tmp` — **один** временный буфер на весь алгоритм, выделяем его в `main` через `malloc((size_t)n * sizeof(int))` и передаём внутрь. Так не выделяем память на каждом слиянии.
- `int mid = lo + (hi - lo) / 2;` — почему не `(lo + hi) / 2`? Сумма `lo + hi` может **переполнить** `int` на огромных индексах. Запись `lo + (hi - lo)/2` даёт ту же середину, но без риска переполнения. Это классический приём (та же ловушка живёт и в бинарном поиске, см. главу 7).
- `a[i] <= a[j]` — именно `<=`, а не `<`: при равенстве берём элемент из **левой** половины, и равные сохраняют исходный порядок. Это и есть **стабильность**.
- три `while` подряд: первый сливает, пока живы обе половины; два следующих дописывают остаток той половины, что ещё не кончилась.

Quicksort или merge sort? Quicksort обычно быстрее на практике (лучше дружит с кэшем, сортирует на месте), но имеет худший случай $O(n^2)$. Merge sort всегда $O(n \log n)$ и стабилен, но тратит $O(n)$ доп. памяти. Для связных списков merge sort даже удобнее — слияние не требует случайного доступа.

Доп. вопросы.

- Сколько памяти под `tmp`? (O(n).) Можно ли слить две половины на месте за O(n)? (Сложно, обычными приёмами — нет.)
- Что такое «восходящий» (bottom-up) merge sort без рекурсии?

Задача 3. Обобщённый стек через `void *`

Условие. Напиши стек (или динамический массив), который умеет хранить элементы **любого** типа — `int`, `double`, структуру, — не переписывая код под каждый тип.

Идея. Шаблонов, как в C++, в Си нет. Универсальность строят на `void *` («указатель на что угодно») и на знании **размера элемента в байтах**. Стек хранит сырой байтовый буфер и `elem_size`. `push` **копирует** `elem_size` байт из переданного адреса в буфер, `pop` — обратно. Сам стек не знает, что за тип внутри: он просто двигает байты через `memcpy`.

Решение (`code/21-03-generic-stack.c`):

```c
typedef struct {
    char *data;          /* буфер из байтов */
    size_t elem_size;    /* размер одного элемента в байтах */
    size_t count;        /* сколько элементов лежит */
    size_t capacity;     /* сколько элементов влезает */
} Stack;

int stack_init(Stack *s, size_t elem_size)
{
    s->elem_size = elem_size;
    s->count = 0;
    s->capacity = 4;
    s->data = malloc(s->capacity * elem_size);
    return s->data != NULL;
}

int stack_push(Stack *s, const void *elem)
{
    if (s->count == s->capacity) {                /* места нет — растём */
        size_t new_cap = s->capacity * 2;
        char *bigger = realloc(s->data, new_cap * s->elem_size);
        if (bigger == NULL) {
            return 0;
        }
```

```
        s->data = bigger;
        s->capacity = new_cap;
    }
    memcpy(s->data + s->count * s->elem_size, elem, s->elem_size);
    s->count++;
    return 1;
}

int stack_pop(Stack *s, void *out)
{
    if (s->count == 0) {
        return 0;
    }
    s->count--;
    if (out != NULL) {
        memcpy(out, s->data + s->count * s->elem_size, s->elem_size);
    }
    return 1;
}
```

Использование одинаково для разных типов:

```
Stack si; stack_init(&si, sizeof(int));
int x = 10; stack_push(&si, &x);

Stack sd; stack_init(&sd, sizeof(double));
double d = 3.14; stack_push(&sd, &d);
```

Объяснение.

- буфер — `char *`, потому что `char` гарантированно 1 байт; адресная арифметика `data + count * elem_size` ходит по буферу побайтно и попадает ровно в начало нужного элемента. По `void *` так шагать **нельзя** — арифметика на `void *` стандартом не определена.
- `const void *elem` в `push` — «адрес элемента, тип неизвестен, менять не буду». Вызывающий передаёт `&x`. Внутри `memcpy` копирует `elem_size` байт — ровно один элемент.
- `pop` пишет результат по адресу `out` (если он не `NULL`) — тем же `memcpy`. Так мы «возвращаем» значение неизвестного типа: через выходной указатель, а не через `return`.
- `realloc` с удвоением вместимости — амортизированная вставка за O(1) (см. растущий массив в главе 11). Результат кладём во **временный** `bigger` и проверяем на `NULL`, иначе при неудаче потеряем старый блок.

Плата за обобщённость. Нет проверки типов: положил `int`, достал как `double` — компилятор смолчит, а ты получишь мусор. И `memcpy` чуть медленнее прямого присваивания. Зато один код работает со всем.

Доп. вопросы.

- Почему буфер `char *`, а не `void *`? (Арифметика указателей.)
- Как сделать обобщённую функцию печати? (Передать ещё указатель на функцию `void print(const void *)` — см. задачу 5.)
- Чем такой стек отличается от `qsort`, которая тоже принимает `void *` и размер элемента?

Задача 4. Свои `atoi` и `itoa` с обработкой краёв

Условие. Напиши `my_atoi` (строка → `int`) и `my_itoa` (`int` → строка). Корректно обработай знак, пробелы, мусор и **переполнение**, включая `INT_MIN`.

Идея. В `atoi` копим число пошагово: `result = result*10 + цифра`. Опасность — переполнение `int`. Поэтому копим в `long` (он шире) и после каждой цифры проверяем выход за `INT_MAX`/`INT_MIN`. В `itoa` главная каверза — `INT_MIN`: его модуль `-INT_MIN` в `int` **не помещается** (диапазон несимметричен: от −2147483648 до 2147483647). Спасает работа в `long`.

Решение (`code/21-04-atoi-itoa.c`):

```c
#include <limits.h>

int my_atoi(const char *s, int *out)
{
    while (*s == ' ' || *s == '\t') {   /* ведущие пробелы */
        s++;
    }

    int sign = 1;
    if (*s == '+' || *s == '-') {
        if (*s == '-') sign = -1;
        s++;
    }

    if (*s < '0' || *s > '9') {          /* нет цифр — не число */
        return 0;
    }
```

```
    long result = 0;                    /* копим в long: он шире int */
    while (*s >= '0' && *s <= '9') {
        result = result * 10 + (*s - '0');
        if (sign == 1 && result > INT_MAX) return 0;
        if (sign == -1 && -result < INT_MIN) return 0;
        s++;
    }

    *out = (int)(sign * result);
    return 1;
}

void my_itoa(int n, char *buf)        /* buf >= 12 байт */
{
    char tmp[12];
    int i = 0;
    int neg = 0;

    long value = n;                   /* long, чтобы взять -INT_MIN */
    if (value < 0) {
        neg = 1;
        value = -value;
    }

    do {                              /* цифры выходят задом наперёд */
        tmp[i++] = (char)('0' + value % 10);
        value /= 10;
    } while (value > 0);

    int j = 0;
    if (neg) buf[j++] = '-';
    while (i > 0) buf[j++] = tmp[--i];   /* разворачиваем в buf */
    buf[j] = '\0';
}
```

Вывод программы целиком:

```
my_atoi("42") = 42
my_atoi("  -17") = -17
```

```
my_atoi("+5") = 5
my_atoi("2147483647") = 2147483647
my_atoi("2147483648") = ошибка
my_atoi("-2147483648") = -2147483648
my_atoi("abc") = ошибка
my_atoi("-2147483649") = ошибка
---
my_itoa(0) = "0"
my_itoa(42) = "42"
my_itoa(-17) = "-17"
my_itoa(2147483647) = "2147483647"
my_itoa(-2147483648) = "-2147483648"
```

Обрати внимание на две соседние строки: `2147483647` разбирается, а `2147483648` — уже нет. Это ровно граница `int`, и проверка на переполнение срабатывает на единицу за ней, а не «примерно там». Симметрично внизу: `-2147483648` разбирается (это минимальный `int`), `-2147483649` — ошибка.

Объяснение.

- `*s - '0'` — перевод символа-цифры в число. Цифры в таблице ASCII идут подряд: `'7' - '0' == 7` (см. главу 10).
- проверки переполнения **после каждой цифры**: для плюса `result` превысил `INT_MAX`, для минуса `-result` ушёл ниже `INT_MIN`. `long` гарантированно вмещает любое промежуточное значение (на 64-битных системах он 8 байт).
- в `my_itoa` цикл `do { } while` обязателен: иначе для `n == 0` мы не выведем ни одной цифры. `do/while` гарантирует хотя бы один проход.
- `value = -value` для отрицательного — безопасно только потому, что `value` это `long`. Сделай мы `int value = n; value = -value;` при `n == INT_MIN` получили бы **UB** (переполнение знакового).
- `tmp[12]` хватает: максимум 10 цифр у `int` + знак + `'\0'` = 12.

Доп. вопросы.

- Почему `int abs(INT_MIN)` — это UB? (Результат не помещается в `int`.)
- Как `my_itoa` обобщить на любую систему счисления (`base`)?
- Что вернёт стандартный `atoi("99999999999")`? (UB при переполнении — поэтому в реальном коде берут `strtol`, который умеет сообщать об ошибке через `errno`.)

Задача 5. Таблица обработчиков на указателях на функции

Условие. Сделай диспетчер: по символу операции (+, −, ∗) вызывается нужная функция. Заодно покажи компаратор для `qsort`.

Идея. Функцию в Си можно передать как аргумент и хранить в переменной — через **указатель на функцию**. Связку «ключ → функция» складываем в массив структур (таблицу). Вместо длинного `switch` пробегаем таблицу и зовём найденный обработчик. Это тот же приём, что делает `qsort` универсальной: ей передают указатель на компаратор.

Решение (`code/21-05-funcptr.c`):

```c
int op_add(int a, int b) { return a + b; }
int op_sub(int a, int b) { return a - b; }
int op_mul(int a, int b) { return a * b; }

typedef struct {
    char symbol;
    int (*fn)(int, int);   /* указатель на функцию (int, int) -> int */
} Handler;

int cmp_desc(const void *pa, const void *pb)
{
    int a = *(const int *)pa;
    int b = *(const int *)pb;
    return (a < b) - (a > b);   /* по убыванию, без вычитания a-b */
}

int main(void)
{
    Handler table[] = { {'+', op_add}, {'-', op_sub}, {'*', op_mul} };
    int n = (int)(sizeof table / sizeof table[0]);

    int x = 6, y = 4;
    char ops[] = {'+', '-', '*'};
    for (int k = 0; k < 3; k++) {
        for (int i = 0; i < n; i++) {
            if (table[i].symbol == ops[k]) {
                int r = table[i].fn(x, y);     /* вызов через указатель */
                printf("%d %c %d = %d\n", x, ops[k], y, r);
```

```
                break;
            }
        }
    }

    int a[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int m = (int)(sizeof a / sizeof a[0]);
    qsort(a, (size_t)m, sizeof a[0], cmp_desc);
    /* ... печать: 9 6 5 4 3 2 1 1 */
    return 0;
}
```

Объяснение.

- `int (*fn)(int, int);` — самая пугающая запись в Си. Читай изнутри: `fn` — это указатель (`*fn`), на функцию, принимающую `(int, int)` и возвращающую `int`. **Скобки вокруг `*fn` обязательны**: без них `int *fn(int, int)` означало бы «функция, возвращающая `int *`» — совсем другое.
- имя функции `op_add` само по себе — уже её **адрес**, поэтому `{'+', op_add}` записывает указатель. Амперсанд не нужен (хотя `&op_add` тоже допустимо и означает то же).
- `table[i].fn(x, y)` — вызов через указатель. Можно писать и `(*table[i].fn)(x, y)` — это эквивалентно.
- `cmp_desc` возвращает `(a < b) - (a > b)`: даёт `+1`, `0` или `–1` без опасного `a - b` (которое может переполнить `int` на больших числах, см. ловушку в главе 8). Для убывания знак подобран так, что больший элемент идёт раньше.

Где это применяют по-настоящему. Таблицы команд в интерпретаторах, обработчики событий, колбэки (callbacks), state-машины, плагины. Указатель на функцию — основа любого «подключаемого» поведения в Си.

Доп. вопросы.

- Что такое `typedef int (*BinOp)(int, int);` и как он упрощает запись?
- Чем `qsort_r` (с доп. контекстом) удобнее обычной `qsort`?
- Можно ли сделать массив указателей на функции без структуры? (Да: `int (*ops[])(int,int) = {op_add, op_sub};`.)

Задача 6. Кольцевой буфер (ring buffer)

Условие. Реализуй кольцевой буфер фиксированного размера: запись в хвост, чтение из головы, переполнение и опустошение — корректно.

Идея. Это очередь (FIFO) поверх массива из главы 11, но оформленная аккуратной структурой. Храним массив, индекс головы `head` и количество `count`. Хвост вычисляем: `(head + count) % CAP`. Остаток от деления `%` заворачивает индекс с конца массива в начало — массив словно свёрнут в кольцо. Пуст при `count == 0`, полон при `count == CAP`.

Решение (`code/21-06-ringbuf.c`):

```c
#define CAP 4

typedef struct {
    int data[CAP];
    int head;     /* индекс первого элемента */
    int count;    /* сколько элементов лежит */
} Ring;

int ring_push(Ring *r, int value)
{
    if (r->count == CAP) {
        return 0;                           /* буфер полон */
    }
    int tail = (r->head + r->count) % CAP;   /* первое свободное место */
    r->data[tail] = value;
    r->count++;
    return 1;
}

int ring_pop(Ring *r, int *out)
{
    if (r->count == 0) {
        return 0;                           /* буфер пуст */
    }
    *out = r->data[r->head];
    r->head = (r->head + 1) % CAP;          /* голова заворачивается */
    r->count--;
    return 1;
}
```

Объяснение.

- почему хранить `count`, а не индекс `tail`? Если держать `head` и `tail`, возникает путаница: при `head == tail` буфер пуст **или** полон — состояния неразличимы. С `count` всё однозначно.
- `(head + count) % CAP` — позиция за последним элементом, она же первое свободное место. Когда сумма дорастает до `CAP`, `%` сбрасывает её в 0.
- `head = (head + 1) % CAP` — после чтения голова шагает вперёд и тоже заворачивается.
- никаких `malloc` — буфер на статическом массиве внутри структуры, освобождать нечего.

Вывод целиком:

```
push 1: ok
push 2: ok
push 3: ok
push 4: ok
push 5: полон
push 6: полон
pop 1
pop 2
pop 3
pop 4
pop 7
pop 8
```

Читается он так. Первые четыре `push` заполнили кольцо (`CAP` равен 4), пятый и шестой уже не влезли. Дальше два `pop` забрали `1` и `2` — и в освободившиеся ячейки легли `7` и `8`, завернувшись в начало массива. Финальный `while` вычерпывает остаток: сначала «старые» `3` и `4`, и только потом `7` и `8`. Порядок FIFO соблюдён, хотя физически в массиве `7` и `8` лежат раньше `3` и `4`.

Зачем нужно. Кольцевые буферы — это потоки аудио/видео, буфер клавиатуры, логи фиксированного размера, обмен данными между потоком- производителем и потоком-потребителем. Размер заранее известен, память не скачет.

Доп. вопросы.

- Чем плох вариант с двумя индексами без `count`? (Неразличимы пусто/полно; лечат «потерей одной ячейки» или отдельным флагом.)
- Почему `CAP` — степень двойки иногда быстрее? (`% CAP` заменяется на `& (CAP–1)`.)
- Как сделать буфер растущим (динамическим)?

Задача 7. Хеш-таблица «строка → счётчик» с цепочками

Условие. Посчитай, сколько раз встретилось каждое слово. Реализуй хеш-таблицу с разрешением коллизий **цепочками**.

Идея. Хеш-таблица — массив «корзин». Функция-хеш превращает строку в число, остаток от деления на число корзин даёт индекс. Разные строки могут попасть в одну корзину — это **коллизия**. Решаем цепочками: каждая корзина это связный список записей, в ней лежащих. Поиск: вычислили корзину, прошли её цепочку, сравнили ключи `strcmp`.

Решение (`code/21-07-hashtable.c`):

```c
#define NBUCKETS 16

typedef struct Entry {
    char *key;
    int count;
    struct Entry *next;   /* цепочка внутри корзины */
} Entry;

typedef struct { Entry *buckets[NBUCKETS]; } HashTable;

unsigned long hash_str(const char *s)        /* djb2 */
{
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char)*s++) != 0) {
        h = h * 33u + (unsigned long)c;
    }
    return h;
}

int table_inc(HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % NBUCKETS;

    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {     /* слово уже есть */
            e->count++;
            return 1;
```

```
        }
    }

    Entry *e = malloc(sizeof(Entry));        /* новое слово */
    if (e == NULL) return 0;
    e->key = malloc(strlen(key) + 1);
    if (e->key == NULL) { free(e); return 0; }
    strcpy(e->key, key);
    e->count = 1;
    e->next = t->buckets[idx];               /* вешаем в начало цепочки */
    t->buckets[idx] = e;
    return 1;
}
```

Объяснение.

- `hash_str` (алгоритм djb2): начинаем с 5381, на каждый символ `h = h*33 + c`. Множитель 33 и старт 5381 подобраны эмпирически — такая функция хорошо «перемешивает» строки. Тип `unsigned long`, чтобы переполнение было определённым (для беззнаковых это не UB, а честный остаток по модулю — см. блок про UB ниже).
- `(unsigned char)*s` — важная деталь: `char` может быть знаковым, и русские/символы с кодом > 127 дали бы отрицательное число и кривой хеш. Приведение к `unsigned char` лечит это.
- коллизии: при `strcmp` мы **сравниваем сами ключи**, а не только хеши. Два разных слова с одинаковым хешем окажутся в одной корзине, но `strcmp` их различит.
- **владение памятью**: ключ копируем своим `malloc(strlen(key)+1) + strcpy`. Нельзя просто сохранить переданный указатель — строка вызывающего может измениться или исчезнуть. На каждый `malloc` есть `free` в `table_free` (сначала `free(e->key)`, потом `free(e)`).
- вставка в **начало** цепочки — O(1), порядок внутри корзины неважен.

Про сложность. При хорошей хеш-функции и небольшом коэффициенте заполнения цепочки коротки, и поиск/вставка — в среднем **O(1)**. Если таблица переполнена (элементов много больше корзин), цепочки длинные, и всё вырождается в **O(n)**. Лечат **рехешированием**: при заполнении увеличивают число корзин и перераскладывают записи.

Доп. вопросы.

- Чем цепочки отличаются от **открытой адресации** (линейное/квадратичное пробирование)? Плюсы и минусы.
- Что такое коэффициент загрузки (load factor) и когда делать рехеш?

- Почему `NBUCKETS` берут простым числом или степенью двойки?

Задача 8. Точка пересечения двух односвязных списков

Условие. Даны два односвязных списка, которые в какой-то момент сливаются в общий хвост. Найди **узел слияния** (по адресу, не по значению). Если не пересекаются — `NULL`.

Идея. Наивно — для каждого узла A проверять весь B: O(n · m). Умнее: если списки сливаются, у них **общий хвост одинаковой длины**. Значит, разница только в длине «головных» частей. Посчитаем длины, проматаем длинный список вперёд на разницу — и теперь оба узла на равном расстоянии до конца. Шагаем параллельно: первый узел, где **адреса совпадут**, и есть точка пересечения. Это O(n + m) времени и O(1) памяти.

Решение (`code/21-08-list-intersection.c`):

```c
typedef struct Node {
    int value;
    struct Node *next;
} Node;

int length(Node *head)
{
    int len = 0;
    for (Node *p = head; p != NULL; p = p->next) len++;
    return len;
}

Node *intersection(Node *a, Node *b)
{
    int la = length(a);
    int lb = length(b);

    while (la > lb) { a = a->next; la--; }   /* выравниваем старты */
    while (lb > la) { b = b->next; lb--; }

    while (a != b) {        /* сравниваем АДРЕСА узлов, не значения */
        a = a->next;
        b = b->next;
    }
```

```
    return a;               /* общий узел или NULL */
}
```

Объяснение.

- ключевая мысль — сравнивать `a != b`, то есть **указатели**, а не `a->value != b->value`. Пересечение списков означает, что с какого-то узла они физически идут по **одной и той же** памяти. Значения могут совпадать и случайно, а адрес узла уникален.
- после выравнивания обоим до конца идти поровну. Если есть общий хвост, бегунки войдут в него синхронно и встретятся на узле слияния. Если пересечения нет, оба одновременно дойдут до `NULL` — и `a == b == NULL`, цикл закончится, вернём `NULL`. Один и тот же код покрывает оба случая.
- освобождение тут с подвохом: общий хвост нельзя `free` дважды. Сначала чистим уникальные части A и B **до** общего узла `tail`, потом — сам хвост один раз. Двойной `free` — это UB (см. блок ниже).

Доп. вопросы.

- Как найти **начало цикла** в списке (алгоритм Флойда «черепаха и заяц», два указателя с разной скоростью)?
- Как решить без подсчёта длин? (Два указателя: дойдя до конца своего списка, перепрыгивают на начало чужого; встретятся на пересечении.)
- Почему сравнение по значению дало бы неверный ответ?

Задача 9. Набросок LRU-кэша

Условие. Реализуй кэш на N записей с вытеснением **давно не использованных** (LRU — Least Recently Used). `get` и `put` — в среднем за O(1).

Идея. Нужны две вещи сразу: быстрый **поиск по ключу** и быстрое определение, **кто давно не использовался**. Поиск даёт хеш-таблица. Порядок использования — **двусвязный список**: голова это самый свежий, хвост — самый старый кандидат на вылет. Каждый узел живёт одновременно в обеих структурах. `get`/`put` находят узел через хеш и переставляют его в голову списка. При переполнении удаляем хвост — и из списка, и из хеша.

Ключевой код (`code/21-09-lru.c`):

```
typedef struct Node {
    int key, value;
    struct Node *prev, *next;   /* список «по свежести» */
    struct Node *hnext;         /* цепочка хеш-корзины */
} Node;
```

```
int lru_get(LRU *c, int key, int *out)
{
    Node *n = hash_find(c, key);
    if (n == NULL) return 0;
    list_unlink(c, n);          /* вынули из текущего места */
    list_push_front(c, n);      /* подняли в голову — самый свежий */
    *out = n->value;
    return 1;
}

void lru_put(LRU *c, int key, int value)
{
    Node *n = hash_find(c, key);
    if (n != NULL) {            /* уже есть — обновим и освежим */
        n->value = value;
        list_unlink(c, n);
        list_push_front(c, n);
        return;
    }
    if (c->size == c->capacity) {   /* полон — вытесняем хвост (старейший) */
        Node *old = c->tail;
        list_unlink(c, old);
        /* убрать old из его хеш-цепочки */
        Node **pp = &c->buckets[bucket_of(old->key)];
        while (*pp != old) pp = &(*pp)->hnext;
        *pp = old->hnext;
        free(old);
        c->size--;
    }
    n = malloc(sizeof(Node));
    if (n == NULL) return;
    n->key = key; n->value = value;
    n->hnext = c->buckets[bucket_of(key)];
    c->buckets[bucket_of(key)] = n;
    list_push_front(c, n);
    c->size++;
}
```

Объяснение.

- узел в **двух** структурах: `prev/next` — порядок свежести, `hnext` — цепочка хеша. Это и даёт O(1): хеш находит узел мгновенно, а двусвязный список позволяет вынуть узел из середины за O(1) (у односвязного пришлось бы искать предыдущий).
- `list_unlink` + `list_push_front` — «освежить»: при каждом обращении узел переезжает в голову. Значит, у хвоста всегда самый давний доступ.
- интересна строка `Node **pp = &c->buckets[...]; while (*pp != old) ...` — **указатель на указатель**. Он позволяет удалить узел из односвязной хеш-цепочки без отдельной проверки «это голова или середина»: `pp` всегда указывает на то поле (`next` предыдущего или сам `buckets[b]`), которое надо перенаправить. Классический senior-приём.
- вывод подтверждает логику: после `get 1` ключ 1 стал свежим, и `put 3` вытеснил именно 2 (давно не трогали), а 1 уцелел:

```
get 1 = 100 (теперь 1 свежий)
get 2 = промах (вытеснен)
get 1 = 100
get 3 = 300
```

Доп. вопросы.

- Почему именно **дву**связный список, а не односвязный? (Удаление узла из середины за O(1).)
- Чем LRU отличается от LFU (вытеснение по частоте) и FIFO?
- Как тот же кэш сделать потокобезопасным? (Мьютекс вокруг операций.)

Задача 10. Разворот числа с защитой от переполнения

Условие. Развернуть цифры знакового `int` (`123 → 321`, `-456 → -654`). Если перевёрнутое число не помещается в `int` — сообщить об ошибке, а не выдать мусор.

Идея. Снимаем последнюю цифру `n % 10`, отбрасываем её `n /= 10`, дописываем в результат `result = result*10 + цифра`. Ловушка та же, что в `atoi`: `result*10` может переполнить `int`. Но здесь нельзя «копить в long и проверить потом» как универсальный приём собеседования — часто требуют остаться в `int`. Поэтому проверяем **до** умножения: не превысит ли `result*10 + цифра` границу.

Решение (`code/21-10-reverse-int.c`):

```c
#include <limits.h>

int reverse_int(int n, int *out)
```

```
{
    int result = 0;
    while (n != 0) {
        int digit = n % 10;   /* знак сохраняется: -123 % 10 == -3 (C99) */
        n /= 10;

        if (result > INT_MAX / 10 ||
            (result == INT_MAX / 10 && digit > INT_MAX % 10)) {
            return 0;         /* переполнение вверх */
        }
        if (result < INT_MIN / 10 ||
            (result == INT_MIN / 10 && digit < INT_MIN % 10)) {
            return 0;         /* переполнение вниз */
        }
        result = result * 10 + digit;
    }
    *out = result;
    return 1;
}
```

Объяснение.

- работаем **без** перевода в `unsigned` или `long` — прямо в `int`. Поэтому `result` хранит уже со знаком: для отрицательного `n` и `digit`, и `result` отрицательны (в C99 деление/остаток для отрицательных округляются **к нулю**: `-123 / 10 == -12`, `-123 % 10 == -3`).
- проверка «влезет ли `result*10 + digit`» сделана **арифметикой без переполнения**: сравниваем `result` с `INT_MAX/10`. Если строго больше — умножение уже переполнит. Если ровно равно — смотрим на последнюю цифру относительно `INT_MAX % 10`. Симметрично для `INT_MIN`. Так мы ни разу не выходим за границы `int` в самих вычислениях.
- `1534236469` развернётся в `9646324351` — это больше `INT_MAX` (2147483647), поэтому функция честно вернёт «переполнение», а не мусор.

Доп. вопросы.

- Почему опасно «развернуть, потом проверить»? (Переполнение знакового — это UB ещё **до** проверки.)
- Как развернуть строку на месте (два указателя навстречу)? (См. главу 10.)
- Чем отличается поведение `%` для отрицательных в C89 и C99?

Часть 2. Каверзы и теория

Здесь код не пишут — объясняют. Эти вопросы отделяют тех, кто «знает синтаксис», от тех, кто понимает Си. Отвечай по существу и с примером.

Каверза 1. Что такое UB и где оно прячется

Неопределённое поведение (undefined behavior, UB) — это ситуации, которые стандарт Си **не описывает вовсе**. Компилятор вправе делать что угодно: выдать ожидаемый результат, выдать мусор, уронить программу или (самое коварное) «оптимизировать» твой код, считая, что UB невозможно. Программа с UB не «иногда работает» — она **некорректна**, даже если сегодня выводит правильное.

Запомни главное правило: **на UB полагаться нельзя**. Поведение может меняться от компилятора, флагов оптимизации, версии и даже соседних строк кода. Частые источники:

```
/* 1. Переполнение знакового int — UB */
int x = INT_MAX;
x = x + 1;              /* НЕ «станет INT_MIN», а UB */

/* 2. Разыменование NULL */
int *p = NULL;
int y = *p;             /* падение или хуже */

/* 3. Выход за границы массива */
int a[3];
a[3] = 5;               /* записали мимо — UB */

/* 4. Неинициализированная память */
int z;
printf("%d", z);       /* читаем мусор — UB */
```

```c
/* 5. Изменение строкового литерала */
char *s = "hello";
s[0] = 'H';            /* литерал в read-only памяти — UB/падение */

/* 6. Использование памяти после free */
free(p);
*p = 7;                /* «use after free» — UB */
```

Почему переполнение **знакового** — это UB, а беззнакового — нет? Для `unsigned` стандарт прямо говорит: считаем по модулю 2^n (заворачивается). А для знакового представление формально не зафиксировано, поэтому переполнение оставили неопределённым — и компиляторы этим пользуются для оптимизаций (например, считают, что `x + 1 > x` всегда истинно).

Лечение: не выходить за границы, инициализировать переменные, проверять указатели на `NULL`, для изменяемых строк заводить `char s[] = "hello";` (массив, а не указатель на литерал), не трогать память после `free`. Под рукой держи санитайзеры: `cc -fsanitize=address,undefined` ловит многие UB на лету.

Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать

Отдельный, неочевидный вид UB. Правило **strict aliasing** гласит: к одному и тому же участку памяти нельзя обращаться через указатели **несовместимых** типов. Компилятор считает, что `int *` и `float *` никогда не указывают на одно место, и оптимизирует, опираясь на это.

```c
float f = 1.0f;
int *p = (int *)&f;    /* смотрим на float как на int */
*p = 0;                /* нарушение strict aliasing — UB */
```

Так часто пытаются «достать биты» числа. Из-за strict aliasing результат непредсказуем при оптимизации. Правильный способ — через `memcpy` или `union`:

```c
int bits;
memcpy(&bits, &f, sizeof bits);   /* законно: memcpy работает с байтами */
```

Исключение из правила — тип `char *` (и `unsigned char *`): через него **можно** смотреть на байты чего угодно. Поэтому обобщённый стек из задачи 3 хранил буфер как `char *` — это безопасно.

Каверза 3. `volatile` — зачем он нужен

`volatile` говорит компилятору: «эта переменная может измениться **в любой момент** способом, которого ты не видишь, — не кэшируй её, не выкидывай обращения, читай каждый раз заново».

Где это критично:

```
volatile int *reg = (volatile int *)0x40000000;  /* аппаратный регистр */
while (*reg == 0) { }   /* ждём, пока железо изменит регистр */
```

Без `volatile` компилятор увидит, что в цикле `*reg` не меняется, и превратит `while (*reg == 0)` в вечный цикл, прочитав регистр один раз. `volatile` запрещает такую оптимизацию.

Второй классический случай — переменная, меняемая в обработчике сигнала:

```
volatile sig_atomic_t stop = 0;   /* флаг «пора остановиться» */
```

Чего `volatile` **не** делает: он **не** обеспечивает потокобезопасность и атомарность. Для многопоточности нужны `_Atomic` / мьютексы, а не `volatile`. Это частая путаница на собеседовании.

Каверза 4. Две роли `static`

Слово `static` в Си означает **разные вещи** в зависимости от места.

1. Внутри функции — переменная живёт от первого вызова до конца программы (а не создаётся заново каждый раз) и сохраняет значение между вызовами:

```
int next_id(void)
{
    static int id = 0;   /* создаётся один раз, помнит значение */
    return ++id;
}
/* вызовы вернут 1, 2, 3, ... */
```

Инициализатор `= 0` срабатывает **один раз**. Память — не на стеке, а в статической области.

2. На уровне файла (у глобальной переменной или функции) — `static` ограничивает видимость **этим файлом** (внутренняя компоновка). Из других `.c`-файлов к ней не достучаться:

```
static int counter;          /* виден только в этом файле */
static void helper(void) { } /* приватная функция модуля */
```

Это способ скрыть детали реализации — аналог `private`. Без `static` глобальные имена видны всей программе, и легко получить конфликт имён при сборке из нескольких файлов.

Каверза 5. Stack vs heap и причины segfault

В программе на Си память делится на области; новичку важны две:

- **Стек** (stack) — здесь живут локальные переменные и кадры вызовов функций. Выделяется и освобождается **автоматически**, быстро, но размер ограничен (обычно несколько МБ). Бесконечная рекурсия его переполняет — **stack overflow** (см. главу 9).
- **Куча** (heap) — память от `malloc`/`calloc`/`realloc`. Большая, живёт, пока сам не вызовешь `free`. За ней следишь **ты**: забыл `free` — утечка; освободил дважды — UB.

```
int arr[1000000];     /* на стеке — рискуешь переполнить стек */
int *arr = malloc(1000000 * sizeof(int));  /* на куче — надёжнее для больших */
```

Segmentation fault («сегфолт») — аварийное завершение при обращении к памяти, которая тебе не принадлежит. Типичные причины:

- разыменование `NULL` или «висячего» указателя (после `free`);
- выход далеко за границы массива;
- запись в строковый литерал;
- переполнение стека глубокой рекурсией.

Сегфолт — это ещё везение: программа хотя бы падает сразу. Хуже, когда UB тихо портит данные.

Каверза 6. Выравнивание и padding: что выведет `sizeof`

```
struct A {
    char  c;    /* 1 байт */
    int   i;    /* 4 байта */
    char  d;    /* 1 байт */
};
printf("%zu\n", sizeof(struct A));   /* НЕ 6, а обычно 12 */
```

Почему не `1 + 4 + 1 = 6`? Процессор читает данные эффективнее, когда они лежат по «круглым» адресам: `int` (4 байта) должен начинаться с адреса, кратного 4. Это **выравнивание** (alignment). Чтобы его соблюсти, компилятор вставляет невидимые **байты-заполнители** (padding):

```
[ c ][pad][pad][pad][  i (4)  ][ d ][pad][pad][pad]
  0    1    2    3    4 5 6 7     8   9   10  11
```

- после `c` (адрес 0) — 3 байта padding, чтобы `i` встал на адрес 4;
- `d` на адресе 8;
- в конце ещё 3 байта, чтобы **весь размер** структуры был кратен выравниванию самого «требовательного» поля (`int`, 4). Это нужно, чтобы в массиве структур каждая следующая тоже была выровнена.

Итого 12. А если переставить поля по убыванию размера — padding почти исчезнет:

```
struct B {
    int   i;    /* 4 */
    char  c;    /* 1 */
    char  d;    /* 1 */
};              /* sizeof == 8: i, c, d, и 2 байта хвоста */
```

Правило: **группируй поля от больших к меньшим** — структура займёт меньше. Точные числа зависят от платформы; узнать выравнивание типа можно через `_Alignof(int)`.

Каверза 7. Опасный макрос `#define SQR(x) x*x`

```
#define SQR(x) x*x
```

Препроцессор делает **текстовую подстановку**, не понимая арифметики. Отсюда два бага.

Баг 1 — приоритет операций. `SQR(a + b)` развернётся в `a + b*a + b`, то есть `a + (b*a) + b`, а вовсе не $(a+b)^2$:

```
int r = SQR(a + b);   /* a + b*a + b — НЕ то, что хотели */
```

Лечится скобками вокруг **каждого аргумента и всего выражения**:

```
#define SQR(x) ((x) * (x))
```

Теперь `SQR(a + b)` → `((a + b) * (a + b))` — верно.

Баг 2 — двойное вычисление аргумента. Даже с правильными скобками `SQR(x)` подставляет `x` **дважды**. Если аргумент имеет побочный эффект, он сработает два раза:

```
int i = 3;
int r = SQR(i++);     /* ((i++) * (i++)) — i увеличится ДВАЖДЫ, результат — мусор */
```

Это UB вдобавок (см. каверзу 9 про порядок). От этого скобки не спасают — лекарство только одно: **не передавай в макросы выражения с побочными эффектами**, а лучше используй `static inline`-функцию:

```
static inline int sqr(int x) { return x * x; }
```

Функция вычислит аргумент один раз и уважает типы. В современном Си макрос для такого уже почти не нужен.

Каверза 8. Идиома `do { ... } while(0)` в макросах

Как написать макрос из нескольких инструкций, чтобы он вёл себя как одна? Наивно:

```
#define SWAP(a, b) { int t = a; a = b; b = t; }
```

Сломается в `if`-без-скобок из-за лишней `;`:

```
if (x) SWAP(p, q); else foo();   /* ; после } разрывает if/else — ошибка */
```

Канонический приём — обернуть тело в `do { ... } while(0)`:

```
#define SWAP(a, b) do { int t = (a); (a) = (b); (b) = t; } while (0)
```

Теперь `SWAP(p, q);` — это один корректный оператор с точкой с запятой на конце, и он спокойно живёт в `if/else`. `while(0)` гарантирует ровно один проход, а компилятор с оптимизацией выкинет фиктивный цикл. Это стандартная идиома многострочных макросов в Си.

Каверза 9. Порядок вычисления аргументов не определён

```
int i = 1;
printf("%d %d\n", i++, i++);   /* что напечатается? */
```

Ответ: **неизвестно**. Стандарт не задаёт, в каком порядке вычисляются аргументы функции — слева направо, справа налево или вперемешку. Может выйти `1 2`, может `2 1`. Более того, изменять `i` дважды без точки следования между изменениями — это **UB**, а не просто «зависит от компилятора».

То же относится к выражениям вроде:

```
a[i] = i++;          /* UB: и читаем, и меняем i без упорядочивания */
foo(i++, i++);       /* UB */
x = f() + g();       /* порядок вызовов f и g не определён */
```

Если `f()` и `g()` имеют побочные эффекты и зависят от порядка — поведение непредсказуемо. Вывод: **не меняй одну переменную несколько раз в одном выражении** и не полагайся на порядок вычисления аргументов. Разнеси по разным строкам.

Каверза 10. Endianness — порядок байтов

Многобайтовое число (`int` — 4 байта) можно хранить в памяти двумя способами:

- **little-endian** — младший байт первым (x86, ARM по умолчанию, большинство ПК);
- **big-endian** — старший байт первым (некоторые сетевые протоколы, старые процессоры; «сетевой порядок» — big-endian).

Проверить порядок на своей машине можно так:

```
int x = 1;
unsigned char *p = (unsigned char *)&x;   /* смотрим на байты числа */
if (p[0] == 1) {
    printf("little-endian\n");   /* младший байт (1) лежит первым */
} else {
    printf("big-endian\n");
}
```

Число `1` в памяти это байты `01 00 00 00` (little) или `00 00 00 01` (big). Читаем первый байт через `unsigned char *` — это **разрешено** strict aliasing (см. каверзу 2). Если там `1` — машина little-endian.

Когда это важно: чтение бинарных файлов и сетевых пакетов, обмен данными между разными машинами. Поэтому в сети числа гоняют в фиксированном порядке через `htons`/`htonl` (host-to-network) и обратно `ntohs`/`ntohl`.

Итог практикума

Ты прошёл три сборника задач с собеседований — Junior, Middle и Senior — и это достойный финал всей книги. Пробежимся по тому, что осело.

На уровне **кода** ты теперь умеешь не просто «сортировать массив», а реализовать quick-sort и merge sort с пониманием их сложности, худших случаев и выбора опорного. Ты собрал **обобщённые** структуры через `void *` и размер элемента, написал хеш-таблицу с разрешением коллизий цепочками, кольцевой буфер, нашёл пересечение списков за O(n+m) и набросал LRU-кэш на связке «двусвязный список + хеш». Это уже не учебные игрушки, а кирпичики, из которых сложены настоящие программы.

На уровне **понимания языка** ты разобрался с тем, что отделяет крепкого Си-разработчика от новичка: что такое UB и почему на него нельзя полагаться, чем опасны strict aliasing и переполнение знакового, зачем нужны `volatile` и две роли `static`, как устроены стек и куча и откуда берётся segfault, почему `sizeof` хитрой структуры больше суммы полей (выравнивание и padding), как не подорваться на макросе `SQR(x)` и зачем макросам идиома `do { } while(0)`, и почему порядок вычисления аргументов и endianness — вещи, о которых нельзя забывать.

Напутствие

Если ты дочитал до этих строк и хотя бы половину задач решил сам — поздравляю, ты уже не новичок. Ты прошёл путь от первого `printf` (глава 1) до LRU-кэша и тонкостей UB. Си не прощает небрежности, и именно поэтому он так многому учит: каждый указатель, каждый `malloc`, каждый байт выравнивания заставляют **понимать**, а не угадывать.

Что делать дальше:

- **пиши код каждый день** — алгоритмы забываются без практики; бери задачи, реализуй, ломай, чини;
- **читай чужой Си** — исходники SQLite, ядра Linux, Redis; там живут лучшие образцы того, как пишут на этом языке;

- **гоняй санитайзеры и предупреждения** — `-Wall -Wextra`, `-fsanitize=address,undefined`, valgrind: пусть инструменты ловят UB за тебя;
- **не бойся низкого уровня** — теперь ты понимаешь, что под капотом, и любой язык повыше будет казаться проще.

Спасибо, что прошёл эту книгу до конца. Главного ты уже добился — научился думать как программист, который **знает, что происходит в памяти**. Остальное — дело практики. Удачи на собеседованиях и в реальных проектах.